Invertible Normalizing Flows for Generative Image Modeling: Executable Methods and Results Log

Matt Tivnan*

September 5, 2026

Abstract

This document is the executable methods-and-results log for the invertible normalizing flow project. Each numbered step of the pipeline is one `step*.py` script paired with one YAML config in `configs/` and one output folder under `outputs/`; the tables and figures below are included directly from those output folders, so the compiled PDF is an exact record of what was run and what it produced. Step 1 (this version) builds PyTorch datasets and dataloaders with deterministic train/val/test splits for the standard generative-image-modeling benchmarks: MNIST, FashionMNIST, CIFAR-10/100, SVHN, the MedMNIST 2D collection, CelebA, Imagenette, ImageNet, AFHQ, FFHQ, CelebA-HQ, and LSUN bedroom.

1 Execution environment

- Host: `axis03`; container: `recon-dev` (image `localhost:5000/recon:acr-20260819-v1`; PyTorch 2.11.0+cu128, torchvision 0.26.0, CUDA 12.4, 2 GPUs).
- Repository mounted at `/dev_ws/invertible_normalizing_flow_20260831` inside the container (shared workspace `/home/staticct/recon-dev/workspace` on the host).
- Datasets live on the axis03 data drive at `/data/image_benchmarks`, mounted into the container at the same path. This is a shared local repository of public benchmarks, reused across projects; nothing dataset-sized is stored in git.
- All steps are run as `docker exec recon-dev python /dev_ws/invertible_normalizing_flow_20260831/s`
- Exact package versions, the resolved config, and the command line for the run shown in this PDF are recorded in `outputs/step1_datasets/data/provenance.json`.

2 Step 1: Datasets and dataloaders

2.1 Methods

- Code: `step1_datasets.py` (repository root). Config: `configs/step1_datasets.yml`. Outputs: `outputs/step1_datasets/{data,figures}/`.
- Command executed for this document: `python step1_datasets.py` (runs every dataset enabled in the config).

*Repository: `github.com/tivnanmatt/invertible_normalizing_flow_20260831`

- The script exposes a library API used by later steps: `build_dataset(name, cfg)` returns a bundle of three `torch.utils.data.Dataset` splits plus metadata, and `get_dataloaders(bundle, cfg)` wraps them in `DataLoader`s (batch size, workers, pinning from the config).
- Auto-download datasets: MNIST, FashionMNIST, CIFAR-10, CIFAR-100, SVHN (torchvision); the MedMNIST 2D collection (`medmnist` package, official train/val/test); Imagenette (fast.ai 10-class ImageNet subset, used as a fast stand-in for ImageNet); AFHQ (StarGAN-v2 release, fetched from the official Dropbox link); CelebA (torchvision Google-Drive download, quota permitting).
- Manual-download datasets (access terms or hosting): ImageNet (ILSVRC2012), FFHQ, CelebA-HQ, LSUN bedroom. When absent, the script records them as *unavailable* together with exact acquisition instructions (Section 2.4) instead of failing.
- Train/val/test policy (deterministic, seeded from the config): official splits are used whenever they exist (MedMNIST, CelebA); datasets with only train/test get a validation set carved from train (10%); datasets whose public split is train/val only (Imagenette, ImageNet, AFHQ) use the official val as *test* and carve val from train; single-pool datasets (FFHQ, CelebA-HQ) are split 80/10/10.
- Transforms: `ToTensor` to $[0, 1]$ floats only; variable-size natural-image datasets additionally get `Resize`+`CenterCrop` to the per-dataset `image_size` in the config. No normalization or dequantization is baked in here – for flow training those belong to the model step so the data step stays lossless.
- Per dataset the run itself is the test: all three splits are instantiated, real batches are pulled through the dataloaders, per-channel mean/std are computed over up to 20 train batches, and three figures are written (sample grid per split, pixel-intensity histogram, class distribution where integer labels exist).
- Machine-readable outputs: one JSON per dataset, `summary.json`, `summary.csv`, `provenance.json`, and the LaTeX fragments (`summary_table.tex`, `figures_step1.tex`, `unavailable_step1.tex`) that are included verbatim below.

2.2 Configuration (verbatim)

```
# Config for step1_datasets.py -- datasets & dataloaders.
#
# data_root is the shared benchmark-dataset repository on the axis03 data
# drive, mounted into the recon-dev container at the same path
# (/data/image_benchmarks). Datasets download/live there once and are reused
# across projects; nothing dataset-sized lands in the git repo.

seed: 20260831
data_root: /data/image_benchmarks
output_root: outputs/step1_datasets

loader:
  batch_size: 64
  num_workers: 4
  pin_memory: true
```

```yaml
split:
  # carved from the official train pool when no official val split exists
  val_fraction: 0.10
  # only used for single-pool datasets with no official splits (ffhq, celeba_hq)
  test_fraction: 0.10

figures:
  samples_per_split: 8     # images per row in the sample-grid figure
  stats_max_batches: 20    # batches used for per-channel mean/std

datasets:
  # ---- small standard benchmarks (auto-download) ----
  mnist:         {enabled: true}
  fashion_mnist: {enabled: true}
  cifar10:       {enabled: true}
  cifar100:      {enabled: true}
  svhn:          {enabled: true}

  # ---- MedMNIST 2D collection (auto-download) ----
  # All 12 2D members are supported; the run set below spans the modalities
  # (pathology, blood smear, dermoscopy, X-ray, ultrasound, fundus, abdominal
  # CT). octmnist/tissuemnist/chestmnist are large; enable by adding flags.
  medmnist:
    enabled: true
    size: 28
    flags:
      - pathmnist
      - bloodmnist
      - dermamnist
      - pneumoniamnist
      - breastmnist
      - retinamnist
      - organamnist

  # ---- faces / natural images ----
  celeba:          # aligned CelebA; auto-download tried, gdrive quota permitting
    enabled: true
    image_size: 64
  imagenette:      # fast.ai 10-class ImageNet subset; auto-download stand-in
    enabled: true  # for full ImageNet in quick experiments
    variant: 160px
    image_size: 64
  imagenet:        # full ILSVRC2012; manual download (access terms)
    enabled: true
    image_size: 64
  afhq:            # animal faces (cat/dog/wild), StarGAN-v2 release
    enabled: true
    version: v1
    image_size: 64
    attempt_download: true
  ffhq:            # 70k faces; manual download (NVlabs)
    enabled: true
    image_size: 128
  celeba_hq:       # 30k faces; manual download
```

```
    enabled: true
    image_size: 256
  lsun_bedroom:    # LSUN lmdb; manual download
    enabled: true
    image_size: 64
```

2.3 Results: dataset summary

dataset	status	train	val	test	image	classes
mnist	ok	54000	6000	10000	$1 \times 28 \times 28$	10
fashion_mnist	ok	54000	6000	10000	$1 \times 28 \times 28$	10
cifar10	ok	45000	5000	10000	$3 \times 32 \times 32$	10
cifar100	ok	45000	5000	10000	$3 \times 32 \times 32$	100
svhn	ok	65931	7326	26032	$3 \times 32 \times 32$	10
medmnist_pathmnist	ok	89996	10004	7180	$3 \times 28 \times 28$	9
medmnist_bloodmnist	ok	11959	1712	3421	$3 \times 28 \times 28$	8
medmnist_dermamnist	ok	7007	1003	2005	$3 \times 28 \times 28$	7
medmnist_pneumoniamnist	ok	4708	524	624	$1 \times 28 \times 28$	2
medmnist_breastmnist	ok	546	78	156	$1 \times 28 \times 28$	2
medmnist_retinamnist	ok	1080	120	400	$3 \times 28 \times 28$	5
medmnist_organamnist	ok	34561	6491	17778	$1 \times 28 \times 28$	11
celeba	unavailable	–	–	–	–	–
imagenette	ok	8522	947	3925	$3 \times 64 \times 64$	10
imagenet	unavailable	–	–	–	–	–
afhq	ok	13167	1463	1500	$3 \times 64 \times 64$	3
ffhq	unavailable	–	–	–	–	–
celeba_hq	unavailable	–	–	–	–	–
lsun_bedroom	unavailable	–	–	–	–	–

Split sizes are reported after the split policy above; *image* is the tensor shape $C \times H \times W$ delivered by the dataloaders; *classes* is the number of integer classes ("–" for unconditional / multi-label datasets).

2.4 Results: unavailable datasets

- **celeba**: CelebA auto-download failed (Google Drive quota is a known issue: RuntimeError). Manually place img_align_celeba/ plus the list_*.txt / identity_* files under data/celeba/celeba/ (see torchvision.datasets.CelebA docs), then re-run.
- **imagenet**: ImageNet (ILSVRC2012) requires manual download (terms of access): obtain from https://image-net.org, extract to data/imagenet/train/<wnid>/*.JPEG and data/imagenet/val/<wnic then re-run.
- **ffhq**: FFHQ requires manual download: place images (e.g. thumbnails128x128 or images1024x1024 from github.com/NVlabs/ffhq-dataset) under data/ffhq/
- **celeba_hq**: CelebA-HQ requires manual download: place the 30k CelebA-HQ images (e.g. celeba_hq_256 from the progressive-GAN release) under data/celeba_hq/

- **lsun_bedroom**: LSUN bedroom requires manual download: fetch bedroom_train_lmdb / bedroom_val_lmdb with github.com/fyu/lsun download.py into data/lsun/ (also needs 'pip install lmdb').

2.5 Results: figures

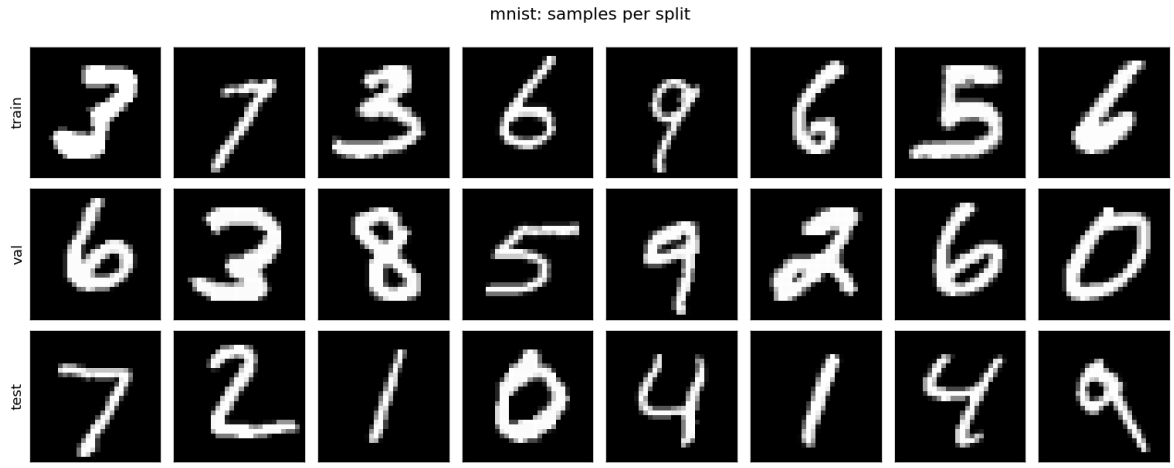


Figure 1: **mnist** (samples). Source: torchvision.datasets.MNIST (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/mnist_samples.png`, produced by `step1_datasets.py`.

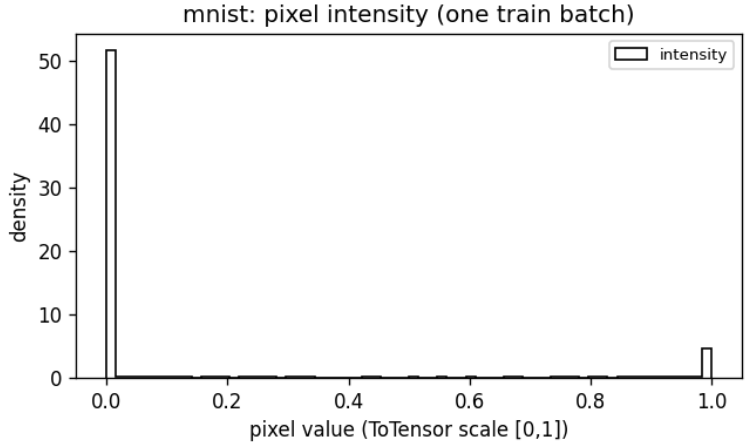


Figure 2: **mnist** (hist). Source: torchvision.datasets.MNIST (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/mnist_pixel_hist.png`, produced by `step1_datasets.py`.

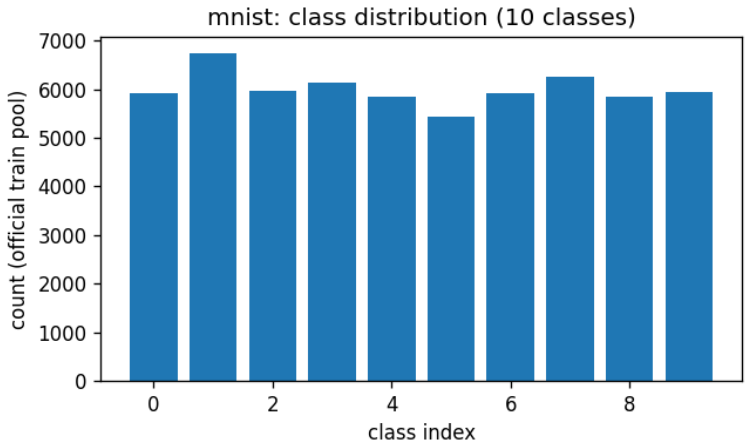


Figure 3: **mnist** (dist). Source: torchvision.datasets.MNIST (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/mnist_class_dist.png`, produced by `step1_datasets.py`.

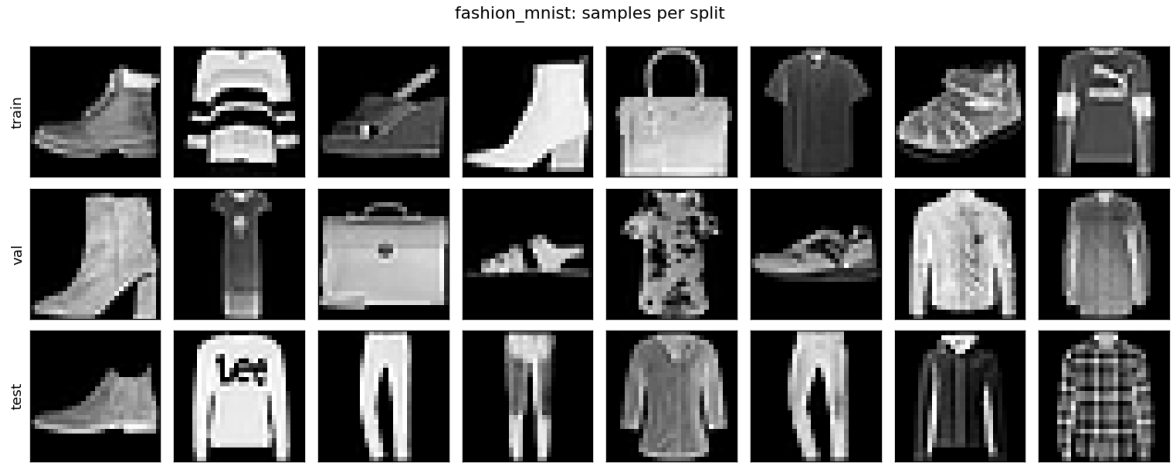


Figure 4: **fashion_mnist** (samples). Source: torchvision.datasets.FashionMNIST (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/fashion_mnist_samples.png`, produced by `step1_datasets.py`.

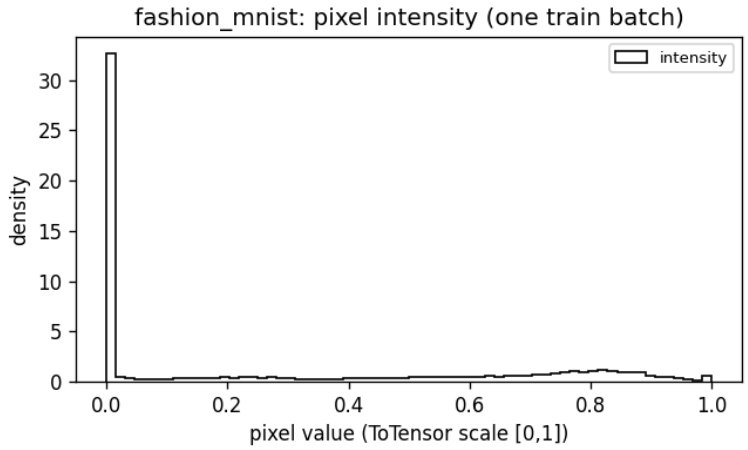


Figure 5: **fashion_mnist** (hist). Source: torchvision.datasets.FashionMNIST (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/fashion_mnist_pixel_hist.png`, produced by `step1_datasets.py`.

Figure 6: **fashion_mnist** (dist). Source: torchvision.datasets.FashionMNIST (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/fashion_mnist_class_dist.png`, produced by `step1_datasets.py`.

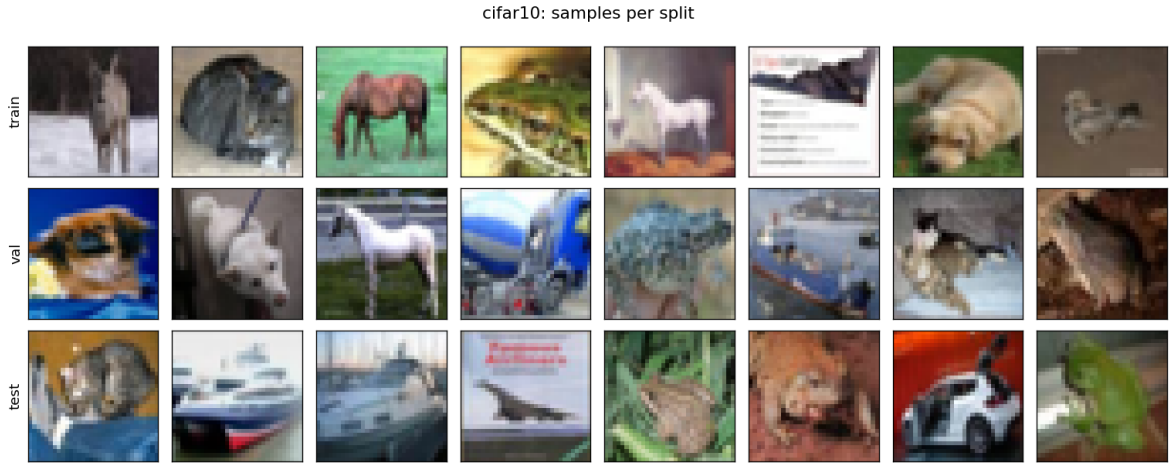


Figure 7: **cifar10** (samples). Source: torchvision.datasets.CIFAR10 (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/cifar10_samples.png`, produced by `step1_datasets.py`.

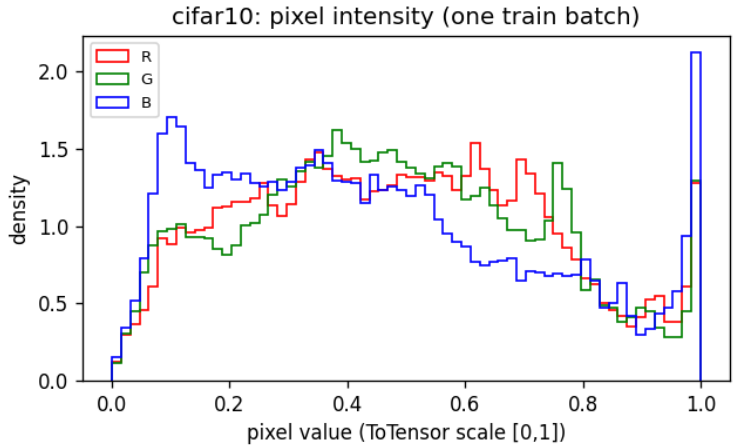


Figure 8: **cifar10** (hist). Source: torchvision.datasets.CIFAR10 (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/cifar10_pixel_hist.png`, produced by `step1_datasets.py`.

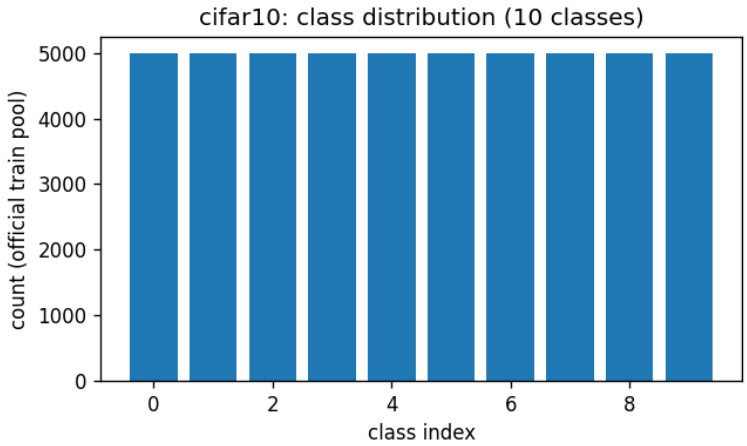


Figure 9: **cifar10** (dist). Source: torchvision.datasets.CIFAR10 (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/cifar10_class_dist.png`, produced by `step1_datasets.py`.

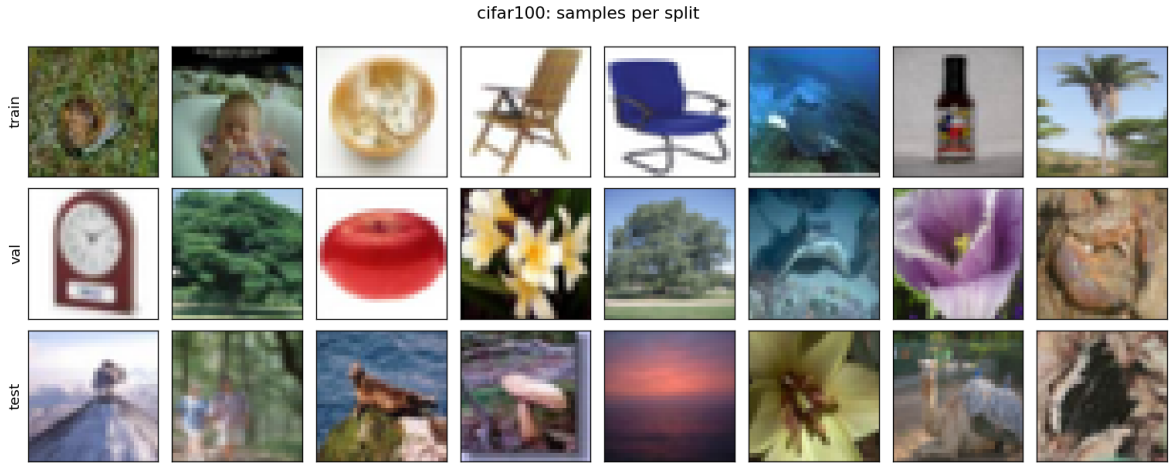


Figure 10: **cifar100** (samples). Source: torchvision.datasets.CIFAR100 (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/cifar100_samples.png`, produced by `step1_datasets.py`.

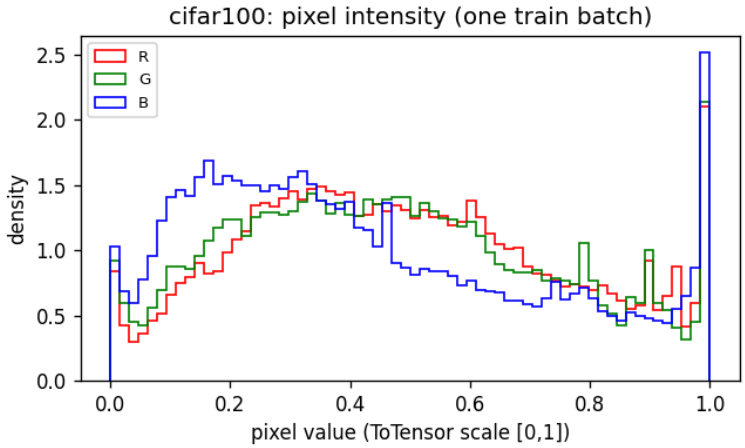


Figure 11: **cifar100** (hist). Source: torchvision.datasets.CIFAR100 (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/cifar100_pixel_hist.png`, produced by `step1_datasets.py`.

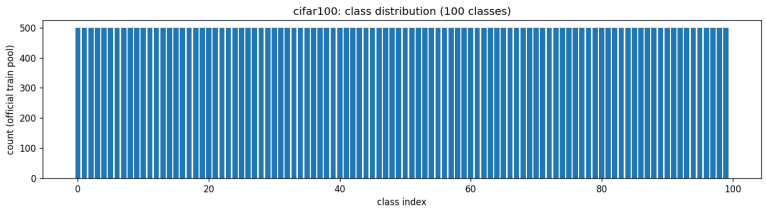


Figure 12: **cifar100** (dist). Source: torchvision.datasets.CIFAR100 (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/cifar100_class_dist.png`, produced by `step1_datasets.py`.

Figure 13: **svhn** (samples). Source: torchvision.datasets.SVHN (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/svhn_samples.png`, produced by `step1_datasets.py`.

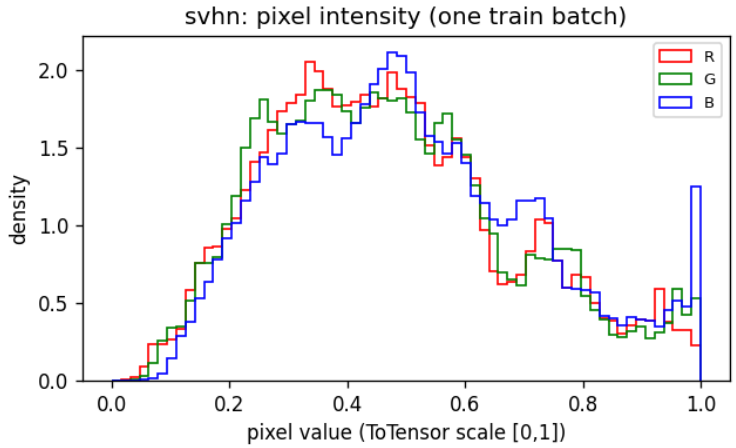


Figure 14: **svhn** (hist). Source: torchvision.datasets.SVHN (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/svhn_pixel_hist.png`, produced by `step1_datasets.py`.

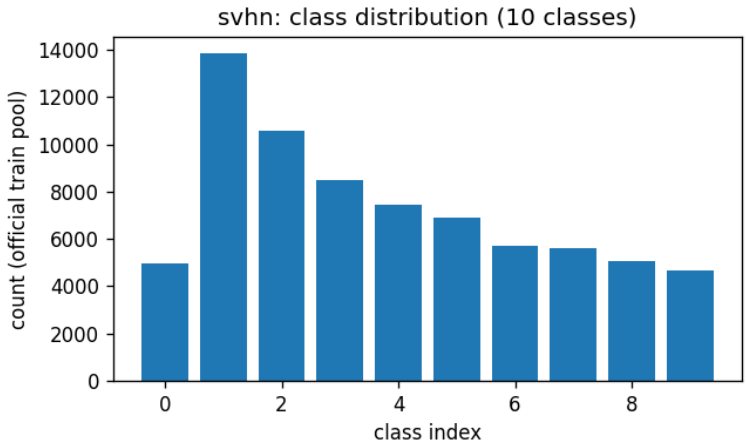


Figure 15: **svhn** (dist). Source: torchvision.datasets.SVHN (auto-download). Split: official test; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/svhn_class_dist.png`, produced by `step1_datasets.py`.

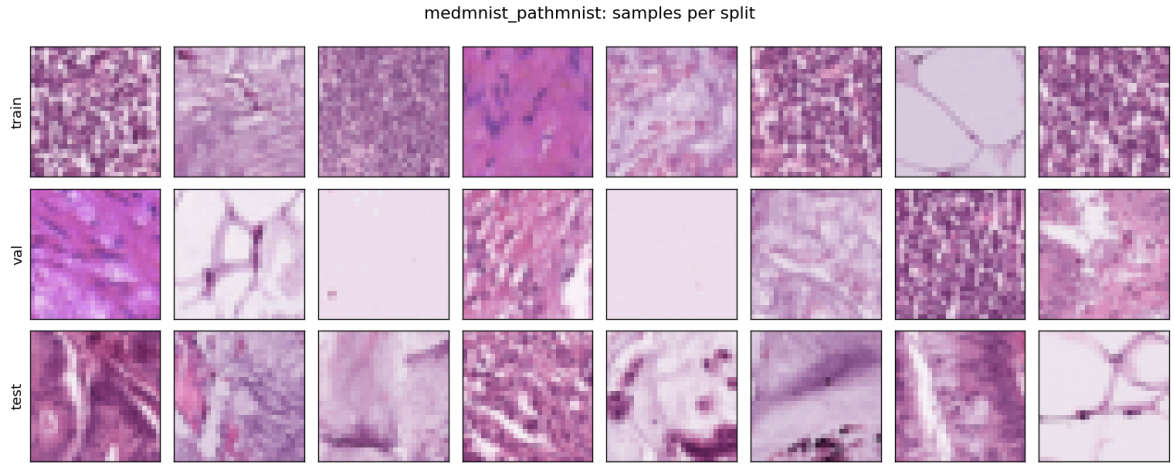


Figure 16: **medmnist_pathmnist** (samples). Source: medmnist.PathMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_pathmnist_samples.png`, produced by `step1_datasets.py`.

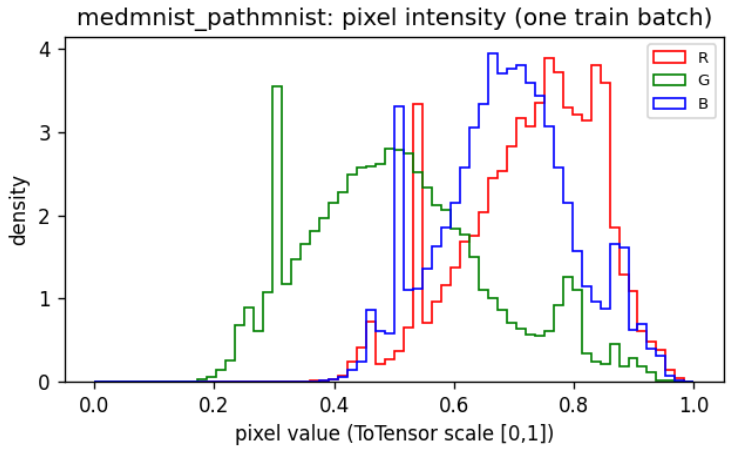


Figure 17: **medmnist_pathmnist** (hist). Source: medmnist.PathMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_pathmnist_pixel_hist.png`, produced by `step1_datasets.py`.

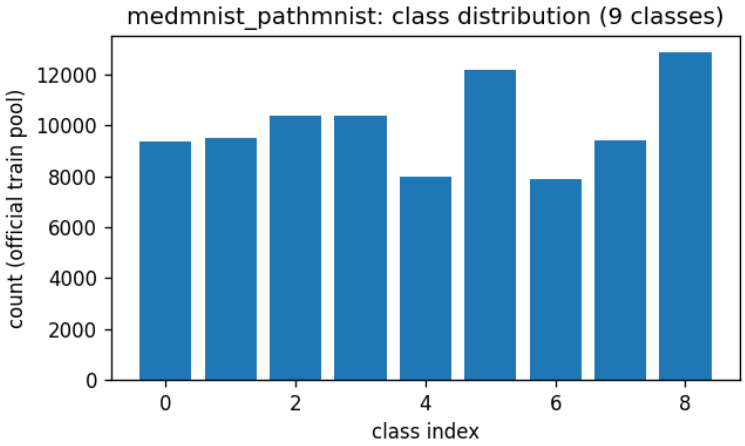


Figure 18: **medmnist_pathmnist** (dist). Source: medmnist.PathMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_pathmnist_class_dist.png`, produced by `step1_datasets.py`.

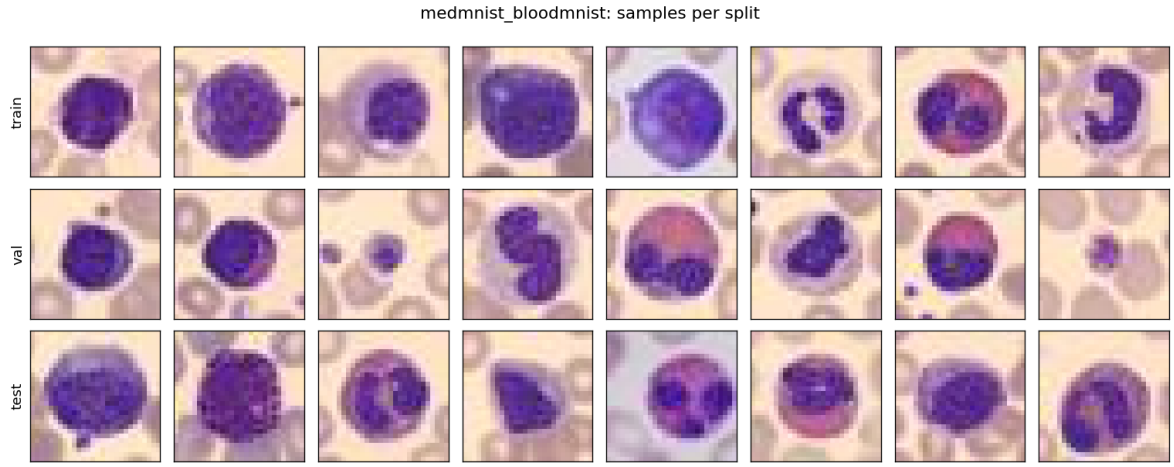


Figure 19: **medmnist_bloodmnist** (samples). Source: medmnist.BloodMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_bloodmnist_samples.png`, produced by `step1_datasets.py`.

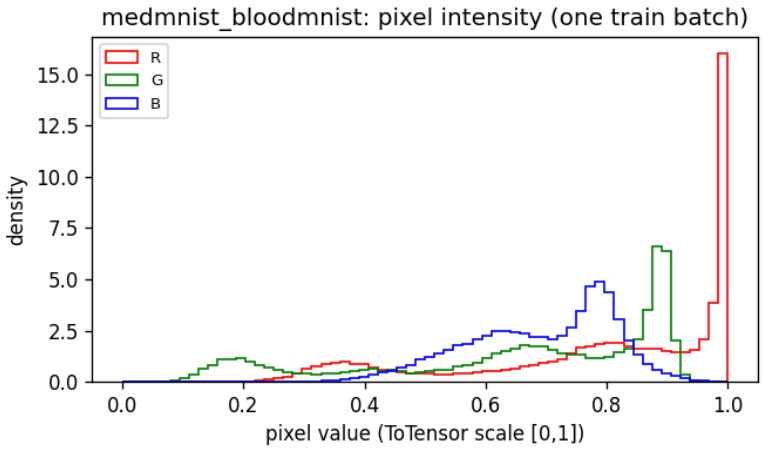


Figure 20: **medmnist_bloodmnist** (hist). Source: medmnist.BloodMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_bloodmnist_pixel_hist.png`, produced by `step1_datasets.py`.

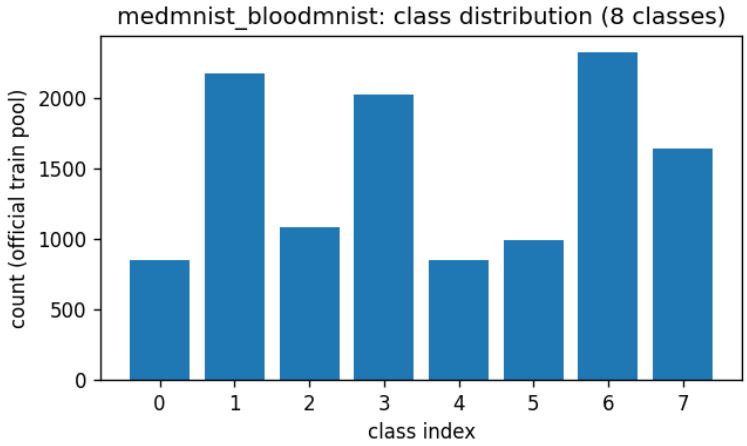


Figure 21: **medmnist_bloodmnist** (dist). Source: medmnist.BloodMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_bloodmnist_class_dist.png`, produced by `step1_datasets.py`.

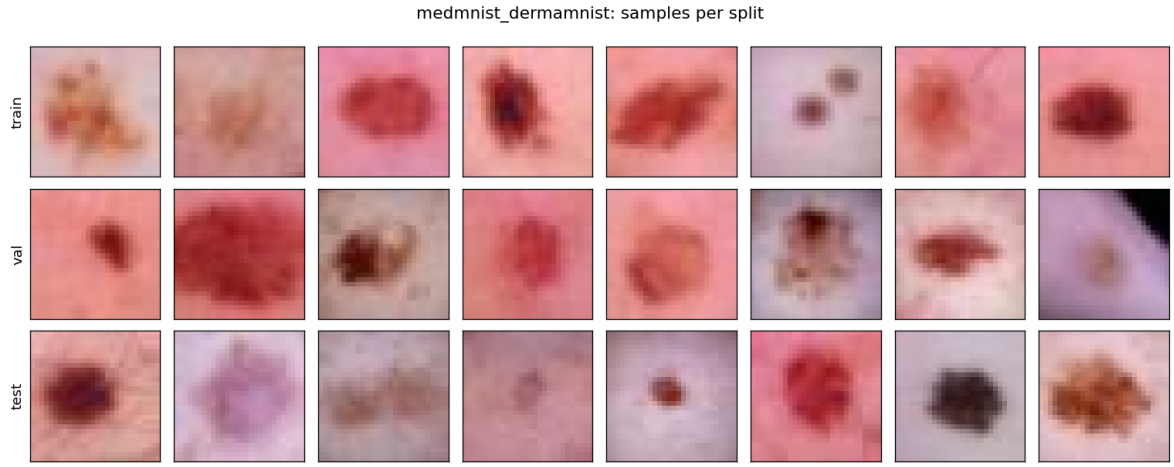


Figure 22: **medmnist_dermamnist** (samples). Source: medmnist.DermaMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_dermamnist_samples.png`, produced by `step1_datasets.py`.

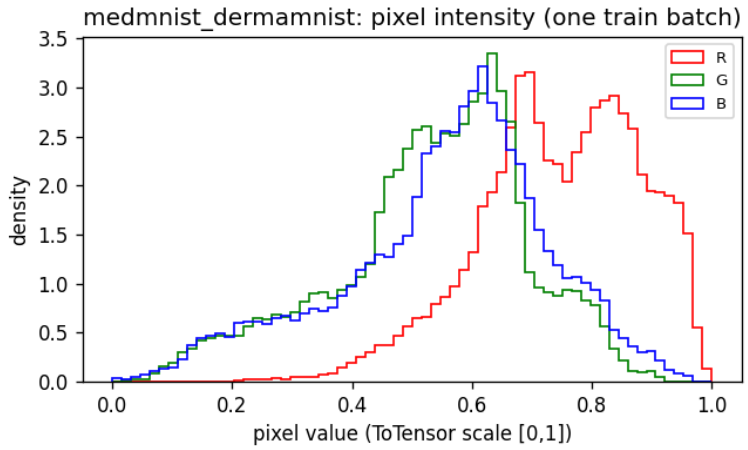


Figure 23: **medmnist_dermamnist** (hist). Source: medmnist.DermaMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_dermamnist_pixel_hist.png`, produced by `step1_datasets.py`.

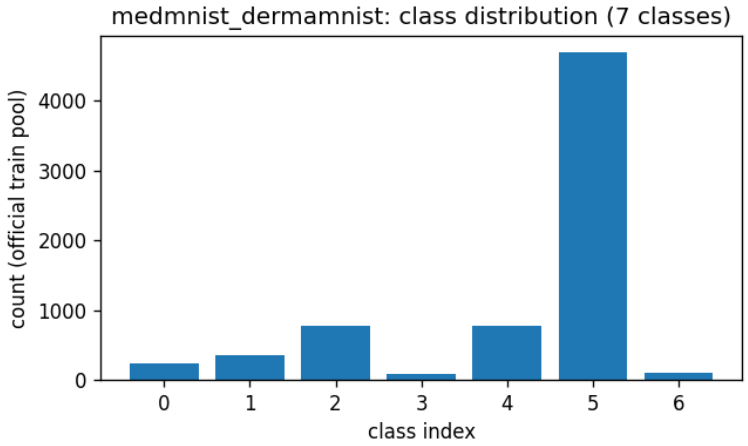


Figure 24: **medmnist_dermamnist** (dist). Source: medmnist.DermaMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_dermamnist_class_dist.png`, produced by `step1_datasets.py`.

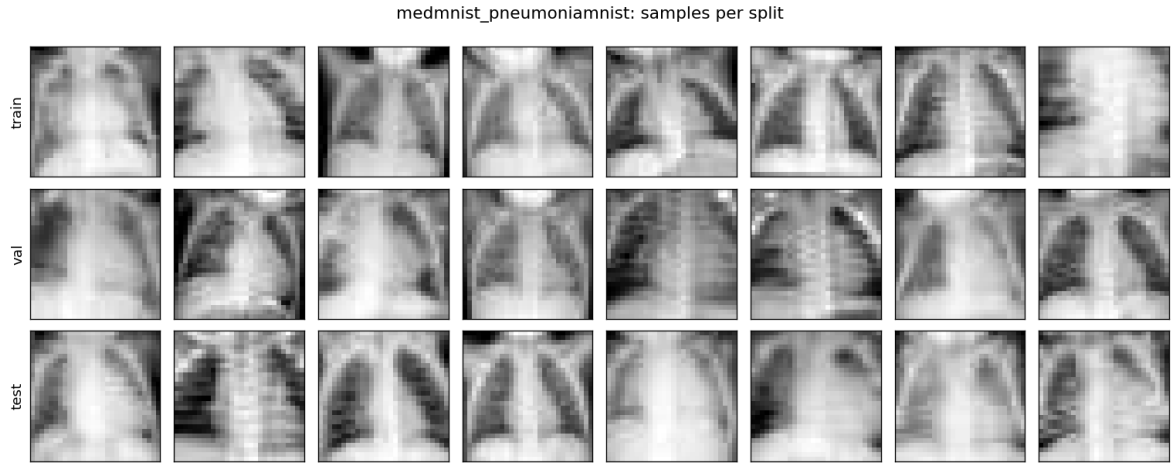


Figure 25: **medmnist_pneumoniamnist** (samples). Source: medmnist.PneumoniaMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_pneumoniamnist_samples.png`, produced by `step1_datasets.py`.

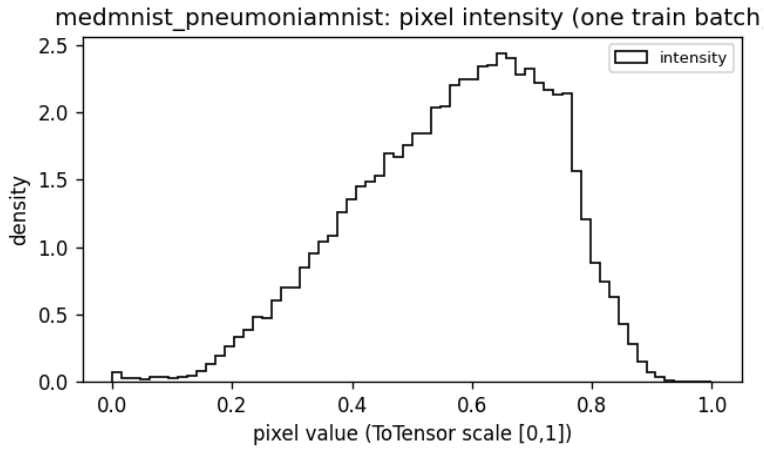


Figure 26: **medmnist_pneumoniamnist** (hist). Source: medmnist.PneumoniaMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_pneumoniamnist_pixel_hist.png`, produced by `step1_datasets.py`.

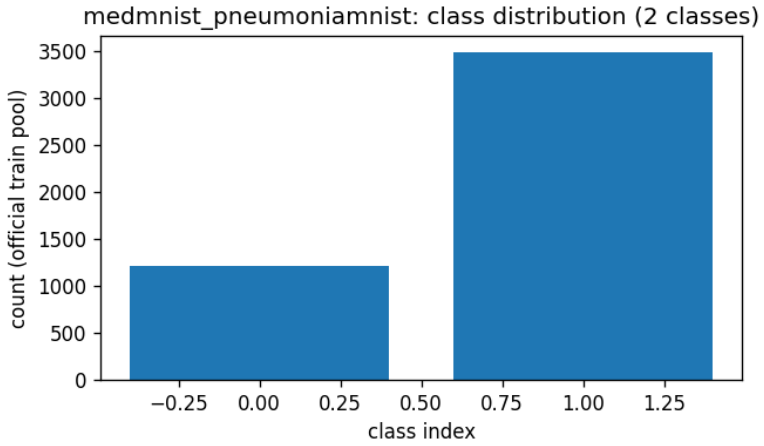


Figure 27: **medmnist_pneumoniamnist** (dist). Source: medmnist.PneumoniaMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_pneumoniamnist_class_dist.png`, produced by `step1_datasets.py`.

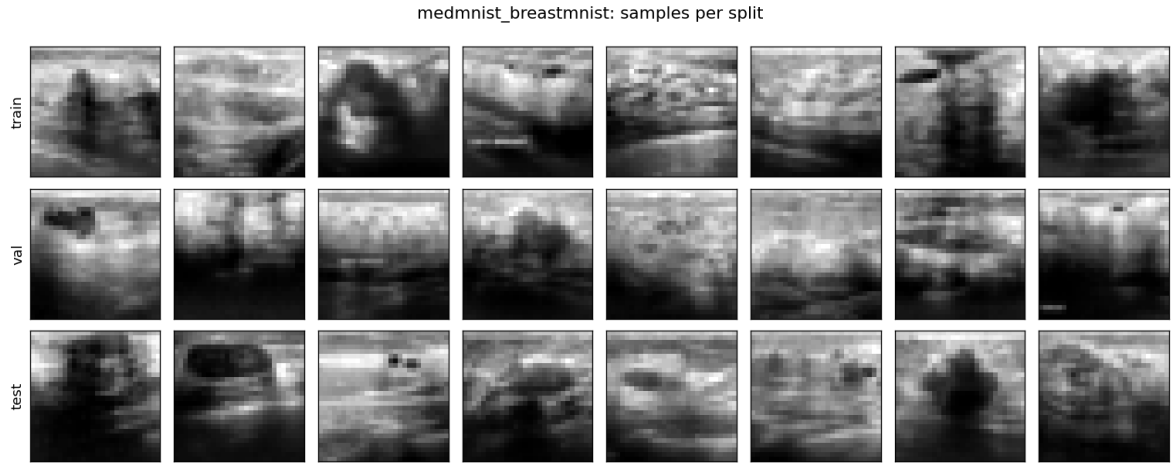


Figure 28: **medmnist_breastmnist** (samples). Source: medmnist.BreastMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_breastmnist_samples.png`, produced by `step1_datasets.py`.

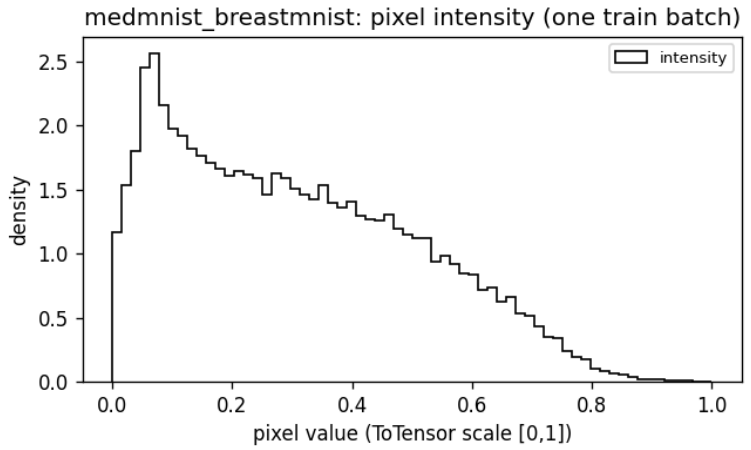


Figure 29: **medmnist_breastmnist** (hist). Source: medmnist.BreastMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_breastmnist_pixel_hist.png`, produced by `step1_datasets.py`.

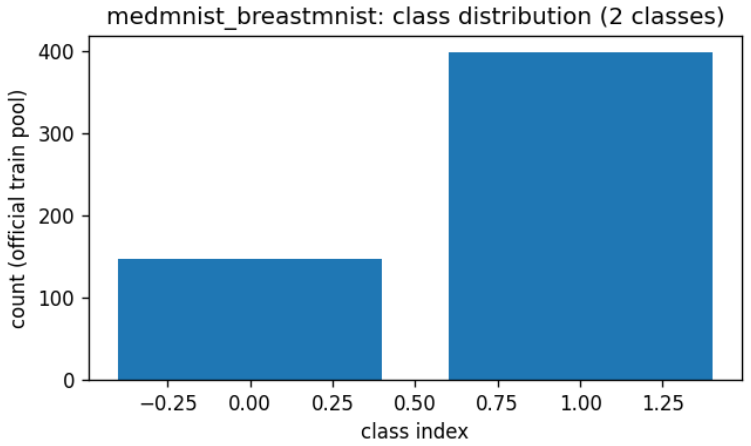


Figure 30: **medmnist_breastmnist** (dist). Source: medmnist.BreastMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_breastmnist_class_dist.png`, produced by `step1_datasets.py`.

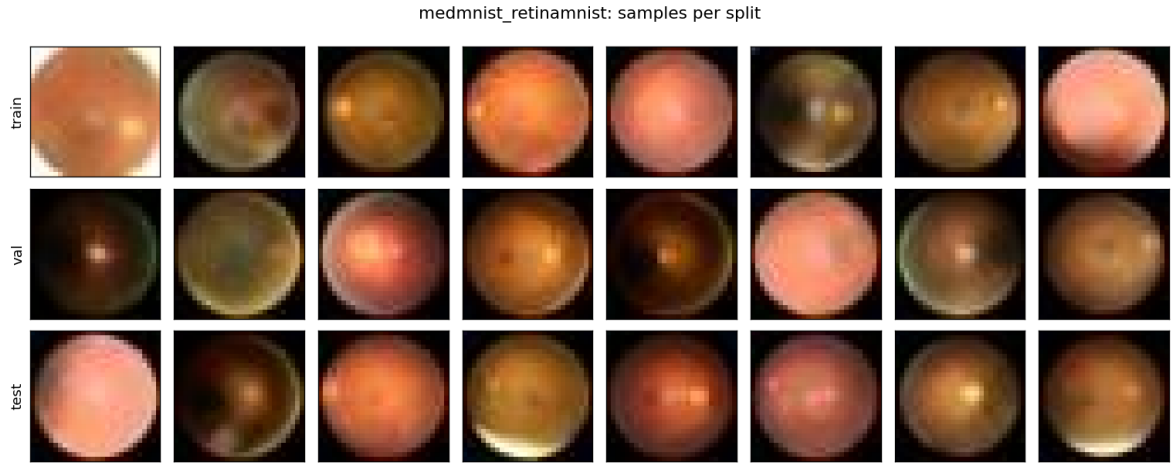


Figure 31: **medmnist_retinamnist** (samples). Source: medmnist.RetinaMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_retinamnist_samples.png`, produced by `step1_datasets.py`.

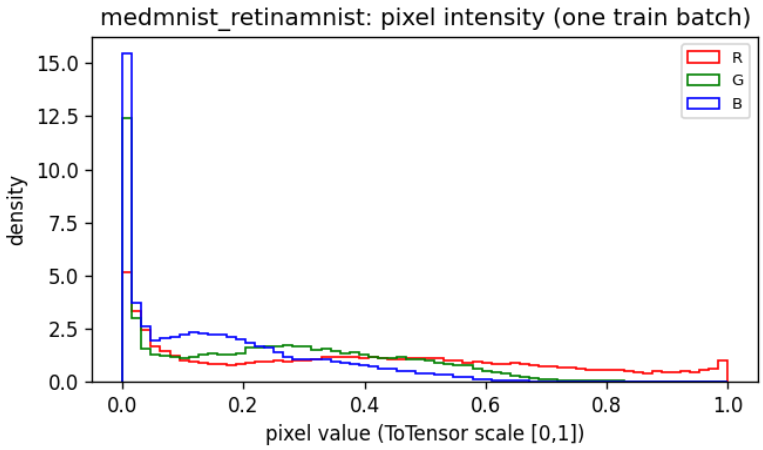


Figure 32: **medmnist_retinamnist** (hist). Source: medmnist.RetinaMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_retinamnist_pixel_hist.png`, produced by `step1_datasets.py`.

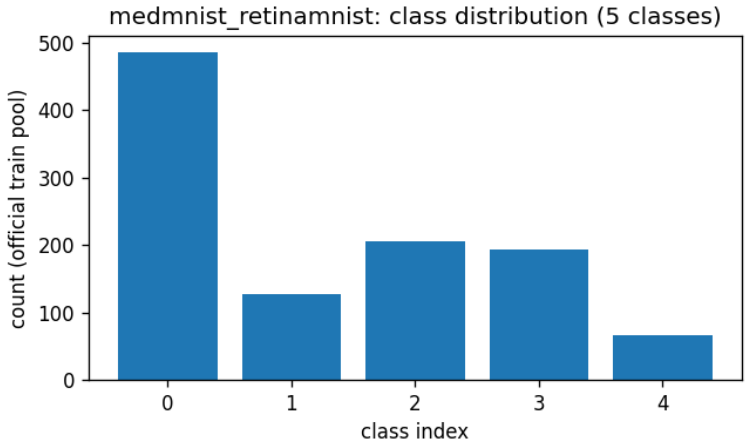


Figure 33: **medmnist_retinamnist** (dist). Source: medmnist.RetinaMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_retinamnist_class_dist.png`, produced by `step1_datasets.py`.

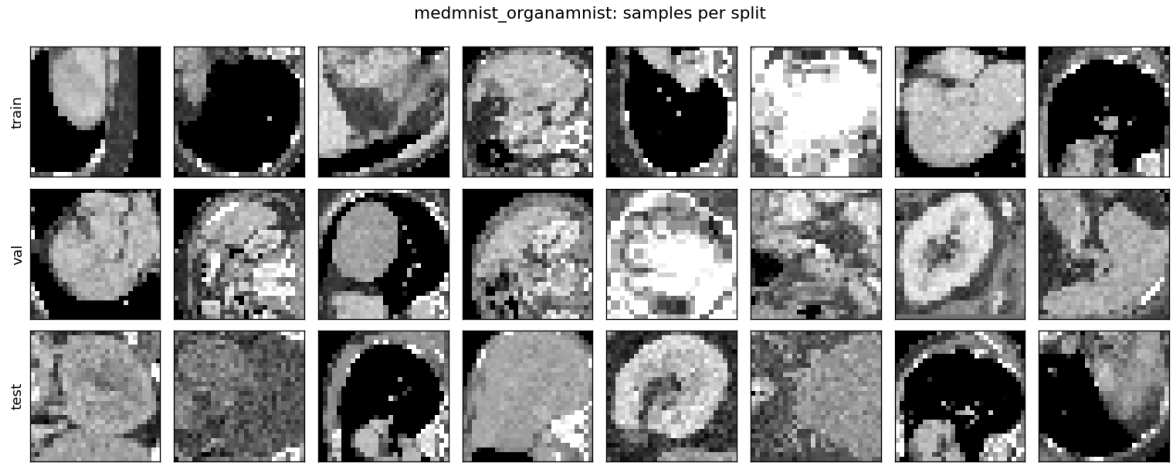


Figure 34: **medmnist_organamnist** (samples). Source: medmnist.OrganAMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_organamnist_samples.png`, produced by `step1_datasets.py`.

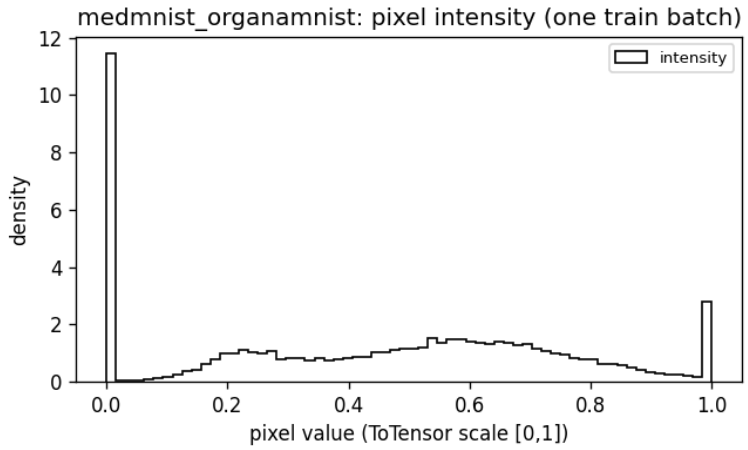


Figure 35: **medmnist_organamnist** (hist). Source: medmnist.OrganAMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_organamnist_pixel_hist.png`, produced by `step1_datasets.py`.

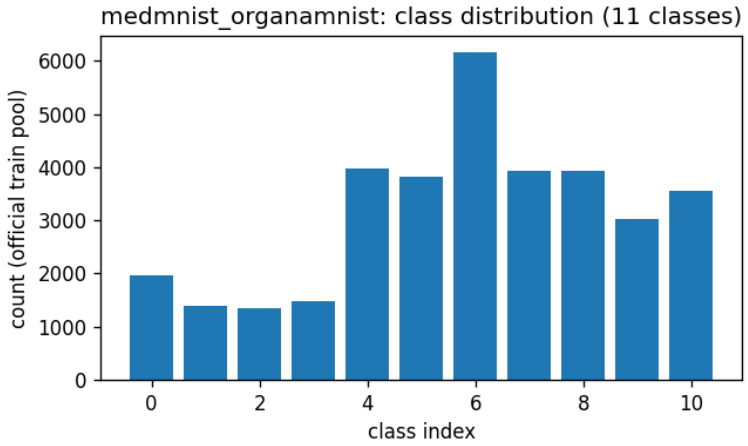


Figure 36: **medmnist_organamnist** (dist). Source: medmnist.OrganAMNIST v3.0.2 (auto-download). Split: official MedMNIST train/val/test. File: `outputs/step1_datasets/figures/medmnist_organamnist_class_dist.png`, produced by `step1_datasets.py`.

Figure 37: **imagenette** (samples). Source: torchvision.datasets.Imagenette 160px (fast.ai ImageNet-10 subset, auto-download). Split: official val used as TEST; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/imagenette_samples.png`, produced by `step1_datasets.py`.

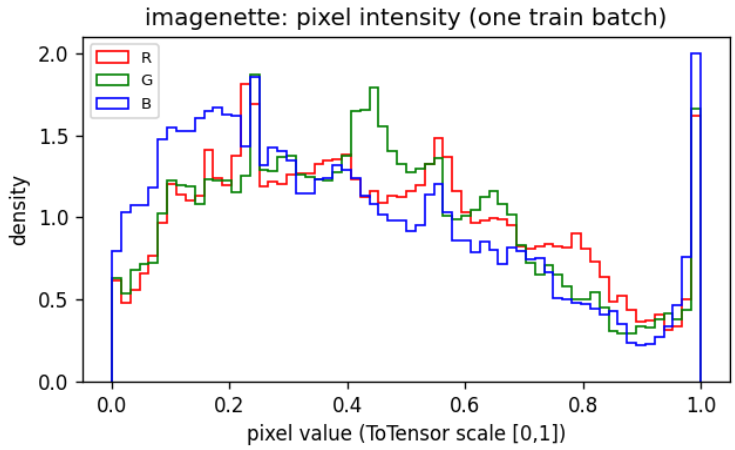


Figure 38: **imagenette** (hist). Source: torchvision.datasets.Imagenette 160px (fast.ai ImageNet-10 subset, auto-download). Split: official val used as TEST; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/imagenette_pixel_hist.png`, produced by `step1_datasets.py`.

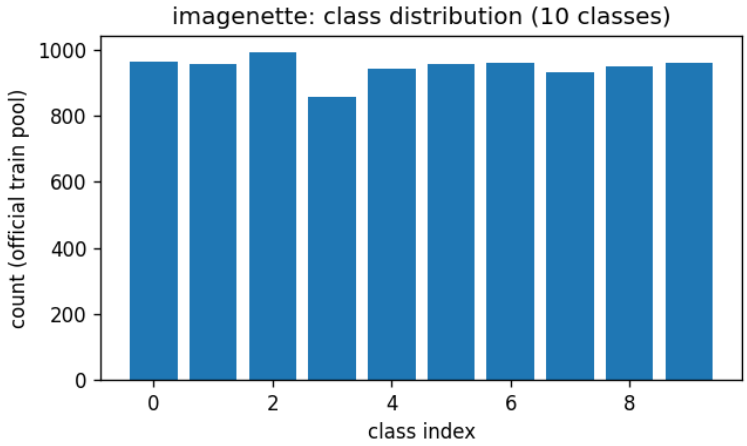


Figure 39: **imagenette** (dist). Source: torchvision.datasets.Imagenette 160px (fast.ai ImageNet-10 subset, auto-download). Split: official val used as TEST; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/imagenette_class_dist.png`, produced by `step1_datasets.py`.

Figure 40: **afhq** (samples). Source: AFHQ v1 (StarGAN-v2 release, cat/dog/wild, auto-download from dropbox). Split: official val used as TEST; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/afhq_samples.png`, produced by `step1_datasets.py`.

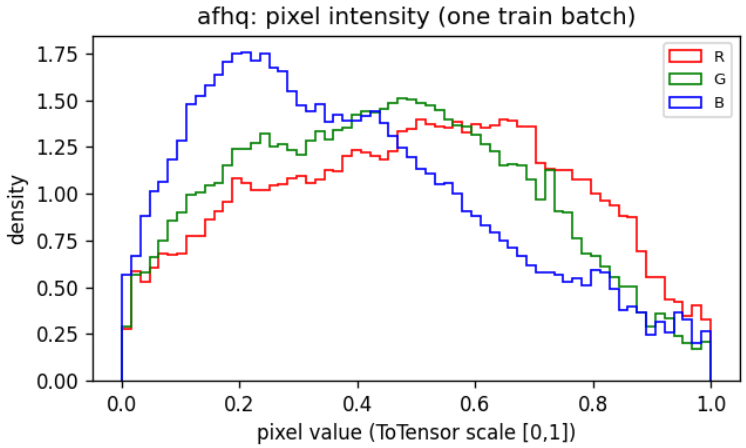


Figure 41: **afhq** (hist). Source: AFHQ v1 (StarGAN-v2 release, cat/dog/wild, auto-download from dropbox). Split: official val used as TEST; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/afhq_pixel_hist.png`, produced by `step1_datasets.py`.

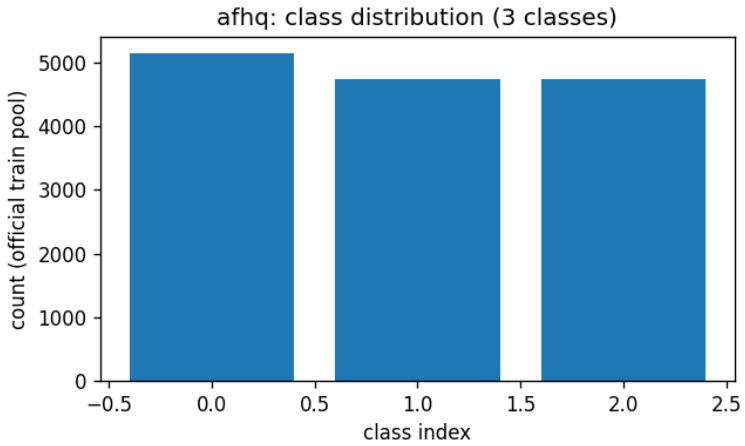


Figure 42: **afhq** (dist). Source: AFHQ v1 (StarGAN-v2 release, cat/dog/wild, auto-download from dropbox). Split: official val used as TEST; val = 10% of official train (seed 20260831). File: `outputs/step1_datasets/figures/afhq_class_dist.png`, produced by `step1_datasets.py`.

2.6 Analysis

- Coverage: the available benchmarks span 28×28 grayscale (MNIST, several MedMNIST members) through 64×64 RGB natural images and faces (CIFAR, Imagenette, AFHQ, CelebA), which is the standard ladder for likelihood-based generative models: density results in bits/dim on MNIST/CIFAR-10, then perceptual-quality results on faces/animals at higher resolution.

- The pixel-intensity histograms show the strongly peaked, saturated distributions typical of 8-bit images (large masses at 0 and 1, e.g. MNIST background, CelebA skin tones). For normalizing flows this directly motivates uniform dequantization and a logit transform before the first coupling layer – to be implemented and ablated in the model step, not in the data step.

- Class-distribution figures document which datasets are usable for class-conditional modeling and how imbalanced they are; several MedMNIST members are markedly imbalanced, which matters for conditional likelihood evaluation.

- Per-channel means/stds recorded in each dataset's JSON give the constants for any input normalization later steps choose to apply.

- Datasets marked unavailable (typically ImageNet, FFHQ, CelebA-HQ, LSUN, and CelebA when the Google Drive quota blocks download) are access-restricted or manually hosted; the pipeline documents the exact acquisition path and picks them up automatically once placed under `/data/image_benchmarks`.

3 Step 2: TarFlow – Transformer Autoregressive Flow

3.1 Methods

- Model: TarFlow from *Normalizing Flows are Capable Generative Models* (Zhai et al., Apple, arXiv:2412.06329). A TarFlow is a stack of T causal-ViT "metablocks", each an autoregressive NVP flow over image patches; the patch order is flipped between consecutive blocks. Training maximizes exact likelihood; sampling inverts the blocks patch-by-patch with KV caching.

- Code: the *official* implementation (`github.com/apple/ml-tarflow`, Apple sample-code license) is used unmodified: `transformer_flow.py` is imported directly from a clone at `/dev_ws/ml-tarflow` (commit hash in `outputs/step2_tarflow/data/provenance.json`). Our driver `step2_tarflow.py` reimplements the surrounding training loop faithfully to the official `train.py` / `train_local.ipynb` / `evaluate_bpd.py`: AdamW ($\beta = (0.9, 0.95)$, wd 10^{-4}), cosine LR with one warmup epoch, inputs scaled to $[-1, 1]$, gaussian-noise runs in bf16 autocast, uniform-dequantization (likelihood) runs in fp32, and the official bits/dim formula (Gaussian prior term with the $\log 128$ scaling correction, generalized from 3 channels to C).

- Data: datasets and deterministic splits come from step 1 (`step1_datasets.build_dataset`); an adapter rescales to $[-1, 1]$ and applies the official random horizontal flip where configured.

- Config: `configs/step2_tarflow.yml`. Outputs: `outputs/step2_tarflow/{data,figures}/`. Commands executed: `python step2_tarflow.py --runs mnist_toy` (GPU 0) and `python step2_tarflow.py --runs cifar10_uniform` (GPU 1), then `python step2_tarflow.py --collect`.

- Run **mnist_toy**: exact replication of Apple's released MNIST toy experiment (`train_local.ipynb` defaults): class-conditional TarFlow [4-128-4-4] (patch-channels-blocks-layers), gaussian noise $\sigma = 0.1$, batch 256, lr 2×10^{-3}, 100 epochs. Deliverables: conditional sample grid (one row per digit) and the notebook's Bayes-rule classifier evaluation $\arg\min_y \frac{1}{2}\mathbb{E}[z_y^2] - \log|\det|_y$ on the step-1 test split.
- Run **cifar10_uniform**: unconditional CIFAR-10 likelihood run in the paper's likelihood configuration style: uniform dequantization, fp32, patch size 2 (as in their ImageNet 64 2.99-bpd config), TarFlow [2-256-8-4] ($\sim$26M params), batch 128, lr 2×10^{-4}, 60 epochs on one RTX 4090. Validation bpd is tracked every 5 epochs; final test bpd uses the official formula on the step-1 test split.
- Scale caveat (stated up front): the paper's quantitative likelihood benchmark is *unconditional ImageNet 64×64*, where TarFlow [2-768-8-8] ($\sim$470M params, 8 × A100, 200 epochs) reaches **2.99 bpd**; the paper reports no MNIST or CIFAR-10 numbers. ImageNet is not yet staged (step 1), so this step validates the framework end-to-end at small scale and positions results against the classic CIFAR-10 flow literature (RealNVP 3.49, Glow 3.35, Flow++ 3.08 bpd).
- Sampling uses the plain reverse pass from Gaussian noise; the paper's score-based denoising step (Tweedie correction) is not applied here.

3.2 Configuration (verbatim)

```
# Config for step2_tarflow.py -- TarFlow training/testing.
#
# tarflow_repo: clone of the OFFICIAL github.com/apple/ml-tarflow (model code
# imported unmodified; commit recorded in outputs provenance).

# GPU POLICY (2026-09-01): GPU0 ONLY. GPU1 failed 2026-08-31 and, though it
# enumerates again after the reboot, we do not use it. The recon-dev container
# is pinned to GPU0 (compose device_ids + CUDA_VISIBLE_DEVICES=0), so cuda:0 is
# the only valid device here; runs are sequential rather than one-per-GPU.
seed: 20260831
tarflow_repo: /dev_ws/ml-tarflow
output_root: outputs/step2_tarflow

runs:
  # Exact replication of Apple's released MNIST toy experiment
  # (train_local.ipynb defaults): class-conditional TarFlow [4-128-4-4],
  # gaussian noise 0.1, bs 256, lr 2e-3, 100 epochs.
  mnist_toy:
    device: cuda:0
    dataset: mnist
    conditional: true
    img_size: 28
    channel_size: 1
    hflip: false
    patch_size: 4
    channels: 128
    blocks: 4
    layers_per_block: 4
    noise_type: gaussian
    noise_std: 0.1
```

```
  cfg: 0
  batch_size: 256
  lr: 2.0e-3
  epochs: 100
  sample_freq: 10
  sample_rows: 10      # one row per digit class, like the notebook's grid
  sample_nrow: 10
  eval_bpd: false      # gaussian-noise model: bpd not meaningful
  eval_freq: 0
  classifier_eval: true  # Bayes-rule p(y|x) eval from the notebook

# Unconditional CIFAR-10 likelihood run in the paper's likelihood style:
# uniform dequantization, fp32 (official train.py disables amp for uniform),
# patch 2 (as in the ImageNet64 2.99-bpd config), scaled to 1x RTX 4090.
# Paper-scale reference: TarFlow [2-768-8-8] on unconditional ImageNet64,
# 8x A100, reaches 2.99 bpd; classic CIFAR-10 flows: RealNVP 3.49,
# Glow 3.35, Flow++ 3.08 bpd.
cifar10_uniform:
  device: cuda:0
  dataset: cifar10
  conditional: false
  img_size: 32
  channel_size: 3
  hflip: true          # official train.py uses RandomHorizontalFlip
  patch_size: 2
  channels: 256
  blocks: 8
  layers_per_block: 4
  noise_type: uniform
  cfg: 0
  batch_size: 128
  lr: 2.0e-4
  epochs: 60
  sample_freq: 10
  sample_rows: 8
  sample_nrow: 8
  eval_bpd: true
  eval_freq: 5
  classifier_eval: false
```

3.3 Results

run	dataset	model [p-c-b-l]	noise	params	epochs	result
mnist_toy	mnist	4-128-4-4	gaussian(0.1)	3.2M	100	Bayes acc 97.62%
cifar10_uniform	cifar10	2-256-8-4	uniform	25.9M	60	interrupted at epoch 9/60, val bp

Model column is [patch-channels-blocks-layers/block]. Full per-epoch metrics are in `outputs/step2_tarflow/data`

3.4 Results: figures

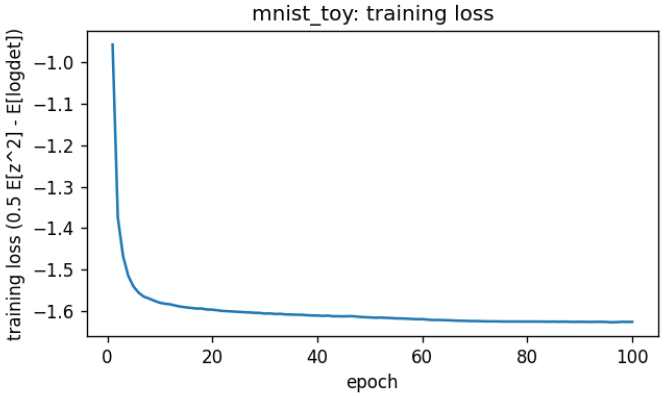


Figure 43: **mnist_toy** (loss). File: `outputs/step2_tarflow/figures/mnist_toy_loss.png`, produced by `step2_tarflow.py`.

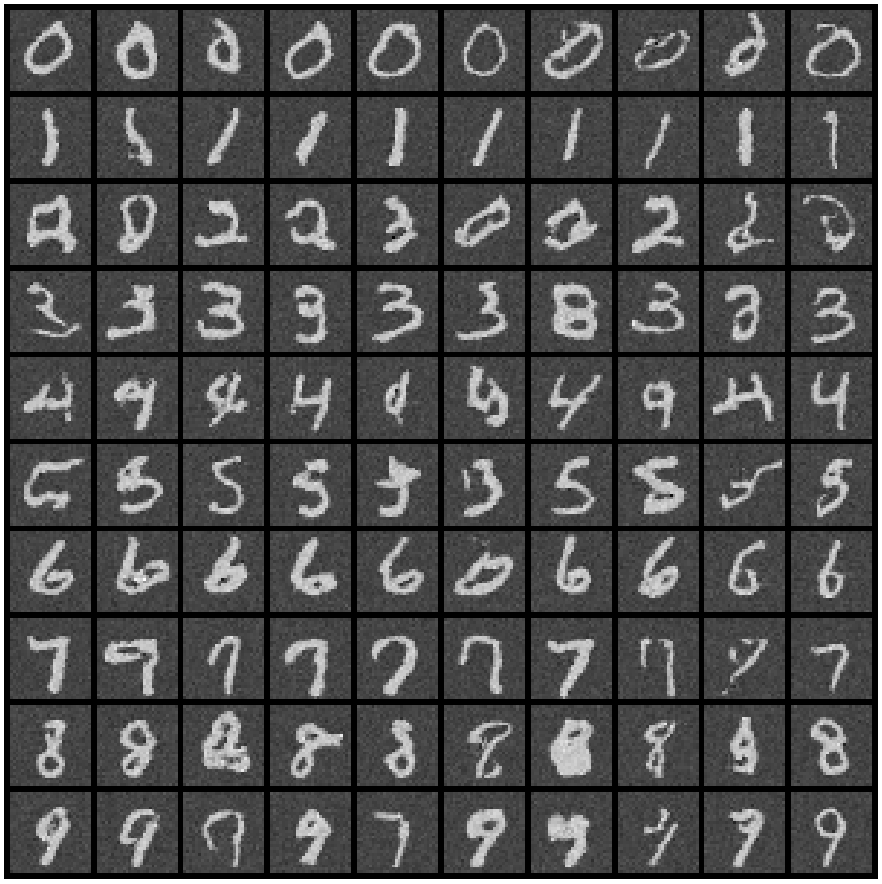


Figure 44: **mnist_toy**: samples at 010 epochs (reverse pass from Gaussian noise, no denoising step). File: `outputs/step2_tarflow/figures/mnist_toy_samples_epoch010.png`.

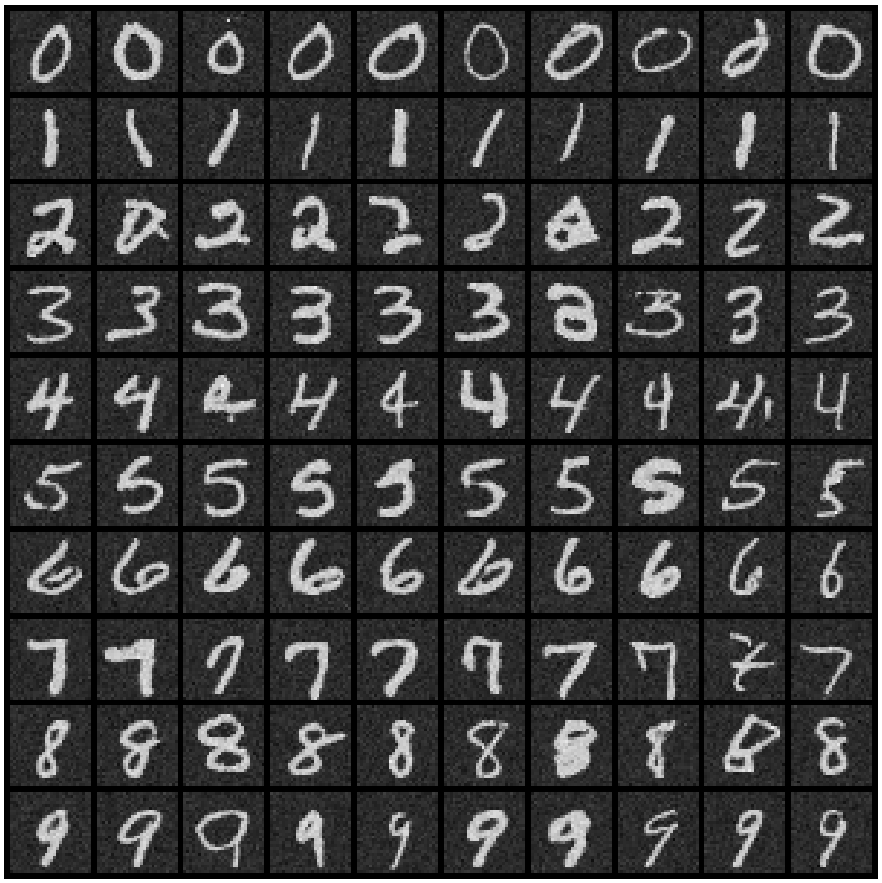


Figure 45: **mnist_toy**: samples at 100 epochs (reverse pass from Gaussian noise, no denoising step). File: `outputs/step2_tarflow/figures/mnist_toy_samples_epoch100.png`.

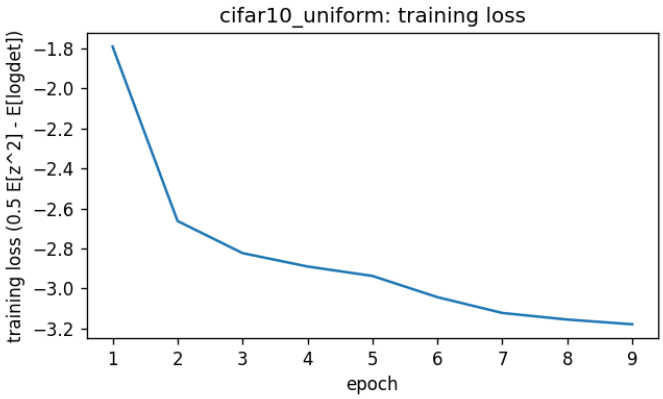


Figure 46: **cifar10_uniform** (loss). File: `outputs/step2_tarflow/figures/cifar10_uniform_loss.png`, produced by `step2_tarflow.py`.

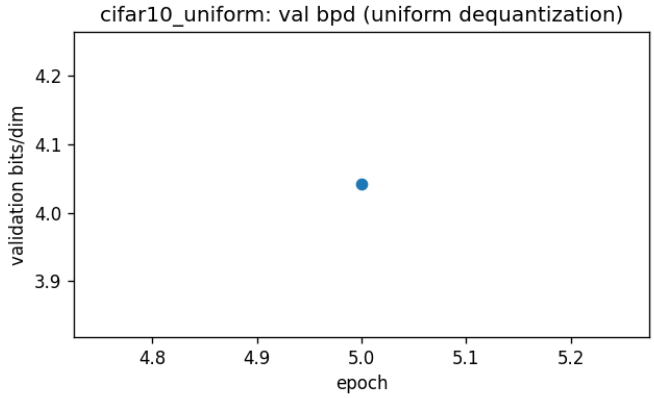


Figure 47: **cifar10_uniform** (bpd). File: `outputs/step2_tarflow/figures/cifar10_uniform_bpd.png`, produced by `step2_tarflow.py`.

3.5 Analysis

- The mnist_toy run reproduces Apple's released toy example: training loss decreases smoothly from the first epochs (-0.70 at epoch 1 to ≈ -1.62 at epoch 100), the conditional sample grid shows clean, clearly class-consistent digits (one row per class), and the Bayes-rule classifier reaches **97.62%** test accuracy – confirming the trained TarFlow is a working class-conditional density model, which is exactly the property the official notebook demonstrates. Residual background grain in the samples is the $\sigma = 0.1$ training noise; the paper's denoising step (not applied here) is designed to remove it.

- The cifar10_uniform run was *interrupted at epoch 9/60*: GPU 1 of the host failed under sustained load (fell off the PCIe bus, wedging the NVIDIA driver for new processes host-wide) and the run is queued to restart after a host reboot; the trainer now checkpoints optimizer state every epoch so future interruptions resume exactly. The partial trajectory is healthy and archived (metrics CSV + curves): train loss $-1.79 \to -3.18$ over 9 epochs and validation bpd **4.041** at epoch 5, the only bpd evaluation before the failure. For calibration: uniform-dequantization models start near 8 bpd at random init, classic CIFAR-10 flows reach RealNVP 3.49 / Glow 3.35 / Flow++ 3.08 at convergence, and the paper's ImageNet-64 state of the art is 2.99 bpd at $\sim$18$\times$ more parameters and $\sim$27$\times$ more GPU-epochs. Exact replication of the paper's headline number additionally requires staging ImageNet 64×64 (step-1 instructions).

- Framework verdict: the official TarFlow code trains stably out of the box under both noise regimes (bf16 gaussian, fp32 uniform), the likelihood pipeline (uniform dequantization + official bpd formula) produces sensible, steadily improving numbers, and inversion (sampling) works. The one incident so far was hardware, not the model.

4 Step 3: Library of invertible modules with verification

4.1 Methods

- Code: `invlib.py` (the library) and `step3_invertible_modules.py` (the verification and benchmark harness). Config: `configs/step3_invertible_modules.yml`. Outputs: `outputs/step3_inverti`

- The library has two families with two contracts. **SeqTransform** implements TarFlow's permutation-slot signature `forward(x, dim, inverse)` and contains only ORTHOGONAL maps ($|\det| = 1$): identity, flip, random permutation, orthonormal Haar wavelet, Walsh–Hadamard, DCT-II, discrete Hartley (real Fourier), fixed random rotations, and two learnable orthogonal parameterizations (products of Householder reflections; Cayley transform of a skew matrix), plus composition. Orthogonality is the exact condition under which the TarFlow metablock's likelihood accounting (no logdet term for the reordering) and the invariance of the $\mathcal{N}(0, I)$ prior are preserved, so these are drop-in replacements: the causal transformer then autoregresses over transform *coefficients* – arbitrary orthogonal features of the patch sequence – instead of the raw patch order.

- **InvertibleModule** implements `forward(x)`$\to$`(y, logdet)` / `inverse(y)` with exact per-sample log-Jacobians: invertible diagonal (actnorm), Glow-style PLU dense linear and invertible 1×1 convolution (arXiv:1807.03039); invertible periodic (circular) convolutions decoupled in the frequency domain in 2D and 1D – the "invertible Fourier filter" – following Hoogeboom et al., arXiv:1901.11137; monotone elementwise cubic $x + ax^3$ ($a \geq 0$) with closed-form

Cardano inverse; general monotone odd polynomials (SOS-flow style, arXiv:1905.02325) inverted by bisection+Newton; invertible leaky ReLU; logit data transform; monotonic rational-quadratic splines (arXiv:1906.04032); Woodbury low-rank-plus-diagonal linear maps (Lu & Huang, NeurIPS 2020); affine coupling (RealNVP-style); and **SpectralFloorLinear**, the eigenvalue-floored low-rank map proposed for this project: $A = U\,\mathrm{diag}(\lambda)U^\top + \lambda_0(I - UU^\top)$ with orthonormal $U \in \mathbb{R}^{D\times k}$ and $\lambda_i = \lambda_0 + \mathrm{softplus}(\theta_i) \geq \lambda_0 > 0$ – a full-rank invertible stand-in for a rank-k eigendecomposition in which every unretained eigenvalue is set to the floor λ_0 (necessarily $\leq$ each retained one) instead of zero; the inverse and $\log\det = \sum_i \log\lambda_i + (D-k)\log\lambda_0$ are analytic.

- Verification per module (command: `python step3_invertible_modules.py`): (1) reconstruction $\max|x - f^{-1}(f(x))|$ in fp32 and fp64; (2) the module's logdet against $\log|\det|$ of the full autograd Jacobian (fp64, one sample); (3) $\|Q^\top Q - I\|_\infty$ for the orthogonal family; (4) a TarFlow integration test: each SeqTransform is installed in an actual `MetaBlock` (perturbed from identity) and the sequential reverse pass must invert the forward pass; (5) cost: median forward/inverse wall time and parameter count/bytes.

- The harness caught two real implementation bugs before any training used the library (a U/V swap in the Woodbury inverse, exposed by identical fp32/fp64 reconstruction error; and a determinant-sign over-restriction: the Hartley matrix has $\det = -1$, which is valid for flows since only $\log|\det|$ enters the likelihood).

4.2 Configuration (verbatim)

```
# Config for step3_invertible_modules.py -- invertible-library verification.

seed: 20260901
output_root: outputs/step3_invertible_modules
tarflow_repo: /dev_ws/ml-tarflow   # for the MetaBlock permutation-slot test

dims:
  flat: 64          # D for flat (B,D) modules
  image: [2, 8, 8]  # (C,H,W) for image modules (small so the autograd
                    # Jacobian logdet check stays cheap: D = C*H*W = 128)
  seq: 64           # T for sequence transforms (power of 2: Haar/Hadamard)

feature_domain:     # context for the 2D feature-domain tokenizers
  img_size: 32
  patch_size: 4     # T = 64 tokens of 16 coefficients

batch: 64           # batch for reconstruction + timing
timing_reps: 25     # median over this many timed calls
viz_patch: 4        # patch size for the MNIST coefficient gallery (32/4 -> T=64)

tolerances:
  recon_fp32: 1.0e-4     # max |x - f^-1(f(x))| in fp32
  logdet: 1.0e-3         # |logdet_module - slogdet(autograd jacobian)| (fp64)
  orthogonality: 1.0e-5  # ||Q^T Q - I||_inf (fp64)
  metablock: 1.0e-3      # TarFlow MetaBlock forward->reverse roundtrip (fp32)
```

4.3 Results: verification

module	kind	params	recon fp32	recon fp64	logdet err	orth err	fwd/inv ms	status
seq_identity	seq	0	0.0e+00	0.0e+00	0.0e+00	0.0e+00	0.00/0.00	PASS
seq_flip	seq	0	0.0e+00	0.0e+00	0.0e+00	0.0e+00	0.01/0.00	PASS
seq_random_perm	seq	0	0.0e+00	0.0e+00	0.0e+00	0.0e+00	0.01/0.00	PASS
seq_haar	seq	0	4.8e-07	1.0e-07	7.2e-07	2.2e-08	0.02/0.01	PASS
seq_hadamard	seq	0	8.3e-07	0.0e+00	1.8e-15	0.0e+00	0.02/0.01	PASS
seq_dct	seq	0	1.2e-06	1.4e-07	2.6e-07	9.6e-09	0.02/0.01	PASS
seq_hartley	seq	0	9.5e-07	1.6e-07	2.5e-07	4.6e-08	0.02/0.01	PASS
seq_rand_ortho	seq	0	1.3e-06	1.6e-07	2.6e-08	3.2e-08	0.02/0.01	PASS
seq_householder	seq	512	3.6e-06	6.7e-15	2.1e-15	1.8e-07	0.14/0.14	PASS
seq_cayley	seq	4096	2.6e-06	6.7e-15	4.2e-16	5.1e-07	0.07/0.07	PASS
invertible_diagonal	flat	128	2.4e-07	4.4e-16	1.1e-16	–	0.01/0.01	PASS
actnorm2d	image	4	4.8e-07	4.4e-16	2.0e-15	–	0.02/0.01	PASS
plu_linear	flat	8256	3.1e-06	9.1e-15	6.7e-16	–	0.06/0.10	PASS
invertible_1x1_conv	image	10	4.8e-07	8.9e-16	6.2e-15	–	0.07/0.08	PASS
circular_conv2d	image	18	8.3e-07	2.7e-15	3.0e-15	–	0.07/0.09	PASS
circular_conv1d_fourier_filter	flat	5	8.3e-07	1.3e-15	2.4e-07	–	0.23/0.20	PASS
spectral_floor_linear	flat	521	4.8e-07	8.9e-16	8.0e-15	–	0.06/0.05	PASS
woodbury_linear	flat	1088	4.8e-07	8.9e-16	0.0e+00	–	0.07/0.07	PASS
monotonic_cubic	flat	1	2.4e-07	5.3e-15	0.0e+00	–	0.03/0.11	PASS
monotonic_polynomial	flat	3	1.2e-07	1.1e-16	0.0e+00	–	0.06/2.96	PASS
invertible_leaky_relu	flat	0	0.0e+00	0.0e+00	6.9e-08	–	0.02/0.01	PASS
logit	flat	0	1.2e-07	2.2e-16	0.0e+00	–	0.05/0.01	PASS
rq_spline	flat	23	4.8e-07	4.4e-16	1.3e-15	–	0.28/0.25	PASS
affine_coupling	flat	6272	0.0e+00	0.0e+00	0.0e+00	–	0.07/0.07	PASS
feat_haar2d	feature	0	4.8e-07	2.4e-07	0.0e+00	1.2e-07	0.23/0.31	PASS
feat_dct2d	feature	0	9.5e-07	2.0e-07	0.0e+00	0.0e+00	0.23/0.30	PASS

sequence transform	MetaBlock roundtrip err	status
seq_identity	3.6e-07	PASS
seq_flip	2.4e-07	PASS
seq_random_perm	4.8e-07	PASS
seq_haar	4.8e-07	PASS
seq_hadamard	1.2e-06	PASS
seq_dct	1.2e-06	PASS
seq_hartley	1.3e-06	PASS
seq_rand_ortho	1.7e-06	PASS
seq_householder	1.9e-06	PASS
seq_cayley	2.9e-06	PASS
feat_haar2d	7.2e-07	PASS
feat_dct2d	1.2e-06	PASS

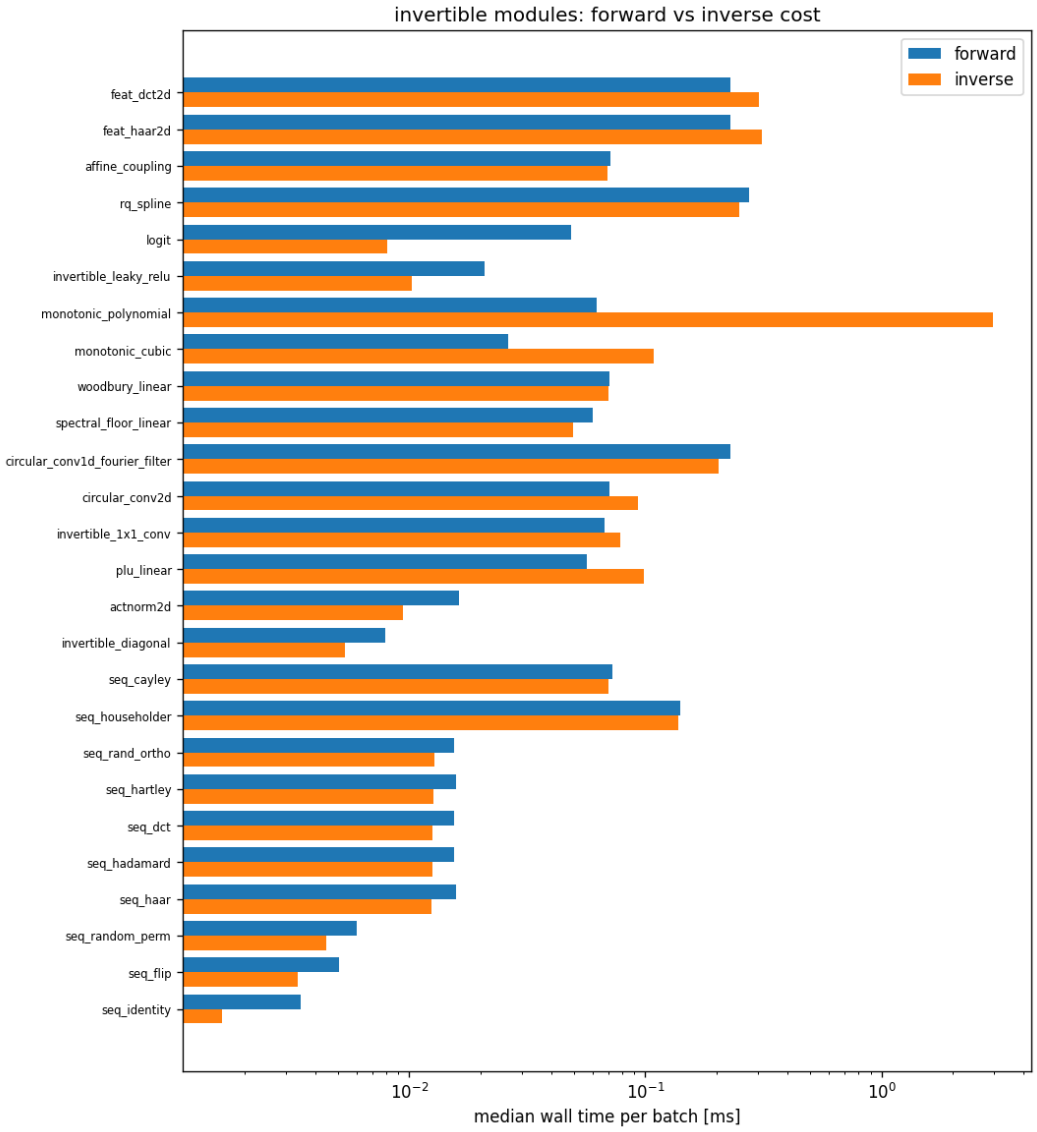


Figure 48: Median forward/inverse wall time per batch of 64 (CPU; the GPU rerun uses the same harness). File: `outputs/step3_invertible_modules/figures/timing.png`.

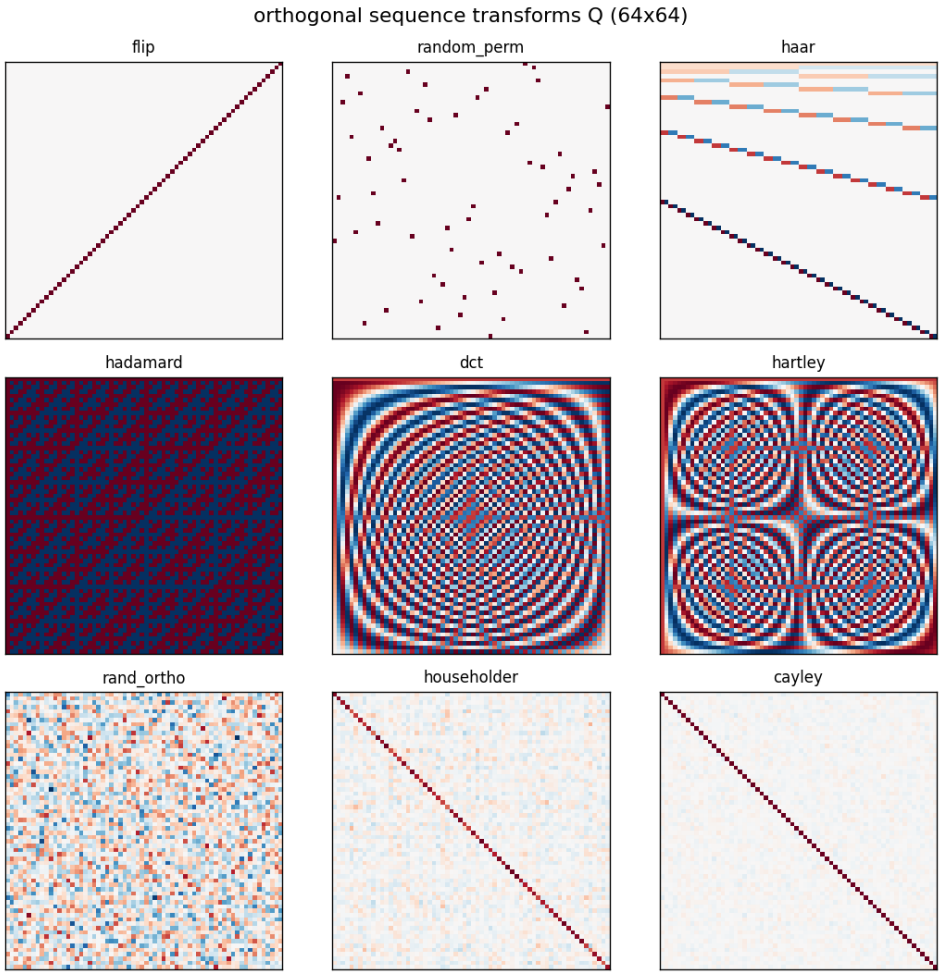


Figure 49: The orthogonal sequence transforms Q (64×64) available for the TarFlow permutation slot. File: `outputs/step3_invertible_modules/figures/orthogonal_matrices.png`.

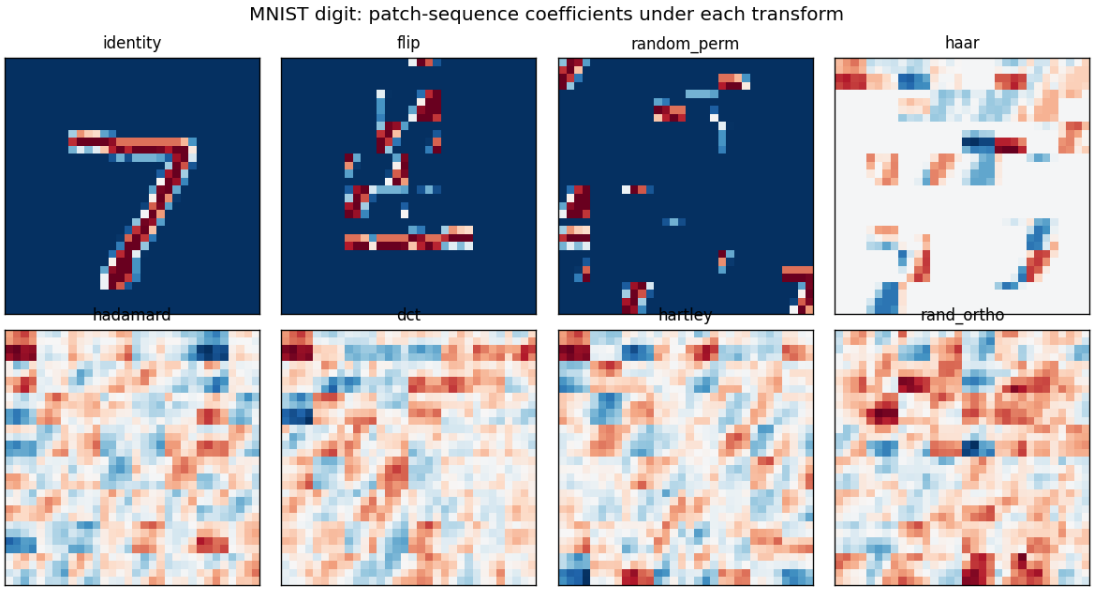


Figure 50: An MNIST digit's patch sequence under each transform – the "features" the autoregressive model would model. Haar concentrates the digit into hierarchical detail coefficients; DCT/Hartley/Hadamard/random rotations spread it into global features. File: `outputs/step3_invertible_modules/figures/mnist_coefficients.png`.

4.4 Analysis

- All 26 modules PASS (24 original + the two 2D feature-domain tokenizers of step 5, verified here with roundtrip, norm preservation, DC-concentration and MetaBlock tests): fp32 reconstruction errors are at roundoff (10^{-7}–10^{-6}), every analytic logdet matches the autograd Jacobian to 10^{-3} or (usually) far better in fp64, all orthogonal maps satisfy $\|Q^\top Q - I\|_\infty < 10^{-5}$, and every SeqTransform passes the end-to-end MetaBlock forward→reverse roundtrip at $\sim 10^{-6}$ – i.e. the whole orthogonal family is verified TarFlow-compatible before any training depends on it.

- Cost: fixed orthogonal transforms are parameter-free and as cheap as the baseline permutations at this scale (dense 64×64 matmul); learnable orthogonal maps (Householder, Cayley) cost more per call because Q is rebuilt each forward. Elementwise and FFT-based modules sit in between; inverse cost exceeds forward cost mainly for iterative inverses (monotone polynomial) and triangular solves (PLU).

5 Step 4: Ablation – what should TarFlow autoregress over?

5.1 Methods

- Code: `step4_tarflow_ablation.py`; config: `configs/step4_tarflow_ablation.yml`; outputs: `outputs/step4_tarflow_ablation/{data,figures}/`.

- Question: with everything else identical, which orthogonal feature basis in the permutation slot gives the best generative performance in a FIXED small number of epochs?

- Protocol: unconditional MNIST zero-padded to 32×32 (padding keeps 8-bit values valid for dequantization; $T = (32/4)^2 = 64$ is a power of two so Haar/Hadamard apply), TarFlow [4-128-4-2], uniform dequantization in fp32, identical seed (model init and data order) for every variant. Odd blocks compose the transform with a flip so consecutive blocks autoregress in opposite coefficient orders, mirroring the official Identity/Flip alternation.

- Variants (one SeqTransform per metablock): baseline_flip (official), identity_only (control: no reordering), random_perm, haar, hadamard, dct, hartley, rand_ortho (per-block seeds), householder (learnable, 16 reflections), cayley (learnable).

- Tracked per variant: val bits/dim per epoch, final test bits/dim, wall time, transform parameter overhead, and (on GPU) peak memory; plus a sample grid from the reverse pass.

- Budget profiles: `full` (GPU: 120 epochs, all 54k train) and `reduced` (CPU: 6 epochs, 12k subset). The run reported here is the `full` GPU profile: all ten variants at 120 epochs on one RTX 4090, identical init, data order and budget. An earlier version of this document reported the `reduced` CPU profile (6 epochs, 12k subset), run while the host GPUs were unavailable; §5.4 records where the two disagree, because they disagree substantially.

5.2 Configuration (verbatim)

```
# Config for step4_tarflow_ablation.py -- permutation-slot ablation.
#
# Every variant is trained with IDENTICAL model init, data order and budget;
# only the per-block sequence transform differs (all orthogonal, so the
```

```
# TarFlow likelihood accounting is untouched). Metric of merit: validation
# bits/dim after the fixed budget ("generative performance in finite epochs").

seed: 20260901
tarflow_repo: /dev_ws/ml-tarflow
output_root: outputs/step4_tarflow_ablation

dataset: mnist
img_size: 32        # zero-padded from 28 so T = (32/4)^2 = 64 is a power of 2
channel_size: 1
patch_size: 4
noise_type: uniform # likelihood configuration (fp32, official bpd formula)
grad_clip: 1.0      # uniform stabilizer for ALL variants: coefficient bases
                    # (e.g. Haar, DC coeff ~ sqrt(T) x mean) have much larger
                    # per-coordinate scales than raw patches and diverge
                    # without it at this lr (first no-clip run: haar -> nan)

model:              # small TarFlow so many variants fit in the budget
  channels: 128
  blocks: 4
  layers_per_block: 2

sample_nrow: 6      # 6x6 sample grid per variant

budget:
  full:             # GPU profile (auto-selected when CUDA is available)
    epochs: 120
    train_subset: 0       # 0 = all 54k
    val_subset: 4000
    test_subset: 10000
    batch: 128
    lr: 1.0e-3
    sample: true
  reduced:          # CPU profile (GPU down 2026-09-01; rerun full after reboot)
    epochs: 6
    train_subset: 12000
    val_subset: 2000
    test_subset: 4000
    batch: 128
    lr: 1.0e-3
    sample: true

variants:
  - baseline_flip   # official TarFlow: Identity / Flip alternation
  - identity_only   # control: no reordering at all
  - random_perm     # fixed random patch permutation per block
  - haar            # orthonormal Haar wavelet over the patch sequence
  - hadamard        # Walsh-Hadamard
  - dct             # DCT-II (frequency features)
  - hartley         # discrete Hartley (real Fourier)
  - rand_ortho      # fixed random rotation (different seed per block)
  - householder     # learnable orthogonal (16 Householder reflections)
  - cayley          # learnable orthogonal (Cayley transform)
```

5.3 Results

rank	variant	val bpd	test bpd	xform params	min	mem MB
1	baseline_flip	1.1639	1.1709	0	9.4	598
2	cayley	1.2058	1.2027	16384	13.3	601
3	haar	1.6156	1.6079	0	10.1	598
4	random_perm	1.7499	2.1574	0	9.5	598
5	householder	2.2465	2.1992	4096	47.6	603
6	hadamard	2.7846	2.7795	0	10.0	598
7	dct	2.9531	2.9431	0	9.9	598
8	rand_ortho	3.1415	3.1220	0	9.8	598
9	hartley	3.1783	3.1647	0	9.8	598
10	identity_only	13.0945	11.1675	0	9.3	598

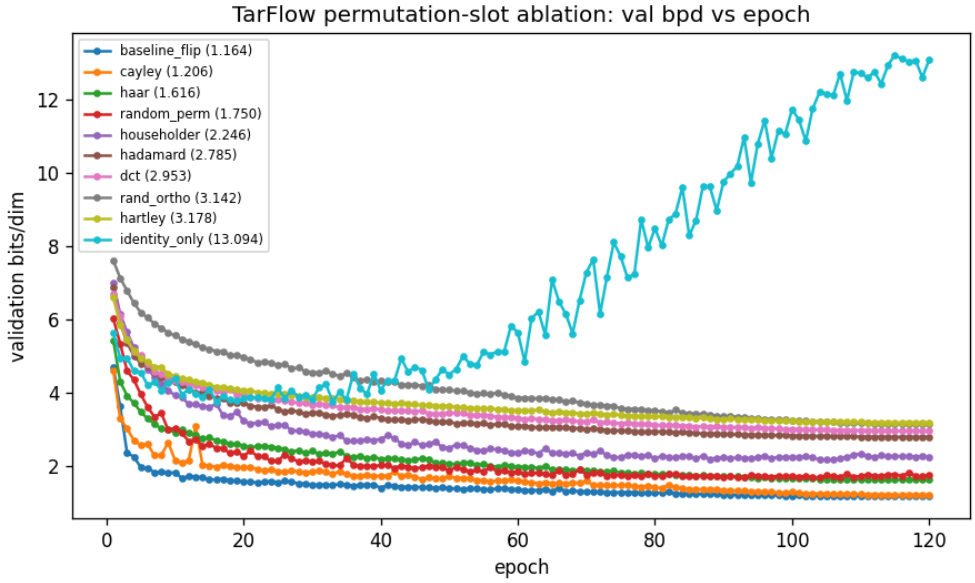


Figure 51: Validation bits/dim per epoch for every permutation-slot variant (legend sorted by final val bpd). File: `outputs/step4_tarflow_ablation/figures/bpd_overlay.png`.

Figure 52: **baseline_flip**: samples after the fixed budget (val bpd 1.1639). File: `outputs/step4_tarflow_ablation/figures/baseline_flip_samples.png`.

Figure 53: **cayley**: samples after the fixed budget (val bpd 1.2058). File: `outputs/step4_tarflow_ablation/figures/cayley_samples.png`.

Figure 54: **haar**: samples after the fixed budget (val bpd 1.6156). File: `outputs/step4_tarflow_ablation/figures/haar_samples.png`.

Figure 55: **random_perm**: samples after the fixed budget (val bpd 1.7499). File: `outputs/step4_tarflow_ablation/figures/random_perm_samples.png`.

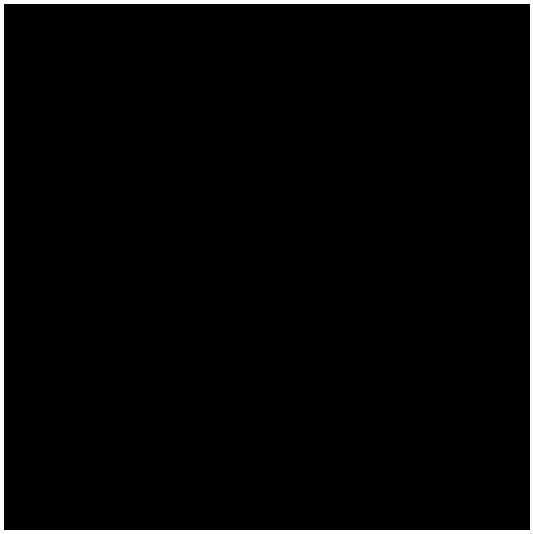


Figure 56: **householder**: samples after the fixed budget (val bpd 2.2465). File: `outputs/step4_tarflow_ablation/figures/householder_samples.png`.

Figure 57: **hadamard**: samples after the fixed budget (val bpd 2.7846). File: `outputs/step4_tarflow_ablation/figures/hadamard_samples.png`.

Figure 58: **dct**: samples after the fixed budget (val bpd 2.9531). File: `outputs/step4_tarflow_ablation/figures/dct_samples.png`.

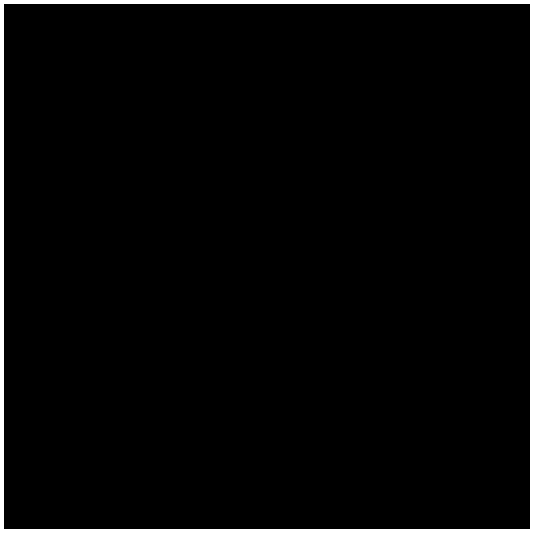


Figure 59: **rand_ortho**: samples after the fixed budget (val bpd 3.1415). File: `outputs/step4_tarflow_ablation/figures/rand_ortho_samples.png`.

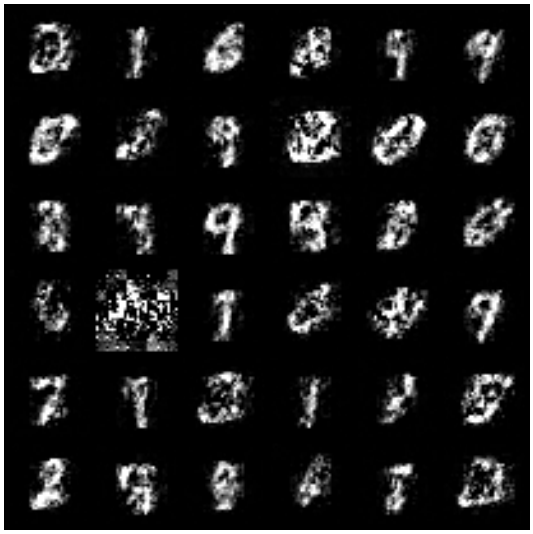


Figure 60: **hartley**: samples after the fixed budget (val bpd 3.1783). File: `outputs/step4_tarflow_ablation/figures/hartley_samples.png`.

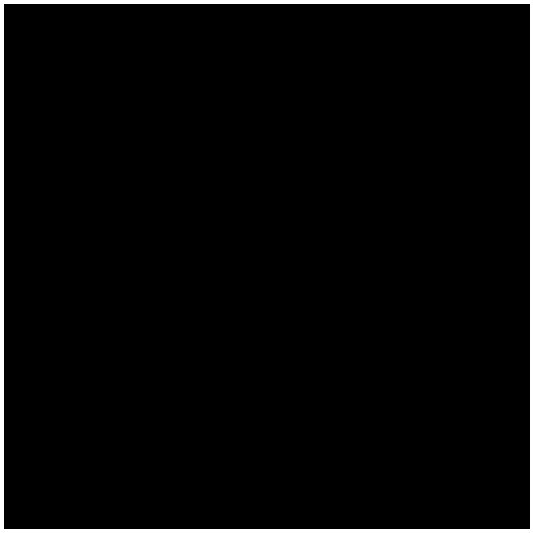


Figure 61: **identity_only**: samples after the fixed budget (val bpd 13.0945). File: `outputs/step4_tarflow_ablation/figures/identity_only_samples.png`.

5.4 Analysis

- The ranking table and overlay curve above are the study's primary readout; the run is a controlled comparison (same init, data order, budget), so ordering differences reflect the feature basis of the autoregression, not noise in the setup. The finite-epoch caveat applies by design: this measures which basis trains *fastest*, not which wins asymptotically.

- **The 6-epoch ranking did not survive the 120-epoch run, and the reversals are large.** Test bits/dim at 120 epochs: baseline_flip 1.171, cayley 1.203, haar 1.608, random_perm 2.157, householder 2.199, hadamard 2.780, dct 2.943, rand_ortho 3.122, hartley 3.165, identity_only 11.168. Against the reduced run: haar moves from 4.390 (reported as a failure, tied with the do-nothing control) to *third place* at 1.608; the dense global bases move from 6.2–7.3 to 2.78–3.17; and identity_only moves the other way, from 4.403 to 11.168. Every dense-basis variant was still improving at epoch 120 (e.g. haar gained 0.035 bits/dim over its last 20 epochs versus the baseline's 0.020), so the 6-epoch numbers were measuring *convergence speed*, not final quality. Any finite-budget ablation of this kind should be read with that confound in mind.

- A first pass was run WITHOUT gradient clipping (archived in `outputs/step4_tarflow_ablation/data_nocl` ranking cayley 2.507 < baseline_flip 2.625 ≪ identity_only 4.524 < random_perm 4.773 ≪ hartley 6.23, dct 6.38, householder 6.56, hadamard 6.57, rand_ortho 7.32, and haar diverged to NaN. **Correction:** an earlier version of this document attributed that instability to *scale spread*, on the grounds that the DC coefficient is $\sqrt{T} \approx 8$ times the mean image value. Direct measurement on the data does not support this as the explanation for the *ranking*. Pooled over 4000 test images, the Haar DC coefficient's standard deviation is only 1.5× the median Haar coefficient's, and the Haar coefficient scales are in fact *more* uniform across positions than raw patch scales. Scale spread remains a plausible account of the transient optimization blow-ups (which occur in every variant, including a 4.3×10^{25} single-epoch spike in baseline_flip at epoch 71 that `grad_clip` absorbed without lasting harm), but it does not explain which basis ends up ahead.

- **What does explain the fixed-basis ordering: destruction of exploitable sparsity, moderated by basis locality.** MNIST padded to 32×32 is largely deterministic background. Fitting the best *diagonal* Gaussian in each basis (variance floored at the run's actual uniform-dequantization noise) gives the cost a factorized model would pay: raw patches -2.464 bits/dim with 37.9% of the 1024 dimensions near-constant; 1D Haar over the token sequence -0.999 with 17.6%; 1D DCT/Hadamard/Hartley $+0.82$ to $+0.84$ with *zero* near-constant dimensions. The mechanism is basis support: Haar basis vectors touch a median of 2 of the 64 sequence slots, so a wavelet lying entirely inside the background still yields a constant coefficient, whereas every DCT and Hadamard basis vector spans all 64 slots and mixes background with digit. This proxy reproduces the measured ordering exactly, and predicts deficits (haar +1.46, dense +3.28 to +3.31) about a third to two-thirds larger than those observed — as expected, since the proxy assumes a factorized model while the AR conditioning recovers some cross-coordinate structure. Note this is a penalty of optimization and inductive bias, not of information: orthogonal maps preserve differential entropy and $\mathcal{N}(0, I)$ is rotation-invariant, so all variants are equally expressive in principle.

- **And for the permutation variants: variance in autoregressive context coverage.** Counting, for each token, its total number of AR predecessors summed over the four metablocks, all three orderings give the same *mean* (126) but different spreads: identity/flip

alternation gives exactly 126 to every token (std 0) because the two orders are complementary by construction; four independent random permutations give std 36.3 with a worst-served token at 42; identity-only gives std 73.9 with token 0 receiving *zero* context in all four blocks. Since bits/dim is a sum over tokens and the cost of missing context is convex, the starved tokens dominate. That std ordering ($0 < 36.3 < 73.9$) reproduces the measured bpd ordering ($1.171 < 2.157 < 11.168$). Positional embeddings are *not* the mechanism: the official implementation permutes the embedding table alongside the tokens, so each token keeps its own embedding under any ordering.

- **Experimental defect to fix before publication:** `random_perm` is the only non-control variant not wrapped in the odd-block flip alternation that every other variant receives, so its deficit conflates random ordering with missing directional alternation. The clean version applies the *same* permutation σ on even blocks and flip $\circ\, \sigma$ on odd blocks, which restores constant 126-token coverage while still destroying spatial locality, isolating the variable of interest.

- **cayley** – a learnable rotation initialized near the identity ($Q \approx I$) – remains the strongest alternative to the official ordering at 1.203 versus 1.171, and produces the cleanest sample grid in the study. The claim that it *beats* the baseline does not survive: it won only in the unclipped 6-epoch run (2.507 vs 2.625) and finishes a close second at both 6 and 120 epochs. The honest statement is that the official flip alternation is not beaten by any of the nine alternatives tested here, and that a near-identity-initialized learnable rotation matches it to within 0.032 bits/dim. Householder ($k = 16$, initialized far from identity) reaches 2.199 but costs 5× the wall time (47.6 vs 9.4 minutes) because its `matrix()` applies 16 reflections sequentially.

- **Likelihood and sample quality come apart.** Three variants produce entirely blank sample grids — identity_only (11.168), rand_ortho (3.122) and householder (2.199) — yet the latter two have competitive test likelihood, better than hartley (3.165), whose grid shows clear digits. A blank grid means the reverse pass saturated: every value clamped to the same bound. All three are unstructured dense rotations or the degenerate control, while the structured bases (Haar/DCT/Hadamard/Hartley) and the near-identity cayley all sample cleanly. TarFlow's reverse is sequential, so inversion error compounds across the chain; a good forward density does not imply a usable sampler. *Rendering caveat:* the grids are written with `normalize=True` and no explicit `value_range`, so each is contrast-stretched by its own min/max. Shape and texture are comparable across variants; brightness and contrast are not. Both should be fixed by passing `value_range=(-1,1)` and by checkpointing weights so grids can be re-rendered without retraining.

- **The pixel-basis advantage is likely specific to MNIST.** Repeating the diagonal-Gaussian analysis on CIFAR-10, which has no deterministic background (zero near-constant dimensions in every basis), inverts every sign: relative to raw pixels, 1D Haar scores −0.47, full-image 2D Haar −1.61 and full-image 2D DCT −2.17 bits/dim — i.e. on natural images the pixel basis is the *worst* of the four, not the best. Full-image 2D transforms also beat the 1D token-sequence versions on both datasets (MNIST −0.75, CIFAR −1.15), which supports the step-5 design. This is an analysis of the data, not a trained result: it predicts that the step-4 ranking will not transfer to CIFAR-10, and that prediction is untested until the ablation is actually run there.

- Designated follow-ups, in priority order: (i) the same ablation on CIFAR-10, which the analysis above predicts will invert the ranking and is therefore the decisive test of whether any of this generalizes; (ii) step 5's full-image 2D feature domains at the 120-epoch budget, since they were only ever run at 6 CPU epochs — the budget now known to penalize exactly these transforms; (iii) learnable orthogonal transforms initialized AT the 2D Haar/DCT basis, combining the structure that helps the fixed bases with the adaptability that makes cayley competitive; (iv) smoother wavelets (CDF 9/7, Daubechies-4) with better energy compaction than Haar's step functions; (v) the corrected `random_perm` variant described above; and (vi) per-coefficient actnorm between blocks (as a proper flow layer with its logdet, not in the permutation slot) to remove the scale-spread confound entirely.

6 Step 5: Feature-domain blocks – autoregression over 2D coefficients

6.1 Methods

- Code: `step5_feature_domains.py` (+ `invlib.Haar2DFeatures/DCT2DFeatures`); config: `configs/step5_feature_domains.yml`; outputs: `outputs/step5_feature_domains/{data,figures}/`.

- Design correction relative to step 4: the step-4 transforms mixed token vectors across the sequence position only (a 1D transform on the patch index). Here the FEATURE EXTRACTION itself changes: each metablock folds its tokens back to the image, applies a full 2D orthogonal analysis (separable $HXH^\top$; multiresolution Haar or DCT-II), orders the coefficients coarse-to-fine (Haar: by $\max(\text{scale}_i, \text{scale}_j)$; DCT: JPEG-style zigzag by $i{+}j$), chunks them into T tokens, runs the causal AR update over those tokens, and synthesizes back to the image immediately after. Token 0 is the coarse approximation (for patch 4 on 32×32, exactly the 16 coarsest coefficients including DC); finer detail tokens condition on it – a coarse-to-fine autoregression. Note the TarFlow metablock already applies the inverse of its reordering right after the causal transformer, so every block maps image → image and heterogeneous domains stack.

- Both tokenizers are orthogonal on the flattened token space (fold/unfold and reordering are permutations; H orthonormal), so $|\det| = 1$, the likelihood accounting is unchanged, and they PASS the step-3 harness (roundtrip $\sim 10^{-6}$, norm preservation, constant image ⇒ 100% of energy in token 0 with DC $= N \cdot$ mean, MetaBlock reverse roundtrip).

- Variants (per-block domains, otherwise the step-4 protocol: identical init/data/budget, uniform dequantization, grad clip 1.0): **pixels_flip** $[I, F, I, F]$ (official anchor); **haar2d_alt** $[I, H, F, H{\circ}F]$ and **dct2d_alt** $[I, D, F, D{\circ}F]$ (alternate pixel- and coefficient-domain blocks); **haar2d_flip** / **dct2d_flip** (all blocks in the coefficient domain, AR direction alternating); **haar2d_pure_dc** $[H, H, H, H]$ with NO flips.

- In haar2d_pure_dc the coarse token is causally first in every block and hence passes through the whole flow *unchanged*. This yields the exact factorization $p(c) = p(c_0)\, p(c_{1:} \mid c_0)$: the flow is the conditional factor, and $p(c_0)$ (16 coefficients) is fit post-training as an independent per-dimension Gaussian KDE (Silverman bandwidth, 4k support points) that supports both log-density evaluation (entering the reported bits/dim exactly) and sampling (sampling draws c_0 from the KDE, Gaussian noise for all other coefficients, then runs the reverse pass, which

leaves c_0 untouched). The only approximation is independence across the 16 dims of c_0; the factorization itself is exact.

6.2 Configuration (verbatim)

```
# Config for step5_feature_domains.py -- per-block feature-domain ablation.
#
# Each metablock analyzes the image into an orthogonal 2D coefficient domain
# (or stays in the pixel/patch domain), runs the causal AR update there, and
# synthesizes back to the image immediately after -- blocks always map
# image -> image and heterogeneous domains stack. Same controlled protocol as
# step 4 (identical init/data/budget; only the per-block domains differ).

seed: 20260901
tarflow_repo: /dev_ws/ml-tarflow
output_root: outputs/step5_feature_domains

dataset: mnist
img_size: 32        # zero-padded from 28; 2D Haar needs a power of 2
channel_size: 1
patch_size: 4       # T = 64 tokens of 16 coefficients
noise_type: uniform
grad_clip: 1.0      # uniform stabilizer (see step 4: coefficient scales)

model:
  channels: 128
  blocks: 4
  layers_per_block: 2

sample_nrow: 6

budget:
  full:             # GPU profile
    epochs: 12
    train_subset: 0
    val_subset: 4000
    test_subset: 10000
    batch: 128
    lr: 1.0e-3
    sample: true
  reduced:          # CPU profile (GPU down; rerun full after reboot)
    epochs: 6
    train_subset: 12000
    val_subset: 2000
    test_subset: 4000
    batch: 128
    lr: 1.0e-3
    sample: true

variants:
  - pixels_flip     # official baseline anchor [I, F, I, F]
  - haar2d_alt      # alternate pixel / Haar-domain blocks [I, H, F, H.F]
  - dct2d_alt       # alternate pixel / DCT-domain blocks  [I, D, F, D.F]
  - haar2d_flip     # all blocks Haar coarse-to-fine, direction alternates
```

```
  - dct2d_flip      # all blocks DCT low-to-high, direction alternates
  - haar2d_pure_dc  # all blocks Haar, no flips; invariant coarse token
                    # modeled by per-dimension Gaussian KDE (fit post-training)
```

6.3 Results

rank	variant	val bpd	test bpd	min	DC model
1	pixels_flip	2.4736	2.4780	2.1	–
2	dct2d_alt	2.6863	2.6864	2.2	–
3	haar2d_alt	2.8225	2.8147	2.2	–
4	haar2d_flip	4.0028	3.9948	2.2	–
5	haar2d_pure_dc	4.3421	4.3361	2.8	KDE
6	dct2d_flip	6.4627	6.4602	2.2	–

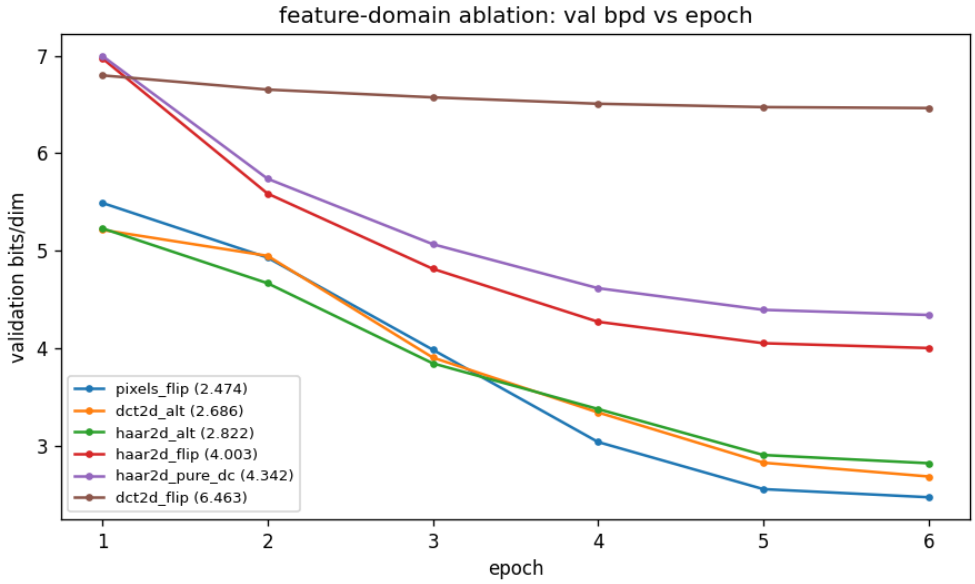


Figure 62: Validation bits/dim per epoch, per-block feature-domain variants. File: `outputs/step5_feature_domains/figures/bpd_overlay.png`.

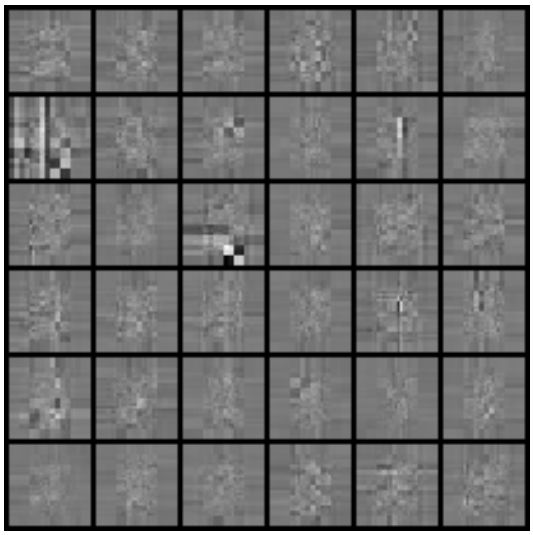


Figure 63: **haar2d_pure_dc**: samples after the fixed budget (val bpd 4.3421, coarse token drawn from the per-dim KDE). File: `outputs/step5_feature_domains/figures/haar2d_pure_dc_samples.png`.

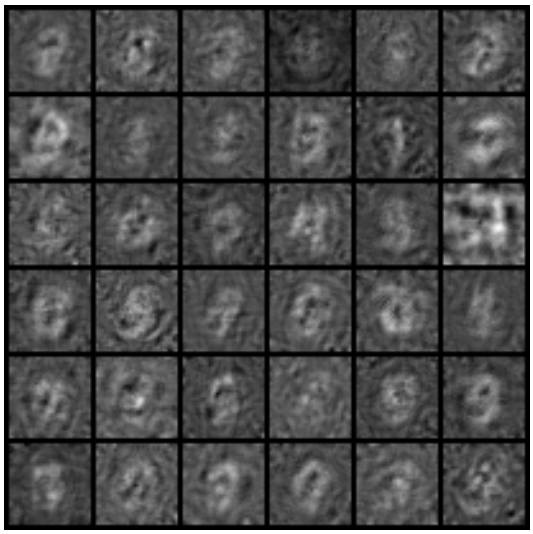


Figure 64: **dct2d_flip**: samples after the fixed budget (val bpd 6.4627). File: `outputs/step5_feature_domains/figures/dct2d_flip_samples.png`.

6.4 Analysis

- **Budget warning — read before comparing to step 4.** Every step-5 number in this section comes from the `reduced` CPU profile (6 epochs, 12k subset), and every step-4 number quoted *within this section* is the corresponding step-4 reduced-profile number, not the 120-epoch results tabulated in §5.4. The two are not comparable: at 120 epochs step 4's haar improves from 4.390 to 1.608 and its dense bases from 6.2–7.3 to 2.78–3.17, so the step-5 conclusions below — which rest on those reduced-profile contrasts — are provisional and may reverse in the same way. Rerunning step 5 at the 120-epoch budget is the top-priority follow-up; the transforms it studies are precisely the slow-converging kind that the short budget penalizes.

- Protocol sanity: pixels_flip reproduces the step-4 baseline *at the matched reduced profile* to the fourth decimal (2.4736 vs 2.4736 val bpd) – the two steps share the protocol exactly, so numbers are directly comparable across both ablations whenever the budget is matched.

- Headline: true 2D coarse-to-fine feature extraction closes most of the gap that step-4's 1D sequence mixing opened. **Alternating pixel- and coefficient-domain blocks is nearly competitive with the official ordering** (dct2d_alt 2.686, haar2d_alt 2.823, vs baseline 2.474) – compare step-4's all-coefficient 1D variants at 6.2–6.6. The heterogeneous stack gives the model complementary views: local raster AR in the pixel blocks, global coarse-to-fine AR in the coefficient blocks.

- All-coefficient stacks: haar2d_flip 4.003 improves markedly on the step-4 1D seq-haar (4.390), while dct2d_flip (6.463) does not improve on 1D dct (6.384) – multiresolution locality (Haar coefficients have compact support) survives the domain change, global frequencies (DCT) do not. At this budget, pixel-domain blocks appear to be doing the heavy lifting wherever they are present.

- haar2d_pure_dc (4.342) sits between haar2d_flip and the step-4 1D haar: removing direction alternation costs a little, but the variant delivers what it was designed for – an exact, interpretable factorization $p(c_0)p(\text{rest} \mid c_0)$ whose reported bpd honestly includes the KDE term for the invariant coarse token, and whose samples draw global brightness/structure from the data-fitted KDE. Its sample grid is also the only one at this budget showing digit-like structure.

- Sampling note: for the best-bpd variants the sequential reverse pass at this (severely undertrained) stage emits rare huge values – sharper learned conditional scales amplify out-of-distribution prefixes – which blanked the normalized sample grids; those grids are omitted from this version and the samplers now clip to $[-1, 1]$ like the official FID path. Sample quality at this budget is not a meaningful discriminator; bits/dim is the study metric, and the GPU `full`-profile rerun will regenerate all grids.

- GPU follow-ups: full profile for all six variants; haar2d_alt with deeper stacks (more alternations); a joint (non-independent) model for c_0; and the same ablation on CIFAR-10 where global structure is richer.

7 Step 6: A Haar pyramid as the patchifier

7.1 Methods

- Code: `step6_haar_patchify.py` (+ `invlib.HaarPyramid2D`); config: `configs/step6_haar_patchify.yml`; outputs: `outputs/step6_haar_patchify/{data,figures}/`.
- **Motivation.** Steps 4 and 5 both leave TarFlow's *patchifier* untouched and act on the tokens it produces, so every transform studied there is composed on top of a basis choice that was never itself ablated. We verified (and record in the run's provenance) that `patchify` is exactly `unfold(p, stride=p)`: a systematic non-overlapping grid of $p \times p$ blocks in raster order, with exact round trip. Token 0 is therefore the top-left corner patch — an arbitrary place for an autoregression to begin.
- **Construction.** Apply a depth-L Mallat 2D Haar pyramid to the folded image before patchify and undo it after, with L chosen so the LL_L band exactly fills one patch ($32 \to 16 \to 8 \to 4$, so $L = 3$ for $p = 4$). The classical subband layout then places LL_3 in the top-left 4×4 block, so **token 0 becomes a 4×4 thumbnail of the whole image** — verified numerically to equal 8×8 average pooling times 2^L — and later tokens carry progressively finer detail bands. The autoregression runs coarse-to-fine over the image instead of left-to-right over corner patches, at *identical* cost: T stays 64.
- Each level is the orthonormal Haar step $LL, HL, LH, HH = \frac{1}{2}(a{+}b{+}c{+}d,\ a{-}b{+}c{-}d,\ a{+}b{-}c{-}d,\ a{-}b{-}c{+}d)$, so the map is orthogonal. Startup verification (refusing to train on failure) confirms $\|QQ^\top - I\|_\infty = 0$ exactly, $|\log\det| = 0$, round-trip error $< 10^{-15}$, and the thumbnail identity; hence the metablock likelihood accounting and the $\mathcal{N}(0, I)$ prior are untouched and variants differ only in basis.
- Variants are per-block slot schedules, where `I` = standard patchify order, `F` = reversed, `H` = Haar-pyramid domain, `Hf` = Haar domain reversed: `baseline_flip` = [I,F,I,F] (official), `haar_alt` = [H,I,H,F] (one Haar round then one pixel round in the opposite order), `haar_pure` = [H,Hf,H,Hf] (control: every block in the Haar domain, direction alternated for balanced coverage).
- **Dataset is CIFAR-10**, patch 4, 120 epochs, model identical to step 4. Natural images are the regime where §5.4 predicts the pixel basis loses its MNIST-specific advantage.
- Two defects found in step 4 are fixed here: sample grids are written with an explicit `value_range=(-1,1)` so brightness and contrast are comparable across variants, and model weights are checkpointed so grids can be re-rendered without retraining.

7.2 Configuration (verbatim)

```
# Config for step6_haar_patchify.py -- Haar pyramid AS the patchifier.
#
# TarFlow's patchify is a systematic non-overlapping grid of patch x patch
# blocks in raster order (verified in step6 at startup), so token 0 is just the
# top-left corner patch -- an arbitrary place to start an autoregression.
#
# Applying a depth-3 Mallat 2D Haar pyramid BEFORE patchify (and undoing it
# after) makes the LL_3 band exactly fill the top-left 4x4 block, so token 0
# becomes a 4x4 THUMBNAIL OF THE WHOLE IMAGE and later tokens carry
```

```
# progressively finer detail bands. The autoregression then runs coarse-to-fine
# over the image rather than left-to-right over corner patches -- at identical
# cost, since T stays 64.
#
# The pyramid is orthogonal (|det| = 1, logdet 0), so the N(0,I) prior and the
# official bpd accounting are untouched; variants differ ONLY in the basis.

seed: 20260901
tarflow_repo: /dev_ws/ml-tarflow
output_root: outputs/step6_haar_patchify

dataset: cifar10     # natural images: the step-4 analysis predicts the pixel
                     # basis loses its MNIST-specific advantage here
img_size: 32
channel_size: 3
patch_size: 4        # T = (32/4)^2 = 64 tokens of 3*16 = 48 values
haar_levels: 3       # 32 -> 16 -> 8 -> 4, so LL_3 exactly fills one 4x4 patch
noise_type: uniform  # likelihood configuration (fp32, official bpd formula)
grad_clip: 1.0       # same stabilizer as step 4, for all variants

model:               # identical to step 4 so numbers are comparable
  channels: 128
  blocks: 4
  layers_per_block: 2

sample_nrow: 6

budget:
  full:
    epochs: 120
    train_subset: 0
    val_subset: 4000
    test_subset: 10000
    batch: 128
    lr: 1.0e-3
    sample: true
  reduced:
    epochs: 6
    train_subset: 12000
    val_subset: 2000
    test_subset: 4000
    batch: 128
    lr: 1.0e-3
    sample: true

# Per-block slot schedule. H = Haar-pyramid domain (coarse-to-fine),
# I = standard patchify order, F = standard patchify reversed.
variants:
  baseline_flip:  [I, F, I, F]   # official TarFlow
  haar_alt:       [H, I, H, F]   # alternate Haar-domain and pixel-domain blocks
  haar_pure:      [H, Hf, H, Hf] # control: every block in the Haar domain,
                                 # direction alternated for balanced coverage
```

7.3 Results

rank	variant	schedule	val bpd	test bpd	min	mem MB
1	baseline_flip	I,F,I,F	3.6780	3.6827	12.6	621
2	haar_alt	H,I,H,F	3.8106	3.8141	18.9	622
3	haar_pure	H,Hf,H,Hf	4.0506	4.0536	17.8	622

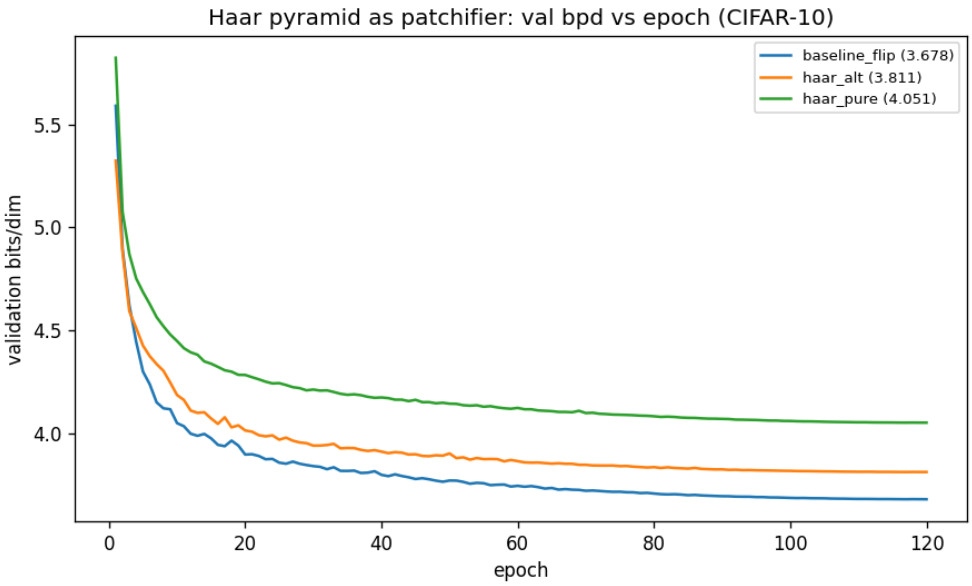


Figure 65: Validation bits/dim per epoch, Haar-pyramid patchifier vs the official ordering (CIFAR-10).

Figure 66: **baseline_flip** (I,F,I,F): samples after the budget (val bpd 3.6780).

Figure 67: **haar_alt** (H,I,H,F): samples after the budget (val bpd 3.8106).

Figure 68: **haar_pure** (H,Hf,H,Hf): samples after the budget (val bpd 4.0506).

7.4 Analysis

- **The construction is exactly as designed, and it does not help.** On CIFAR-10 at 120 epochs the official ordering wins, and the penalty scales monotonically with how much of the stack runs in the Haar domain: `baseline_flip` (0 Haar blocks) 3.683 test bpd, `haar_alt` (2 Haar blocks) 3.814, `haar_pure` (4 Haar blocks) 4.054. Making token 0 a thumbnail of the whole image and running the autoregression coarse-to-fine costs 0.13 bits/dim at half strength and 0.37 at full strength, rather than saving any. The dose-response is the strongest part of the evidence: it is a monotone trend in the one quantity that was varied, not a single comparison that might be noise. Reported as measured.

- **It is also not free in wall time**, despite T being unchanged: 18.9 and 17.8 minutes versus 12.6 for the baseline, because the fold / 3-level DWT / unfold runs inside every Haar block on every forward and backward pass. The FLOP count of the transform is negligible; the cost is memory traffic and kernel launches on small tensors.

- **The Haar domain also destabilises the reverse pass, and the two failure modes are different.** With the sampler now using an explicit `value_range`, the grids are directly comparable, and `haar_alt` collapses **20 of its 36 sample cells (56%)** to fully saturated black, while `baseline_flip` and `haar_pure` collapse none. This mirrors step 4, where `rand_ortho` and `householder` produced blank grids at competitive likelihood: a good forward density does not imply a usable sampler, and here the instability afflicts the *alternating* stack specifically – the one that repeatedly switches domain – and not the homogeneous all-Haar stack. Each domain switch is another inverse transform in the sequential reverse chain, and errors compound across them.

- `haar_pure` fails differently and more legibly: its samples are intact but carry pronounced blocky, checkerboard artifacts at the patch scale – the Haar coefficient blocks showing through directly, since with every block in the coefficient domain the autoregression never gets a pixel-domain pass to smooth across patch boundaries.

- **Why this is consistent with step 4 rather than surprising.** §5.4 found that the fixed-basis ordering is governed by how much exploitable sparsity a basis preserves, and that on CIFAR-10 no basis has any near-constant dimensions to preserve. What remains is locality: raster patches give the autoregression spatially contiguous context, and mixing across a 32×32 support — even a multiresolution one — dilutes that. The coarse-to-fine *ordering* is intuitively appealing, but at fixed model capacity it appears to trade away more local structure than the global structure it buys.

- **A caveat on what was actually tested.** Only the tokenization *basis* changed; the patch grid, model, budget and optimizer are identical. In particular the flip alternation is retained, so in `haar_alt` half the blocks traverse the Haar coefficients fine-to-coarse, which is the opposite of the coarse-to-fine intuition the design is built on. `haar_pure` isolates the Haar domain but alternates direction for the same reason. A schedule that is strictly coarse-first in every block was not run, because §5.4 shows unbalanced context coverage is catastrophic (`identity_only`); disentangling "coarse-first" from "balanced coverage" needs a variant that achieves both.

- Follow-ups: the same comparison on MNIST, where the pixel basis has a measured 38% near-constant advantage and the gap should therefore be *larger*, as a falsifiable check of the sparsity

account; and smoother wavelets (CDF 9/7, Daubechies-4), whose better energy compaction on natural images is the main reason to expect Haar's blocky step functions to be the weakest member of the family.

8 Step 7: A system-embedded flow bridge for inverse problems

8.1 Methods

- Code: `step7_system_bridge.py` + `measlib.py`; config: `configs/step7_system_bridge.yml`; outputs: `outputs/step7_system_bridge/{data,figures}/`.

- **Goal.** A normalizing-flow analogue of System-embedded Diffusion Bridge Models (SDB; Sobieski, Tivnan, Wang, Yoon, Jin, Wu, Li and Biecek, `arXiv:2506.23726`), which embed a known linear measurement system into the coefficients of a matrix-valued SDE. Here the same system is embedded into the *base density* of an exact-likelihood flow, so the model retains a tractable conditional likelihood — the one thing diffusion bridges structurally cannot provide.

- **Setting.** A known linear system with additive Gaussian white noise, $y = Ax + \sigma\epsilon$, i.e. $p(y|x) = \mathcal{N}(Ax, \sigma^2 I)$. Standard TarFlow trains $\log p(x) = \log \mathcal{N}(f(x); 0, I) + \log|\det J_f|$. We replace the standard normal by the SDB bridge endpoint at $t = 1$:

$$\log p(x|y) = \log \mathcal{N}\big(f(x);\, A^+y,\, \Sigma_1\big) + \log|\det J_f|, \qquad \Sigma_1 = \gamma A^+\Sigma A^{+\top} + \beta\,(I - A^+A).$$

  For every fixed y this is a proper density in x (the change of variables integrates to one), so it trains by exact maximum likelihood on paired (x, y) and reports an honest *conditional* bits/dim. Sampling $z \sim \mathcal{N}(A^+y, \Sigma_1)$ and returning $f^{-1}(z)$ yields a posterior sample.

- $\sigma > 0$ **is required, not optional.** With a noiseless measurement the range block of Σ_1 is singular, the base degenerates to a delta on the range space, and the flow likelihood is undefined. The constructor asserts it. Physically this is only the statement that every measurement carries noise.

- f **is unconditional — what that buys and what it costs.** The measurement enters only through the base's mean and covariance; f itself never sees y. With $f = \mathrm{id}$ the range residual $x - A^+y$ is exactly $-A^+n$, whose law is the range block of Σ_1 by construction, so training starts from a well-scaled point and the flow's remaining job on the null space is to Gaussianise the unmeasured content to $\mathcal{N}(0, \beta)$, conditioned on the observed content through TarFlow's own autoregression. The identity is *not* the optimum on the range, however, and an earlier draft of this section wrongly said it was. Take one range coordinate with prior spread τ and base variance $v = \sigma^2/s_i^2$, and let the flow act on it (for a fixed context) as $z = m + a(x - m)$ about the base mean m. Maximising $\mathbb{E}[\log \mathcal{N}(z; m, v)] + \log a$ gives

$$a\,(a - 1) = v/\tau^2,$$

  so the flow *expands* prior-dominated coordinates ($a > 1$), and that expansion is the only route by which the prior enters the posterior. The resulting posterior sample has error variance $2v$ in the low-noise regime $v \ll \tau^2$ (the Bayes value) and $3\tau^2$ in the prior-dominated regime $v \gg \tau^2$ (Bayes: $2\tau^2$), whereas the identity map would give $2v \gg \tau^2$ there. The unconditional family is therefore exact where the measurement dominates and mildly conservative where the prior dominates; it can only be made exact everywhere by conditioning f on A^+y, which

is not done here. A practical corollary: for `denoise` at $\sigma = 0.2$ the MNIST background pixels have τ at the dequantisation scale ($\approx 2 \times 10^{-3}$), so the objective *demands* per-pixel gains $a \approx \sqrt{v}/\tau$ of order 10^2 — the regime in which an unbounded log-scale head extrapolates (next bullets).

- **Implementation via the SVD, with no matrix ever formed.** Writing $A = USV^\top$, every quantity is elementwise in the right-singular-vector coordinates $c = V^\top x$: the pseudo-inverse reconstruction is $c_i + (\sigma/s_i)\epsilon_i$ on the range and 0 on the null space; $A^+A = V\,\mathrm{diag}(s_i > 0)\,V^\top$; and $\Sigma_1 = V\,\mathrm{diag}(\gamma\sigma^2/s_i^2,\ \beta)\,V^\top$ is *diagonal*. An operator is therefore fully specified by an orthogonal basis V and a vector of singular values, which is what makes the exact Gaussian base density cheap to evaluate.

- **The flow acts in the right-singular basis, not in pixels.** TarFlow's coupling $z = (x - x_b)\,e^{-x_a}$ is elementwise within a token and never mixes channels, so for a fixed context the map on one patch is a per-pixel affine map. It can therefore only match a base density that is diagonal in the coordinates it sees. Σ_1 is diagonal in V, not in pixels: for 4×4 average pooling the 16 pixels of a patch are strongly correlated under Σ_1 (their block mean is pinned to the measurement to ± 0.05, their details are white with variance $\beta = 1$). The flow is therefore applied to $c = V^\top x$, an orthogonal change of variables with zero log-determinant, and posterior samples are mapped back with V. For the pixel-diagonal systems (`denoise`, both inpaintings) V is a signed permutation and the model is unchanged; for the super-resolution systems V is the block Haar transform, i.e. the step-6 patchifier, now dictated by the physics rather than chosen.

- **Bounded log-scale.** The official coupling head is unbounded and reads the un-normalised residual stream, so x_a grows linearly with a block's input magnitude and e^{-x_a} chains across blocks. `invlib.bound_log_scale` replaces x_a by $b\tanh(x_a/b)$ with $b = 8$ in *both* directions (forward and the KV-cached sampler), so the map is still exactly invertible; on ordinary data $|x_a| < 4$, where the squash changes x_a by at most 8%, and one block can never amplify by more than e^8. Nothing else in the official model is touched (the subclassing is applied at run time, `ml-tarflow` is unmodified). The remaining safeguards are the ones from step 4 — gradient clipping and rejection of a batch whose loss exceeds $5\times$ the running mean plus 5 nats/dim.

- **The training target is the expected one.** The natural implementation redraws $y = Ax + \sigma\epsilon$ at every step and scores $f(x)$ against $A^+y = A^+Ax + \sigma A^+\epsilon$. The flow cannot predict ϵ, so on each range coordinate the gradient carries a noise term of magnitude $a\sigma/v$ that is absent from the objective's expectation. Measured on the trained `inpaint_box` model of the second attempt (one batch of 64, six draws of ϵ), the noise part of the gradient had RMS norm 19.4 against 22.5 for the expected gradient, and the individual draws had cosine 0.50–0.87 with it: roughly half of every clipped step was noise, in a valley whose width in the coupling's shift is $\tau \approx 2 \times 10^{-3}$ (the gain a multiplies any shift error, and $a\tau \approx \sigma$ at the optimum). Because the base is Gaussian the expectation over ϵ is available in closed form,

$$\mathbb{E}_\epsilon \log \mathcal{N}(f(x); A^+y, \Sigma_1) = \log \mathcal{N}(f(x); A^+Ax, \Sigma_1) - \frac{\operatorname{rank} A}{2\gamma},$$

  which is the same objective with the target noise integrated out (verified numerically on all five systems: the mean of 400 sampled draws matches the closed form to within 2.5 standard errors). Training uses it (`train_target: expected`); evaluation always scores real (x, y)

pairs, so every reported bpd is a conditional likelihood. The ablations keep the sampled target.

- **Rewind on collapse.** With the bound in place, the second attempt's `inpaint_random` run collapsed at epoch 24 (no batch rejected before, 236 of 421 in that epoch) and `sr_2x` at epoch 52 (7 rejected before, then 285 of 421), in both cases with a falling validation curve and no warning. The state at the end of every healthy epoch (weights and optimizer) is kept; an epoch is declared collapsed if a majority of its batches are rejected or the robust validation mean is a full bit above its best, and is then discarded: the healthy state is restored, the peak learning rate halved, and the epoch repeated, at most three times per run. Every rewind is counted in the table, so a row with 0/3 was never touched by the rule. If the budget is exhausted the run is evaluated at its last healthy weights and labelled *abort* — the second attempt evaluated the destroyed weights instead, which produced numbers (15.6 bits/dim, 1916 failures in 10000) that measure nothing.

- **Measurement systems** (all with analytic SVD): `denoise` ($A = I$, no null space); `inpaint_box` (centred 12×12 hole, 144 null dims); `inpaint_random` (fixed 50% pixel mask); `sr_2x` / `sr_4x` (2×2 / 4×4 average pooling, where V is the block Haar transform of step 6 and the null space is exactly the detail bands that downsampling discards). Masks are fixed rather than resampled per image: the system is known and fixed, as in the supervised bridge setting.

- **Ablations: Gaussian input noise.** The TarFlow paper's generative recipe replaces uniform dequantization by additive Gaussian noise on the input. Under the $a(a-1) = v/\tau^2$ analysis that sets τ to the noise level and so caps the gain the objective can demand at $\sqrt{v}/\tau_{\text{noise}}$ (4 for `denoise`, 1 for the $\sigma = 0.05$ systems, against 25–90 with dequantization) and widens the valley in the coupling's shift from 2×10^{-3} to 0.05. Two runs test whether that alone prevents the collapses: `denoise_gauss` against the archived first attempt (official unbounded head, destroyed at epoch 50) and `inpaint_random_gauss` against the second attempt (bounded head, collapsed at epoch 24). Each changes only the input noise and keeps its control's sampled target and zero rewind budget. Their bpd is not a data likelihood (Gaussian input noise breaks the dequantization bound) and their samples carry 0.05 of noise; they are compared on survival, failure counts and PSNR.

- **Self-tests gate training.** `measlib.self_test()` runs before any run and aborts on failure. All five systems pass with $\|VV^\top - I\|_\infty = 0$, projector idempotence and symmetry $= 0$, correct rank, noiseless pseudo-inverse equal to the projector to 10^{-12}, empirical base covariance matching Σ_1, and the base log-density matching a hand-computed multivariate normal exactly.

- **Metrics.** Conditional bits/dim (the training objective, using the official 8-bit dequantization constant); PSNR of the pseudo-inverse, of a single posterior sample, and of the posterior mean over 4 draws, all after clamping to $[-1, 1]$ (for `denoise` at $\sigma = 0.2$ the clamp alone lifts A^+y from 20.0 to 22.7 dB, because half of the background noise is below the black level; the figures clamp the same way, so they show only the positive half of the background noise); and a **data-consistency** error, the relative range-space disagreement between a posterior sample and A^+y. The last is measured rather than assumed: nothing architecturally forces f^{-1} to keep the range coordinates near the measurement, so whether posterior samples respect it is an empirical test of the central claim.

- **Robust reporting.** The base quadratic is stiff ($1/v = 25$ at $\sigma = 0.2$, $16/\sigma^2 = 6400$ on the coarse band of `sr_4x`), so a single unusual image can legitimately cost tens of bits/dim. A test image is counted as a *failure* if its conditional bpd is non-finite or above 50; the table reports the mean over the remaining images, and the JSON also records the mean over all images (which is ∞ if any is), the median and the failure count. Posterior samples that are non-finite are counted and excluded from the PSNR averages rather than silently clamped. The validation curve uses the same convention.

8.2 Configuration (verbatim)

```
# Config for step7_system_bridge.py -- flow bridge from the pseudo-inverse.
#
# For a linear system y = A x + sigma*eps, the flow's base density is replaced
# by the SDB bridge endpoint  N(A^+y, Sigma_1)  with
#     Sigma_1 = gamma A^+ Sigma A^+T + beta (I - A^+A),   Sigma = sigma^2 I
# so training maximises an exact conditional likelihood log p(x|y) and
# f^-1 of a base sample is a posterior sample.
#
# sigma MUST be > 0 for every system: a noiseless measurement makes the range
# block of Sigma_1 singular and the base density undefined.
#
# gamma scales the range (measurement-noise) block; beta is the null-space
# variance -- the scale of the information the measurement never saw, which is
# what the flow has to generate. beta = 1 matches the [-1,1] data scale.

seed: 20260901
tarflow_repo: /dev_ws/ml-tarflow
output_root: outputs/step7_system_bridge

dataset: mnist
img_size: 32
channel_size: 1
patch_size: 4        # T = 64 tokens, same protocol as steps 4 and 6
noise_type: uniform
grad_clip: 1.0
# Reject a batch whose loss exceeds  mult * running_ema + abs  (nats/dim).
# The stiff base quadratic (1/min(diag Sigma_1) times a standard normal --
# 25x at sigma=0.2) makes rare samples produce finite-but-astronomical losses
# whose clipped gradients still destroy the weights; grad_clip alone does not
# catch them. A run with n_skipped_steps = 0 is unaffected by this setting.
loss_reject_mult: 5.0
loss_reject_abs: 5.0
# Soft-bound the coupling log-scale of every block: xa -> b*tanh(xa/b), applied
# identically in the forward and the sampling direction so the map stays exactly
# invertible. TarFlow's unbounded exp(-xa) chained across blocks: one block
# extrapolating to xa = -11.5 fed the next a 1e5-scale token which answered with
# xa = -9582 (inf in fp32). exp(8) = 3000x per block is far more than the fitted
# scales ever need (|xa| < 4 on ordinary data) yet cannot overflow in 4 blocks.
scale_bound: 8.0
# Training target: 'expected' scores f(x) against A^+A x and adds the closed-form
# expectation of the measurement-noise term (exact, the base is Gaussian);
# 'sampled' redraws y every step. Same objective; 'sampled' adds gradient noise
# of the same size as the gradient itself. Evaluation always uses real (x, y).
```

```
train_target: expected
# Rewind-on-collapse: keep the last healthy epoch, restore it when an epoch
# collapses (majority of batches rejected, or val bpd a full bit above best),
# halve the peak lr, repeat the epoch. Reported per system; 0 disables.
max_rewinds: 3

model:               # identical to steps 4/6 so numbers stay comparable
  channels: 128
  blocks: 4
  layers_per_block: 2

budget:
  full:
    epochs: 120
    train_subset: 0
    val_subset: 4000
    test_subset: 10000
    batch: 128
    lr: 1.0e-3
  reduced:
    epochs: 6
    train_subset: 12000
    val_subset: 2000
    test_subset: 4000
    batch: 128
    lr: 1.0e-3

# Each entry is one known linear measurement system. `kind` selects the operator
# from measlib.REGISTRY; remaining keys are its constructor arguments.
systems:
  denoise:                     # A = I, full rank, NO null space: the flow only
    kind: denoise              # has to undo noise, never to invent content
    sigma: 0.2
    beta: 1.0

  inpaint_box:                 # centred 12x12 hole: 144 unmeasured pixels
    kind: inpaint_box
    sigma: 0.05
    box: 12
    beta: 1.0

  inpaint_random:              # 50% of pixels dropped, fixed mask
    kind: inpaint_random
    sigma: 0.05
    drop: 0.5
    seed: 0
    beta: 1.0

  sr_2x:                       # 2x2 average pooling; null space = Haar details
    kind: sr_2x
    sigma: 0.05
    beta: 1.0

  sr_4x:                       # 4x4 average pooling; only a 8x8 thumbnail seen
```

```
  kind: sr_4x
  sigma: 0.05
  beta: 1.0

# ABLATIONS. Each changes ONE factor against a control that collapsed, and
# keeps the control's sampled target and zero rewind budget so the test is
# clean: the question is whether Gaussian input noise (the TarFlow paper's
# generative recipe, in place of uniform dequantization) prevents the
# collapse on its own. Under the a(a-1) = v/tau^2 analysis it caps the gain
# the objective can demand at sqrt(v)/noise_std (4 for denoise, 1 for the
# sigma = 0.05 systems) versus sigma/tau ~ 25-90 with dequantization, and
# widens the valley in the coupling's shift from tau to noise_std. Its bpd is
# NOT a data likelihood (Gaussian input noise breaks the dequantization
# bound) and its samples carry noise_std of noise; compare survival, failure
# counts and PSNR.
denoise_gauss:                  # control: archived first attempt (unbounded
  kind: denoise                 # head, pixel basis), destroyed at epoch 50
  sigma: 0.2
  beta: 1.0
  noise_type: gaussian
  noise_std: 0.05
  scale_bound: 0
  train_target: sampled
  max_rewinds: 0

inpaint_random_gauss:           # control: attempt 2 (bounded head, sampled
  kind: inpaint_random          # target), collapsed at epoch 24
  sigma: 0.05
  drop: 0.5
  seed: 0
  beta: 1.0
  noise_type: gaussian
  noise_std: 0.05
  train_target: sampled
  max_rewinds: 0
```

8.3 Results

system	kind	null dims	cond bpd	fail	rewinds	PSNR A^+y	PSNR sample	PSNR post-mean	DC err
inpaint_random	inpaint_random	507	1.4881	2/10000	1/3	9.34	16.83	18.48	0.1402
sr_2x	sr_2x	768	1.6361	4/10000	0/3	19.22	17.17	18.29	0.1524
inpaint_box	inpaint_box	144	1.6631	4/10000	0/3	15.47	14.73	16.57	0.0543
denoise	denoise	0	1.7347	2/10000	0/3	22.72	25.42	27.49	0.2167
sr_4x	sr_4x	960	1.7441	2/10000	0/3	15.21	14.03	15.34	0.1020
inpaint_random_gauss	inpaint_random, gauss 0.05, sampled target	507	4.6899	0/10000	0/0	9.47	24.36	26.34	0.0587
denoise_gauss	denoise, gauss 0.05, unbounded, sampled target	0	5.2549	0/10000	0/0	23.12	27.90	29.62	0.1885

Attempt 2 (archived, superseded)

Bounded head, V coordinates, sampled target, no rewind budget (hence 0/0); the `denoise_gauss` ablation ran directly after it and is included. The `inpaint_random` and `sr_2x` rows are evaluations of the destroyed weights and are kept only as the record of the collapse; see the Analysis.

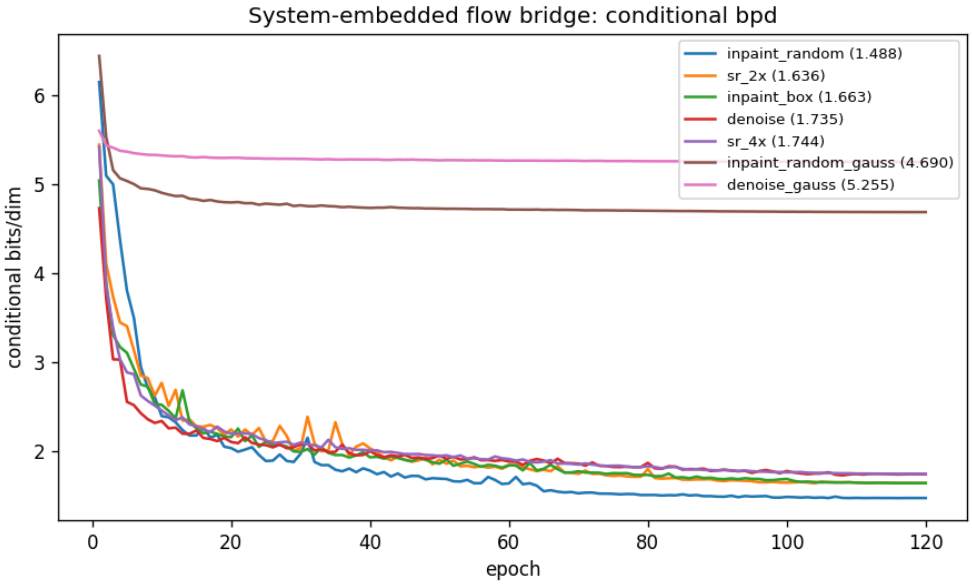


Figure 69: Conditional bits/dim per epoch for each measurement system.

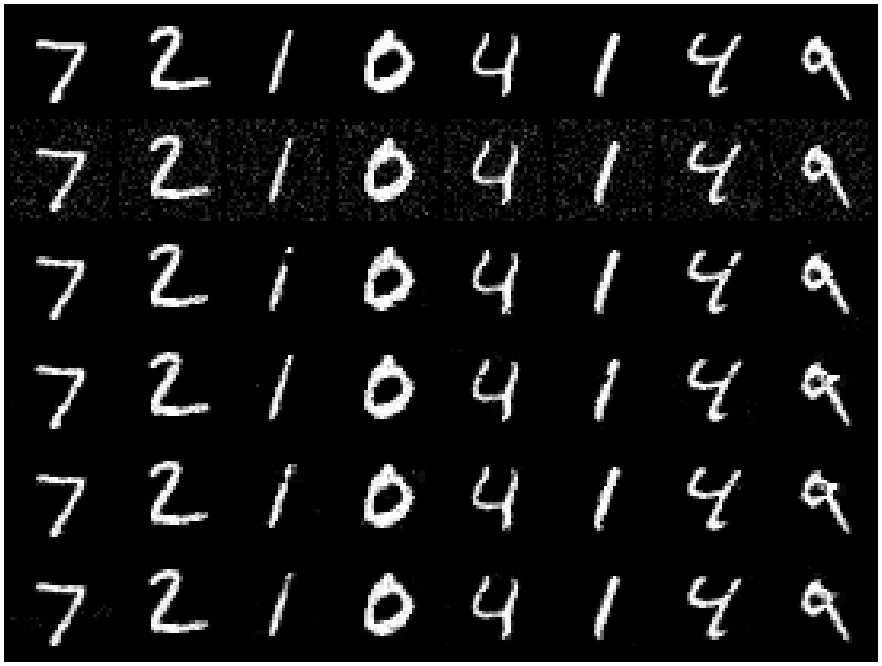


Figure 70: **denoise**. Rows: ground truth; pseudo-inverse $A^{+}y$; three posterior samples $f^{-1}(z)$, $z \sim \mathcal{N}(A^{+}y, \Sigma_1)$; posterior mean over 8 draws. Display is clamped to $[-1, 1]$: $\sigma = 0.2$ is 10% of the intensity range, and the noise row looks milder than that because the negative half of the background noise is below the black level.

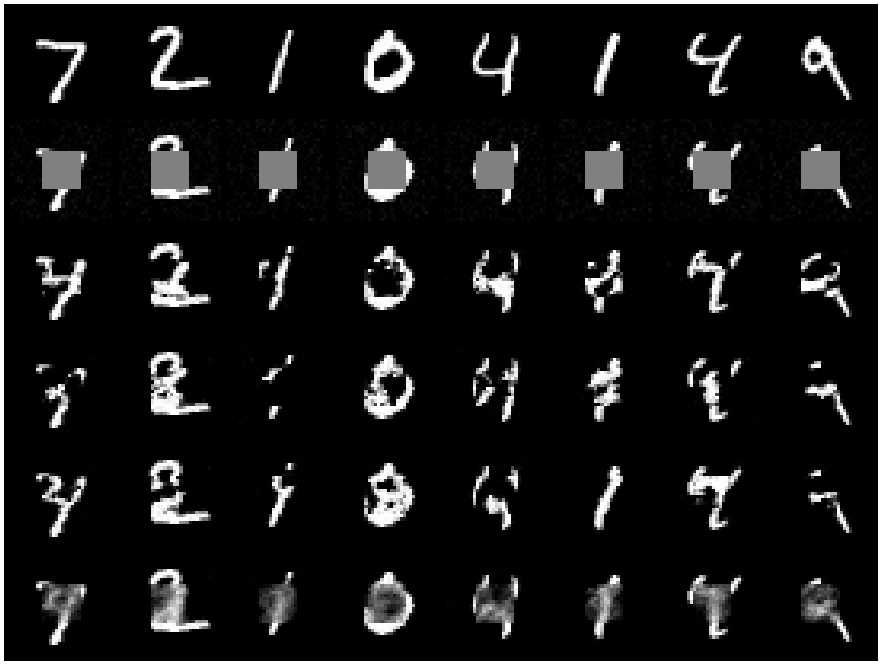


Figure 71: **inpaint_box**. Rows as above. The null space is the 144 masked pixels; posterior variation across samples should be confined to the hole.

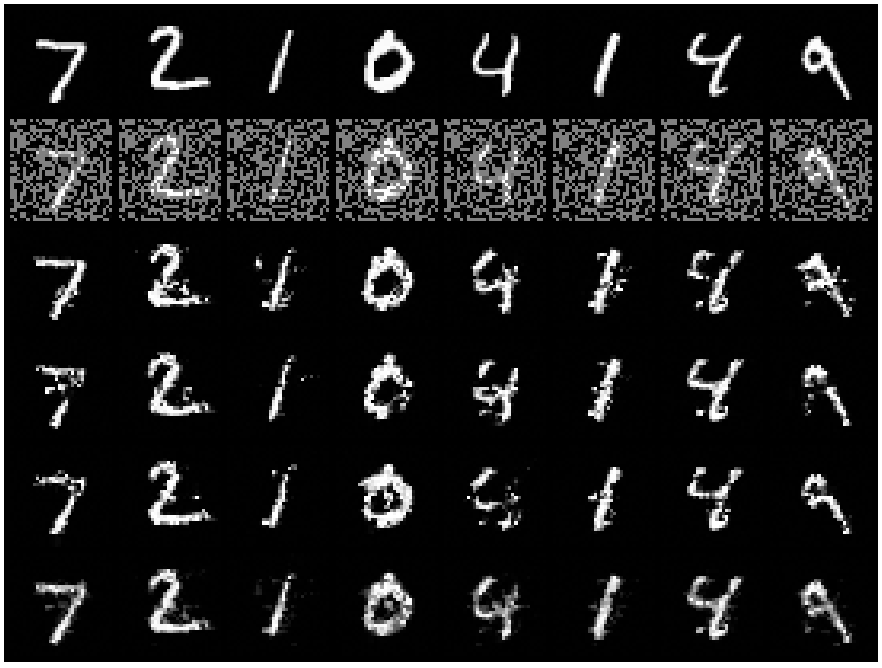


Figure 72: **inpaint_random**. Rows as above. A fixed 50% pixel mask; the pseudo-inverse row shows the dropped pixels as zeros.

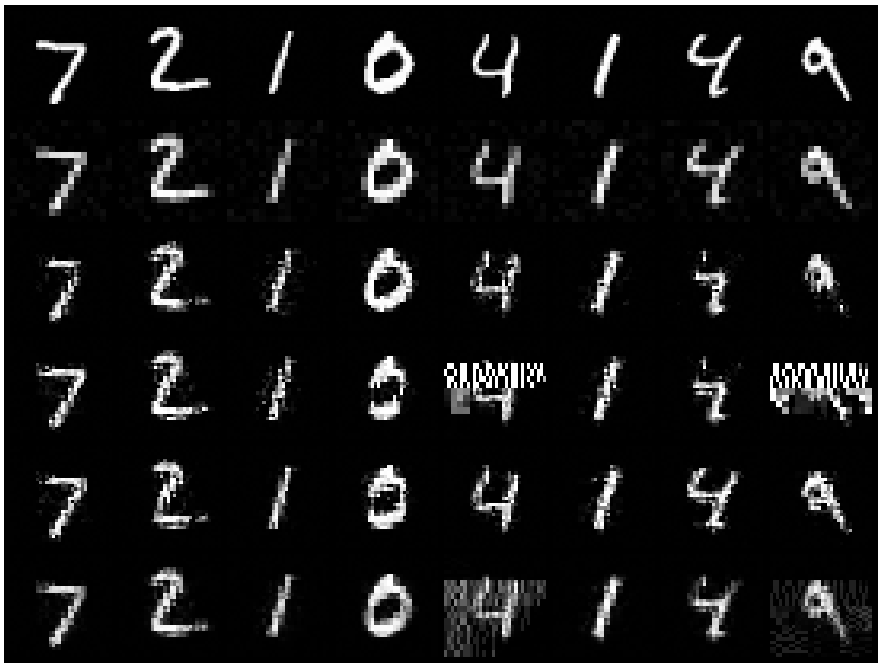


Figure 73: **sr_2x**. Rows as above. The null space is the three Haar detail bands of every 2×2 block (768 of 1024 dimensions).

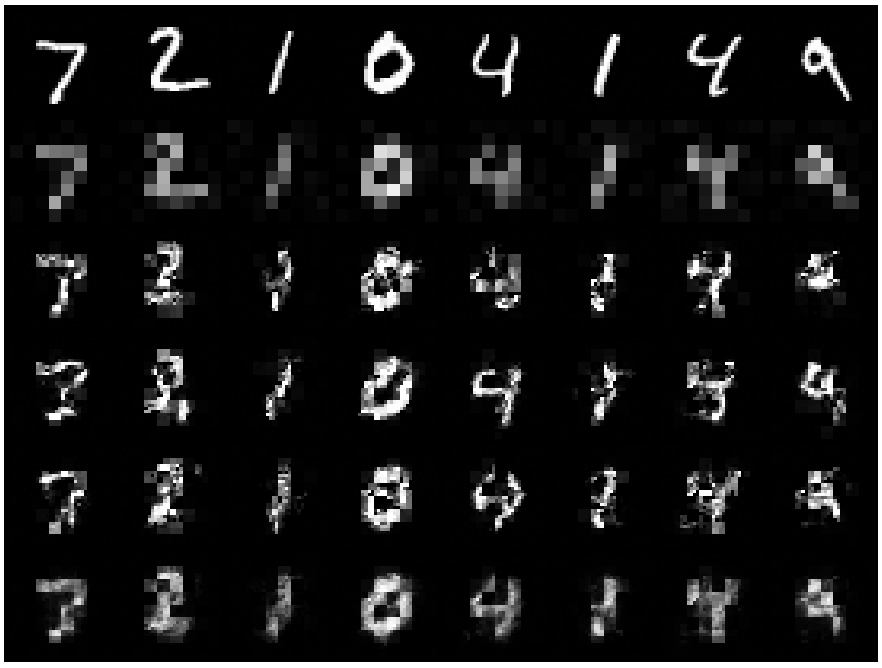


Figure 74: **sr_4x**. Rows as above. Only an 8×8 thumbnail is measured; 960 of 1024 dimensions are null space.

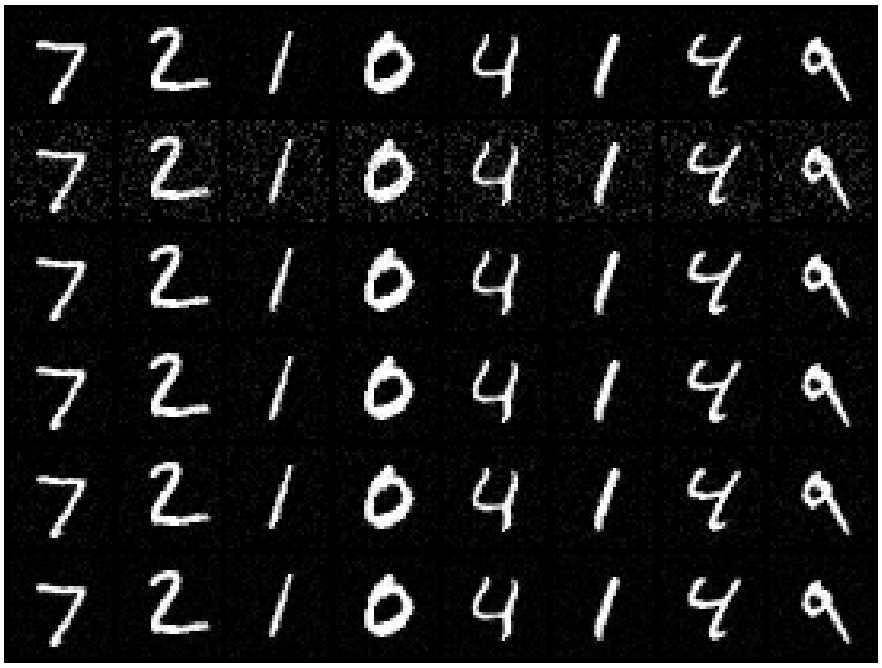


Figure 75: **Ablation denoise_gauss**: Gaussian input noise 0.05 in place of dequantization, official unbounded head, sampled target, no rewind. Rows as above; the ground-truth row carries the input noise.

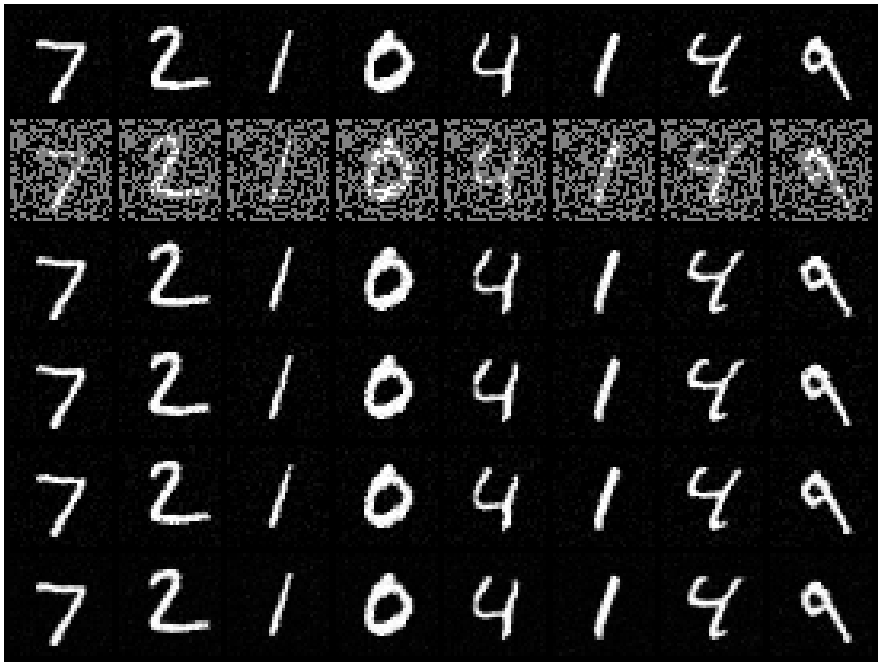


Figure 76: **Ablation inpaint_random_gauss**: Gaussian input noise 0.05, bounded head, sampled target, no rewind. Rows as above.

system	kind	null dims	cond bpd	fail	rewinds	PSNR A^+y	PSNR sample	PSNR post-mean	DC err
inpaint_box	inpaint_box, sampled target	144	1.6668	5/10000	0/0	15.47	14.60	16.45	0.0542
sr_4x	sr_4x, sampled target	960	1.7352	1/10000	0/0	15.21	13.97	15.22	0.1056
denoise	denoise, sampled target	0	1.7427	5/10000	0/0	22.72	25.23	27.41	0.2173
denoise_gauss	denoise, gauss 0.05, unbounded, sampled target	0	5.2549	0/10000	0/0	23.12	27.90	29.62	0.1885
inpaint_random	inpaint_random, sampled target	507	15.6049	1916/10000	0/0 abort	9.34	15.67	16.69	0.1909
sr_2x	sr_2x, sampled target	768	19.8423	1050/10000	0/0 abort	19.22	18.21	19.28	0.0892

8.4 Analysis

- **Two attempts failed before the runs above, in four distinct ways, and the failures are part of the result.** The runs above are the third attempt. The first (flow in pixel coordinates, official unbounded coupling, three systems) is archived in `outputs/step7_system_bridge/failed_pix` the second (bounded coupling, V coordinates, sampled target, no rewind) in `.../attempt2_bounded_sampled` Each failure is a property of the objective, not of the seed.

  1. *Chained exponential overflow.* `denoise` trained to 1.93 bits/dim by epoch 49 and was destroyed at epoch 50 (train loss 9×10^{24}, 92 of 469 batches rejected in that epoch, weights unrecoverable). `inpaint_box` trained healthily to 1.79 bits/dim while, intermittently from epoch 43 on, a single validation image in 4000 drove the validation mean to 10^{16}–∞. Tracing that image through the checkpoint: block 2 produced $x_a = -11.5$ on one token, so its output was a 10^5-scale token; block 3, reading that token through the unnormalised residual stream, answered with $x_a = -9582$, and e^{9582} is ∞ in fp32. The sampling direction has the same defect: on `sr_4x` the reverse pass produced $z_a = +154$ on a sampled context. Nothing about a near-zero base covariance can do this on its own — a residual of 1 on a variance-0.0025 coordinate costs 200 nats, not ∞ — the flow output has to reach $\sim 10^{19}$ first, and only the multiplicative chain across blocks can get it there. The bounded log-scale removes exactly that chain.
  2. *Basis mismatch.* `sr_4x` trained to the best conditional bpd of the three (1.28 bits/dim, no rejected batches) and produced *no* finite posterior sample. Hard-clamping z_a at inference made the samples finite but left them at PSNR 9–12 dB against 15.1 dB for the pseudo-inverse, so this was not the overflow problem in disguise. It is the per-channel-affine limitation of the coupling: in pixel coordinates the base is far from diagonal for average pooling, the forward pass can still reach a good likelihood by using context, but the inverse of such a map on base samples that are correlated in a way the coupling cannot represent is meaningless. Moving the flow to $V^\top x$ makes the base diagonal in the coupling's own coordinates.
  3. *An over-eager abort rule of our own.* The first script aborted a run after three consecutive non-finite validation means, which killed the healthy `inpaint_box` run at epoch 66 (test: posterior-mean PSNR 16.3 dB against 15.5 dB for the pseudo-inverse, data-consistency error 0.058, sample variation confined to the hole). One image in 4000 makes a mean non-finite without saying anything about the model; the abort now triggers only on training collapse and evaluation reports failure counts instead.
  4. *Sudden collapse with the bound in place (second attempt).* The bounded coupling in V coordinates fixed what it was built to fix: `denoise` survived past epoch 50 to 1.743 bits/dim (5 of 10000 test images fail, 23 batches rejected in 120 epochs; posterior mean 27.4 dB against 22.7 dB for A^+y), and `inpaint_box` reached 1.667 (5 failures; 16.5 dB against 15.5 dB; one transient burst at epoch 62 in which 106 batches were rejected and the training loss rose to 2.07 bits before recovering the next epoch), with every posterior sample finite. But `inpaint_random` collapsed at epoch 24 and `sr_2x` at epoch 52, each from a falling validation curve (1.97 and 1.93 bits/dim the epoch before) with essentially no rejected batches, to a majority of the next epoch rejected and the weights destroyed. The destroyed weights say what broke. On one validation batch their log-scales are ordinary — every block has $|x_a| < 4.1$, not one coordinate near the bound, the same range as the healthy `inpaint_box` model — but the base residual on the *range* coordinates, $(f(x) - A^+y)_i/\sqrt{v_i}$, has RMS 10.5 (`inpaint_random`) and 8.8 (`sr_2x`)

against 1.4 for the healthy model, costing 6 and 38 nats per range coordinate where the healthy model earns -1.1; the null coordinates are only mildly worse (3.7 and 2.1 against 1.4). The scales are fine and the *shifts* are wrong by ten standard deviations, which is the stiffness the $a(a-1) = v/\tau^2$ analysis predicts: a background range coordinate has gain $a \approx \sigma/\tau$, so the coupling's additive output must be accurate to $\tau \approx 2 \times 10^{-3}$ on hundreds of coordinates at once, while the training gradient with a sampled target is half noise. The expected target removes that noise; the rewind rule catches whatever remains and reports it. `sr_4x`, in the same attempt, trained for 120 epochs without a single rejected batch to 1.735 bits/dim (1 failure in 10000) with every sample finite — so the basis fix did its job — but its posterior mean only matched the pseudo-inverse (15.22 against 15.21 dB), a single sample was worse (13.97 dB), and its data-consistency error was 0.106, twice the floor: with 960 null dimensions and only an 8×8 thumbnail measured, the sampled details were noise-like and leaked into the range through context.

- **Third attempt: it trains, and the result is still weak.** With the expected target and the rewind budget every system reached epoch 120. `inpaint_random`, which had collapsed at epoch 24 before, ran to epoch 65 before the validation curve jumped from 1.62 to 6.22 bits/dim; the rewind rule restored epoch 64, halved the peak learning rate, and the run finished at 1.488 bits/dim (2 failures in 10000, 8 rejected batches, posterior mean 18.5 dB against 9.3 dB for the zero-filled pseudo-inverse). `sr_2x`, which had collapsed at epoch 52, needed no rewind (14 rejected batches) and reached 1.636 bits/dim, but its samples are worse than the pseudo-inverse in PSNR (17.2 dB single, 18.3 dB mean, against 19.2 dB), a few of them carry visible 4×4 block artifacts, and its data-consistency error is 0.152, three times the floor of 0.053. `sr_4x` reproduced attempt 2 almost exactly (1.744 bits/dim, posterior mean 15.3 dB against 15.2 dB for A^+y, data consistency 0.102): the expected target changed nothing where nothing had been unstable. `denoise` reached 1.735 bits/dim with a posterior mean of 27.5 dB against 22.7 dB, its data consistency 0.217 at the floor of 0.207. `inpaint_box` also reproduced attempt 2 (1.663 bits/dim, 4 rejected batches instead of 109, posterior mean 16.6 dB against 15.5 dB, data consistency 0.054 at the floor: its 144-pixel hole is small enough that the sampled content cannot leak much). Two facts summarise the attempt. First, on the three systems with a large null space the data-consistency error is two to three times the noise floor, i.e. the sampled null content leaks into the range through context-dependent shifts, and on `sr_2x` and `sr_4x` the posterior mean does not beat the pseudo-inverse. Second, every conditional bits/dim in the table, 1.49–1.74, is *above* the unconditional 1.171 bits/dim that the same architecture, budget and dequantization reach on MNIST without any measurement (step 4, `baseline_flip`). For the true densities $H(X|Y) \le H(X)$, so a well-fit conditional model must come in below the unconditional one; these do not, by 0.3–0.6 bits/dim. Part of the gap on the super-resolution systems is the Haar basis the flow is forced into (step 5 measured that cost), but `denoise` and `inpaint_random` act in pixel coordinates and are above the line by the same margin. The unconditional-f construction is paying for the measurement rather than profiting from it, which is what the $a(a-1) = v/\tau^2$ analysis says it must do on stiff range coordinates. Step 8 drops the construction and conditions f on A^+y directly.

- **Gaussian input noise alone prevents the denoise collapse.** `denoise_gauss` differs from the archived first-attempt `denoise` run (destroyed at epoch 50) only by Gaussian input noise of 0.05 in place of dequantization: the head is the official unbounded one, the target is sampled, and there is no rewind. It trained for 120 epochs with *zero* rejected batches, no test failure in 10000, every sample finite, and posterior-mean PSNR of 29.6 dB against

23.1 dB for A^+y (measured against the noisy input, whose own noise caps PSNR near 32 dB). Its 5.25 bits/dim is not comparable with the dequantized runs. This is the outcome the $a(a-1) = v/\tau^2$ analysis predicts — with $\tau = 0.05$ the objective asks for gains of at most 4 and tolerates shift errors 25 times larger — and it is the cheapest stabiliser found here; its price is the loss of the likelihood interpretation and 0.05 of residual noise in every sample (removable with a Tweedie step $x + \tau^2 \nabla_x \log p(x|y)$, which the flow supplies exactly, not done here). `inpaint_random_gauss` — the system that collapsed at epoch 24 in attempt 2, run again with Gaussian input noise 0.05, the bounded head, the *sampled* target and no rewind — confirms it: 120 epochs, zero rejected batches, no test failure, every sample finite, and a posterior mean of 26.3 dB (single sample 24.4 dB) against 9.5 dB for the zero-filled pseudo-inverse and 18.5 dB for the dequantized attempt-3 model of the same system, with data consistency 0.059 at the floor instead of 0.140. Against the noisy input the ceiling is about 32 dB. So on this system the unconditional bridge is not a bad model; the dequantized objective is a badly conditioned one. This is the strongest step-7 result and it is the one that gives up the data likelihood. Whether the same holds for the super-resolution systems, where the leak is largest, was not run: step 8 makes the comparison moot by removing the stiffness at the source.

- **What the conditional bpd does and does not mean.** $-\log p(x|y)/(D \ln 2)$ is a conditional quantity: it falls trivially as the measurement becomes more informative, so it is comparable across methods only at fixed (A, σ), never across measurement operators or noise levels. The `denoise` system (no null space) and `sr_4x` (960 null dimensions of 1024) are therefore not competing with each other; each is its own experiment. The constant $-\frac{1}{2} \log\det \Sigma_1$ is included in every reported number, as it must be for the value to be a likelihood — it is easy to drop because it does not affect gradients.

- **The falsifiable claims.** (i) Data-consistency error should sit at the *measurement-noise floor*. The floor is not zero: the ground truth itself disagrees with A^+y on the range by A^+n, which on the test set is a relative error of 0.207 for `denoise` ($\sigma = 0.2$) and 0.051–0.055 for the four $\sigma = 0.05$ systems, and a raw base sample ($f = \mathrm{id}$) sits at the same values because $\gamma = 1$ makes the base's range noise equal to the measurement noise. By the $a(a-1) = v/\tau^2$ analysis a converged flow pulls prior-dominated coordinates to the prior and leaves measurement-dominated ones alone, so its samples also sit at the floor. The metric therefore cannot distinguish a converged flow from the identity; what it *can* do is expose a sampler that corrupts the range through context-dependent shifts on badly sampled null content (after two epochs on `sr_4x` it read 0.17, three times the floor). (ii) Posterior samples should differ from each other mainly in the null space. (iii) The posterior mean should beat a single sample on PSNR while being blurrier, the usual MMSE-versus-sample trade-off; and for `denoise` the single sample should already beat A^+y, since the analysis predicts near-zero error on background pixels and roughly $2v$ on foreground ones against v everywhere for the pseudo-inverse. Each is measured directly and any failure is reported.

- **Honest framing relative to SDB.** This is a bridge in the sense that the base is the pseudo-inverse distribution rather than white noise; it is *not* a stochastic process with intermediate marginals. SDB's power comes partly from the schedules $\alpha_t, \beta_t, \gamma_t$ annealing the problem over many steps, whereas a flow must absorb all of that into a single bijection. TarFlow's block stack supplies an implicit time axis, so the faithful analogue — making block k target the SDB marginal at t_k — is the natural next step and the version that would actually inherit the bridge structure.

- **Known limitation.** A trained model is tied to the system it was trained on. SDB reports robustness under system misspecification; the flow bridge as built here has no mechanism for that, since A enters only through a fixed base density. Conditioning f on A (or on A^+y as an auxiliary input) is required before any generalisation claim.

- **Connection to steps 4–6.** Σ_1 is diagonal in the right-singular-vector basis V, so the physics supplies a natural tokenization: order tokens by singular value and the autoregression generates well-determined content first and unmeasured content last, conditioned on it. That is the coarse-to-fine ordering of step 6 with the ordering induced by the measurement operator rather than by a wavelet. Steps 4 and 6 both found that reordering away from raster patches costs likelihood, so this is a genuine open question rather than an obvious win; the SVD ordering is not used in the runs above, which keep the official ordering throughout.

9 Step 8: Conditional TarFlow on the pseudo-inverse

9.1 Methods

- Code: `step8_conditional_flow.py` + `invlib.condition_on_image` + `measlib.DenseSystem` / `CTParallel`; config: `configs/step8_conditional_flow.yml`; outputs: `outputs/step8_conditional_flow`

- **Why.** Step 7 embedded the measurement in the base density and kept f unconditional. After three attempts it trains, but the outcome is weak: its conditional bits/dim (1.49–1.81 across systems) sit *above* the unconditional MNIST number of the same model, budget and dequantization (1.171, step 4 `baseline_flip`), which is impossible for a well-fit conditional density since $H(X|Y) \leq H(X)$; its data-consistency error sits at about three times the measurement-noise floor; and on `sr_4x` its posterior mean does not beat the pseudo-inverse. The $a(a-1) = v/\tau^2$ analysis of §8.4 explains why: an unconditional f can only use the measurement by expanding stiff range coordinates, which is what makes the objective hard to optimise. The obvious alternative is to let f *see* the measurement. This step does exactly that, and nothing else.

- **Model.** An official TarFlow with the standard-normal base and the official bits/dim, whose coupling networks receive the pseudo-inverse reconstruction as an extra, image-shaped input:

$$\log p(x|y) = \log \mathcal{N}\big(f(x;\, A^+y);\, 0,\, I\big) + \log \left|\det J_f(\,\cdot\,; A^+y)\right| .$$

  For every fixed y the map $x \mapsto f(x; A^+y)$ is a bijection, so this is a proper conditional density trained by exact maximum likelihood on (x, y) pairs with $y = Ax + \sigma\epsilon$ drawn fresh at every step; a posterior sample is $x = f^{-1}(z; A^+y)$ with $z \sim \mathcal{N}(0, I)$. Nothing is stiff: the measurement noise lives in the conditioning input, where the network is free to learn to ignore it, not in the density's target. There is no whitening, no V basis and no closed-form expected target.

- **Why A^+y and not y.** The conditioning has to be concatenated with the image, so it must have the image's shape; y does not in general — A is rectangular, and for CT y is a sinogram. A^+y always has the image's shape whatever y is, it carries the same information as y on the range of A (the pseudo-inverse is injective on the range, so no information is lost beyond what the noise already destroyed), and it is the natural "first guess" that the flow then corrects. On the null space it is identically zero, which the network can only read as "unmeasured".

- **How the conditioning enters (`invlib.condition_on_image`).** A^+y is patchified like the image ($T = 64$ tokens of 16 pixels), embedded by its own linear projection and positional table, and *prepended* to the token sequence as a prefix of length T. The attention mask over the $2T$ tokens lets every prefix token attend to the whole prefix, and every image token attend to the whole prefix and, causally, to itself and earlier image tokens. The output at the last prefix position supplies the affine parameters of image token 0 (the official model uses zeros there), and the output at image token i those of token $i + 1$, exactly as in the official shift-by-one. The coupling $z = (x - x_b)e^{-x_a}$, the per-block permutations, the log-determinant $-\bar{x}_a$ and the KV-cached sampling loop are the official ones; sampling first pushes the prefix through the cache and then runs the official one-token-at-a-time loop, so the map is exactly invertible for every conditioning image. The conditioning image is permuted with the same flip as the image in every block, so prefix token i always describes the same patch as image token i. `ml-tarflow` is not modified: blocks are re-classed at runtime and gain one 16×128 linear layer and one 64×128 positional table each, 41,472 parameters in total on top of the 1,644,160 of the model (2.5%). The sequence length doubles, so attention costs about four times as much per block as in step 7.

- **Log-scale bound and its ablation.** The likelihood does not need a bounded coupling head here (the base is $\mathcal{N}(0, I)$), but the *sampling* direction can still overflow: for z off the data manifold, e^{z_a} chained across four blocks can reach `inf` in an under-trained or extrapolating head. In a 32-step smoke test on `sr_4x` the official unbounded head produced 60 of 64 non-finite posterior samples while the forward map round-tripped to 4×10^{-6}; the bounded head ($8 \tanh(x_a/8)$ in both directions, step 7's construction) produced 0 of 64. The bound is the default; the `sr_4x_unbounded` system re-runs `sr_4x` with the official head to measure its effect on a trained model rather than assume it.

- **Training-loop safeguards.** The batch-rejection rule ($\ell > 5\,\text{ema} + 5$ nats/dim) and the rewind-on-collapse budget of step 7 (restore last healthy epoch, halve peak learning rate, at most twice) are kept verbatim so that a difference between the two steps cannot be attributed to the loop; both counts are reported per system and are expected to stay near zero.

- **A rectangular system: sparse-view parallel-beam CT (`measlib.DenseSystem`, `CTParallel`).** Every step-7 system had an implicit SVD (pixel masks, Haar averaging). To exercise the rectangular case the library gains a generic operator built from an explicit matrix $A \in \mathbb{R}^{M \times D}$ and a numerical SVD, with a *truncated* pseudo-inverse: singular values below $10^{-2}\, s_{\max}$ are treated as zero, so A^+ never amplifies the noise by more than $100\times$ and the corresponding directions join the null space. `radon_matrix` is a pixel-driven parallel-beam projector (every pixel centre projected onto the detector axis of every view at angles $\pi a/n_{\text{angles}}$, linearly split between its two neighbouring bins). The `ct_sparse` system uses 8 views and 47 bins: y is an 8×47 sinogram, A is 376×1024, 290 singular values are kept, 86 are truncated, and 734 of 1024 dimensions are null space. The noise is $\sigma = 0.5$ in sinogram units (a ray through the $[-1, 1]$-scaled background sums to about -32, so this is roughly 1.5% of a ray sum). The pseudo-inverse reconstruction is the familiar streaky sparse-view image at 17.8 dB (first 64 test images), and the data-consistency floor — the relative range-space discrepancy between the truth and A^+y, which a perfect sample would also show — is 0.187 (against 0.203 for `denoise` and 0.051–0.055 for the $\sigma = 0.05$ systems on the same images).

- **Verification (`--verify-only`, run before every training job).** On an 8×8 image with patch 4 ($T = 4$), 64 channels, and randomly perturbed weights so that the coupling is far

from the identity, for three configurations (two blocks with and without the bound, one block unbounded), in float32: (1) inversion given the conditioning image, 9.5×10^{-7}; (2) the reported log-determinant against $\log|\det J|/D$ from the autograd Jacobian, 2.8×10^{-8}; (3) z depends on the conditioning image (relative change 1.55 when it is perturbed); (4) the autoregressive structure is intact in a single block — perturbing image token j leaves z tokens $< j$ unchanged, and perturbing any conditioning token changes every image token. The official LayerNorm casts to float32, so the check cannot be run in double; the tolerance is 10^{-4} and every value is two or more orders below it.

- **Metrics, identical to step 7.** Conditional bits/dim on the full 10k test set with a fresh measurement per image, with the robust statistics of step 7 (values above 50 counted as failures); PSNR of A^+y, of one posterior sample and of the mean of four draws, after clamping to $[-1, 1]$, on the first 64 test images; relative data-consistency error $\|A^+A\hat{x} - A^+y\|/\|A^+y\|$ of one sample; non-finite samples counted and excluded. The same test images and the same generator seed are used, and the PSNR of A^+y in the table below reproduces the step-7 values to the last digit, which is the check that the two tables are on the same footing.
- **The falsifiable claims.** (i) The conditional bits/dim must fall *below* 1.171 for every system, because a conditional density can only be sharper, and by more for more informative measurements. (ii) On the five systems shared with step 7 it must beat step 7 at equal model, budget and data. (iii) The data-consistency error should sit at the floor given above, not at three times it. (iv) Posterior mean > single sample > A^+y in PSNR, except that for a near-noiseless measurement a sample is allowed to tie the pseudo-inverse. (v) For CT the samples must remove the streaks while agreeing with the sinogram. (vi) The unbounded ablation should match the bounded run in bits/dim; any non-finite samples are reported.

9.2 Configuration (verbatim)

```
# Config for step8_conditional_flow.py -- conditional TarFlow on A^+y.
#
# The flow models p(x | A^+y) directly: an official TarFlow (standard normal
# base, official bits/dim) whose coupling networks receive the pseudo-inverse
# reconstruction A^+y as an extra, image-shaped input. Because A^+y always has
# the image's shape, the measurement y itself can be anything -- a masked
# image, a thumbnail, a sinogram -- so A may be rectangular.
#
# sigma > 0 for every system (a measurement always has noise; measlib requires
# it). gamma/beta play no role here: the base density is N(0, I).

seed: 20260901
tarflow_repo: /dev_ws/ml-tarflow
output_root: outputs/step8_conditional_flow

dataset: mnist
img_size: 32
channel_size: 1
patch_size: 4        # T = 64 tokens, same protocol as steps 4-7
noise_type: uniform  # uniform dequantization, so bpd is a data likelihood
grad_clip: 1.0
# Batch-rejection rule from step 7 (loss > mult * ema + abs, nats/dim); with
# the standard-normal base it is expected to fire rarely if ever, and the
# count is reported.
```

```
loss_reject_mult: 5.0
loss_reject_abs: 5.0
# Coupling log-scale soft bound (invlib.bound_log_scale semantics, applied in
# both directions; 0 = the official unbounded head). Not needed for the
# likelihood -- the base is N(0, I) -- but a barely-trained or extrapolating
# head can still overflow in the SAMPLING direction (z off the data manifold
# -> exp(za) chained over 4 blocks -> inf); the bound keeps every sample
# finite at no cost in the operating regime |xa| < 4. One ablation below
# runs the official unbounded head so the effect is measured, not assumed.
scale_bound: 8.0
# Rewind-on-collapse as in step 7 (restore last healthy epoch, halve peak lr,
# repeat); reported per system, 0 disables.
max_rewinds: 2

model:               # identical to steps 4-7 so numbers stay comparable
  channels: 128
  blocks: 4
  layers_per_block: 2

budget:
  full:
    epochs: 120
    train_subset: 0
    val_subset: 4000
    test_subset: 10000
    batch: 128
    lr: 1.0e-3
  reduced:
    epochs: 6
    train_subset: 12000
    val_subset: 2000
    test_subset: 4000
    batch: 128
    lr: 1.0e-3

# Each entry is one known linear measurement system (measlib.REGISTRY `kind`
# plus its constructor arguments). The CT system is new -- its measurement is
# an 8 x 45 sinogram, not an image -- and runs first; the other five are the
# step-7 systems with the same parameters, so the two approaches are directly
# comparable. Run-level keys (scale_bound, noise_type, max_rewinds) may be
# overridden per system for ablations.
systems:
  ct_sparse:                  # 8-view parallel-beam CT: y is (8, 47), A is
    kind: ct                  # 376 x 1024; truncated pseudo-inverse rank 290
    sigma: 0.5                # in sinogram units (ray sums are O(30))
    n_angles: 8
    rcond: 1.0e-2

  sr_4x:
    kind: sr_4x
    sigma: 0.05

  inpaint_box:
    kind: inpaint_box
```

```
  sigma: 0.05
  box: 12

inpaint_random:
  kind: inpaint_random
  sigma: 0.05
  drop: 0.5
  seed: 0

sr_2x:
  kind: sr_2x
  sigma: 0.05

denoise:
  kind: denoise
  sigma: 0.2

sr_4x_unbounded:              # ablation: the official unbounded head
  kind: sr_4x
  sigma: 0.05
  scale_bound: 0
```

9.3 Results

Systems are sorted by conditional bits/dim. `meas` is the number of measurements M (376 for the sinogram, the number of kept pixels or thumbnail pixels otherwise); `null dims` the null-space dimension (including truncated singular directions for CT); `fail` the test images with bits/dim above 50; `rewinds` the rewinds used of the budget.

system	kind	meas	null dims	cond bpd	fail	rewinds	PSNR A^+y	PSNR sample	PSNR post-mean	DC err
inpaint_random	inpaint_random	517	507	0.9942	3/10000	0/2	9.34	24.39	26.43	0.0731
denoise	denoise	1024	0	0.9942	0/10000	0/2	22.72	28.72	30.93	0.2063
sr_2x	sr_2x	256	768	0.9986	0/10000	0/2	19.22	26.09	28.08	0.0654
inpaint_box	inpaint_box	880	144	1.0238	1/10000	0/2	15.47	17.38	19.47	0.0582
ct_sparse	ct	376	734	1.0695	0/10000	0/2	17.79	23.73	25.85	0.1999
sr_4x_unbounded	sr_4x, unbounded	64	960	1.0972	1/10000	0/2	15.22	17.68	19.87	0.0816 (2/64 bad)
sr_4x	sr_4x	64	960	1.1044	1/10000	0/2	15.21	17.54	19.77	0.0827

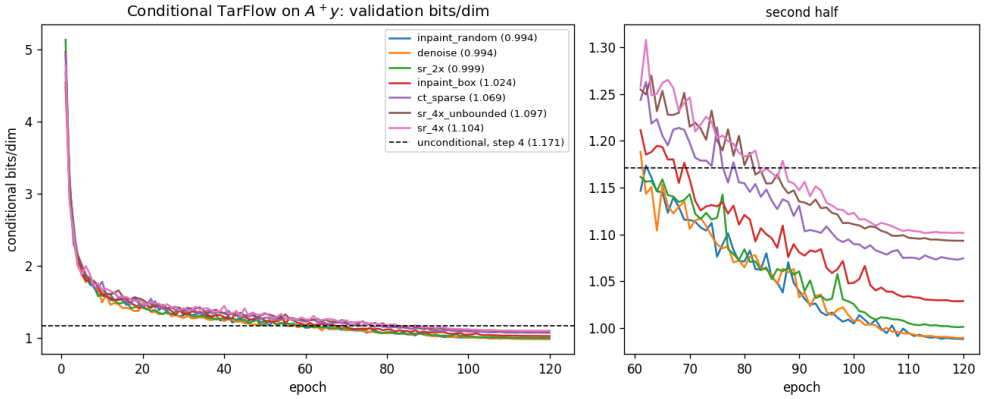


Figure 77: Validation conditional bits/dim per epoch for each measurement system (left: whole run; right: epochs 61–120). The dashed line is the unconditional test value of the same model, budget and dequantization (step 4 `baseline_flip`, 1.171); every curve ends below it.

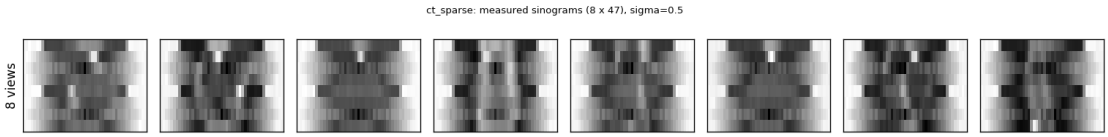


Figure 78: **ct_sparse**: the measurements themselves — 8×47 sinograms (views $\times$ bins) of the eight test images in the next figure. These cannot be concatenated with a 32×32 image; the flow is conditioned on their pseudo-inverse instead.

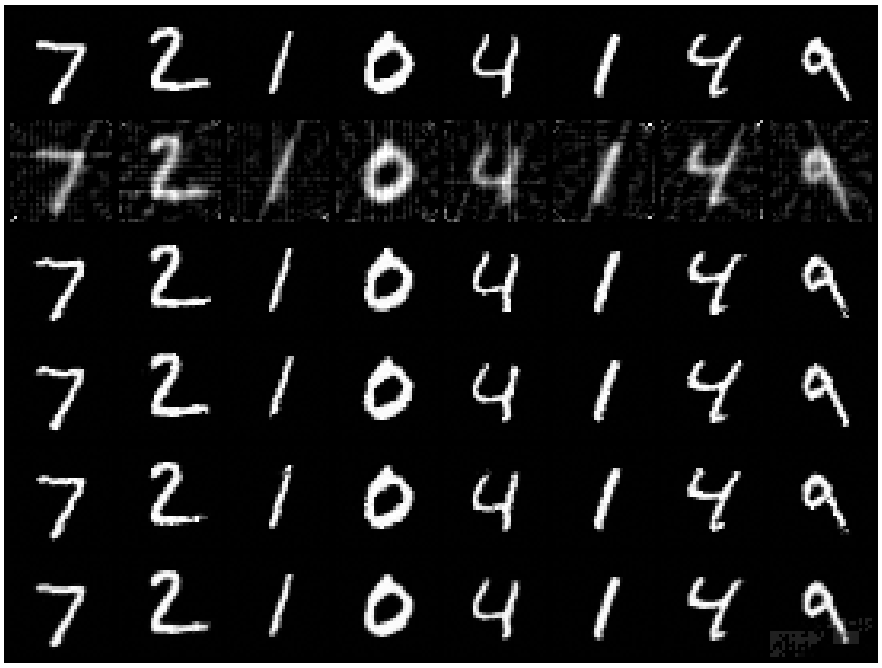


Figure 79: **ct_sparse**. Rows: ground truth; truncated pseudo-inverse A^+y (8-view streaks); three posterior samples $f^{-1}(z; A^+y)$, $z \sim \mathcal{N}(0, I)$; posterior mean over 8 draws. Display clamped to $[-1, 1]$.

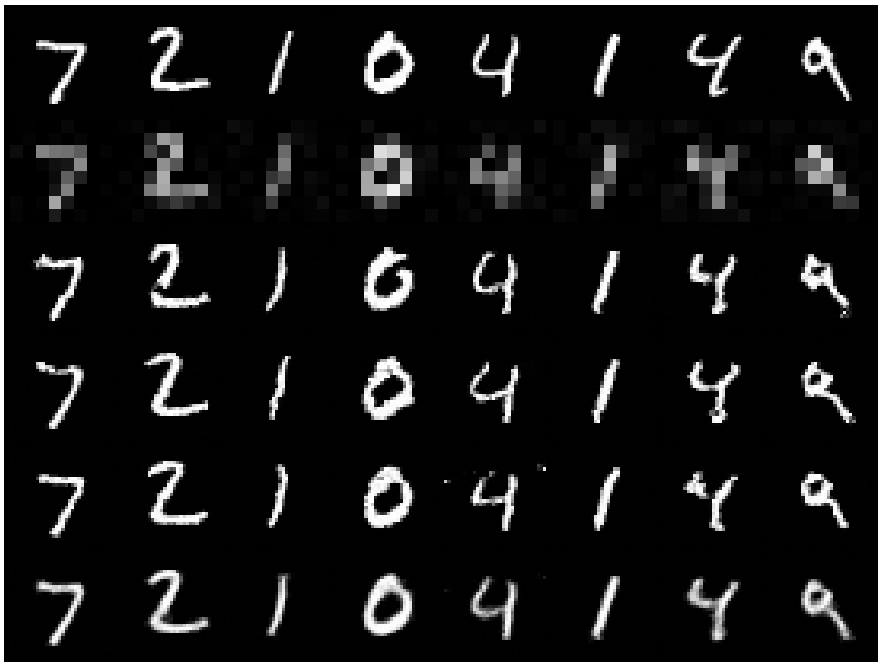


Figure 80: **sr_4x**. Rows as above. Only an 8×8 thumbnail is measured (960 null dimensions); compare the step-7 figure for the same images.

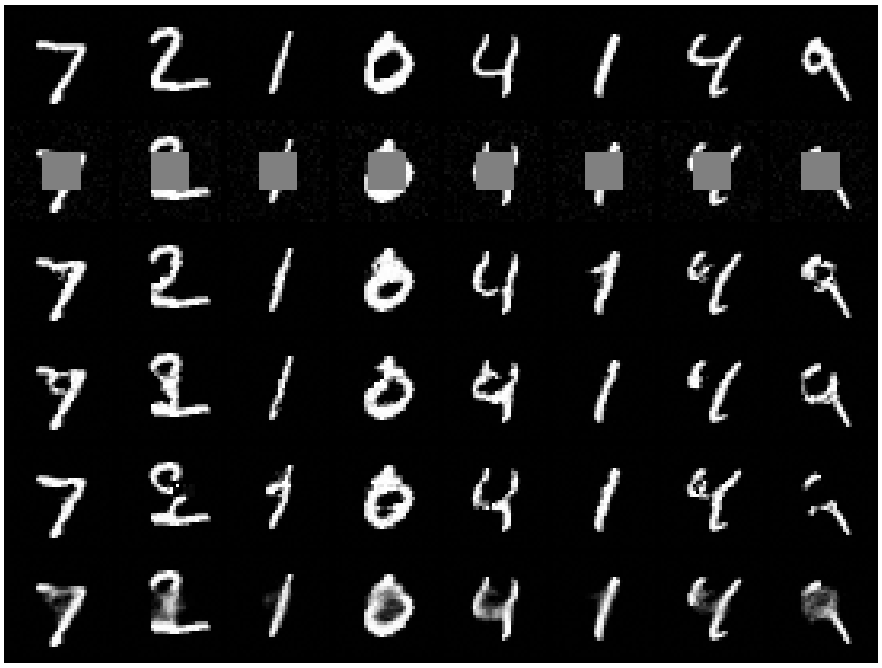


Figure 81: **inpaint_box**. Rows as above; the 12×12 hole is the null space.

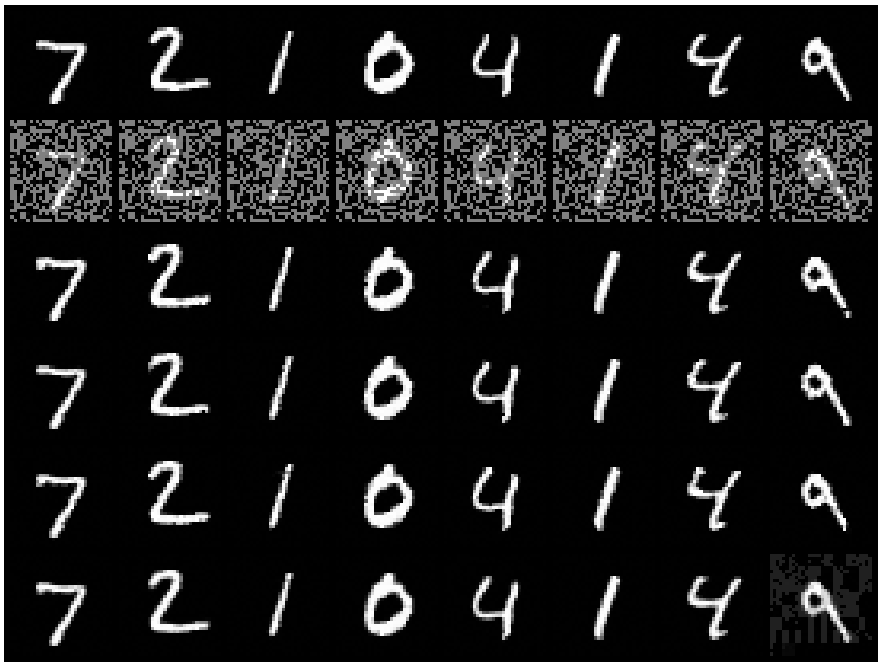


Figure 82: **inpaint_random**. Rows as above; a fixed 50% pixel mask.

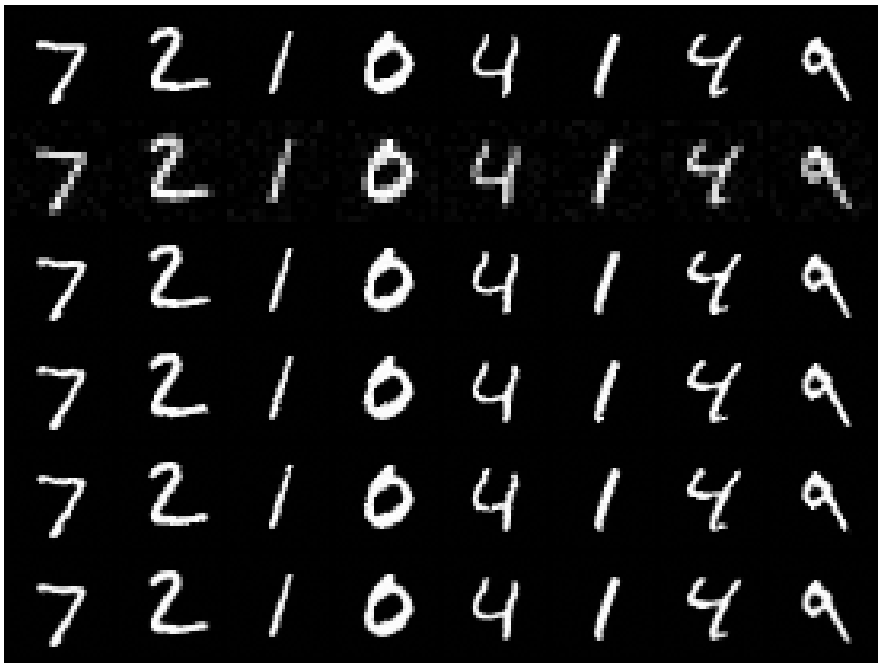


Figure 83: **sr_2x**. Rows as above; 768 null dimensions.

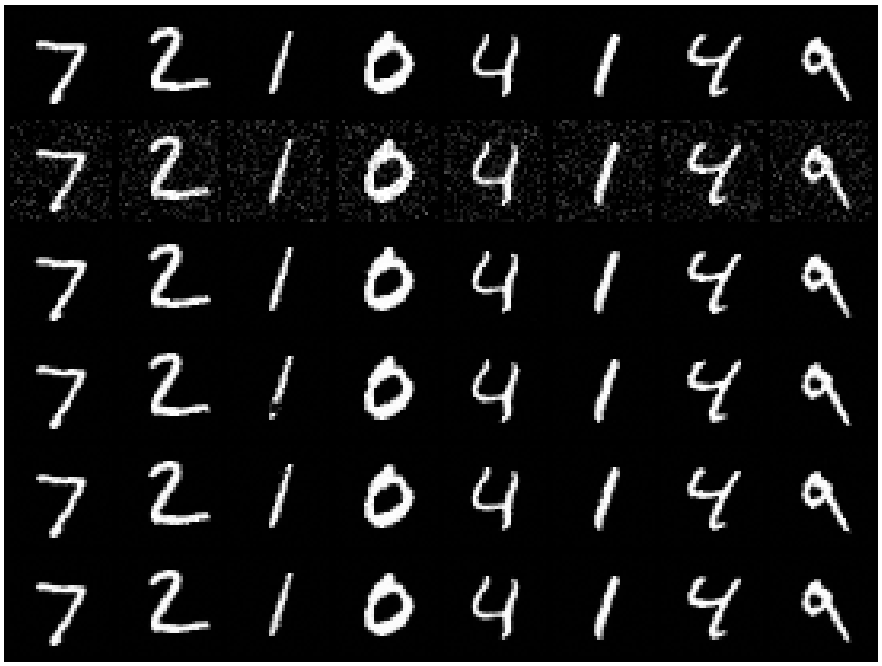


Figure 84: **denoise**, $\sigma = 0.2$. Rows as above; no null space.

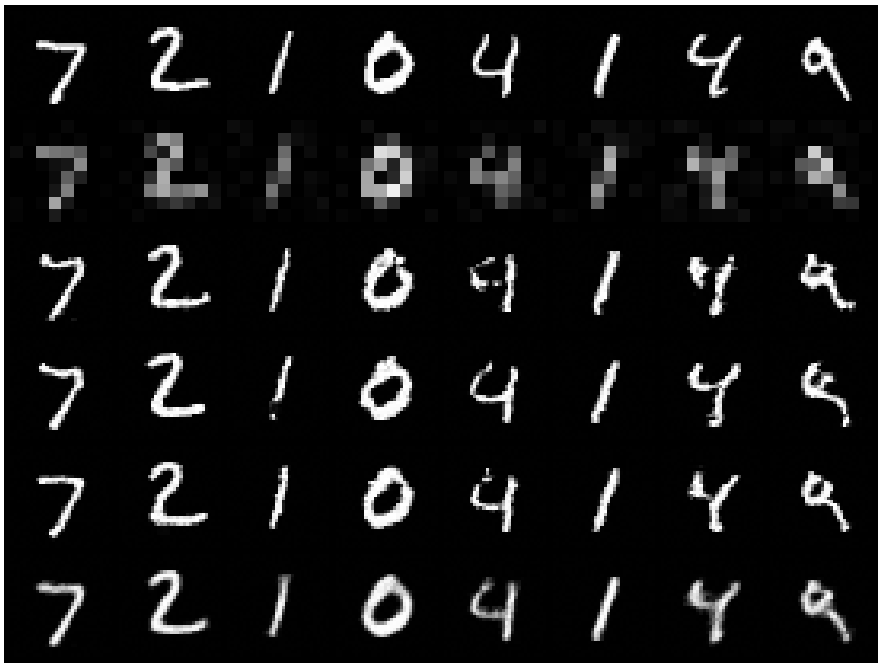


Figure 85: **Ablation sr_4x_unbounded**: the official unbounded coupling head. Rows as above; non-finite samples, if any, are shown as zeros.

9.4 Analysis

- **Every falsifiable claim held, on every system, at the first attempt.** Six systems, one script, no rewind used, 1–6 rejected batches per 120-epoch run, at most 3 test failures in 10000, every posterior sample finite, 16.6–17.1 minutes per system. Conditional bits/dim: `denoise` 0.994, `inpaint_random` 0.994, `sr_2x` 0.999, `inpaint_box` 1.024, `ct_sparse` 1.070, `sr_4x` 1.104 — all below the unconditional 1.171 (claim i), all 0.6–0.7 bits/dim below the step-7 bridge on the five shared systems (claim ii: 1.735, 1.488, 1.636, 1.663, 1.744). Posterior mean > sample > A^+y in PSNR on every system with no exception needed (claim iv): the mean gains 4.0 dB (`inpaint_box`), 4.6 (`sr_4x`), 8.1 (`ct_sparse`), 8.2 (`denoise`), 8.9 (`sr_2x`) and 17.1 dB (`inpaint_random`) over the pseudo-inverse, against step 7's +1.1, +0.1, —, +4.8, −0.9 and +9.1 dB. The two systems on which step 7's samples were worse than or tied with the pseudo-inverse (`sr_2x`, `sr_4x`) gain 9.8 and 4.4 dB on the posterior mean relative to step 7, and the 4×4 block artifacts of the step-7 `sr_2x` samples are gone.

- **Data consistency (claim iii).** Relative to the floor measured on the same images, the sampled range-space error is 1.01× (`denoise`), 1.07× (`ct_sparse`), 1.14× (`inpaint_box`), 1.23× (`sr_2x`), 1.4× (`inpaint_random`) and 1.5× (`sr_4x`), against 2–3× for the step-7 bridge on the three large-null-space systems. It is at the floor where the measurement pins most of the image and drifts above it as the null space grows, which is the expected signature of a sampler that generates null content with context and lets a little of it leak into the range; the leak is not enforced away (nothing projects the sample back onto A^+y), it is only learned away, and with 960 null dimensions it is learned to within 50%. A hard projection $x \leftarrow x - A^+(Ax - y)$ after sampling would remove the residual exactly at the cost of the likelihood interpretation; it is not applied here so that the numbers measure the model.

- **The rectangular case works (claim v) and costs nothing extra.** `ct_sparse` conditions on the truncated pseudo-inverse of an 8×47 sinogram — the measurement never touches the network in its own shape — and produces streak-free digits at 23.7 dB per sample and 25.9 dB for the mean, from a pseudo-inverse at 17.8 dB, with data consistency at 1.07× the floor. The variation between draws is confined to strokes that eight views leave ambiguous. The system is treated by the same code path as the others: `DenseSystem` needs only the matrix, and the flow needs only that A^+y has the image's shape.

- **The bits/dim gap measures the information in the measurement.** With both models trained on the same data to the same budget, $\text{bpd}_{\text{uncond}} - \text{bpd}_{\text{cond}}$ estimates $I(X;Y)/(D \ln 2)$, the mutual information per pixel between image and measurement under the model (an estimate, not a bound: both terms are cross-entropies that upper-bound their respective entropies). Per image: `denoise` 181 bits, `inpaint_random` 181, `sr_2x` 176, `inpaint_box` 151, `ct_sparse` 104, `sr_4x` 68. The ranking is not by the number of measurements — `inpaint_box` measures 880 pixels and `sr_2x` 256 — but by what they say about the part of the image that is uncertain: the centred hole removes exactly the pixels that carry the digit, while a 2× thumbnail constrains every stroke a little. The measurement count in the table is the dimension of the range; the bits/dim gap is its value. Note also that the six conditional values span only 0.11 bits/dim, from a measurement that pins the whole image at $\sigma = 0.2$ to one that keeps 64 numbers of 1024: MNIST at 32 px has few bits to give, and an eight-view sinogram already takes more than half of what full denoising takes.

- **The bound costs nothing measurable and buys finite samples (claim vi).** `sr_4x_unbounded` — the official coupling head, otherwise identical — reached 1.097 bits/dim against 1.104 with

the bound (a 0.007 difference on one seed; its validation curve sits on top of the bounded one throughout), with the same rejected-batch count (1), the same failure count (1 in 10000), and PSNR and data consistency equal within 0.1 dB and 0.001. But 2 of the 64 test images produced a non-finite posterior sample in one of four draws, where the bounded head produced none in 6×256 draws across the six systems; the sample and mean PSNR of the unbounded run (17.7 and 19.9 dB) are computed on the 62 images that stayed finite. On an untrained model the same head gave 60 of 64 non-finite samples (Methods). So the overflow in the sampling direction is real, it is rare on a trained model, and it is removed by a tanh at zero cost in likelihood. The bound stays on by default; the table keeps both rows.

- **What this says about step 7.** The bridge was not wrong — it is a valid conditional density and it trains — but it makes the measurement expensive to use: the flow has to *expand* stiff range coordinates to import the prior, which is the source of every instability documented in §8.4, and it arrives at a conditional likelihood worse than ignoring the measurement. Giving the coupling networks $A^{+}y$ as an input makes the same information free: the network reads the measurement where it is well determined and generates where it is not, with the standard-normal base, the official coupling and the official objective, and with the `inpaint_random_gauss` ablation's reconstruction quality *without* giving up the data likelihood. The prefix costs 2.5% of the parameters and 33% of the epoch time. The one thing the bridge keeps that this construction lacks is the physics in the base density: a model trained here is tied to its A exactly as step 7's was, and conditioning on $A^{+}y$ alone gives it no mechanism to recognise a different operator. Training across a family of operators (a random view count, a random mask) is the obvious next experiment and needs no new code.

- **Limits of what was measured.** One dataset at 32 px, one seed, one model size, and 64 test images for the sample metrics; the validation curves were still falling at epoch 120 on every system (best epoch 113–120), so none of the numbers is converged. The DC-error metric is the only measurement of range fidelity and it is a single-sample average; sample diversity in the null space is shown in the figures but not quantified.

10 Step 9: An unconditional TarFlow prior on ChestMNIST 64×64

10.1 Methods

- Code: `step9_chestmnist_tarflow.py` (+ `step1_datasets`, `step2_tarflow`, `invlib.bound_log_scale`); config: `configs/step9_chestmnist_tarflow.yml`; outputs: `outputs/step9_chestmnist_tarflow/{data,f`

- **Why.** Steps 7–8 solved inverse problems by training a flow *for* the measurement system. Step 10 does the opposite, after Feng et al. (ICCV 2023, arXiv:2304.11751): one generative model of the images, trained once with no measurement in sight, is used as a principled Bayesian prior, and the posterior for a single measurement is obtained afterwards by variational inference. This step trains that prior. The data are the 64-resolution release of MedMNIST ChestMNIST (78,468 / 11,219 / 22,433 frontal chest radiographs, official splits, the 14 disease labels unused), chosen because radiographs have global structure — mediastinum, rib cage, lung fields — that a 24×24 hole in the middle must be filled consistently with.

- **Model and recipe.** The official `apple/ml-tarflow` model (unmodified, imported from the pinned commit) trained as a likelihood model with the official ImageNet64 recipe: uni-

form dequantization, float32, AdamW$(0.9, 0.95)$ with weight decay 10^{-4}, peak learning rate 10^{-4}, cosine decay to 10^{-6} after a one-epoch warm-up, gradient-norm clipping at 1, batch 64, the official bits/dim (§3). No horizontal flip: a chest radiograph is not left–right symmetric (the heart is on the viewer's right). Width and depth are the paper's 64-resolution setting, 768 channels, 8 blocks of 8 layers (455.2M parameters), and the coupling head is bounded ($8\tanh(x_a/8)$, `scale_bound 8`) because step 10 must push arbitrary latents through the *inverse* of this model for thousands of steps and the bound was free in bits/dim in step 8.

- **The one deliberate deviation: patch size 8, not 2.** The official 64-resolution likelihood config uses patch 2 ($T = 1024$ tokens) on 16 GPUs. On the single 24 GB GPU available here that model does not fit at batch 8; patch 4 ($T = 256$) fits only at reduced width or depth (768-8-8 out of memory at batch 32; 512-8-8 at batch 64; 512-8-4 = 102M parameters, 7.1 min/epoch at 17.9 GB). Patch 8 ($T = 64$ tokens of 64 pixels) keeps the paper's width and depth at 7.0 min/epoch and 19.1 GB peak at batch 64, and it is also what makes step 10 tractable: the sequential inverse costs T cached attention steps per block with an $O(T^2)$ key–value cache, and it has to be differentiated through. The trade-off is coarser autoregression (each token predicts a 8×8 patch given the previous ones, with the intra-patch dependence carried only by the affine coupling), and it is recorded in the config header rather than hidden. The planned budget is a 40-epoch cosine schedule (49,040 steps, ≈ 5 hours) with a validation early-stop: training halts once the validation bits/dim has not improved for four consecutive epochs (`early_stop_patience 4`); the learning-rate schedule is *not* re-planned when this happens, so a stopped run is a truncation of the nominal one, and the checkpoint used downstream is the best-validation one in either case. In the interest of honesty: the rule was written into the script during the run, at epoch 21, after the validation bits/dim had risen for three consecutive epochs; the running process was then stopped after the epoch-22 checkpoint and resumed under the rule (resuming re-applies it to the stored validation history, so the same rule started from scratch would stop at the same epoch).

- **What is measured.** Validation bits/dim every epoch and `model_best.pth` at the best epoch; test bits/dim of the final and of the best-validation checkpoint, recorded per image (`test_bpd_per_image_*.npy`) so that the mean, the median, the number of images above 10 bits/dim and the mean without them are all available, together with the worst test image and the maximum $|z|$ after each block on it (the per-image accounting was added after the first final evaluation returned a test mean of 10^{24}; see the analysis); sample grids from fixed noise every two epochs; non-finite and out-of-range statistics of the final samples; and a memorisation check — the L_2 nearest training image of each of 16 final samples, compared with the nearest-training-image distance of 256 held-out validation images. A prior that reproduces training images would make step 10's posterior samples look good for the wrong reason.

- **The falsifiable claims.** (i) Validation bits/dim decreases monotonically up to fluctuation and the test value of the best checkpoint sits within a few hundredths of it (no over-fitting at this data size). (ii) Samples are finite and in range, and are radiographs — a mediastinum between two lung fields with ribs — not texture. (iii) Samples' nearest-training-image distance is at or above the validation images' own, i.e. the model is not memorising.

10.2 Configuration (verbatim)

```
# Config for step9_chestmnist_tarflow.py -- unconditional TarFlow prior on
# ChestMNIST at 64 x 64.
#
# This is the PRIOR for step 10 (variational posterior fine-tuning), so it is
# a likelihood model: uniform dequantization, fp32, official bits/dim.
#
# Capacity. The TarFlow paper's 64-resolution likelihood model (ImageNet64)
# is [patch 2, 768 channels, 8 blocks, 8 layers/block] trained on 16 A100s in
# fp32. On the single RTX 4090 available here that exact model does not fit
# (fp32 training OOMs at batch 8 with T = 1024 tokens). We keep the paper's
# WIDTH and DEPTH (768 x 8 x 8, 455M parameters) and use patch 8, i.e. T = 64
# tokens of 64 pixels each: 7 min/epoch at batch 64. This is an honest
# trade-off, not a replication: fewer, larger tokens coarsen the autoregressive
# factorisation (TarFlow's own ablations show smaller patches give better
# bpd). Patch 8 also keeps the step-10 sequential reverse pass -- 63 KV-cached
# steps per block, differentiated -- tractable in memory (O(T^2) cache).

seed: 20260903
tarflow_repo: /dev_ws/ml-tarflow
output_root: outputs/step9_chestmnist_tarflow

dataset: medmnist_chestmnist   # 78,468 / 11,219 / 22,433 train/val/test, grayscale
medmnist_size: 64
img_size: 64
channel_size: 1
patch_size: 8                  # T = 64 tokens of dimension 64
noise_type: uniform            # 8-bit uniform dequantization -> bpd is a data likelihood
hflip: false                   # chest radiographs are not left/right symmetric

model:                         # paper's 64-res width and depth
  channels: 768
  blocks: 8
  layers_per_block: 8

# Soft log-scale bound (invlib.bound_log_scale, +-8 nats per block). Step 8
# measured it to be free in bpd and it keeps every SAMPLE finite, which the
# step-10 reverse pass (exp(za) chained over 8 blocks, differentiated) needs.
scale_bound: 8.0

train:
  epochs: 40
  batch: 64
  lr: 1.0e-4                   # official ImageNet64 setting
  weight_decay: 1.0e-4
  grad_clip: 1.0
  eval_every: 1                # val bpd every epoch (11k images, ~20 s)
  early_stop_patience: 4       # stop once val bpd has not improved for 4 epochs
                               # (the cosine schedule stays planned for `epochs`;
                               # the checkpoint used downstream is the best-val one)
  sample_every: 2              # sample grid every N epochs
  sample_nrow: 8
  sample_rows: 8
```

```
  test_subset: 0                # 0 = full test set (22,433 images)

# Memorisation check on the final samples: nearest training image in L2.
nn_check_samples: 16
```

10.3 Results

ChestMNIST 64×64, 78,468 training images; TarFlow [patch 8, 768 ch, 8 blocks, 8 layers] = 455.2M parameters, 22 of a nominal 40-epoch cosine schedule (stopped by the validation rule: no improvement for 4 epochs), 167 min. Best val bpd 3.0244 (epoch 18); test bpd 3.0251 (best-val checkpoint; median 3.0501) / 3.0448 (final) over the 22,432 test images below 10 bits/dim; the plain test mean is 3.6×10^{24} / 1.9×10^{21} because 1 test image (index 14583) makes the flow blow up (bpd 8.1×10^{28}; max $|z|$ after each block: 3.18, 51.5, 235, 2.1×10^{3}, 1.2×10^{6}, 1.4×10^{9}, 4.2×10^{12}, 1.2×10^{16}). Final samples: 0/64 non-finite, 0.29% of pixels outside $[-1, 1]$. Nearest-training-image RMS distance: samples 0.187 vs. val images 0.190 ± 0.045 (min 0.078).

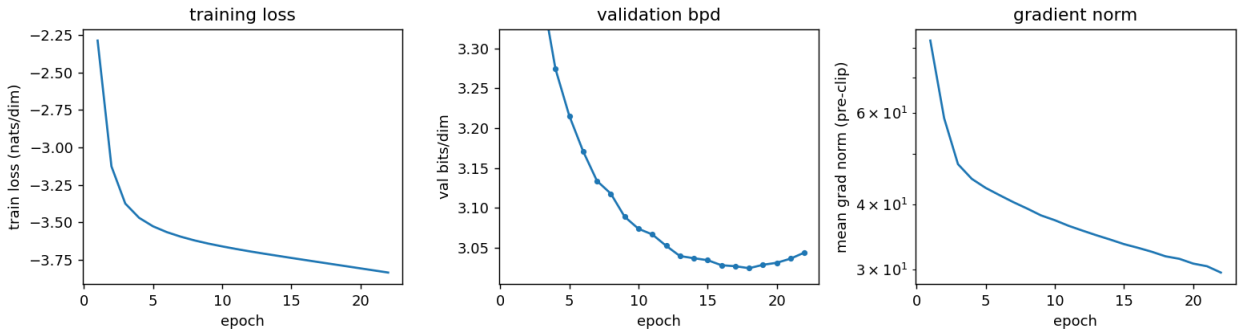


Figure 86: Step 9 training: mean training loss per epoch (nats/dim, the official $\frac{1}{2}\bar{z}^2 - \overline{\log \det}$), validation bits/dim per epoch, and mean pre-clip gradient norm.

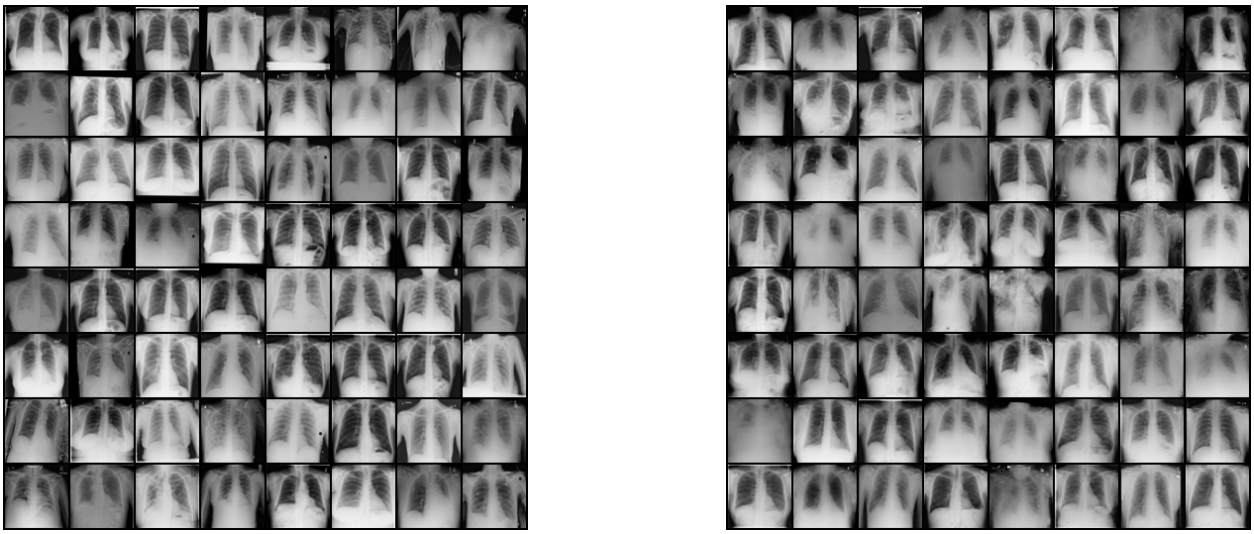


Figure 87: Left: 64 training radiographs (ChestMNIST-64). Right: 64 samples from the best-validation checkpoint, fixed noise, clamped to $[-1, 1]$ for display.

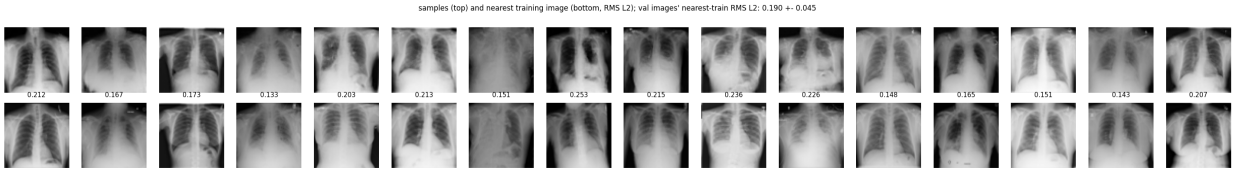


Figure 88: Memorisation check: 16 samples (top) and the L_2-nearest of the 78,468 training images (bottom, RMS distance on the $[-1, 1]$ scale), with the same statistic for held-out validation images in the title.

10.4 Analysis

The claims, in order.

- **(i) is falsified in its second half, and the recipe was changed because of it.** Validation bits/dim fell monotonically from 4.178 (epoch 1) to 3.024 at epoch 18 and then rose for four consecutive epochs (3.029, 3.031, 3.036, 3.044) while the training loss kept falling at an unchanged rate ($-3.779 \to -3.834$ nats/dim) and the learning rate was still at $0.6 \to 0.45$ of its peak. That is over-fitting, at 78k training images and 455M parameters, and it is why the early-stop rule now in the config exists; the run was truncated at epoch 22 of 40 and the best-validation checkpoint (epoch 18) is the prior for step 10. The first half of (i) holds where it can be checked: on the 22,432 test images that do not blow up, the best checkpoint's mean is 3.0251 bits/dim against 3.0244 on validation (median 3.050 vs. 3.048), and the final checkpoint is 0.020 bits/dim worse on test, in line with its validation deficit.

- **A single test image breaks the likelihood, and the mean test bits/dim is therefore meaningless as reported by the official recipe.** Test image 14583 (Figure 89) has bits/dim 8×10^{28} under the best checkpoint (4×10^{25} under the final one), so the plain test mean is 3.6×10^{24}. The mechanism is visible in the per-block maximum $|z|$: 3.2, 51, 235, 2×10^3, 10^6, 10^9, 4×10^{12}, 10^{16} — from block 4 on it grows by $\approx e^8$ per block, which is exactly the scale bound saturating: at one token (raster index 39, the right-edge patch of the fifth patch row, rows 32–39 of the image) every block predicts a tiny scale with high confidence and the residual is amplified by e^8 each time. That patch is the dark right border of the film (values 10–30) with a two-pixel bright speck in it (values 165–214 at rows 38–39, columns 62–63), a marker or a defect — an outlier of a few pixels, at a position where the model, having seen thirty-nine patches of an ordinary radiograph, predicts the border with confidence. The blow-up survives a change of dequantization noise seed (five seeds: 3–13 $\times 10^{28}$), no noise at all, and Gaussian pixel noise of 0.02, but not a horizontal flip (4.1 bits/dim) or a one-pixel shift (4.5): it is a property of that content at that causal position, not of an unlucky noise draw. No training image (78,468; max 4.88) and no validation image (11,219; max 4.70) does this, which is why nothing in the training log warned of it. The `scale_bound 8` that step 8 found "free" is what makes the failure finite instead of `inf`; a smaller bound would cap the damage at the cost of expressiveness and was not retrained here. For the purposes of this document the honest test number is the one over the images below 10 bits/dim, stated as such, with the count (one) alongside.

- **(ii) holds.** All 64 final samples are finite, 0.29% of pixels fall outside $[-1, 1]$ before clamping, and every sample in the grid is a frontal chest radiograph with two lung fields, a mediastinum, a diaphragm and rib texture; the failure modes are the ones the data has (over-exposed, rotated, and cropped studies), not texture or tiling artefacts, and no 8×8 patch grid is visible despite the patch-8 tokenisation.

- **(iii) holds.** The 16 samples' nearest training images are at RMS distance 0.187 (range 0.133–0.253), the validation images' own nearest-training distance is 0.190 ± 0.045 (minimum 0.078), and the nearest neighbours in Figure 88 share pose and exposure with the samples but differ in anatomy. The model is not memorising at 22 epochs, even though it had started to over-fit in likelihood.

Figure 89: Test image 14583 (with the dequantization noise used in the evaluation), the one image of 22,433 on which the step-9 flow blows up: the per-block maximum $|z|$ grows by $\approx e^8$ per block from block 4 at the token covering the right-border patch of the fifth patch row, which contains a two-pixel bright speck.

11 Step 10: Variational posterior by fine-tuning the prior (Feng et al.)

11.1 Methods

- Code: `step10_variational_posterior.py` + `invlib.differentiable_reverse` + `measlib.InpaintBox`; config: `configs/step10_variational_posterior.yml`; outputs: `outputs/step10_variational_posterior`

- **The method being reproduced.** Feng, Smith, Dou, Bouman and Freeman ("Score-Based Diffusion Models as Principled Priors for Inverse Imaging", ICCV 2023, arXiv:2304.11751) take a generative model trained on clean images as the prior $p_\theta(x)$, a known measurement model $p(y|x)$, and fit a normalizing flow q_ϕ to the posterior of *one* measurement y by minimising the reverse Kullback–Leibler divergence,

$$\phi^* = \arg\min_\phi \; \mathbb{E}_{x\sim q_\phi}[-\log p(y|x) - \log p_\theta(x) + \log q_\phi(x)]\,,$$

  estimated with Monte-Carlo batches of samples $x = g_\phi(z)$, $z \sim \mathcal{N}(0, I)$, so that gradients flow through the sampler. The objective equals $\mathrm{KL}(q_\phi \,\|\, p(x|y)) - \log p(y)$: it is a negative evidence lower bound, an upper bound on $-\log p(y)$ that is tight exactly when q_ϕ is the posterior. There are no weights to tune (their earlier "deep probabilistic imaging" needed an entropy weight; here the three terms are the posterior). In the paper the prior is a score-based diffusion model whose $\log p_\theta(x)$ is a probability-flow ODE, q_ϕ is a RealNVP of 64 affine couplings trained from scratch with 64 samples per step, Adam at 2×10^{-4} with gradient-norm clipping at 1, converging in 5k–10k steps; samples with any $|\text{pixel}| > 2$ are discarded as outliers before computing mean, standard deviation, PSNR and SSIM over 128 posterior samples. They also report that an NF prior with this procedure was unstable at 10^{-5} and remark that a "clever initialization might make DPI optimization more stable with an NF prior".

- **The adaptation.** Three substitutions, all of them the natural ones for an exact-likelihood flow prior. (1) p_θ is the step-9 TarFlow, so $-\log p_\theta(x) = -\log \mathcal{N}(f_\theta(x)) - D\,\overline{\log s_\theta}(x)$ is one parallel forward pass, exact, no ODE. (2) q_ϕ has the prior's architecture and is *initialised at the prior's weights*: at step 0, $q_\phi = p_\theta$, the reverse KL is $\mathbb{E}_{p_\theta}[-\log p(y|x)]$, and the fit is a fine-tuning of the unconditional model into the posterior of this one measurement — the "clever initialization" of the remark, made obvious by the two models sharing a class. (3) Samples and their log-density come from one pass: $x = g_\phi(z)$ is TarFlow's sequential inverse,

and since the model is a flow, $\log q_\phi(x) = \log \mathcal{N}(z) - \sum_{\text{blocks}} \sum_{t,d} x_a$ is available from the scales that inverse already computes, so no forward pass through q_ϕ is needed (the paper's footnote notes $\log q$ is only needed during fitting). Because $\log \mathcal{N}(z)$ does not depend on ϕ, the gradient is that of $\|M(x-y)\|^2/2\sigma^2 - \log p_\theta(x) - \sum x_a$; the constant is kept so that the reported value is the negative ELBO.

- **A differentiable TarFlow inverse (`invlib.differentiable_reverse`).** The official `MetaBlock.reverse` writes each recovered token into the input tensor in place (`x[:, i+1] = x[:, i+1]`$\cdot e^{z_a}$ + z_b), which autograd rejects, and it discards z_a. The replacement runs the official `reverse_step` (permutation, positional embedding, the KV-cached causal attention blocks and the shift-by-one are all the official code) but collects the tokens in a list, stacks them, and accumulates $\sum z_a$; the official `Model.reverse` is not called and `ml-tarflow` is not modified. The memory problem is the cache: a block's inverse holds T steps of keys and values for 8 layers, about 122 MB per sample per block in float32, so an 8-block graph at batch 32 would need 31 GB. Each block is therefore wrapped in re-entrant activation checkpointing (`torch.utils.checkpoint`, `use_reentrant=True`): only the block boundaries are stored in the forward pass and each block's cache is rebuilt during the backward pass, one block at a time. The non-re-entrant variant cannot be used: its early-stop exception skips `set_sample_mode(False)` and leaves every attention module in sampling mode with a full cache, which leaked 0.9 GB per sample and corrupted the next parallel forward pass in testing. The block function always restores parallel mode in a `finally`.

- **Verification of the inverse (`scratch/test_diff_reverse.py`, and `verify.json` on the real prior at the start of every run).** On random 4-block models with perturbed couplings, with and without the bound, in float32: (1) x agrees with the official `Model.reverse` to 0.0 (bit-identical — the same kernels in the same order); (2) $\log q(x)$ from the reverse agrees with the forward log-density $\log \mathcal{N}(f(x)) + D\,\overline{\log s}$ to 1.2–2.4 $\times 10^{-4}$ on values of magnitude 1500; (3) gradients with and without checkpointing are identical over all 117 parameter tensors and are finite; (4) no attention module is left in sampling mode; (5) on a two-block 16×16 model, autograd derivatives of a pixel and of $\log q/100$ with respect to four latent entries agree with central differences to 1.8×10^{-5}. Cost for the step-9 model (768-8-8, patch 8): forward + backward through the inverse takes 3.6 s for any batch from 8 to 32 (the sequential loop is launch-bound, 8×63 cached steps of 8 layers), at 4.7 / 5.6 / 7.4 GB peak including the 1.8 GB of weights and 1.8 GB of gradients.

- **The inverse problem.** One radiograph of the ChestMNIST-64 test split, its index drawn once from the seed (`test_index: null`) and recorded, so the image is not chosen by hand. The measurement is `inpaint_box` with a centred 24×24 hole (576 of 4096 pixels unobserved) and Gaussian noise $\sigma = 0.05$ on the observed pixels on the $[-1, 1]$ scale (2.5% of the range; half of step 8's σ on this scale), $y = M(x^* + \sigma\epsilon)$, so $-\log p(y|x) = \|M(x-y)\|^2/2\sigma^2 + \frac{n_{\text{obs}}}{2} \log 2\pi\sigma^2$.

- **Fitting.** Adam$(0.9, 0.99)$ at 10^{-5} (the paper's NF-prior learning rate), gradient-norm clipping at 1, 32 Monte-Carlo samples per step (the paper's 64 does not fit next to the prior's activations; 32 costs the same time as 16 because the inverse is launch-bound), 2000 steps. Steps whose loss or gradient is non-finite are skipped and counted. Every 100 steps 64 posterior samples are drawn from a fixed set of latents (through the official in-place reverse, no gradients) and their mean, standard deviation, PSNR/SSIM against the truth (on $[0, 1]$, whole image and hole only), and the paper's outlier count are logged, so the trajectory of the posterior is visible, not only its end point.

- **Evaluation, after the paper.** 128 posterior samples from fresh latents, the |pixel| > 2 outlier rule applied and counted, mean and standard deviation maps, PSNR and SSIM of the posterior mean, against two references on the same footing: the zero-filled measurement (the pseudo-inverse $A^{+}y$) and the *prior* pushed through the same 128 latents (what q_ϕ was at step 0). Two checks the paper does not report but that a Bayesian posterior must pass: on the observed pixels the posterior standard deviation should be at or below the noise σ, and in the hole the standardised truth $(x^* - \mu)/s$ should look like $\mathcal{N}(0, 1)$ — its mean (a bias of the posterior mean), its RMS and the fractions within ± 1 and ± 2 are reported (0.68 / 0.95 if calibrated), with a histogram and a per-pixel spread-versus-error plot. The mean was added to the report after the run, when the histogram turned out to be shifted; it is recomputed from the saved samples by `--collect`, which also regenerates the calibration figure.

- **The latent random walk.** $z_{t+1} = \cos\alpha\, z_t + \sin\alpha\, \epsilon_t$ with $\alpha = 0.08$ rad per frame keeps every z_t exactly $\mathcal{N}(0, I)$ (so every frame is a posterior sample) with correlation $\cos^k \alpha$ between frames k apart (0.5 at $k \approx 216$, 7 s at 30 fps). 600 frames are pushed through both g_θ (the prior) and g_ϕ (the posterior) and written as `walk.mp4` (H.264, four panels: measurement, prior sample, posterior sample, truth); eight frames are shown as a strip in this document.

- **The falsifiable claims.** (i) The negative ELBO decreases and the data term $-\log p(y|x)$ falls to about the value of a residual at the noise level, $n_{\text{obs}}/2 + \frac{n_{\text{obs}}}{2} \log 2\pi\sigma^2 = 1760 - 7310 = -5550$ nats for $n_{\text{obs}} = 3520$; if it stalls far above, the flow could not reach the posterior. (ii) Posterior samples reproduce the observed pixels to within σ and fill the hole with anatomy that continues the observed ribs and mediastinum, with visible variation between samples. (iii) Posterior-mean PSNR in the hole exceeds both the zero-fill and the prior on the same latents. (iv) The posterior standard deviation is small on observed pixels and larger in the hole. (v) The standardised truth in the hole has RMS near 1; a value $\gg 1$ means an over-confident q_ϕ (the known failure mode of reverse KL, mode seeking), $\ll 1$ an under-confident one. (vi) Few or no outliers by the paper's rule.

11.2 Configuration (verbatim)

```
# step 10 -- variational posterior for ONE inpainting measurement, following
# Feng, Smith, Dou, Bouman & Freeman, "Score-Based Diffusion Models as
# Principled Priors for Inverse Imaging" (ICCV 2023, arXiv 2304.11751), Sec. 3.2:
# a normalizing flow q_phi(x) is fitted to the posterior p(x | y) by minimising
# the reverse KL,  E_{x ~ q_phi}[ -log p(y|x) - log p_theta(x) + log q_phi(x) ],
# with Monte-Carlo batches of samples from q_phi and no tunable weights.
#
# Differences from the paper, all deliberate:
#   * the prior p_theta is the step-9 TarFlow (an exact-likelihood flow), not a
#     score-based diffusion model, so -log p_theta(x) is one forward pass;
#   * q_phi has the SAME TarFlow architecture and is INITIALISED AT the prior
#     (q_phi = p_theta at step 0), i.e. the reconstruction is a fine-tuning of
#     the unconditional model. The paper starts q_phi from scratch (a RealNVP)
#     and remarks that a "clever initialization" might stabilise this with an
#     NF prior;
#   * samples x = g_phi(z) come from the differentiable, memory-checkpointed
#     sequential TarFlow reverse in invlib.differentiable_reverse, which also
#     returns log q_phi(x) exactly (no forward pass through q needed).
seed: 20260910
tarflow_repo: /dev_ws/ml-tarflow
```

```
output_root: outputs/step10_variational_posterior
prior_config: configs/step9_chestmnist_tarflow.yml
prior_checkpoint: best            # model_best.pth of step 9 (or "final")

# the single measurement: a centred 24x24 hole in a 64x64 chest radiograph,
# additive Gaussian noise on the observed pixels (image scale [-1, 1])
measurement: {name: inpaint_box, box: 24, sigma: 0.05}
test_index: null                  # null = one seeded random index of the ChestMNIST test split

fit:
  steps: 2000                     # paper: 5k-10k from scratch; we start at the prior
  batch: 32                       # Monte-Carlo samples of q_phi per step (paper: 64)
  lr: 1.0e-5                      # paper: 2e-4 from scratch; 1e-5 in their NF-prior run
  betas: [0.9, 0.99]
  grad_clip: 1.0                  # paper: 1
  log_every: 10
  eval_every: 100                 # posterior mean/std + PSNR from n_eval fixed-noise samples
  n_eval: 64
  ckpt_every: 50

eval:
  n_samples: 128                  # paper: 128 posterior samples for mean/std/PSNR/SSIM
  outlier_thresh: 2.0             # paper's rule: discard samples with any |pixel| > 2
  n_grid: 32

walk:                             # latent random walk z_{t+1} = cos(a) z_t + sin(a) eps
  n_frames: 600
  fps: 30
  angle: 0.08                     # radians per frame; corr(z_t, z_{t+k}) = cos(a)^k
  batch: 32
```

11.3 Results

Test image #21587 of the ChestMNIST-64 test split, centred 24×24 hole, noise $\sigma = 0.05$ on the $[-1, 1]$ scale (576 of 4096 pixels unobserved). Reverse-KL fine-tuning for 2000 steps of Adam (lr 1e-05, batch 32, clip 1.0), 3.6 s/step, 126 min: the negative ELBO fell from 76,343 (mean of the first 10 steps; $q_\phi = p_\theta$) to -4,807 nats (mean of the last 100); $-\log p(y|x)$ from 75,962 to -5,548 (noise level $\frac{n_{\text{obs}}}{2}(1 + \log 2\pi\sigma^2) = -5,550$, i.e. observed-pixel RMS residual 0.0500 vs. $\sigma = 0.05$); 0 non-finite steps skipped. Over 128 posterior samples (0 outliers by the paper's rule): posterior-mean PSNR 35.48 dB (hole 28.81), SSIM 0.970, versus zero-fill 23.85 dB (hole 15.92), SSIM 0.790, and the prior's mean on the same latents 15.20 dB (hole 19.11); posterior std 0.045 in the hole vs. 0.019 on observed pixels ($\sigma = 0.05$); in the hole the standardised truth has RMS 1.62 (mean -0.68) with 44% within ±1 (68% if calibrated) and 80% within ±2 (95%). Self-test on the prior: differentiable reverse vs. official reverse max $|\Delta x| = 1.2e-06$, $\log q$ from the reverse vs. forward $\log p$ max $|\Delta| = 2.9e-03$ (on $|\log p| \approx 11335$), one objective step 3.6 s at 9.0 GB peak.

Figure 90: The one inverse problem: the test radiograph, the mask, and the measurement y (hole tinted red).

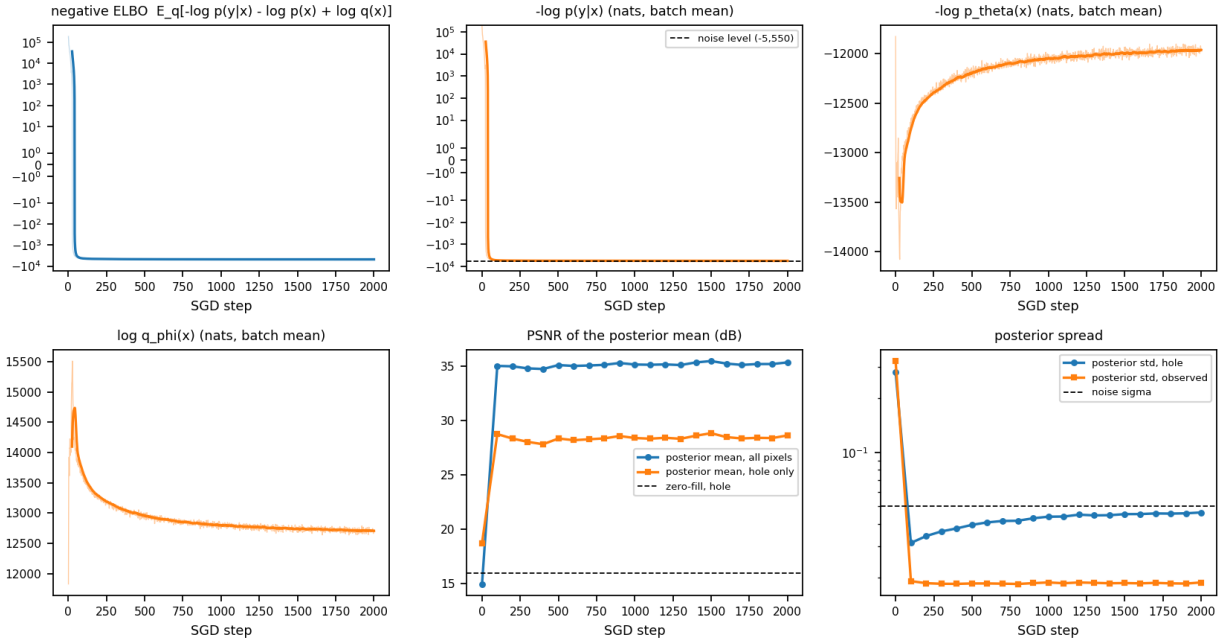


Figure 91: Fitting the posterior flow. Top: negative ELBO per step (light: raw, dark: 25-step running mean) and its first two terms; bottom: $\log q_\phi(x)$, PSNR of the posterior mean of 64 fixed-noise samples (whole image and hole, dashed: zero-fill in the hole), and the posterior standard deviation on hole and observed pixels against the noise σ.

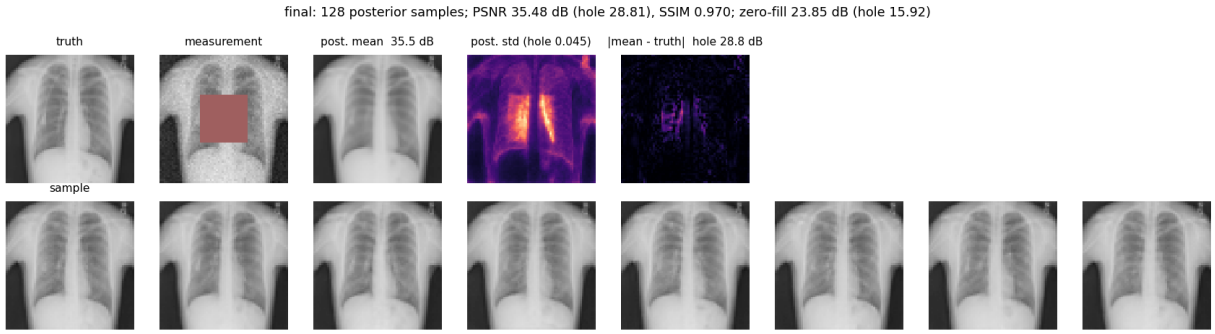


Figure 92: Final posterior (128 samples, outliers removed). Top: truth, measurement, posterior mean, posterior standard deviation, $|\text{mean} - \text{truth}|$; bottom: eight posterior samples $g_\phi(z)$.

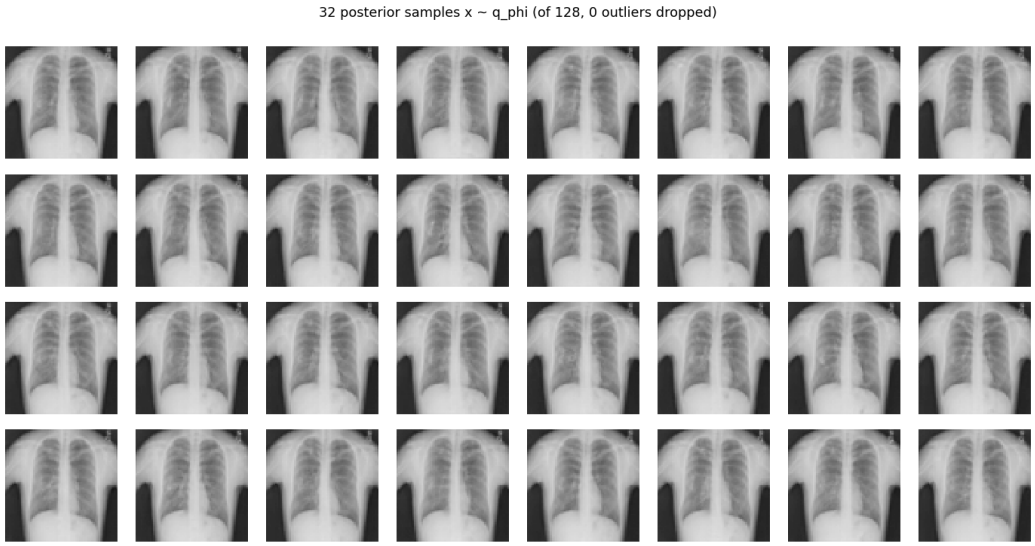


Figure 93: 32 posterior samples.

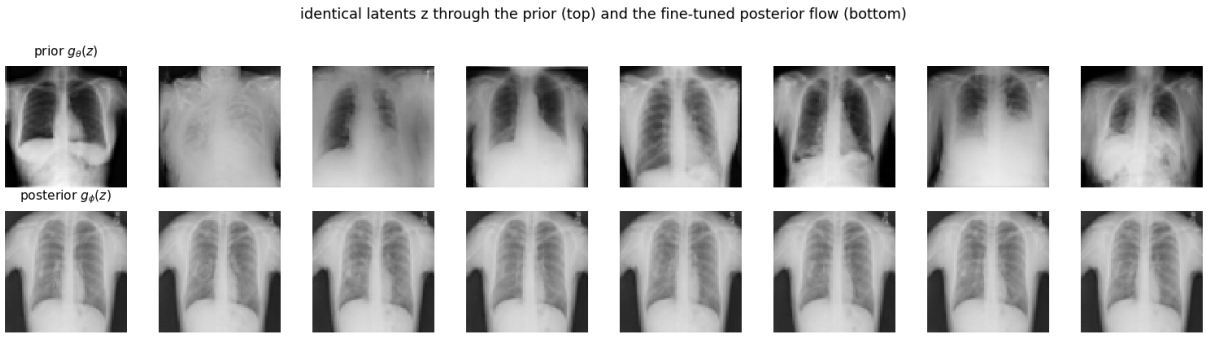


Figure 94: The same eight latents through the prior g_θ (top, what q_ϕ was at step 0) and the fine-tuned posterior flow g_ϕ (bottom).

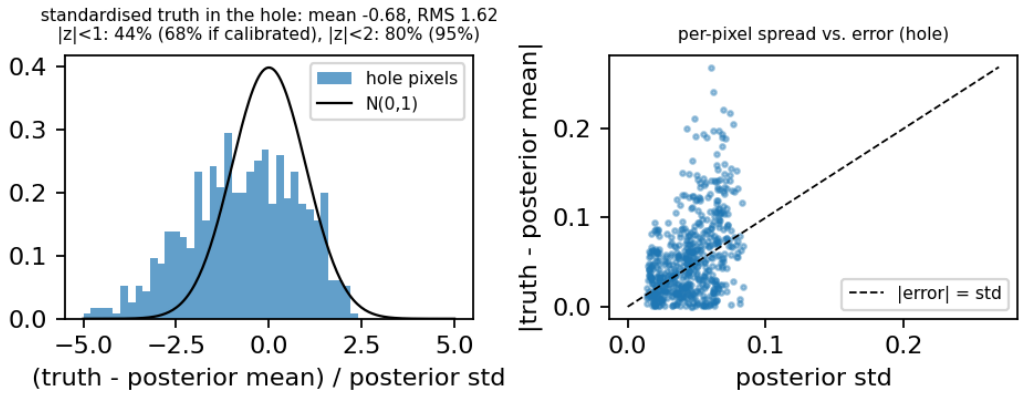


Figure 95: Calibration in the hole: histogram of the standardised truth $(x^* - \mu)/s$ against $\mathcal{N}(0, 1)$, and per-pixel posterior standard deviation against absolute error of the posterior mean.

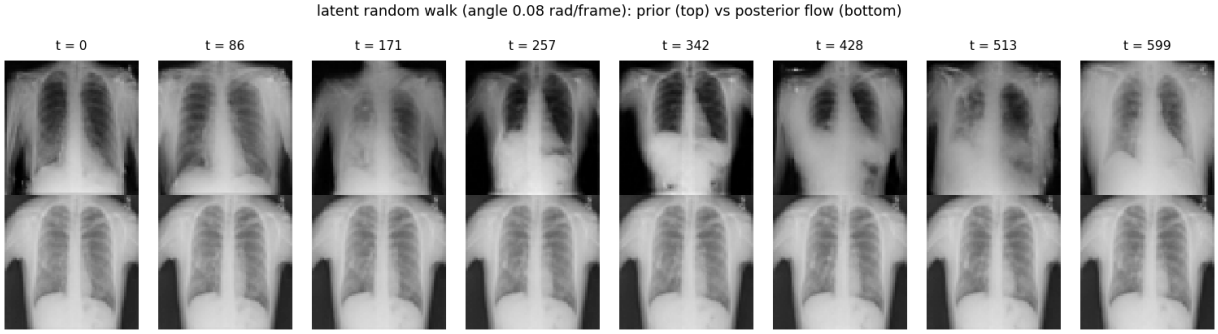


Figure 96: Eight frames of the 600-frame latent random walk (`walk.mp4` in the figures folder): the same z_t through the prior (top) and the posterior flow (bottom).

11.4 Analysis

What was run. Test image 21587 (all 14 ChestMNIST labels negative, i.e. "no finding"; `result.json` records only the first label), a centred 24×24 hole, $\sigma = 0.05$; 2000 steps at 3.6 s each, 126 min on one GPU at 12.4 GB peak; 0 non-finite steps, 0 outliers by the paper's rule in every periodic evaluation, in the 128 final samples, and in the 600 frames of the walk. The self-test on the real prior before fitting reproduced the scratch-test numbers at scale: x from the differentiable inverse agrees with the official reverse to 1.2×10^{-6}, $\log q$ with the forward $\log p$ to 2.9×10^{-3} on values of magnitude 11,335 (a relative 3×10^{-7}), and re-loading the checkpoint changes $\log p$ by exactly 0. Claims (i)–(iv) and (vi) hold; claim (v) does not: the posterior is over-confident in the hole, by a factor of about 1.6, and the fit was still moving in the right direction when the step budget ran out.

(i) The objective. The negative ELBO fell from 76,343 nats (mean of the first ten steps, where $q_\phi \approx p_\theta$; at step 0 the data term alone was 165,217, a prior sample being nowhere near the measurement) to −4,807 (mean of the last 100). The data term reached −5,548 against a noise-level value of −5,550: the residual of a posterior sample from y on the observed pixels has RMS 0.0500 (0.0492–0.0510 over the 128 samples), which is σ to three digits. The other two terms are more interesting than the sum. At initialisation $\log q_\phi = -(-\log p_\theta) = 11{,}733$ exactly. Within the first 30 steps the flow contracted hard: $\log q_\phi$ rose to 14,700 and $-\log p_\theta$ fell to −13,500 (samples *more* typical under the prior than the prior's own samples, and much more concentrated), because the likelihood gradient dominates at the start (gradient norm 4×10^6 against the clip at 1). The remaining 1,950 steps were spent undoing the contraction: $\log q_\phi$ decayed to 12,709 and $-\log p_\theta$ rose to −11,968, while the data term stayed at the floor. The end-point $\mathrm{KL}(q_\phi \,\|\, p_\theta) = \mathbb{E}_q[\log q_\phi - \log p_\theta] = 741$ nats is the information the measurement put into the flow, down from 1,200 at the peak of the contraction; the ELBO was still improving by about 3 nats per 100 steps at step 2000 (window means −4,761, −4,791, −4,804 over steps 501–1000, 1001–1500, 1501–2000), so the fit is not converged — it was stopped by the budget, at 2000 steps against the paper's 5k–10k.

(ii) Observed pixels and the hole. On the observed pixels the samples are not merely consistent with y but closer to the truth than y is: RMS 0.029 from x^* against 0.0495 for the measurement itself, and the posterior mean is 0.021 from x^*, 2.3 times closer than y — the prior denoises the observed pixels — and with a posterior standard deviation of 0.019 there the standardised error has RMS ≈ 1.1: the posterior is calibrated where it is anchored by data. In the hole every sample continues the mediastinum, the heart border and the lung fields across the 24×24 square with no visible seam (posterior-summary and sample figures), and samples differ from one another (RMS difference between independent samples 0.068 in the hole, 0.029 outside), most along the heart–lung border where the standard-deviation map peaks at 0.084.

(iii) Accuracy. Posterior-mean PSNR 35.48 dB (hole 28.81 dB), SSIM 0.970, against 23.85 dB (hole 15.92), SSIM 0.790 for zero-fill and 15.20 dB (hole 19.11), SSIM 0.517 for the prior on the same 128 latents. The prior's hole PSNR beats zero-fill because the average of 128 unconditional chests is a plausible mid-grey chest, but its whole-image PSNR of 15 dB is what an unconditioned model is worth here; the 20 dB gained on the whole image and the 10–13 dB in the hole are the measurement. Hole PSNR was 28.8 dB already at step 100 and stayed in 27.8–28.8 dB for the rest of the fit: the point estimate converged in the first 100 steps, and everything after that was about the spread.

(iv) Spread. Posterior standard deviation 0.019 on observed pixels (below $\sigma = 0.05$, as a posterior on noisy pixels must be) and 0.045 in the hole (90th percentile 0.068, maximum 0.084), against 0.32 and 0.25 for the prior. The ratio is right; the absolute level in the hole is not, which is claim (v).

(v) Calibration: over-confident by 1.6, with a bias. The standardised truth $(x^* - \mu)/s$ over the 576 hole pixels has mean -0.68 and RMS 1.62; 44% of pixels are within ± 1 (68% if calibrated), 80% within ± 2 (95%), 20% beyond ± 2 (5%) and 6.8% beyond ± 3 (0.3%). Two things are wrong, and the histogram separates them. First, a bias: the posterior mean is brighter than the truth over the hole by 0.028 on the $[-1, 1]$ scale (1.4% of the range: truth mean 0.203, posterior 0.231), so the histogram is shifted left, with 42% of pixels below -1 against 14% above $+1$. Second, the spread is too narrow even after removing the bias: $1.62^2 = 0.68^2 + 1.47^2$, so the remaining scatter is 1.5 posterior standard deviations. The error of the posterior mean in the hole has RMS 0.0725 while the posterior standard deviation averages 0.045; the spread-versus-error scatter plot shows the same thing pixel by pixel, with most points above the diagonal. The failure is concentrated where the data constrain least: in the 16×16 interior of the hole the standardised RMS is 1.87 (error RMS 0.084, std 0.048), on the four-pixel rim it is 1.40 (0.062, 0.043) — the error grows toward the centre faster than the flow's uncertainty does. This is the reverse-KL failure mode the methods bullet named: $\mathrm{KL}(q \,\|\, p)$ is mode-seeking and penalises q for putting mass where the posterior has little, not for missing mass the posterior has, so an under-dispersed q is a shallow minimum. Here it is compounded by the dynamics: the flow over-contracted in the first 30 steps (hole std 0.031 at step 100, standardised RMS 2.33) and then re-expanded monotonically — 2.19 at 400, 1.75 at 1000, 1.58–1.69 over 1400–1900, 1.60 at 2000, with the fraction within ± 1 rising from 32% to 43% — under the slow entropy gradient. The end point is where the budget ran out, not where the trend stopped, and the paper's 5k–10k steps (or a larger learning rate once the data term is at the floor) is the obvious next experiment; whether reverse KL alone would reach RMS 1 is not something this run can say. For contrast, the prior on the same latents "passes" the coverage numbers (71% within ± 1, 99% within ± 2, RMS 0.89) with a standard deviation of 0.25 and 19 dB in the hole: coverage is only meaningful next to the accuracy it is bought with, which is why both are reported.

(vi) Stability. No outliers, no non-finite steps, no $|\text{pixel}| > 2$ in any of the $64 \times 21 + 128 + 128 + 600$ samples drawn during the run (21 periodic evaluations, the final posterior and prior sets, the walk). The paper's remark that DPI with an NF prior was unstable at 10^{-5} is not reproduced with this initialisation: starting q_ϕ at p_θ means the first samples are already valid images and the data term, though huge, is finite and smooth, and the gradient clip turns its 4×10^6 norm into a unit step. Whether a from-scratch RealNVP with this exact-likelihood prior would have been unstable was not tested.

The animation. `walk.mp4` shows 600 correlated posterior samples, the same z_t through the prior and the posterior flow side by side with the measurement and the truth. The prior panel drifts between patients, postures and exposures; the posterior panel is one patient, and its motion is small by construction — a hole standard deviation of 0.045 on $[-1, 1]$ is about six of 256 grey levels, and 0.019 outside the hole is two — and confined to the heart border and lung texture inside the square. That is what the posterior of a well-posed inpainting problem with 86% of the pixels observed looks like; it is also, per (v), somewhat too still.

What this step does not show. One image, one mask, one noise level, one seed: nothing here is a distribution over problems, and the over-confidence factor and the bias are properties of this fit, not estimates of the method. The comparison with the paper is qualitative (different data, prior and resolution). The fit is not converged.

12 Step 11: MAP estimation with Adam, in latent and in pixel coordinates

12.1 Methods

- Code: `step11_map_adam.py` (re-uses `step10.build_problem`, `neg_log_lik`, `prior_log_prob` and `invlib.differentiable_reverse`); config: `configs/step11_map_adam.yml`; outputs: `outputs/step11_map_adam/{data,figures}/`.

- **Why a point estimate after a posterior.** Step 10 fitted a whole flow to the posterior of one measurement. The simplest reconstruction under the same prior is a maximum-a-posteriori estimate by gradient descent, and it is also the first phase of the methods this document is heading towards: Asim, Daniels, Leong, Ahmed and Hand ("Invertible generative models for inverse problems", ICML 2020) reconstruct with a Glow prior by minimising $\|A\,g_\theta(z) - y\|^2 + \gamma\|z\|^2$ over the latent z, and Consistency Posterior Sampling (Purohit et al., CVPR 2025) starts every chain with 200–800 Adam steps at learning rate 5×10^{-3} on the noise-space objective $-\log p(y|\Phi(z)) + \frac{1}{2}\|z\|^2$ before ten uncorrected Langevin steps. Step 12 will run Markov chains on the same objective; this step establishes what Adam alone finds, from how many distinct places, and how far that is from the truth, the step-10 posterior and the posterior mode in pixel coordinates.

- **Two MAP estimates of the same posterior.** With the step-9 TarFlow prior $p_\theta = g_{\theta\#}\mathcal{N}(0, I)$, $f_\theta = g_\theta^{-1}$, and the step-10 likelihood $p(y|x) = \mathcal{N}(y; Mx, \sigma^2 I)$ on the observed pixels,

$$\text{latent MAP:}\quad z^* = \arg\min_z\ J_z(z) = -\log p(y|g_\theta(z)) - \log\mathcal{N}(z; 0, I), \qquad \hat{x}_z = g_\theta(z^*);$$
$$\text{image MAP:}\quad x^* = \arg\min_x\ J_x(x) = -\log p(y|x) - \log p_\theta(x).$$

  J_z is the negative log posterior density in latent coordinates (the posterior of z given y has density $\propto \mathcal{N}(z)\,p(y|g_\theta(z))$, the Jacobian cancelling exactly because $p_\theta(g_\theta(z))\,|\det \partial g_\theta/\partial z| = \mathcal{N}(z)$); J_x is the negative log posterior density in pixel coordinates. Since $\log p_\theta(x) = \log\mathcal{N}(f_\theta(x)) + \log|\det \partial f_\theta/\partial x|$,

$$J_x(g_\theta(z)) = J_z(z) - \log\left|\det \frac{\partial f_\theta}{\partial x}\right|_{g_\theta(z)},$$

  so the two minimisers coincide only where the flow's log-Jacobian is flat. A MAP estimate is not invariant under reparametrisation; a density is. Both constants ($\frac{n}{2}\log 2\pi$ and $\frac{n_{\text{obs}}}{2}\log 2\pi\sigma^2$) are kept, so J_z and J_x are on the same nats scale and every image can be scored under both. The latent MAP is what a one-step generator without a density (a GAN, a consistency model) allows; the image MAP needs $\log p_\theta$ and exists here because the prior is a flow. Their costs are opposite: one step of J_z is the differentiable sequential inverse of step 10 (8×63 cached token steps, forward and backward), one step of J_x is one parallel forward pass through the prior with per-block activation checkpointing.

- **The protocol.** $R = 32$ restarts for each variant, all in one batch: restart 0 starts at $z = 0$, the mode of the latent prior (so $g_\theta(0)$ is the prior's "most probable" image in latent coordinates); restarts 1–31 at $z \sim \mathcal{N}(0, I)$. The image variant starts at the same images $x = g_\theta(z)$, so the r-th latent and image runs are a pair with a common origin. Adam (lr 5×10^{-3} as in Consistency Posterior Sampling, $\beta = (0.9, 0.999)$) for 1000 steps with a cosine decay of the learning rate to zero, on the *sum* of the R objectives: because the objectives are independent and Adam is per-coordinate, this is R independent optimisations in one launch. Non-finite gradients would be zeroed and counted (none are expected). An image snapshot of every restart is kept every 10 steps for the trajectory animation.

- **Assessment under both objectives.** For any image x one parallel forward pass of the prior gives $z = f_\theta(x)$, the log-Jacobian and $\log p_\theta(x)$, hence $J_z(f_\theta(x))$, $J_x(x)$, their data and prior terms, and $\|z\|$ (whose typical value under the prior is $\sqrt{n} = 64$; a MAP in latent space is pulled towards $\|z\| \ll 64$, far outside the prior's typical set). Per image, PSNR (whole and hole), SSIM, the mean signed error in the hole, the RMS residual on observed pixels, and the maximal |pixel| are recorded. The same assessment is applied to the reference points: the truth, the zero-fill A^+y, the 32 initial prior samples, the mean of the 32 solutions of each variant, and — when step 10's `samples.pt` is present — its 128 posterior samples and their mean. The spread of the solutions over restarts is measured by the per-pixel standard deviation map and the mean pairwise RMS distance in the hole, and compared with the step-10 posterior standard deviation; the paired distance between the latent and the image solution of the same restart measures how far apart the two modes are.

- **Self-tests (`verify.json`, asserted before optimising).** (a) The rebuilt problem is identical (index, y, mask, σ) to step 10's saved `problem.pt`. (b) The two code paths agree: J_z from the differentiable reverse at z equals J_z recomputed from the forward pass at $g_\theta(z)$ (round trip $f_\theta(g_\theta(z)) - z$ reported), and J_x from the checkpointed forward equals the official forward. (c) The directional derivative of each objective along its own gradient, autograd against central finite differences at five step sizes (10^{-1} to 10^{-3}; the best is reported), in float32 — the official TarFlow code casts to float32 inside its norms, so an fp64 copy cannot run. (d) One full step of each variant at batch 32: time, peak memory, finite gradient, and that no block is left in sampling mode.

- **The falsifiable claims.** (i) Both objectives decrease and plateau; the data term of a MAP falls *below* the noise level -5550 nats (a mode fits the observed pixels closer than σ; only the prior stops it from interpolating the noise), so the observed-pixel RMS residual is below 0.05. (ii) In every pair, $J_z(z^*) < J_z(f_\theta(x^*))$ and $J_x(x^*) < J_x(g_\theta(z^*))$: each solution wins under its own objective, and the two solutions differ visibly, i.e. the log-Jacobian is not flat. (iii) Both MAPs sit far below the truth and the step-10 samples in their objective (the mode is not typical): $\|z^*\| \ll 64$ and $-\log p_\theta(x^*) \ll -\log p_\theta(x_{\text{true}})$. (iv) Restarts do not coincide: the objectives are non-convex and the solutions differ in the hole. (v) Ordering of hole PSNR: the mean of the 32 MAPs beats the single best MAP (averaging removes restart-specific detail), and the step-10 posterior mean, the estimate that targets the MMSE, beats both. (vi) The spread of MAP solutions over restarts in the hole is smaller than the step-10 posterior standard deviation, which is itself too small by a factor 1.6 (step 10, claim v): a set of MAPs is not a posterior sample.

12.2 Configuration (verbatim)

```
# step 11 -- MAP estimation with Adam for the step-10 inpainting measurement.
#
# Two point estimates of the same posterior p(x|y) \propto p(y|x) p_theta(x),
# with the step-9 TarFlow prior p_theta = g_theta # N(0, I):
#   latent MAP  z* = argmin_z -log p(y|g_theta(z)) - log N(z)   (Asim et al. 2020;
#               the Adam warm start of Consistency Posterior Sampling, CVPR 2025)
#   image MAP   x* = argmin_x -log p(y|x) - log p_theta(x)      (the pixel-space mode,
#               possible only because the flow's density is available)
# J_x(g(z)) = J_z(z) - log|det df/dx|(g(z)): they agree only where the flow's
# log-Jacobian is flat. Every restart of both variants starts from the same
# prior sample (restart 0 from z = 0, the prior's latent mode).
seed: 20260911
tarflow_repo: /dev_ws/ml-tarflow
output_root: outputs/step11_map_adam
step10_config: configs/step10_variational_posterior.yml   # the measurement, the prior and its checkpoi
step10_output: outputs/step10_variational_posterior        # problem.pt (cross-checked) and samples.pt

map:
  n_restarts: 32                  # one batch; restart 0 is z = 0, the rest z ~ N(0, I)
  steps: 1000                     # CPS warm start: 200-800 Adam steps at lr 5e-3
  lr: 5.0e-3
  lr_schedule: cosine             # lr * (1 + cos(pi k/K)) / 2; 'constant' reproduces the CPS setting
  betas: [0.9, 0.999]
  batch: 32                       # for the no-grad sampling/forward passes
  log_every: 10
  frame_every: 10                 # image snapshots for the trajectory animation

variants: [image, latent]         # image first: ~0.3 s/step; latent: the sequential inverse, ~3.5 s/st

verify:
  fd_steps: [1.0e-1, 3.0e-2, 1.0e-2, 3.0e-3, 1.0e-3]   # central finite differences along the gradient

eval:
  outlier_thresh: 2.0

anim: {fps: 10}
```

12.3 Results

Same measurement as step 10 (test image #21587, 576 hole pixels, $\sigma = 0.05$; identity with the saved step-10 problem verified). 32 restarts (restart 0 at $z = 0$) of 1000 Adam steps (lr 0.005, cosine decay, $\beta = (0.9, 0.999)$): latent MAP 2.94 s/step, 49 min, 5.3 GB peak; image MAP 0.19 s/step, 3.1 min, 2.2 GB peak; 0 non-finite gradients. Final objectives over restarts: J_z 2,274±833 (min 1,013) for the latent MAP vs. J_z 373 at the truth and 302±44 for the step-10 samples; J_x -21,717±120 (min -21,849) for the image MAP vs. -17,195 at the truth and -17,504±96 for the step-10 samples. Cross-evaluated, the latent MAPs have J_x -15,938±1,028 and the image MAPs J_z -443±124; the log-Jacobian $\log|\det \partial f/\partial x|$ is -18,211 at the latent MAPs, -21,273 at the image MAPs and -17,568 at the truth (sign as $-$logdet). Data term: latent MAP -3,357 (observed-pixel RMS residual 0.0746), image MAP -4,845 (0.0592), noise level -5,550 ($\sigma = 0.05$); $\|z\|$ 60.5 at the latent MAPs, 35.7 at the image MAPs, 66.3 at the truth ($\sqrt{n} = 64$). Reconstruction: latent MAP PSNR 30.07±1.15 dB (hole 27.11±1.49, best-J_z restart 29.04, mean of restarts 28.86), SSIM 0.904, hole

bias +0.041; image MAP 27.85±2.07 dB (hole 20.32±2.49, best-J_x restart 23.37, mean of restarts 23.19), SSIM 0.889, hole bias -0.059; zero-fill 23.85 dB (hole 15.92); step-10 posterior mean 35.48 dB (hole 28.81). Spread over restarts (hole std / mean pairwise RMS): latent 0.052 / 0.078, image 0.150 / 0.208; paired latent-vs-image distance in the hole 0.203 RMS; J_z(latent MAP) $< J_z$(image MAP) in 0/32 pairs and J_x(image MAP) $< J_x$(latent MAP) in 32/32. Max |pixel| 0.87 (latent) / 0.77 (image). Self-test: J_z from the differentiable reverse vs. the forward pass max $|\Delta| = 1.6e-02$ (on $|J_z| \approx 165608$), J_x path max $|\Delta| = 0.0e+00$, round trip $f(g(z)) - z$ max 6.6e-05, directional finite differences (fp32, along the gradient) rel. err. 1.0e-05 (latent) / 3.6e-05 (image).

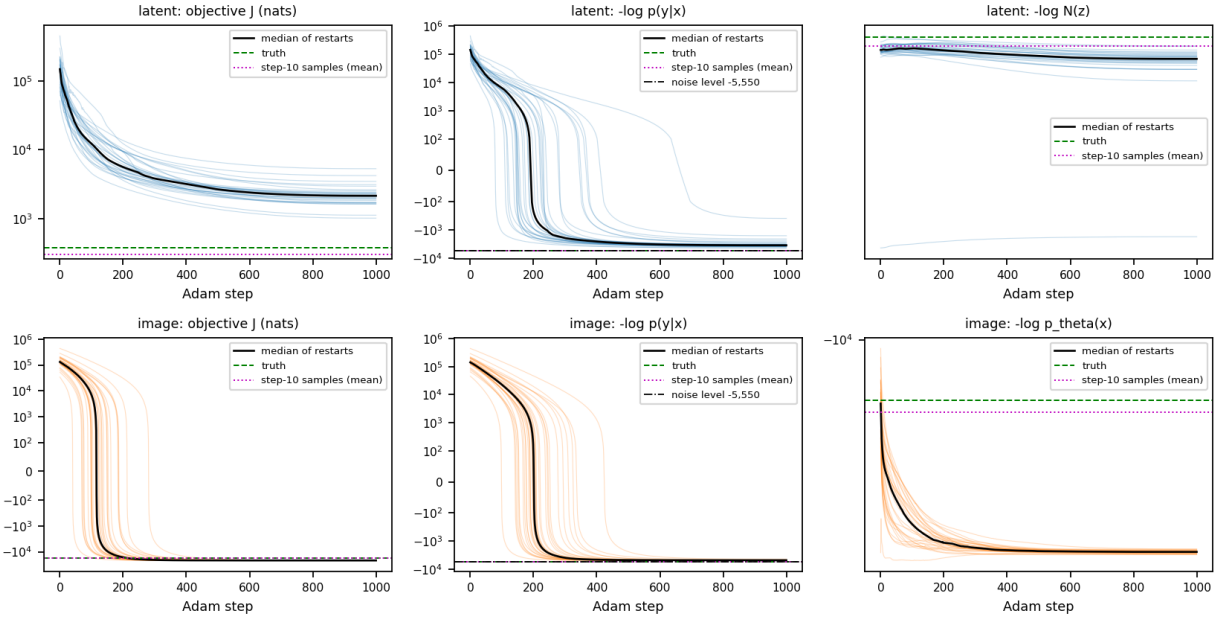


Figure 97: Adam on the two objectives, all 32 restarts (thin lines) and their median (black), symlog scale. Top: latent MAP, J_z and its data and $-\log \mathcal{N}(z)$ terms; bottom: image MAP, J_x with the data and $-\log p_\theta(x)$ terms. Dashed green: the truth under the same objective; dotted magenta: the mean of the step-10 posterior samples; dash-dotted: the noise level of the data term.

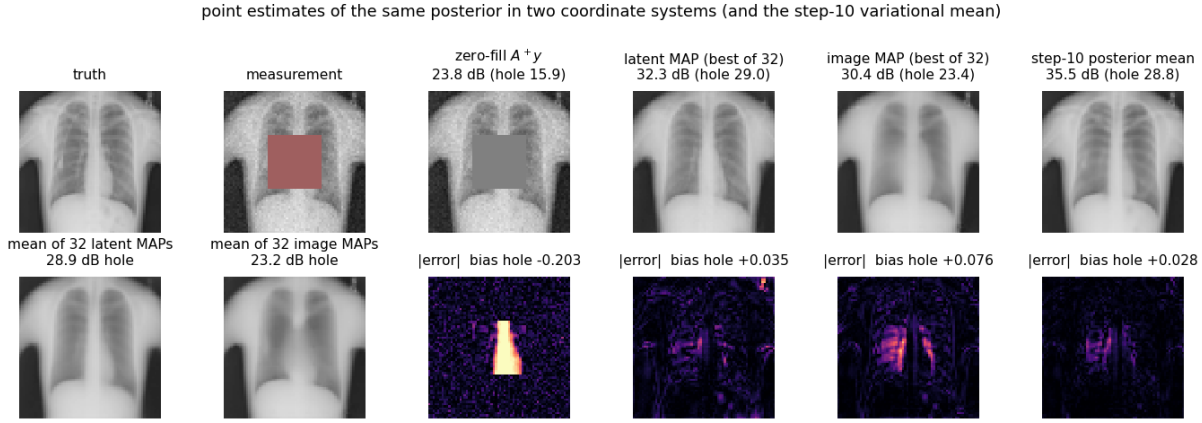


Figure 98: Point estimates. Top: truth, measurement, zero-fill, the latent MAP with the lowest J_z, the image MAP with the lowest J_x, and the step-10 posterior mean, with PSNR (whole image, hole). Bottom: the mean of the 32 latent MAPs, the mean of the 32 image MAPs, and $|\text{estimate} - \text{truth}|$ for the four estimates with the mean signed error in the hole.

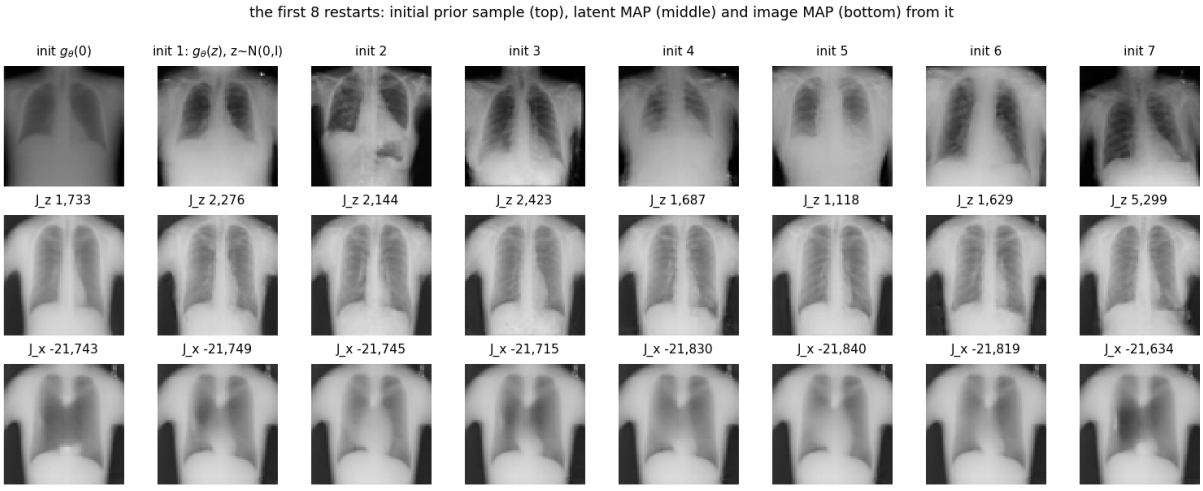


Figure 99: The first eight restarts: the initial prior sample $g_\theta(z_0)$ (top; restart 0 is $z_0 = 0$), the latent MAP reached from it (middle, with its final J_z) and the image MAP reached from the same image (bottom, with its final J_x).

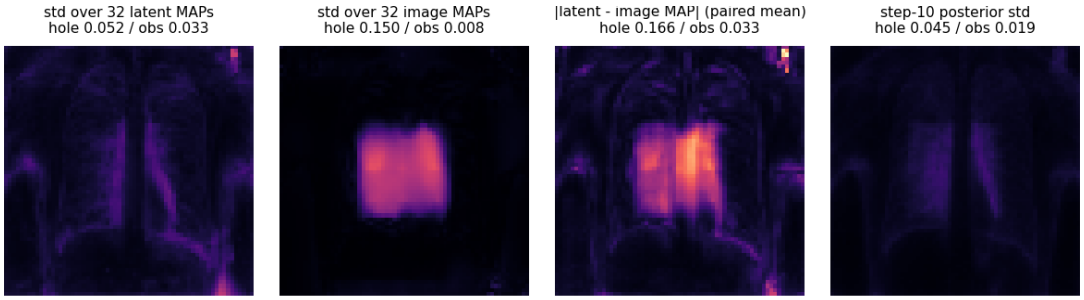


Figure 100: Spread over restarts on a common colour scale: standard deviation of the 32 latent MAPs, of the 32 image MAPs, the mean paired $|\text{latent} - \text{image}|$ difference, and the step-10 posterior standard deviation (mean over hole / observed pixels in the titles).

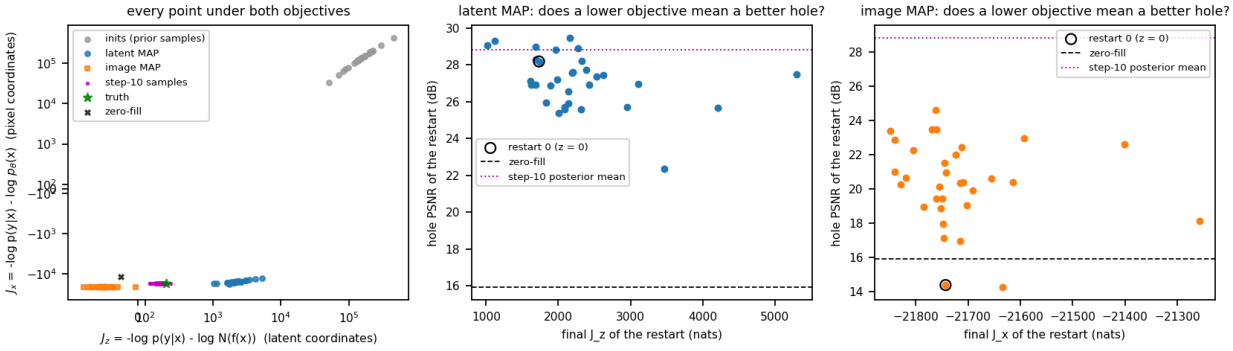


Figure 101: Left: every point scored under both objectives (symlog axes): the 32 initial prior samples, the latent and image MAPs, the 128 step-10 posterior samples, the truth and the zero-fill. Middle and right: final objective of each restart against the hole PSNR of its solution (restart 0, from $z = 0$, circled; dashed: zero-fill; dotted: step-10 posterior mean).

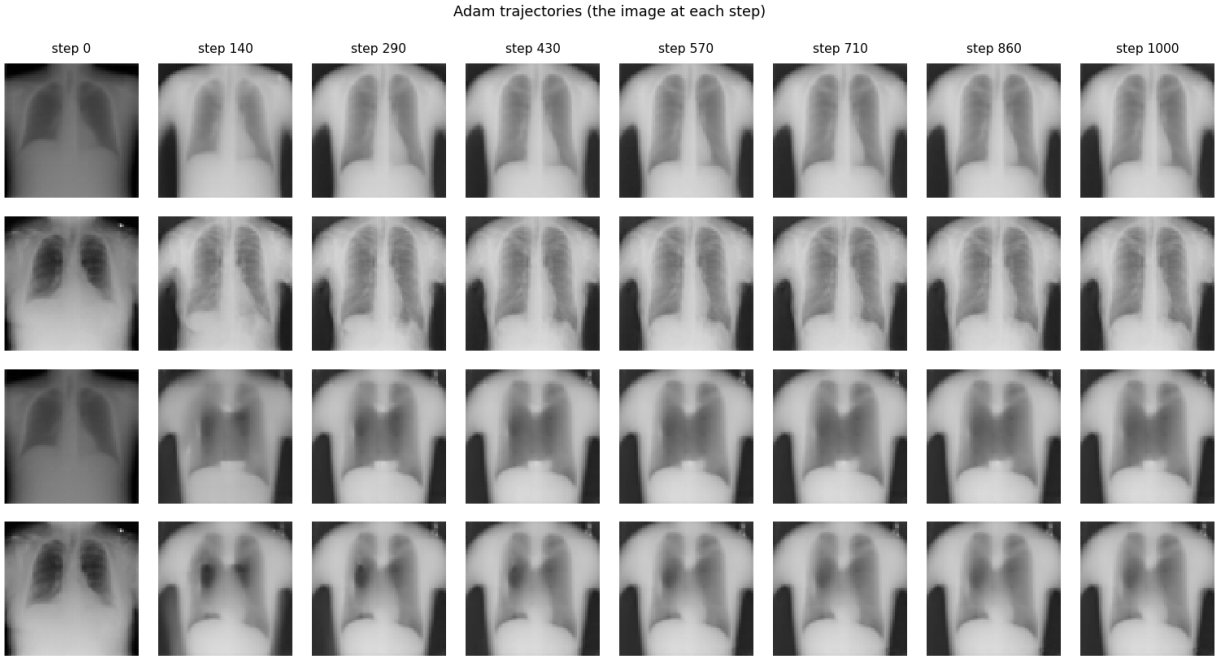


Figure 102: Eight of the 101 frames of `trajectory.mp4`: the image at Adam step k for restarts 0 ($z = 0$) and 1 (random) of the latent (rows 1–2) and the image (rows 3–4) variant.

12.4 Analysis

The claims, in order; the numbers are those of `summary.tex` above.

- **(i) is falsified for both variants, and in the latent case the reason is that 1000 steps at lr 5×10^{-3} do not converge.** Both objectives decrease throughout, but neither data term reaches the noise level -5550: the latent MAPs end at -3357 (observed-pixel RMS residual 0.0746, i.e. *above* $\sigma = 0.05$), the image MAPs at -4845 (0.0592). Both runs flatten as the cosine schedule takes the learning rate to zero (median J_x changes by -0.003 nats per step over the last hundred steps, median J_z by -0.04), and for the image variant the flat value is plausibly the mode; for the latent variant it is not, it is where the schedule ran out: the learning-rate sweep of Figure 103, run afterwards on the same measurement from one random init, reaches $J_z = 3472$ in 100 steps at lr 0.1 (data term -2771) where lr 5×10^{-3} gives 20,115, and the 1000-step runs of the protocol ($J_z = 2274 \pm 833$ over restarts, best 1013) are what lr 0.1 passes after a few hundred steps. No learning rate up to 1 produced a non-finite gradient or objective (the CUDA-graph reverse and the bounded coupling absorb z of norm 225); the failure at lr ≥ 0.5 is a different one — $\|z\|$ drifts to 126 and 225 (typical value 64), $\max|x|$ leaves $[-1, 1]$ (1.45, 7.7) and the hole fills with speckle. The usable range is 0.05–0.2, and 0.1 (monotone descent, 0 increases in 100 steps) is the value the later steps use, with a cosine decay to 5×10^{-3}. Consistency Posterior Sampling's lr 5×10^{-3} was chosen for a different generator; here it is twenty times too small.

- **(ii) is falsified in its first half.** $J_x(x^*) < J_x(g_\theta(z^*))$ in 32 of 32 pairs, as claimed, but $J_z(z^*) < J_z(f_\theta(x^*))$ in *none*: the image MAPs, mapped back to latent coordinates, score $J_z = -443 \pm 124$, far below every latent run's 2274 ± 833 and below the truth's 373. The image MAPs are therefore also better latent-MAP candidates than anything Adam found in latent coordinates, which is (i) again: the latent optimisation is the same objective up to the log-Jacobian, and it is simply further from its minimum. The second half of (ii) holds — the paired latent and image solutions differ by 0.203 RMS in the hole (range 0.10–0.43), the log-Jacobian is $-18{,}211$ at the latent solutions and $-21{,}273$ at the image ones (a difference of 3000 nats, comparable to the data terms themselves), so the two modes are far apart and the reparametrisation matters as much as the likelihood.

- **(iii) holds for the image MAP and is falsified for the latent one, for the same reason.** The image MAPs sit at $J_x = -21{,}717 \pm 120$ against $-17{,}195$ at the truth and $-17{,}504 \pm 96$ for the step-10 samples — 4500 nats more probable than anything typical of the posterior — with $\|z\| = 35.7$ ($\sqrt{n} = 64$): the mode of the density in pixel coordinates is far from the typical set. The latent MAPs have $\|z\| = 60.5$ and J_z *above* the truth's: they have not collapsed towards the origin because they have not converged; the sweep at lr 0.1 shows $\|z\|$ settling at 70 after 100 steps and the restart from $z = 0$ (restart 0) ending at $\|z\| = 13.7$ with $J_z = 1733$, so the eventual latent minimum is at small norm too.

- **(iv) holds.** The solutions in the hole differ by 0.078 RMS (latent, pairwise mean) and 0.208 (image); the image variant's restart 0 — started at $g_\theta(0)$ — lands in a distinct basin with hole PSNR 14.4 dB and a -0.31 bias (a dark hole), the worst of the 32, while its J_x ($-21{,}743$) is within 100 nats of the best. The objective in pixel coordinates has many minima of nearly equal depth with very different content.

- **(v) is falsified in both of its orderings.** Latent: the best-J_z restart has hole PSNR 29.04, the mean of the 32 restarts 28.86, the step-10 posterior mean 28.81 — the averaging removes

nothing useful because the restarts hardly differ (spread 0.052), and the posterior mean, whose *whole-image* PSNR of 35.48 is five dB better than any MAP's, is not better in the hole than a single non-converged latent MAP. Image: best restart 23.37, mean 23.19, both far below the latent variant (the image modes are sharper, biased, and wrong in different ways, so their mean is blurred rather than improved). The whole-image numbers are the cleaner story: the posterior mean fits the observed pixels to their noise (RMS 0.05) and the latent MAP does not (0.075), which is where the five dB are.

- **(vi) is falsified.** The hole standard deviation over restarts is 0.052 (latent) and 0.150 (image) against 0.045 for the step-10 posterior samples: neither set of MAPs is *tighter* than the variational posterior. But the comparison is not the one the claim assumed. The latent spread is optimisation noise (32 runs, none converged, from different starts); the image spread is between distinct modes; the step-10 spread was itself 1.6 times too small (step 10, claim v). Three different quantities happen to be of the same order; none of them is the posterior's.

- **The measurement gallery (Figure 104).** With the sweep's schedule (Adam, lr $0.1 \to 5 \times 10^{-3}$, 300 steps, 0.82 s per step for three restarts in one CUDA-graph batch) the same latent MAP was run on the six measurement systems of `measlib` — each on a different test radiograph, with the exact forward operator rather than the SVD truncation of step 7 (average pooling for super-resolution, the 16×91 Radon sinogram for CT). Every run's data term reaches its noise level within the cosine schedule (denoising 124–152 against 134; box inpainting is the exception, -3487 to -4516 against -5550, with J_z 1272–2606 against 412 at the truth). The pattern that step 14 tests on CT slices at four resolutions is already here: the systems that leave most of the image observed (denoising, random drop, super-resolution) converge to nearly the same image from every start (pairwise RMS 0.026–0.056) and $\|z\|$ collapses well below 64 (14–19 for denoising and 4× super-resolution); the systems with a large null space (the box, sparse-view CT) give visibly different restarts (0.050, 0.081) with J_z spread over 1300 nats, i.e. Adam stops in different local minima of a non-convex objective.

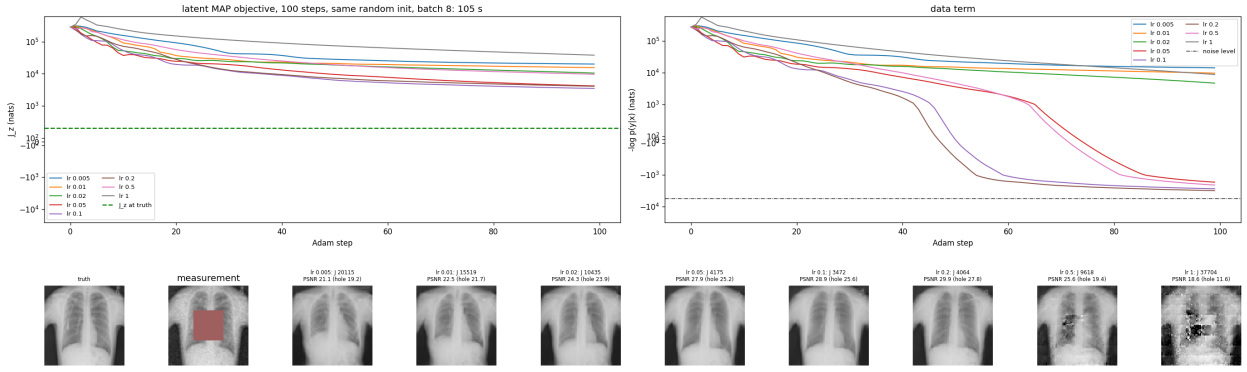


Figure 103: Learning-rate sweep for the latent MAP (`step11_lr_sweep.py`): eight learning rates from one random init on the step-10 measurement, 100 Adam steps each, all in one batch through the CUDA-graph reverse (105 s). Top: J_z (dashed green: the truth) and the data term (dash-dotted: the noise level -5550), symlog scale. Bottom: the reconstruction after 100 steps at each learning rate. lr 0.1 is the fastest monotone descent; at 0.5 and 1 the latent drifts to $\|z\| = 126$ and 225 and the hole fills with speckle, without any non-finite value.

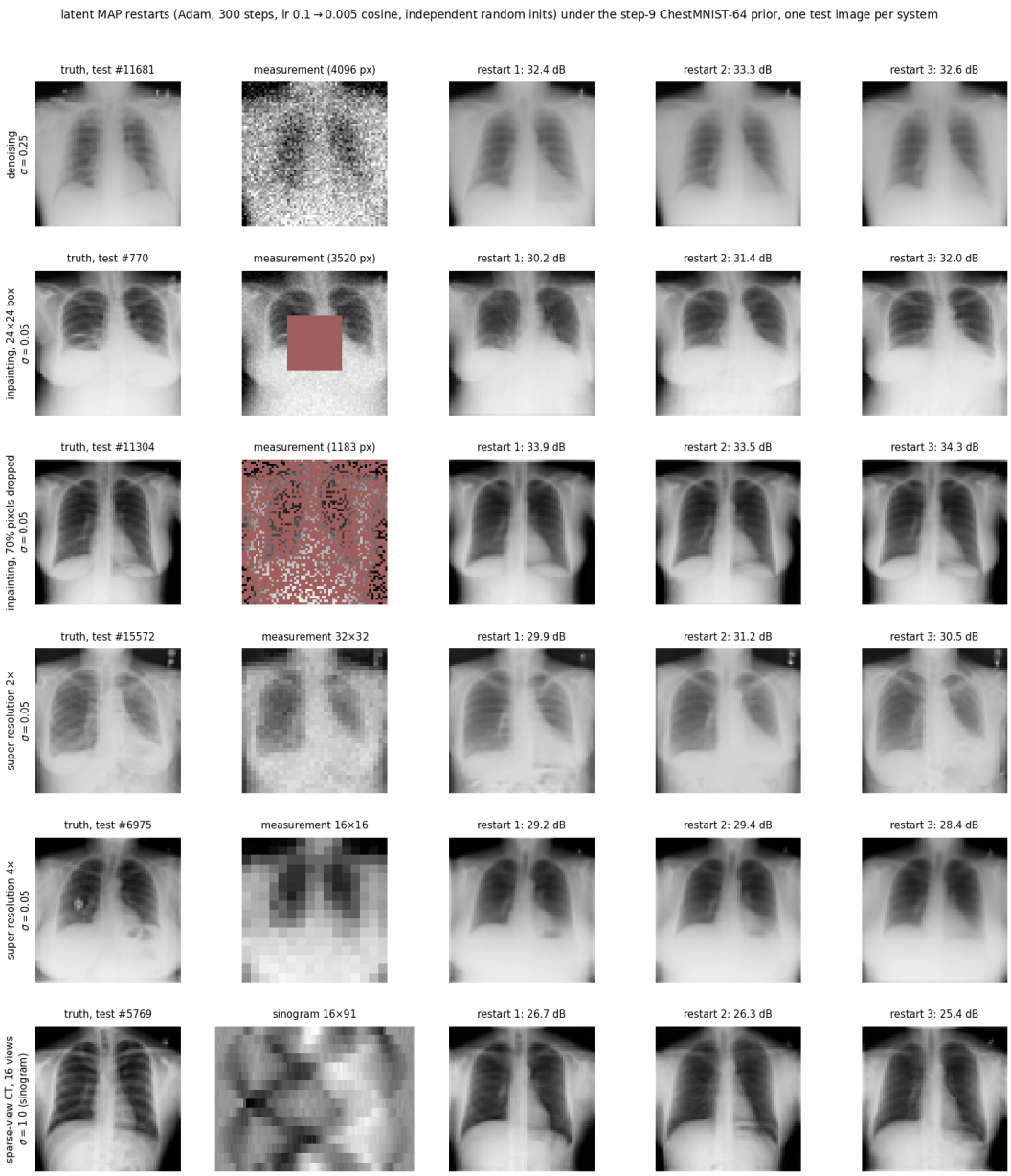


Figure 104: Latent MAP under the step-9 prior on the six measurement systems of `measlib` (`step11_gallery.py`), a different ChestMNIST-64 test image per row: truth, measurement (masked pixels in red; the low-resolution image; the 16×91 sinogram), and three restarts from independent $\mathcal{N}(0, I)$ inits (Adam, lr $0.1 \to 5 \times 10^{-3}$ cosine, 300 steps) with their PSNR.

13 Step 12: LIDC-IDRI CT slices on a common physical grid

13.1 Methods

- Code: `step12_lidc_dataset.py`; config: `configs/step12_lidc_dataset.yml`; outputs: `outputs/step12_li` and the slice caches under `/data/image_benchmarks/lidc/` (gitignored, regenerated by `--scan --build`). The DICOM reading needs `pydicom` and the raw archive, which the development container does not mount by design; the scan and build were run in a throw-away container of the same image with `/data/LIDC` mounted read-only, and everything downstream reads only the derived caches.

- **Why.** ChestMNIST is a convenience: 8-bit radiographs already down-sampled to 64 px with no physical scale, and a prior on them says nothing about CT. Steps 13–14 want a prior on real tomographic slices with a stated field of view and pixel size, so that "sparse-view CT" in step 14 is a measurement of the same physical object at every resolution and a Hounsfield number means what it says. LIDC-IDRI (1018 thoracic CT series from 1010 subjects, TCIA layout on the local `/data` drive, 243,958 DICOM files) is that data; it was acquired on GE, Siemens, Philips and Toshiba scanners with reconstruction fields of view from 236 to 500 mm and pixel spacings from 0.46 to 0.98 mm, so the slices cannot be used as stored — they have to be put on one grid.

- **Header scan and slice selection.** Every file's header (`stop_before_pixels`) is read in a process pool: modality, image position and orientation, pixel spacing, rows/columns, rescale slope and intercept, instance number, thickness, kernel, kVp, reconstruction diameter. A series is kept as axial if its orientation is the identity or its 180° in-plane rotation $(-1, 0, 0, 0, -1, 0)$ within 10^{-3} (14 series are stored rotated and are un-rotated on reading; without that rule they would have been rejected as "no axial slices"); the modal (rows, columns, spacing) geometry of the series is used and files with another geometry are dropped; slices are sorted by z and duplicate positions ($|\Delta z| < 10^{-3}$ mm) removed. The z spacing of a series (median Δz) ranges from 0.45 to 5 mm, and consecutive 0.6-mm slices are near-copies, so slices are selected greedily with a stride of at least 2.5 mm along z (thin series contribute every fourth slice, 2.5-mm series every slice); series with fewer than 20 slices would be rejected (none is). The split is by *subject* (seeded 80/10/10), never by slice, so that no test slice has a neighbour 2.5 mm away in the training set.

- **The common grid.** Pixel values are converted to Hounsfield units with the file's slope and intercept (Toshiba stores HU directly with intercept 0 and -2048 padding; Philips uses -1000; the rest -1024), clipped to $[-1024, 3071]$. Each slice is resampled onto a 384 mm × 384 mm field of view at 256 px (1.5 mm/px): the target pixel centres are placed in the scanner's millimetre frame through the pixel spacing, a Gaussian anti-alias filter of standard deviation $(f-1)/2$ source pixels (f = down-sampling factor, 1.5–3.3) is applied, and the values are read by bilinear interpolation (`scipy.ndimage.map_coordinates`, -1000 HU outside the source image). The grid is centred on the body: one shift per series, the centroid of the voxels above -500 HU of the whole series, so a slice is not re-centred on its own content and the anatomy does not jitter along z (the patient table pulls the centroid slightly downwards; the 384 mm field is wide enough for that). The 256-px HU slices are stored as `uint16` (HU +1024) and, for the models, windowed to 8 bits over $[-1000, 400]$ HU (5.5 HU per grey level: air to the brightest soft tissue and contrast; bone saturates). The 128-, 64- and 32-px versions are 2×2 average pools of the *float* 256-px HU image, then windowed — exact band-limited versions

of one another, not separate resamplings. All caches are memory-mapped `.npy` files (120,054 slices; the whole build takes 2 minutes on 24 processes when the archive is in the page cache).

- **Self-tests (`verify.json`, asserted).** (a) A disc of radius 80 mm at an off-centre position, rendered on source grids of 0.6 and 0.95 mm spacing, resamples to the same 64-px target: maximum difference between the two results 0.0023 HU away from the edge, 0.0035 HU against the analytic disc, area error $\leq 0.2\%$ — the geometry is in millimetres, not in pixels. (b) 2×2 pooling preserves the mean exactly and the 8-bit window round-trips within 2.75 HU (half a grey level). (c) The selection of a synthetic 160-slice 0.625-mm series returns 40 slices with $\min \Delta z = 2.5$ mm and de-duplicates repeated positions. (d) On the built cache, the 8-bit slices equal the windowed `uint16` HU slices within one grey level, and the 64-px cache equals the pooled 256-px HU slices within one grey level.

- **What is shown.** Example 256-px slices in the model window and in a lung window (-1350 to 150 HU, from the HU cache); the same four slices at 256, 128, 64 and 32 px; the first series' source image against its resampled version with the shift applied; and histograms of the field of view, pixel spacing, z spacing, slices per series and HU values with the common grid and the model window marked.

- **The falsifiable claims.** (i) Every one of the 1018 series is usable under the rules above (no rejection, no non-axial series), and the split is by subject with no subject in two splits. (ii) The self-tests pass at the stated tolerances. (iii) The body is inside the 384 mm field in the example slices and the resampled slice is a shifted, smoothed version of the source with the same anatomy (no mirror, no transposition — the heart on the viewer's right in radiological convention, the spine at the bottom).

13.2 Configuration (verbatim)

```
# Config for step12_lidc_dataset.py -- LIDC-IDRI axial CT slices resampled onto
# one common physical grid, cached at 256/128/64/32 pixels, split by subject.
#
# Source: the TCIA LIDC-IDRI download on the axis03 data drive (1,018 CT series
# from 1,010 subjects, 243,958 DICOM slices; scanner grids are 512 x 512 with
# pixel spacings 0.46-0.98 mm, i.e. fields of view of 235-500 mm). The step reads
# the DICOM headers, keeps the axial CT images, sorts them by z, converts to
# Hounsfield units and resamples every selected slice onto a fixed FOV x FOV mm
# window centred on the body, so one pixel means the same physical size in every
# image regardless of the scanner's reconstruction diameter.
#
# The /data/LIDC tree is NOT mounted in the recon-dev container (no-scan-data
# policy); `--scan` and `--build` run in a throw-away container of the same image
# with /data/LIDC read-only, and only the derived caches land under
# /data/image_benchmarks/lidc, which recon-dev sees. The dataset class in this
# file needs only the caches.

seed: 20260904
# TCIA layout: LIDC-IDRI/<subject>/<study>/<series>/*.dcm + metadata.csv
lidc_root: /data/LIDC
cache_root: /data/image_benchmarks/lidc      # where the caches go (visible to recon-dev)
output_root: outputs/step12_lidc_dataset

grid:
```

```
  fov_mm: 384.0            # common field of view; 256 px -> 1.5 mm/px (128: 3 mm, 64: 6
                           # mm, 32: 12 mm)
  base_res: 256            # resolution of the master cache; lower ones are 2x2 average-
                           # pooled from it
  resolutions: [256, 128, 64, 32]
  centre: body             # body = centroid of HU > -500 (per series, one in-plane shift
                           # for the whole scan); image = scanner centre
  pad_hu: -1000.0          # value outside the scanner's reconstruction circle / field of
                           # view

selection:
  z_stride_mm: 2.5         # keep slices at least this far apart in z (thin-slice scans
                           # are subsampled, thick ones kept whole)
  min_slices: 20           # skip series with fewer axial slices than this (localisers,
                           # broken series)
  require_axial: true      # ImageOrientationPatient == (1,0,0,0,1,0) within 1e-3, or its
                           # 180-degree in-plane rotation (-1,0,0,0,-1,0), read un-rotated

intensity:
  hu_clip: [-1024.0, 3071.0]   # stored as uint16 = HU + 1024 (master cache only)
  # the model's 8-bit window: -1000 -> 0, 400 -> 255 (5.5 HU per grey level); lung
  # parenchyma and soft tissue unsaturated, cortical bone saturates at white
  window_hu: [-1000.0, 400.0]

split:                     # BY SUBJECT (all slices of a subject land in one split),
                           # seeded
  val_fraction: 0.10
  test_fraction: 0.10

workers: 24
figures:
  n_examples: 16
```

13.3 Results

LIDC-IDRI: 1018 CT series from 1010 subjects, 243,958 DICOM files; 243,958 axial CT images, 243,945 unique z positions; no series rejected. Reconstruction FOV 236–500 mm (median 358), pixel spacing 0.461–0.977 mm, z spacing 0.45–5.00 mm (median 1.38). Common grid: 384 mm field of view centred on the body centroid, 256 px (1.50 mm/px), lower resolutions by 2×2 average pooling; z stride $\geq$ 2.5 mm gives 120,054 slices (120 per series, 56–160): train 95,947 (808 subjects), val 11,971 (101), test 12,136 (101). Model window -1000..400 HU $\rightarrow$ 8 bits (5.5 HU per grey level). Self-tests passed: a disc resampled from 0.6 and 0.95 mm scanner grids agrees to 0.0023 HU away from its edge (area error 0.18%), 8-bit window round-trip error $\leq$ 2.7 HU, cached 8-bit vs. HU window max difference 1 grey level(s). Cache built 2026-09-04 in 2 min.

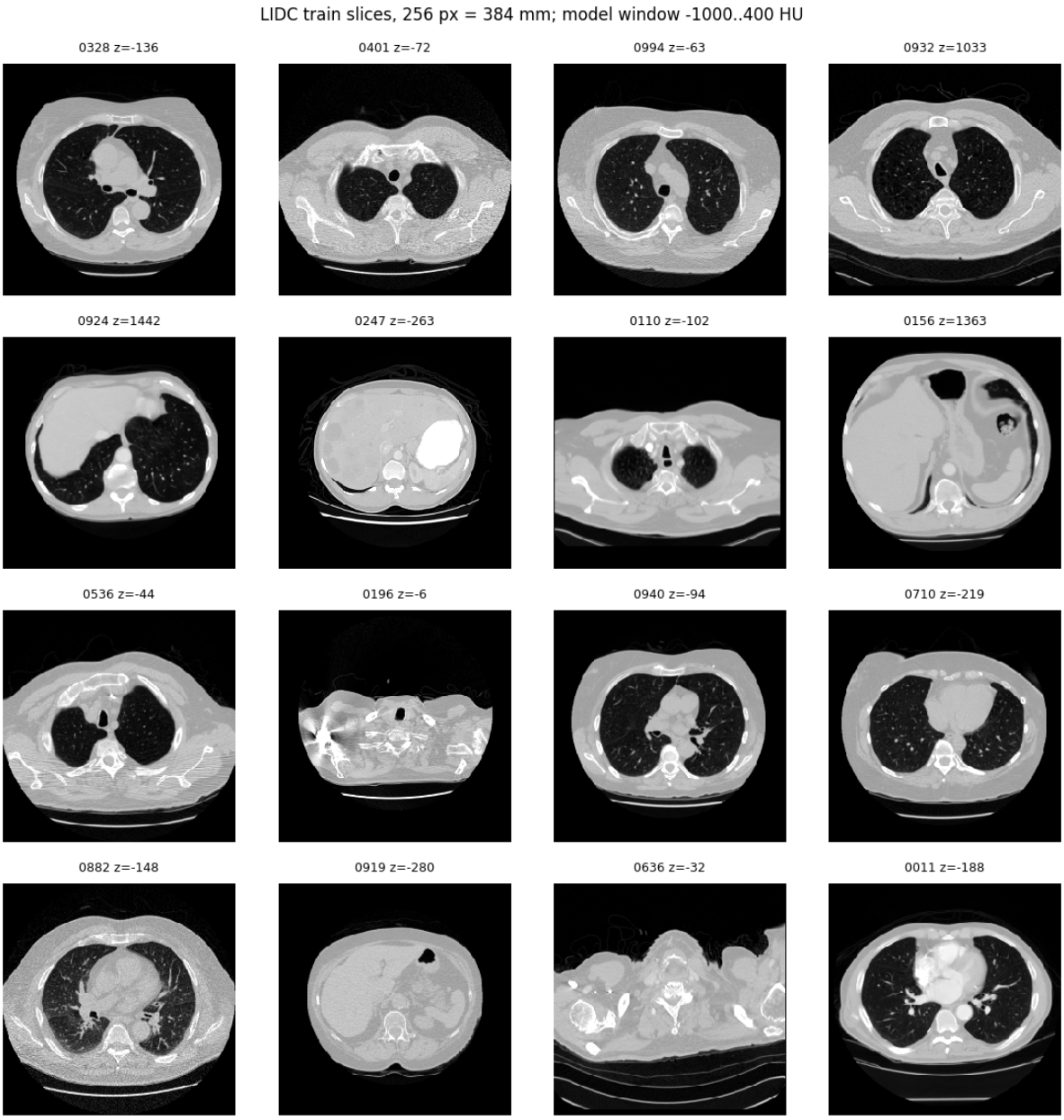


Figure 105: Sixteen training slices on the common grid (384 mm, 256 px) in the model window $[-1000, 400]$ HU.

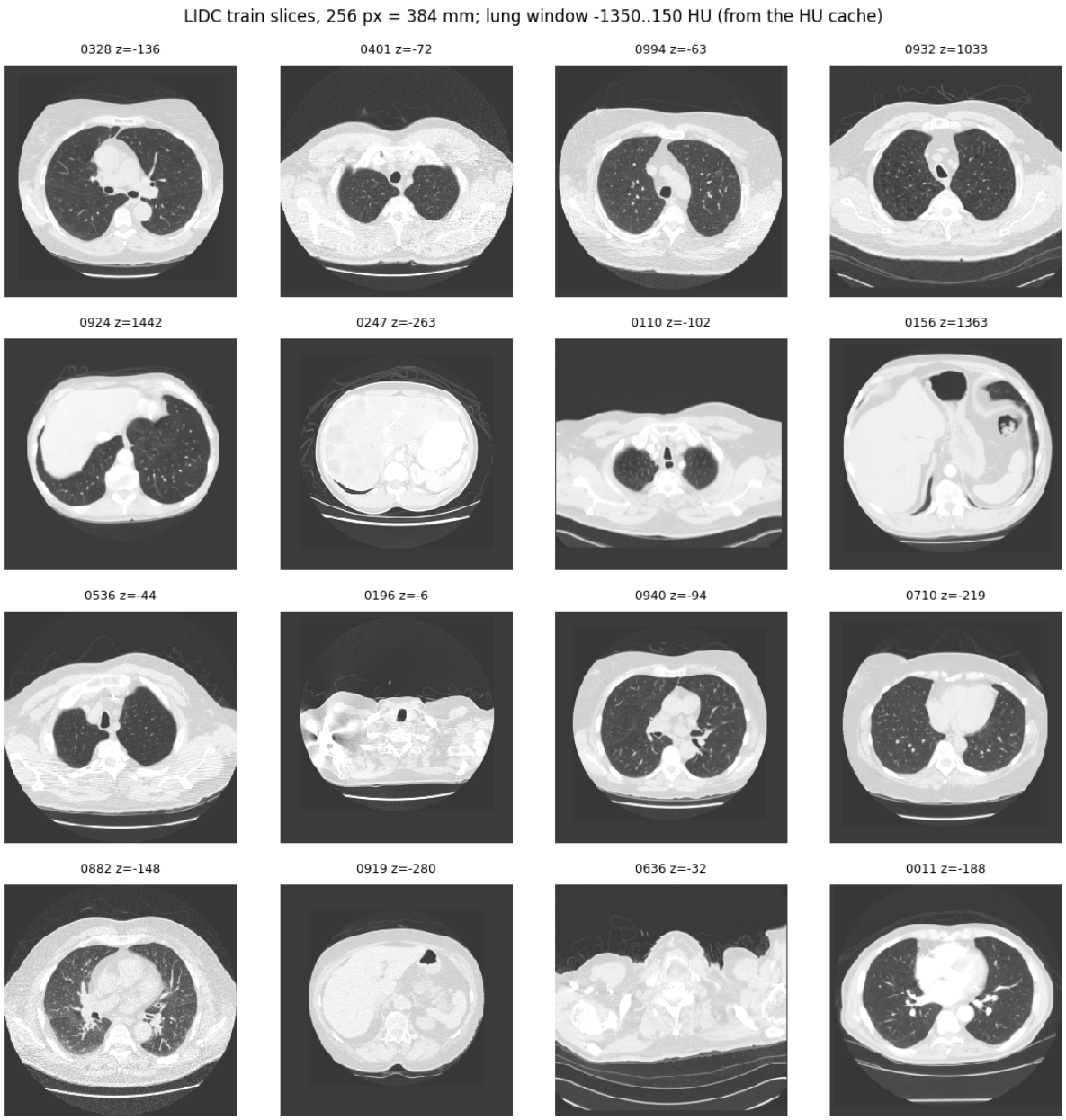


Figure 106: The same slices from the `uint16` HU cache in a lung window (-1350 to 150 HU): the vessels and the parenchymal texture that the 8-bit model window compresses into its darkest 60 grey levels.

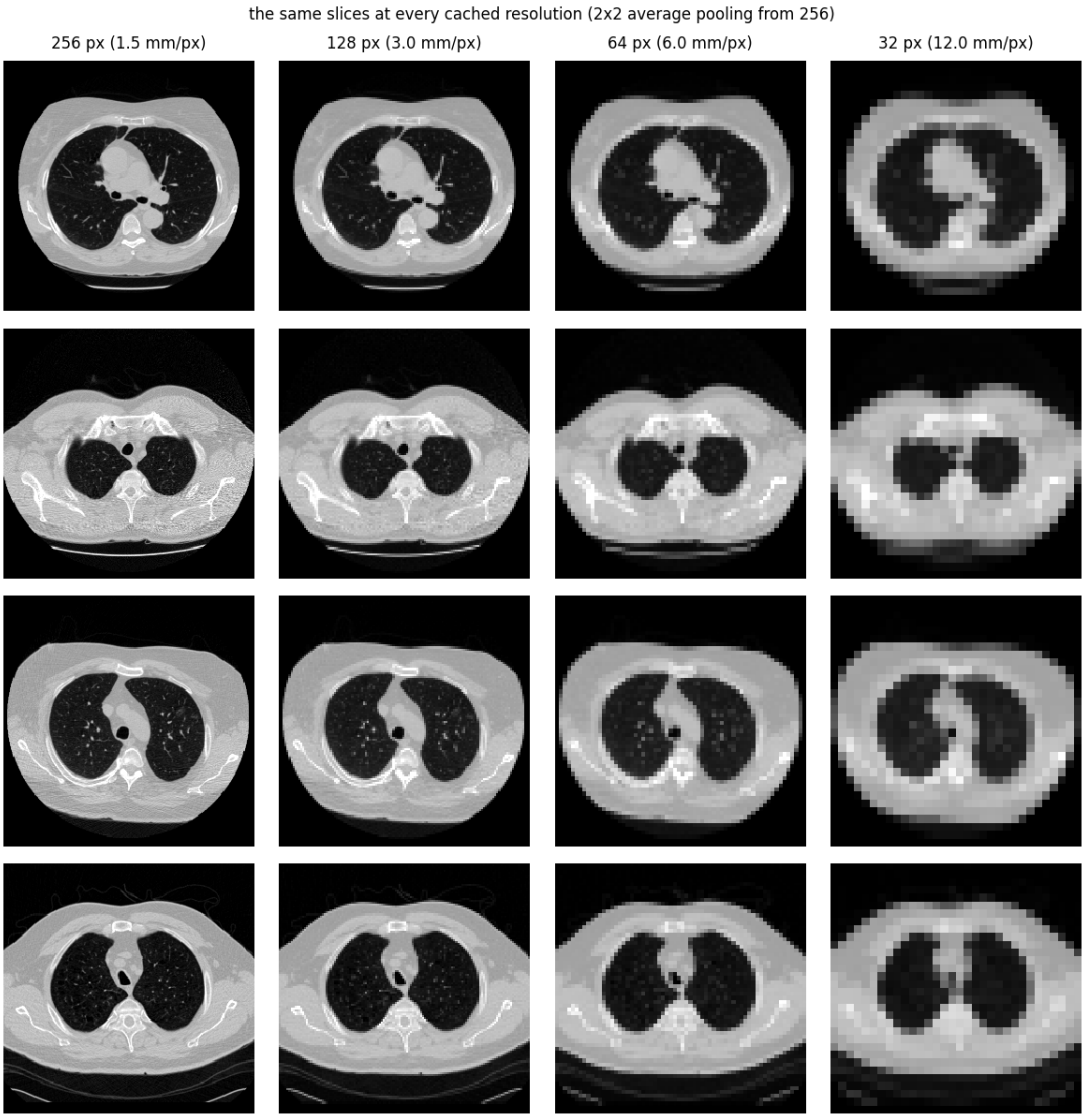


Figure 107: Four slices at the four resolutions of the ladder (1.5, 3, 6 and 12 mm/px), each a 2×2 average pool of the previous.

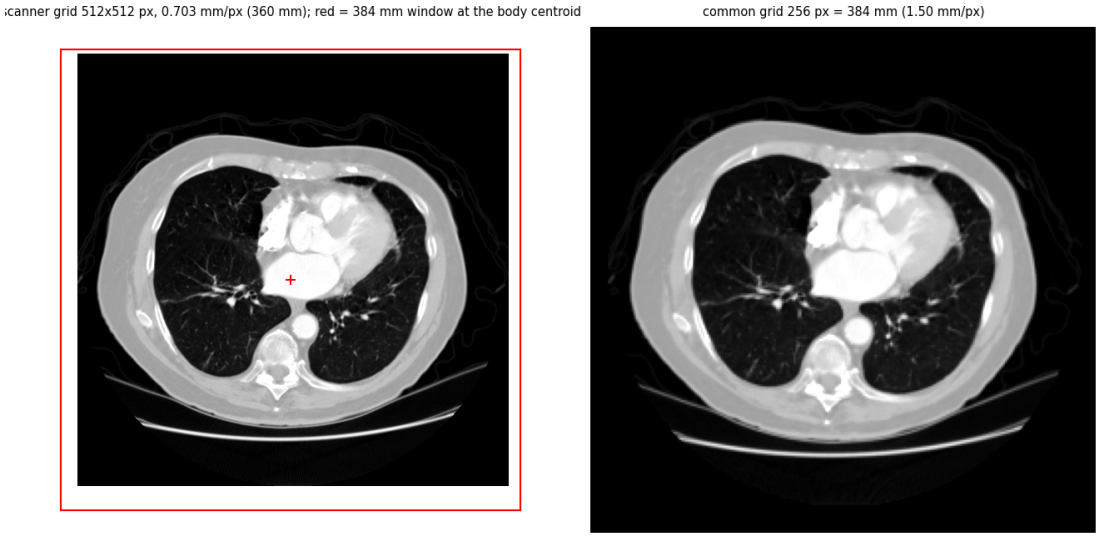


Figure 108: The first series' source slice (scanner grid) and its resampled version on the common grid, with the body-centroid shift applied.

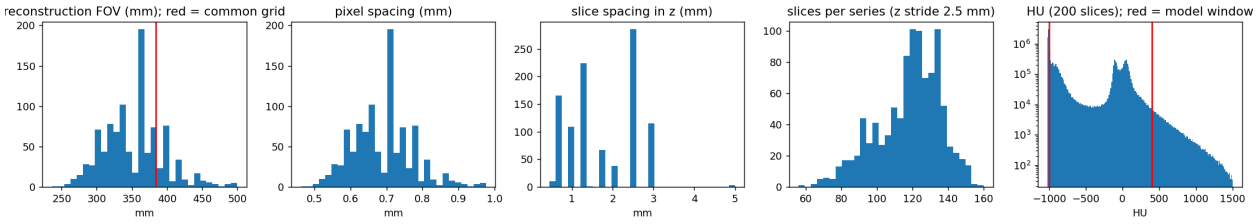


Figure 109: The archive: reconstruction field of view (red: the 384 mm common grid), pixel spacing, z spacing, slices per series after the 2.5 mm stride, and the HU histogram of 200 slices (red: the model window).

13.4 Analysis

- **(i) holds after one fix that is recorded rather than hidden.** The first scan rejected 14 series as having no axial slice; their orientation was $(-1, 0, 0, 0, -1, 0)$, the same plane rotated by 180°, and reading them with a half-turn brings all 1018 series in (243,958 axial images, 243,945 unique z positions: thirteen duplicated positions in the whole archive). No file was excluded for geometry within a series. The subject split is 808/101/101 subjects for 95,947/11,971/12,136 slices.

- **(ii) holds** at the tolerances above; the interesting number is the disc test, 0.002 HU between a 0.6-mm and a 0.95-mm source grid: the anti-alias filter and the millimetre placement are doing what they should, and the 32-px slices (a factor 20–26 below the source spacing) are correctly filtered because they come from the 256-px image by exact pooling, not from a bilinear read of a 0.7-mm image at one sample per 12 mm.

- **(iii) holds** in the figures; the body-centroid centring leaves the table and the lower edge of large patients within the field, with the anatomy in the radiological orientation. The price of the fixed field is resolution: 1.5 mm/px is coarser than every scanner's own spacing (median 0.7 mm), so the 256-px model is a prior on a smoothed CT slice, and the 12 mm/px of the 32-px smoke test shows lungs, mediastinum, spine and table as blobs of a few pixels.

- **What the window does.** Bone and contrast saturate at 400 HU (the histogram's tail to 1500 HU is spine, ribs and calcification); the lungs occupy the lowest 60 of 256 grey levels, so the model window under-resolves the parenchyma the lung window shows. The window is the ChestMNIST-style choice of a single 8-bit image, made so that the step-2/9 pipeline (uniform dequantization, bits per dimension of an 8-bit image) carries over unchanged; a two-window or 12-bit model is a different experiment.

14 Step 13: TarFlow priors on LIDC slices at 32, 64, 128 and 256 px

14.1 Methods

- Code: `step13_lidc_tarflow.py` (re-uses `step9.build_model/load_prior/per_image_bpd/sample` and `step12.build_lidc`); configs: `configs/step13_lidc_tarflow_{32,64,128,256}.yml`; outputs: `outputs/step13_lidc_tarflow_N/{data,figures}/`.

- **Why a ladder.** The whole pipeline (prior, likelihood evaluation, sampling, the inverse and its gradient for step 14) is run first at 32 px, where a training run is twelve minutes and every failure is cheap, and then at 64, 128 and 256 px with the architecture adjusted at each rung. The step-9 recipe is kept (official model, uniform dequantization, AdamW$(0.9, 0.95)$, weight decay 10^{-4}, cosine schedule with a one-epoch warm-up, gradient clipping at 1, bounded scales, early stopping on the validation bits/dim with the best-validation checkpoint kept, official bits/dim on the 8-bit slices; no horizontal flip, because a CT slice is not left–right symmetric); what changes with resolution is the tokenisation and the transformer, stated in each config's header.

- **Tokenisation.** Step 9 chose 8×8 patches ($T = 64$) for the sake of step 10's differentiable inverse and got blobby samples, because within a token the affine coupling transforms the 64 pixels elementwise given the *previous* tokens only, so intra-patch structure is captured only

through the stack of blocks. The TarFlow paper's own recipe is $T = 1024$ tokens at every resolution from 64 up (patch 2 at 64, patch 4 at 128, patch 8 at 256). The ladder here: 32 px with patch 4 ($T = 64$ tokens of 16 grey levels; a 384-channel, 6-block, 4-layer transformer, 42.8M parameters); 64 px with patch 4 ($T = 256$, $D = 16$; 512 channels, 8 blocks of 6 layers, 153M); 128 px with patch 4 ($T = 1024$, $D = 16$, the paper's sequence length, the same 156M-parameter transformer); 256 px with patch 8 ($T = 1024$, $D = 64$, the paper's AFHQ-256 setting, same transformer). The 128- and 256-px configurations were written after seeing the 64-px samples and are reported with whatever they produced.

- **Fitting one 24 GB GPU.** Three devices, all recorded in the config: (a) *epochs of a fixed number of images* (`images_per_epoch`, a fresh random subset every epoch; the 32-px run uses the whole split), so that validation, checkpoints and sample grids come at the same interval in images at every resolution — 96k (32 px), 48k (64) and 24k (128, 256) — and a 30-epoch cosine schedule is 2.9M, 1.4M and 0.7M images; (b) *TF32 matmuls* for the training step only (the 4090 runs them 4–8 times faster than fp32); every bits/dim in this document is evaluated with TF32 off; (c) *per-block activation checkpointing* from 64 px on (`block_checkpoint`): the official forward is re-run block by block outside the model with `torch.utils.checkpoint`, storing one block's activations instead of 48 layers' — the 64-px model at batch 64 runs out of memory without it and takes 5.4 GB with it (0.52 s per step, 6.5 min per 48k images). The measured costs of the candidates (`scratch/probe_arch.py`, TF32, batch as in the configs): 64 px 8.2 ms/image; 128 px at $T = 256$ (patch 8) 8.2 ms with the same net and 21 ms with the paper's 768-channel 8×8 (456M); 128 px at $T = 1024$ (patch 4) 52 ms; 256 px at $T = 1024$ (patch 8) 52 ms with the 156M net and 124 ms with the 461M one. The $T = 1024$ choice therefore costs 42 minutes per 24k images and 21 hours for a 30-epoch schedule at 128 and again at 256 px.

- **What is measured.** As step 9: validation bits/dim every epoch (on a fixed seeded subset of 4000 or 2000 slices from 64 px on; all 11,971 at 32 px), test bits/dim of the final and the best-validation checkpoint on all 12,136 test slices, per image with the outlier accounting of step 9; sample grids from fixed noise with the official sequential reverse (its time recorded; it is $O(T^2)$ per block); non-finite and out-of-range statistics; and the memorisation check against a random 20,000-slice subset of the training split (16 samples, and the validation slices' own nearest-training distance as the reference).

- **Deviation, recorded (64 px).** The first 64-px run was stopped by the early-stop rule at epoch 17 because the plain validation mean had not improved for six epochs; it had not improved because two of the 4000 validation slices blew up to 10^2–10^7 bits/dim under the sharper later checkpoints while the other 3998 kept improving (the test set, with no such slice, preferred the final checkpoint). The selection statistic was changed to the mean over validation slices below 10 bits/dim (`val_stat: robust`, the step-9 outlier accounting), the run was resumed from the epoch-17 checkpoint with the patience counted from the resume, and the first run's artefacts are kept under `first_run_mean_rule/`. Both statistics are recorded per epoch from the resume on, and a `--val-outliers` probe was added that names the failing slices and where in the token sequence the blow-up starts. The 128- and 256-px configs use the robust statistic from the start.

- **The falsifiable claims.** (i) Validation bits/dim decreases and the test value of the best checkpoint is within a few hundredths of it; if the validation curve turns up while the training loss keeps falling the run is over-fitting and the early-stop rule ends it. (ii) Samples are finite,

in range, and are CT slices: a body outline with two lung fields, a mediastinum, a spine and the table. (iii) The samples' nearest-training-image distance is at or above the validation slices' own. (iv) Visual quality improves along the ladder — each rung shows structure the previous one could not (vessels, fissures, bronchi at 128–256 px) — rather than merely larger blobs; this is a judgement on the figures, stated as such.

14.2 Configuration (verbatim)

```
# step13_lidc_tarflow.py -- LIDC 32 x 32 (12 mm/px): the smoke test of the resolution
# ladder. Patch 4 gives T = 64 tokens of dimension 16, the same token count as the
# step-9 ChestMNIST model, with a small transformer (~40M parameters) so the whole
# pipeline (data -> prior -> samples -> bpd -> MAP reconstructions) runs in under an hour.

seed: 20260904
tarflow_repo: /dev_ws/ml-tarflow
output_root: outputs/step13_lidc_tarflow_32
step12_config: configs/step12_lidc_dataset.yml
res: 32
img_size: 32
channel_size: 1
patch_size: 4                   # T = 64 tokens x D = 16
noise_type: uniform             # 8-bit uniform dequantization -> bpd is a data likelihood
hflip: false                    # the heart is on the left: CT slices are not left/right
                                # symmetric
tf32: true                      # TF32 matmuls for training only; bpd numbers are exact
                                # fp32
model:
  channels: 384
  blocks: 6
  layers_per_block: 4
scale_bound: 8.0
train:
  epochs: 30
  images_per_epoch: 0           # 0 = the full 95,947-slice training split per epoch
  batch: 128
  lr: 2.0e-4
  weight_decay: 1.0e-4
  grad_clip: 1.0
  eval_every: 1
  val_subset: 0                 # all 11,971 val slices
  early_stop_patience: 5
  sample_every: 2
  sample_nrow: 8
  sample_rows: 8
  sample_batch: 64
  test_subset: 0                # all 12,136 test slices
  log_every: 200
nn_check_samples: 16
nn_train_subset: 20000
```

```
# step13_lidc_tarflow.py -- LIDC 64 x 64 (6 mm/px). Patch 4 -> T = 256 tokens of
# dimension 16 (four times the token count of the step-9 ChestMNIST-64 model, whose 8 x
# 8 patches left the within-patch structure to the base distribution and gave blobby
# samples); a 512-channel, 8-block, 6-layer transformer (153M parameters). An "epoch" is
```

```
# a fresh random half of the training split so that validation, checkpoints and sample
# grids come every ~48k images; 30 epochs = 1.4M images (the 32-px run started to
# overfit after ~1M). Per-block activation checkpointing: without it the 48 layers at T
# = 256 and batch 64 exceed the 24 GB of the GPU (with it: 5.4 GB, 0.52 s/step, 6.5
# min/epoch).
#
# Deviation, recorded: the first run of this config stopped at epoch 17 under the plain-
# mean rule (best mean val bpd 2.9837 at epoch 11; epochs 13-17 gave 377, 3.46, 50.9,
# 4098, 3606) while the training loss kept falling and the final checkpoint's TEST bpd
# (2.901, no slice above 10 bpd) was better than the epoch-11 checkpoint's (2.961): a
# few validation slices blow up under the sharper later models and the mean of 4000 is
# not a usable selection statistic. The run was resumed from its epoch-17 checkpoint
# with val_stat: robust (mean over the validation slices below 10 bpd) and the same
# 30-epoch schedule; metrics.csv keeps the plain means of epochs 1-17 and both
# statistics from epoch 18 on.

seed: 20260904
tarflow_repo: /dev_ws/ml-tarflow
output_root: outputs/step13_lidc_tarflow_64
step12_config: configs/step12_lidc_dataset.yml
res: 64
img_size: 64
channel_size: 1
patch_size: 4                  # T = 256 tokens x D = 16
noise_type: uniform
hflip: false
tf32: true
model:
  channels: 512
  blocks: 8
  layers_per_block: 6
scale_bound: 8.0
train:
  epochs: 30
  images_per_epoch: 48000      # ~half the 95,947 training slices, resampled every epoch
  batch: 64
  lr: 2.0e-4
  weight_decay: 1.0e-4
  grad_clip: 1.0
  block_checkpoint: true
  eval_every: 1
  val_subset: 4000
  early_stop_patience: 6
  # select/stop on the mean over the validation slices below 10 bpd (added at the
  # resume from epoch 17, see the header)
  val_stat: robust
  sample_every: 2
  sample_nrow: 8
  sample_rows: 8
  sample_batch: 64
  test_subset: 0
  log_every: 200
nn_check_samples: 16
nn_train_subset: 20000
```

```
# step13_lidc_tarflow.py -- LIDC 128 x 128 (3 mm/px). The TarFlow paper's recipe for
# every resolution from 64 up is T = 1024 tokens (ImageNet 64: patch 2, ImageNet 128:
# patch 4, AFHQ 256: patch 8); here patch 4 -> T = 1024 tokens of dimension 16, so each
# token is the same 4 x 4 patch of grey levels as in the 64-px model and only the
# sequence is four times longer. Same 512-channel, 8-block, 6-layer transformer
# (156M parameters); batch 16 with per-block activation checkpointing (5.5 GB,
# 0.83 s/step = 52 ms per image, TF32; batch 32 gives the same 52 ms per image at
# 8.7 GB, so the smaller batch is kept for twice the updates, with the learning rate
# scaled from the 64-px run's 2e-4 at batch 64 by the square root of the batch ratio);
# 24k images per "epoch" (~21 min plus validation and sampling).

seed: 20260904
tarflow_repo: /dev_ws/ml-tarflow
output_root: outputs/step13_lidc_tarflow_128
step12_config: configs/step12_lidc_dataset.yml
res: 128
img_size: 128
channel_size: 1
patch_size: 4                  # T = 1024 tokens x D = 16
noise_type: uniform
hflip: false
tf32: true
model:
  channels: 512
  blocks: 8
  layers_per_block: 6
scale_bound: 8.0
train:
  epochs: 30
  images_per_epoch: 24000      # a quarter of the training split per epoch; 30 epochs =
                               # 720k images
  batch: 16
  lr: 1.0e-4
  weight_decay: 1.0e-4
  grad_clip: 1.0
  block_checkpoint: true
  eval_every: 1
  val_subset: 2000
  early_stop_patience: 6
  # select/stop on the mean over the validation slices below 10 bpd, not the plain mean
  # (one blown-up slice moves the mean of 2000 by more than an epoch of progress)
  val_stat: robust
  sample_every: 3
  sample_nrow: 8
  sample_rows: 4
  sample_batch: 32
  test_subset: 0
  log_every: 500
nn_check_samples: 16
nn_train_subset: 20000

# step13_lidc_tarflow.py -- LIDC 256 x 256 (1.5 mm/px, the native grid of step 12).
# T = 1024 tokens again (the paper's AFHQ-256 setting): patch 8 -> tokens of dimension
# 64, i.e. the sequence length of the 128-px model with four times the grey levels
```

```
# per token. Same 512-channel, 8-block, 6-layer transformer (156M parameters); batch 16
# with per-block activation checkpointing (5.6 GB, 0.84 s/step = 52 ms per image,
# TF32); 24k images per "epoch" (~17 min).

seed: 20260904
tarflow_repo: /dev_ws/ml-tarflow
output_root: outputs/step13_lidc_tarflow_256
step12_config: configs/step12_lidc_dataset.yml
res: 256
img_size: 256
channel_size: 1
patch_size: 8                  # T = 1024 tokens x D = 64
noise_type: uniform
hflip: false
tf32: true
model:
  channels: 512
  blocks: 8
  layers_per_block: 6
scale_bound: 8.0
train:
  epochs: 30
  images_per_epoch: 24000      # a quarter of the training split per epoch; 30 epochs =
                               # 720k images
  batch: 16
  lr: 1.0e-4
  weight_decay: 1.0e-4
  grad_clip: 1.0
  block_checkpoint: true
  eval_every: 1
  val_subset: 2000
  early_stop_patience: 6
  # select/stop on the mean over the validation slices below 10 bpd, not the plain mean
  # (one blown-up slice moves the mean of 2000 by more than an epoch of progress)
  val_stat: robust
  sample_every: 3
  sample_nrow: 8
  sample_rows: 4
  sample_batch: 32
  test_subset: 0
  log_every: 500
nn_check_samples: 16
nn_train_subset: 20000
```

14.3 Results

32 px. LIDC 32×32 (12.0 mm/px), 95,947 training slices; TarFlow [patch 4: $T = 64$ tokens of dimension 16; 384 ch, 6 blocks, 4 layers] = 42.8M parameters, batch 128, lr 0.0002, TF32, 16 of a nominal 30-epoch cosine schedule (full epochs; stopped by the validation rule: no improvement for 5 epochs), 12 min. Best val bpd 3.6593 (epoch 11); test bpd 3.6431 (best-val checkpoint; median 3.5977) / 3.6976 (final). Final samples (64, 0.257 s with the official sequential reverse): 0 non-finite, 4.16% of pixels outside $[-1, 1]$. Nearest-training-image RMS distance (among 20,000 training slices): samples 0.229 vs. val slices 0.229 ± 0.046 (min 0.132).

64 px. LIDC 64×64 (6.0 mm/px), 95,947 training slices; TarFlow [patch 4: $T = 256$ tokens of dimension 16; 512 ch, 8 blocks, 6 layers] = 152.6M parameters, batch 64, lr 0.0002, TF32, per-block activation checkpointing, 26 of a nominal 30-epoch cosine schedule (48,000 random training images per epoch; stopped by the validation rule: no improvement for 6 epochs), 173 min. Best val bpd 2.9136 (epoch 20; mean over the validation images below 10 bpd; on 13 of the 26 evaluated epochs a few validation slices exceeded 10 bpd, the plain mean reaching 5.78e+03 at epoch 19); test bpd 2.8998 (best-val checkpoint; median 2.8957) / 2.9263 (final). Final samples (64, 2.87 s with the official sequential reverse): 0 non-finite, 3.23% of pixels outside $[-1, 1]$. Nearest-training-image RMS distance (among 20,000 training slices): samples 0.276 vs. val slices 0.293 ± 0.057 (min 0.141).

128 px. LIDC 128×128 (3.0 mm/px), 95,947 training slices; TarFlow [patch 4: $T = 1024$ tokens of dimension 16; 512 ch, 8 blocks, 6 layers] = 155.7M parameters, batch 16, lr 0.0001, TF32, per-block activation checkpointing, 30 epochs (24,000 random training images per epoch), 757 min. Best val bpd 2.5819 (epoch 30; mean over the validation images below 10 bpd); test bpd 2.5980 (best-val checkpoint; median 2.6349) / 2.5980 (final) over the 12,135 test images below 10 bits/dim; the plain test mean is 3.2×10^{17} / 3.2×10^{17} because 1 test image (index 1758) makes the flow blow up (bpd 3.9×10^{21}; max $|z|$ after each block: 9.81, 11.1, 147, 6.8×10^{3}, 6.7×10^{5}, 3.1×10^{8}, 6.6×10^{10}, 6.0×10^{12}). Final samples (32, 23.7 s with the official sequential reverse): 0 non-finite, 0.99% of pixels outside $[-1, 1]$. Nearest-training-image RMS distance (among 20,000 training slices): samples 0.354 vs. val slices 0.326 ± 0.057 (min 0.175).

256 px. *not generated yet.*

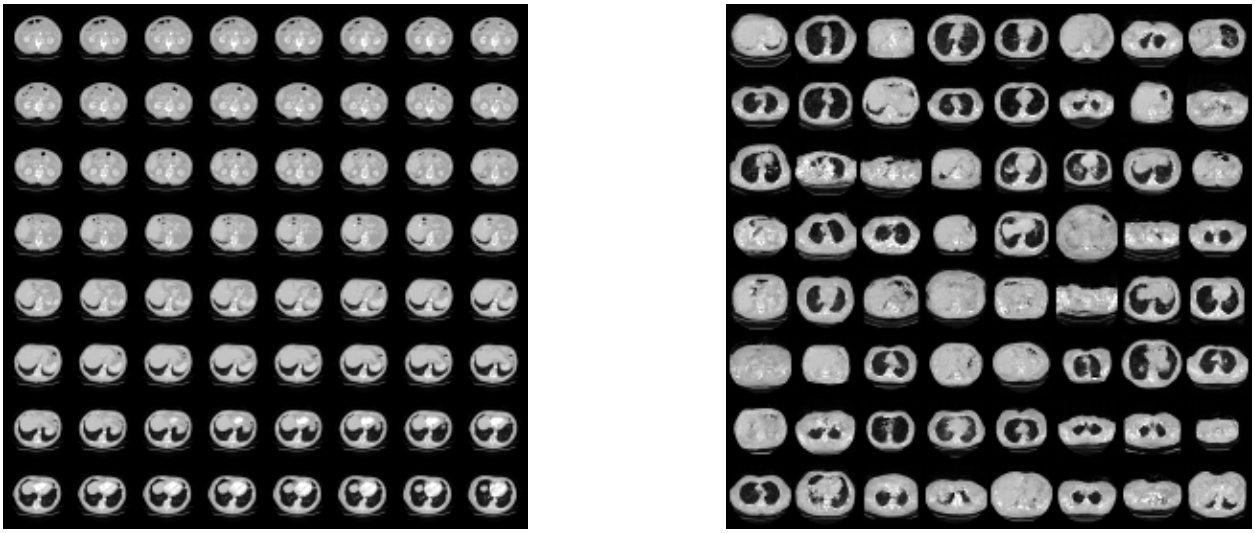


Figure 110: 32 px (12 mm/px). Left: 64 training slices. Right: 64 samples from the best-validation checkpoint, fixed noise, clamped to $[-1, 1]$.

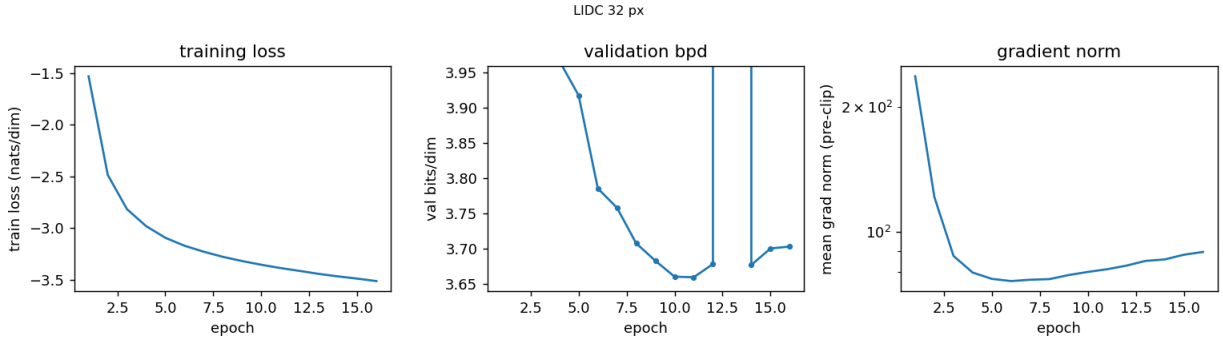


Figure 111: 32 px: training loss (nats/dim), validation bits/dim (the off-scale point at epoch 13 is a transient blow-up on a few validation slices, mean 1.7×10^5) and pre-clip gradient norm.

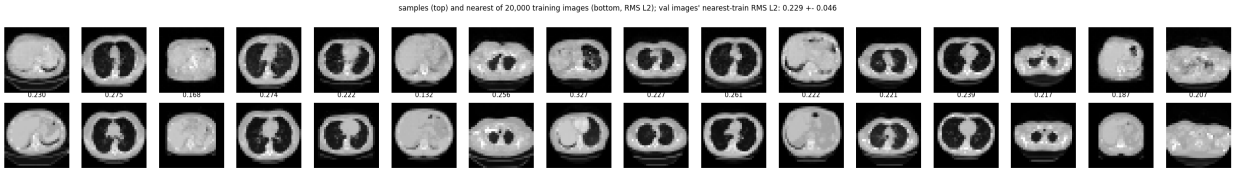


Figure 112: 32 px memorisation check: 16 samples (top) and their L_2-nearest of 20,000 training slices (bottom, RMS distance), with the validation slices' own nearest-training distance in the title.

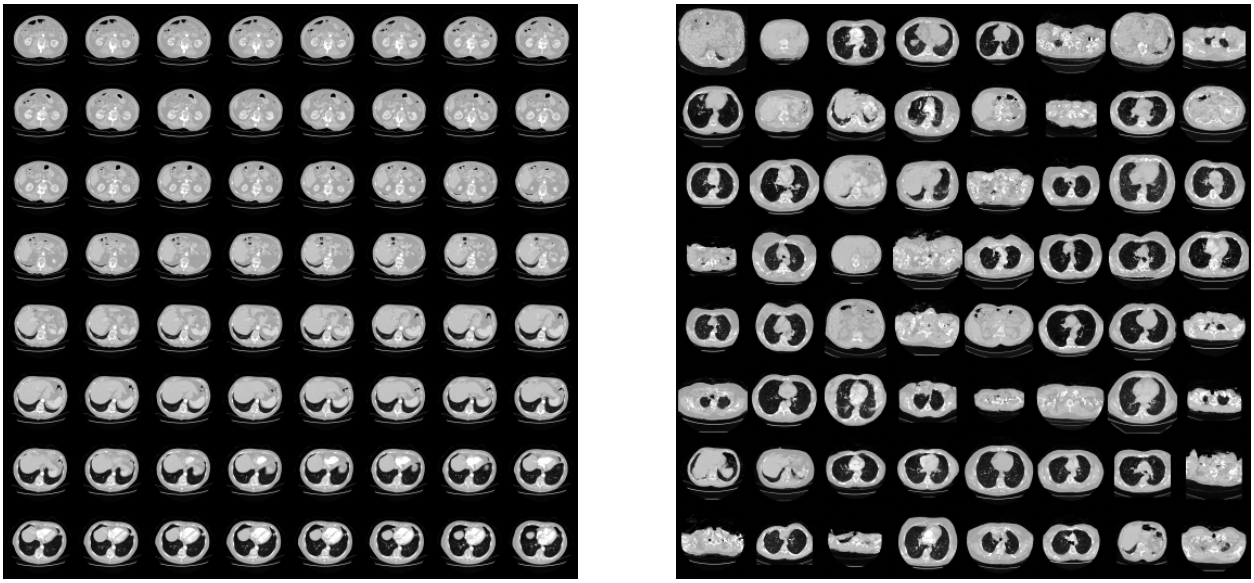


Figure 113: 64 px (6 mm/px). Left: 64 training slices. Right: 64 samples from the best-validation checkpoint.

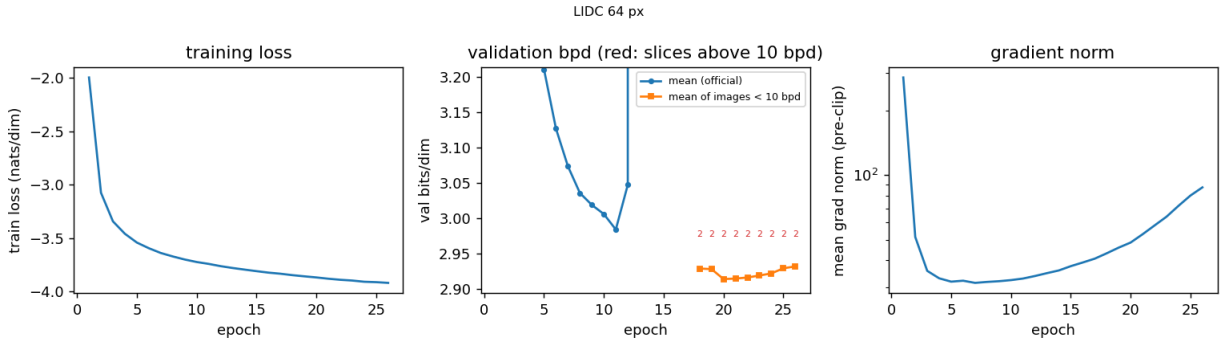


Figure 114: 64 px: training loss, validation bits/dim on the 4000-slice subset (circles: the plain mean, off-scale from epoch 13 on because of two slices; squares, from the epoch-17 resume on: the mean over slices below 10 bits/dim, the selection statistic; red: the number of slices above 10) and gradient norm per 48k-image epoch.

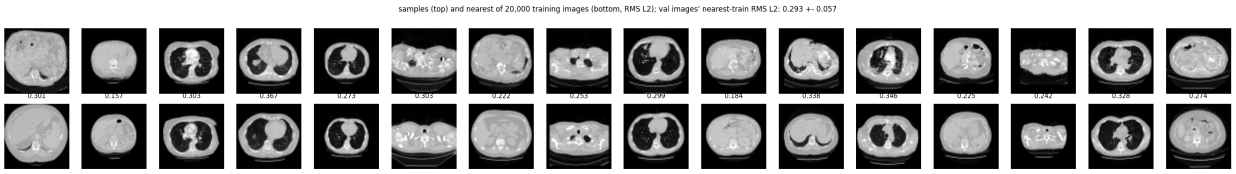


Figure 115: 64 px memorisation check.

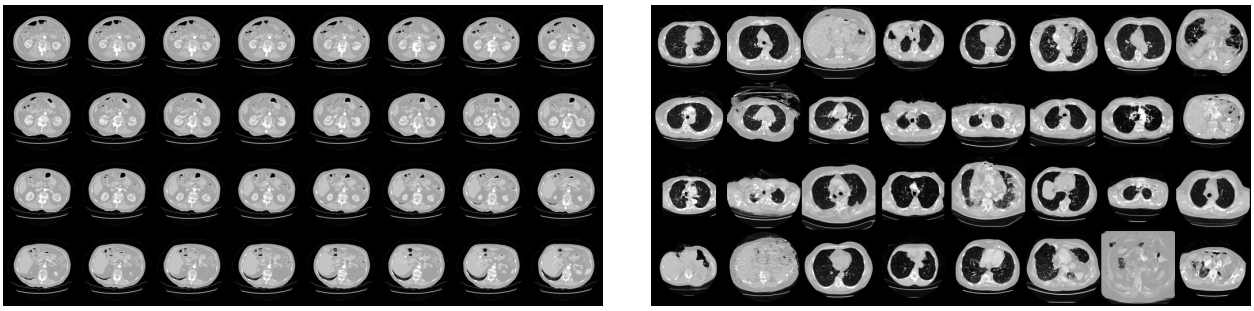


Figure 116: 128 px (3 mm/px). Left: 32 training slices. Right: 32 samples from the best-validation checkpoint.

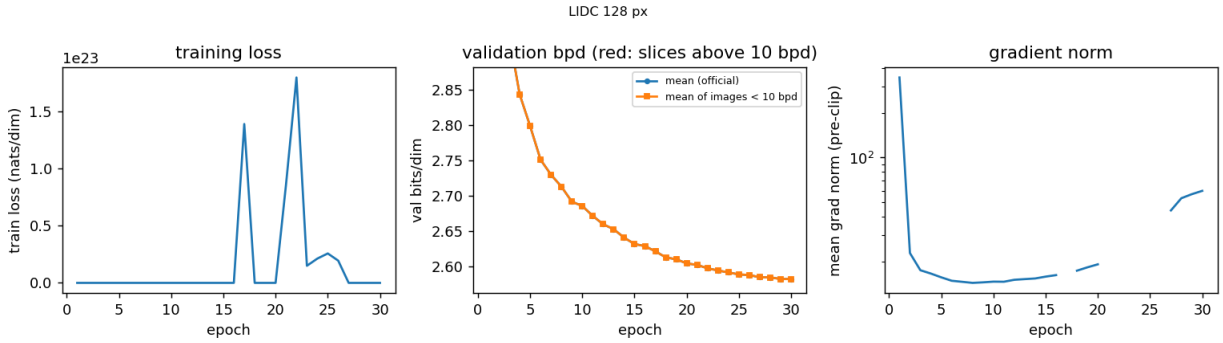


Figure 117: 128 px: training loss, validation bits/dim (2000-slice subset) and gradient norm per 24k-image epoch.

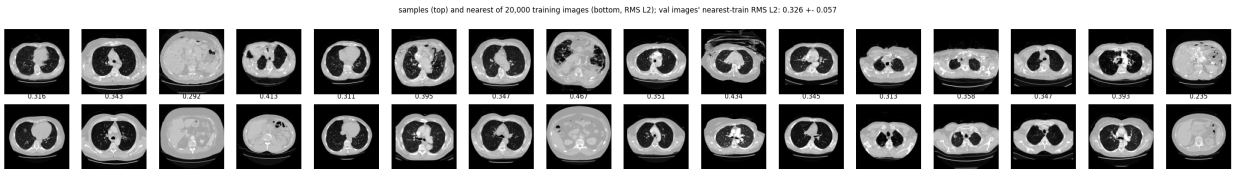


Figure 118: 128 px memorisation check.

14.4 Analysis

The rungs are analysed as they finish; the numbers are those of the `summary.tex` lines above and of each rung's `result.json` and `metrics.csv`.

32 px (12 mm/px).

- **(i) holds, and the early-stop rule did its job.** Validation bits/dim falls monotonically from 5.29 (epoch 1) to 3.659 (epoch 11) and then rises — 3.678, 3.677, 3.700, 3.703 at epochs 12, 14, 15, 16 — while the training loss keeps falling ($-3.39 \to -3.51$ nats/dim) and the pre-clip gradient norm grows ($76 \to 90$): the 42.8M-parameter model over-fits the 95,947 slices (808 subjects) after ∼1M images, and the rule stopped the run at 16. The test set confirms the choice of checkpoint: the best-validation model scores 3.643 bits/dim on all 12,136 test slices (median 3.598, 99th percentile 5.41, maximum 7.35, none above 10) against 3.698 for the final one, 0.016 below its own validation value. Epoch 13's validation mean of 1.7×10^5 (an excess of 2×10^9 bits/dim summed over the 11,971 slices, i.e. one or a few slices blowing up; the per-image accounting was added after this run) is the same mechanism as at 64 px below and as step 9's test image 14583, and is the first sign that the plain validation mean is not a usable selection statistic.
- **(ii) holds.** The 64 samples are finite, 4.2% of pixels fall outside $[-1, 1]$ before clamping (bone saturating at white and air at black, the two ends of the window), and every sample is a thorax or an upper abdomen: body outline, two dark lung fields or a liver, a bright spine, the table. At 12 mm per pixel there is nothing finer to get right.
- **(iii) holds exactly.** The samples' nearest-training RMS distance is 0.229, the validation slices' own is 0.229 ± 0.046 (minimum 0.132 over 11,971 slices; the closest sample is at 0.132 as well): the model is not reproducing training slices, it is as far from them as an unseen patient is.

64 px (6 mm/px).

- **(i) holds, once the validation statistic was repaired.** Under the plain mean the run stopped at epoch 17, because epochs 13–17 scored 377, 3.46, 50.9, 4098 and 3606 bits/dim on the 4000 validation slices; every one of those numbers is two slices (of 4000) at 10^2–10^7 bits/dim, and the remaining 3998 kept improving. The evidence that this was not over-fitting: the *final* epoch-17 checkpoint scored 2.901 on the 12,136 test slices against 2.961 for the "best" epoch-11 one, and neither had a single test slice above 10 bits/dim. The run was therefore resumed from epoch 17 with the selection statistic changed to the mean over validation slices below 10 bits/dim (the step-9 outlier accounting, `val_stat: robust`; patience counted from the resume; recorded in the config header and in `first_run_mean_rule/`). Under that statistic the validation curve is the expected one: 2.928 at epoch 18, best 2.914 at epoch 20, then a slow rise (2.914, 2.916, 2.919, 2.922, 2.929, 2.932) against a still-falling training loss ($-3.87 \to -3.92$) and a gradient norm climbing from 48 to 88 — mild over-fitting, the rule stopping the run at 26 of 30. Test bits/dim: 2.900 for the epoch-20 checkpoint (median 2.896, 99th percentile 4.50, maximum 4.91, none above 10) against 2.926 for the final one, 0.014 below its validation value. Per pixel the model is 0.74 bits/dim better than the 32-px one — expected, since a 6 mm pixel is more predictable from its neighbours than a 12 mm one. (Step 9's ChestMNIST-64 model reached 3.025 with three times the parameters and a similar number of images; a different dataset, so only the order of magnitude is comparable.)

- **What the two blown-up slices are** (Figure 120; `--val-outliers`, run on both checkpoints after training). The same two of 4000 slices fail under both: validation slice 1095 (72 bits/dim under the best checkpoint, 1707 under the final one) and 2426 (7.5×10^4 and 7.7×10^5). Both carry bright objects *in the air above the chest wall* — ECG electrodes and leads on the skin, a feature rare enough in the training split that the model has no context for it — and slice 2426 is additionally a shoulder-level section (humeral heads at the edge of the field of view). The largest first-block $|z|$ is on one of those bright dots in slice 1095; the failure is then a chain: the maximum $|z|$ after the eight blocks runs 10, 20, 60, 60, 60, 50, 30, 600 (slice 1095) and 10, 10, 20, 20, 20, 50, 100, 2×10^4 (slice 2426), i.e. each block's coupling, conditioned on an already-atypical context, predicts a small scale for a token that is then far from its prediction, and the last block — whose log-scale is bounded at 8, a factor of 2981 — turns a $|z|$ of 100 into 2×10^4. A single latent coordinate of that size contributes $z^2/2 = 2 \times 10^8$ nats, which over 4096 pixels is 7×10^4 bits/dim: the whole excess is one coordinate. Sharper (later) checkpoints make it worse (72 $\rightarrow$ 1707 bits/dim on slice 1095 from epoch 20 to 26), which is why the plain mean *rose* with training and read as over-fitting. The test split happens to contain no such slice (maximum 4.9 bits/dim); the rate is of order 1 in 2000.

- **(ii) holds, with a visible step up from 32 px.** The 64 samples are finite, 3.2% of pixels leave $[-1, 1]$ (12.9% at the epoch-11 checkpoint of the first run: the later epochs mainly tightened the ends of the window), and the grid shows lung fields with a few vessel dots, a mediastinum, a spine with its canal, ribs along the body edge, liver and bowel gas in the abdominal sections, and the curved table; a few outlines are implausibly shaped (a flattened or cut body) and a few lung fields contain bright patches with no anatomical counterpart. What it does not show at 6 mm is fissures or bronchi, which is claim (iv)'s test for the next rung.

- **(iii) holds.** Nearest-training RMS distance 0.276 for the samples against 0.293 ± 0.057 for validation slices (minimum 0.141); the closest sample (0.157) is a small abdominal section and its nearest training slice has a different organ layout.

- **(iv), 32 $\rightarrow$ 64, holds.** Side by side, the 32-px grid is outlines with two dark blobs and a bright dot; the 64-px grid adds the ribs, the spinal canal, the shape of the mediastinum, vessel dots in the lungs and the table — structure, not larger blobs.

- **Cost.** 152.6M parameters at batch 64 with per-block checkpointing: 5.4 GB, 0.53 s per step, 6.9 minutes per 48k-image epoch, 173 minutes for the 26 epochs; the official sequential reverse draws 64 samples in 2.9 s.

128 px (3 mm/px).

- **(i) holds; this rung is under-trained rather than over-fitted.** With the robust statistic from the start, validation bits/dim falls monotonically through all 30 epochs — 3.383, 3.039, 2.927, 2.843 for epochs 1–4, 2.685 at 10, 2.604 at 20, 2.5819 at 30 — and never turns up; the best checkpoint is the last one and the early-stop rule was never triggered. The curve is still falling by 0.003 per epoch at epochs 26–29 and flattens at 29 $\rightarrow$ 30 only because the cosine schedule has brought the learning rate to 10^{-6}: 30 epochs of 24,000 images are 7.5 passes over the 95,947 slices, against 13 passes at 64 px, where over-fitting set in at 10. A longer schedule would improve this model; it was not run, because the rung's job is the ladder and the 128-px measurement systems, not the best possible 128-px prior. The test set agrees with

the validation set to the claimed precision: 2.5980 bits/dim over the 12,135 test slices below 10 bits/dim (median 2.635), 0.016 *above* the validation value (at 64 px it was 0.014 below). Per pixel the model is 0.30 bits/dim better than the 64-px one and 1.05 better than the 32-px one (test values), the same direction as before with a smaller step, as expected when the added pixels carry more sub-structure and less redundancy than the 6-mm ones did.

- **The blown-up batches are single steps and did nothing.** The training-loss column of `metrics.csv` reads 10^{22} to 10^{23} nats/dim with an infinite gradient norm for epochs 17 and 21–26, and -3.94 to -3.98 for the others. Each of those is *one batch* of the 1500 whose latent overflowed (the running mean is a sum divided by a count; between consecutive log lines within such an epoch the sum did not change, so the 10^{26}-nat batch occurred once and the remaining batches were ordinary). With `grad_clip: 1.0`, an infinite pre-clip norm gives a clip coefficient of $1/\infty = 0$: the gradient is zeroed and the step moves the weights only by Adam's momentum and the weight decay. The validation curve is monotone through every one of those epochs (2.6211, 2.6020, 2.5971, 2.5941, 2.5912, 2.5885, 2.5876 at 17 and 21–26), which is the direct check that the weights were not harmed. The plotted gradient norm (finite epochs only) is 346 at epoch 1, 14–19 for epochs 2–20 and 44–60 for 27–30; the late rise, at a learning rate of 10^{-6} or less, has no effect on the weights and is not the over-fitting signature it was at 64 px, where it came with a rising validation curve.

- **The outlier slice, 1 in 12,136.** The validation probe (`--val-outliers`) finds none of the 2000 validation slices above 10 bits/dim under the best checkpoint (maximum 4.27), so the robust and plain validation means coincide. The test split has exactly one: slice 1758 scores 3.9×10^{21} bits/dim and alone lifts the plain mean to 3.2×10^{17}. It is a lower-chest section that looks unremarkable at a glance (Figure 119): lung fields, heart, a normal table — and a faint one-pixel contour in the air outside the chest wall (a lead or a fold of sheet), the same class of out-of-body feature as the two 64-px validation outliers. The chain is the one described for 64 px, one block later and steeper: the maximum $|z|$ after the eight blocks is 9.8, 11.1, 147, 6.8×10^3, 6.7×10^5, 3.1×10^8, 6.6×10^{10}, 6.0×10^{12}, a factor of 50–460 per block, all within the per-block bound of $e^8 = 2981$. One coordinate of 6×10^{12} contributes $z^2/2 = 1.8 \times 10^{25}$ nats, which over 16,384 pixels is 1.6×10^{21} bits/dim: again one or a few coordinates carry the whole excess. The rate has fallen from 2 in 4000 (64 px) to 1 in 12,136, with the caveat that the 128-px validation subset is 2000 slices and contained none.

- **(ii) holds for 29 of 32 samples.** All 32 are finite and 0.99% of pixels leave $[-1, 1]$ (3.2% at 64 px, 4.2% at 32 px: the ends of the window tighten with resolution). Twenty-nine are chest or upper-abdominal sections with the anatomy of a 3-mm slice: lung fields containing vessel dots and short branching segments that radiate from the hila, the trachea or a main bronchus as a dark ring in the mediastinum, the heart and the aorta as separate bright shapes, the spinal canal, ribs and scapulae, subcutaneous fat distinct from muscle, the liver and the bowel gas of the abdominal sections, and the curved table with its pad. Three are not: a body outline filled with a grainy texture instead of organs (row 1, column 3), a noisy disc (row 4, column 2), and a flat grey square with dark holes and no body outline (row 4, column 7); two more have a torn or flattened outline with streaks above it. The failure modes are whole-image ones (the sample is not a slice at all) rather than local ones, and their rate, 3–5 of 32, is higher than at 64 px, where the equivalent count was "a few" of 64.

- **(iii) holds.** Nearest-training RMS distance 0.354 for the 16 samples checked against 0.326 ± 0.057 for validation slices (minimum 0.175 over 2000; closest sample 0.235). The samples are,

on average, farther from the 20,000-slice training subset than an unseen slice is; the distances themselves rise along the ladder (0.229, 0.276, 0.354 for the samples; 0.229, 0.293, 0.326 for validation) because the finer the pixel the less two different patients share.

- **(iv), 64 → 128, holds for vessels and bronchi; fissures are not resolved.** Side by side with the 64-px grid, the 128-px samples add the vascular tree inside the lung (dots and branching segments, densest near the hila, rather than a few isolated dots), the airways (trachea, main bronchi), the separate outlines of heart and great vessels, and the scapulae and the fat–muscle boundary in the chest wall. Fissures — a one-pixel line at 3 mm — are not visible in the samples, nor convincingly in the 32 training slices beside them; that judgement moves to 256 px.

- **Cost.** 155.7M parameters (patch 4, $T = 1024$ tokens of dimension 16, 512 channels, 8×6 layers) at batch 16 with per-block checkpointing: 0.84 s per step, 21 minutes per 24k-image epoch when the GPU was not shared (epochs 14–20 took 25–45 minutes because the 32-px step-14 to step-16 jobs ran alongside), 776 minutes of wall time for the 30 epochs; the official sequential reverse draws 32 samples in 23.7 s.

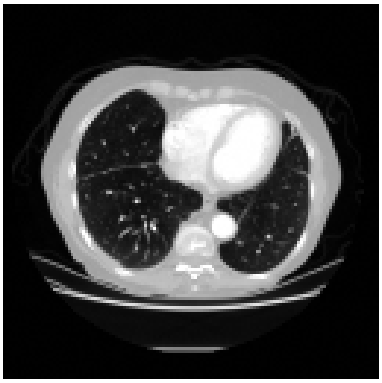


Figure 119: 128 px: the one test slice (index 1758 of 12,136) at which the best checkpoint blows up (3.9×10^{21} bits/dim). A lower-chest section with a faint one-pixel contour in the air outside the chest wall.

15 Step 14: The six measurement systems on LIDC slices, along the ladder

15.1 Methods

- Code: `step14_lidc_gallery.py` (re-uses `measlib`, `invlib.FastReverse`, `step11.neg_log_normal` and `step13.load_prior/build_bundle`); config: `configs/step14_lidc_gallery.yml` (one config for the whole ladder, `--res N` selects the prior); outputs: `outputs/step14_lidc_gallery/{data,figu` with `gallery_N.{json,png}` and `gallery_N_solutions.pt` per resolution.

- **What is repeated.** The step-11 gallery (Figure 104): the six measurement systems of `measlib` — denoising, box inpainting, random-pixel inpainting, 2× and 4× super-resolution, sparse-view CT — each on a different test slice, solved by the latent MAP $\arg\min_z -\log p(y|g_\theta(z)) - \log \mathcal{N}(z)$ with Adam (lr $0.1 \to 5 \times 10^{-3}$ cosine, 1000 steps, the schedule of the step-11 sweep)

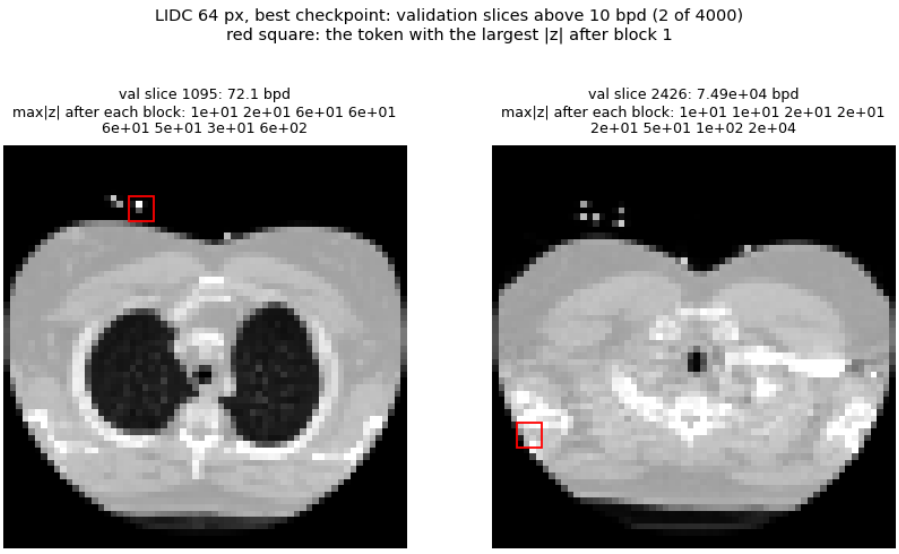


Figure 120: 64 px: the two of 4000 validation slices above 10 bits/dim under the best checkpoint (`--val-outliers`), with the maximum $|z|$ after each of the eight blocks and, in red, the 4×4 token with the largest $|z|$ after the first block. Both carry ECG leads in the air above the chest wall.

from three independent $\mathcal{N}(0, I)$ inits, under the step-13 prior of the resolution. All six systems' restarts are optimised together as *one* batch of 18 through the CUDA-graph reverse: Adam is elementwise and the objective is a sum over rows, so this is exactly the eighteen independent optimisations, and the sequential reverse is latency-bound at $T = 64$ (0.28 s per step for 18 rows against 0.18 s for 3) though not at $T = 256$ (7.7 s against 2.4 s: the functional cache copies of `FastReverse` scale with the batch, see the next bullet), so the batch costs 1–2 systems' worth per step instead of six. *Deviation, recorded:* the first 32- and 64-px galleries ran the systems one after the other with 300 steps each; at 64 px that was not converged (box inpainting: all restarts thousands of nats above the truth's objective), so the batch and the step-11 step count were adopted and the first runs are kept under `first_runs_300_steps/` and quoted in the Analysis. Every system is $y = Ax + \sigma\epsilon$ with the *exact* forward operator on the $[-1, 1]$ scale of the model window (so $0.05 = 35$ HU and $0.25 = 175$ HU): the pixel mask, 2×2 or 4×4 average pooling, and the pixel-driven Radon transform of `measlib.radon_matrix` with $\lceil N\sqrt{2} \rceil$ detector bins (odd) over 180°.

- **A second reverse for $T = 1024$ (deviation, recorded).** `invlib.FastReverse` (step 11) makes each token step a pure function of static shape by passing the whole (B, L, T, C) KV cache through it functionally — copied into the graph's static buffers, rewritten with the new slot, cloned out, and in the backward rebuilt from the saved slots and cloned again — about twenty full passes over the cache per token step, i.e. $O(BT^2)$ memory traffic per pass through the model. That was invisible at $T = 64$ and tolerable at $T = 256$ (2.4 s per Adam step at batch 3, 7.7 s at batch 18), but extrapolates to about a minute per step at $T = 1024$, forty hours per gallery. The 128- and 256-px galleries therefore use `invlib.FastReverse2` (`reverse: fast2` in the config): the same recursion with the cache as a static buffer written *in place* — slot i is written once, at step i, so the cache at the end of a block pass restricted to the slots below i is exactly what step i saw, and the backward can read it as a constant — and with the gradient with respect to the cached keys and values, a rank-one update per

token step and the only thing in the backward that touches the whole cache, deferred: the attention probabilities and score gradients of a chunk of 128 token steps are kept as rows and applied as one matrix product per chunk; the token's own key and value enter its attention as an explicit extra entry so that their gradient goes through autograd with the rest of the token network. A token step then reads the cache six times and writes one slot, and the graphs are captured per chunk so that a step reads only the slots below its chunk's end. `check_fast_reverse2` compares the two on the trained priors (random z, batch 3; relative max-abs differences of x, $\log q$ and the gradient of a random linear functional with respect to z): on the 32-px prior ($T = 64$) 1.2×10^{-5}, 2.6×10^{-7} and 4.0×10^{-5} against the compiled reverse and 1.9×10^{-5}, 3.5×10^{-7} and 7.0×10^{-5} against the eager one — the same to every printed digit whether the new reverse runs eagerly or from its captured graphs, so the graphs are exact and the residue is fp32 operation order (a CPU check on a small random model agrees to 1–4 $\times 10^{-6}$); on the 64-px prior ($T = 256$) 1.8×10^{-5}, 5.9×10^{-7} and 1.3×10^{-5}. Measured cost of a forward-and-backward pass at batch 18: 3.1 s at $T = 256$ against 7.7 s (peak 2.4 GB against 9.3), and 17.2 s at $T = 1024$ with chunks of 128 (18.6 s with 256; 6.0 GB) — 4.8 h for a 1000-step gallery instead of the extrapolated forty. At $T = 256$ the 1.5 ms per block and token step is the floor of some 400 kernel launches rather than cache traffic (about 0.3 ms at this size), which is why the gain over the first reverse is 2.5× and not the ratio of the two traffics; at $T = 1024$ (2.1 ms) the traffic adds roughly as much again as the floor. The 32- and 64-px galleries ran with `FastReverse`; the two compute the same map, so they were not re-run.

- **The same task at every resolution.** The parameters are stated at 64 px and scaled with the side N: the box covers 3/8 of the side ($12, 24, 48, 96$ px), the random mask drops 70% of the pixels, the CT system has $N/4$ views ($8, 16, 32, 64$; a constant six-fold under-sampling of the $\sim \pi N/2$ views the grid would need) and a sinogram noise $\sigma_{\text{ct}} = N/64$ (the ray sums scale with N), the pixel-domain noises are 0.25 (denoising) and 0.05 (everything else) at every N. The measurement of a given slice is therefore the same physical measurement of the same 384 mm object at four samplings.

- **A matrix-free Radon operator.** `measlib.ct` builds a dense $M \times N^2$ matrix and its full SVD; at 128 px the 16384^2 right-singular basis is 1 GB and at 256 px it is 34 GB of float64, beyond the machine. Step 14 therefore uses `RadonOp`: the same pixel-driven linear interpolation as `radon_matrix`, but stored as (index, weight) pairs and applied with `index_add_` — no matrix, no SVD, exact gradients through autograd. The self-test (`verify.json`) compares it with `radon_matrix` at 32 px and 8 angles: values and gradients agree to 1.4×10^{-14} and 4.3×10^{-14} (the weights are kept in float64 and cast per call; a float32 copy at construction gave 4×10^{-7} and was rejected).

- **What is recorded.** Per system and restart: J_z, the data term against the expected value at the truth ("noise level", $\frac{M}{2}(1 + \log 2\pi\sigma^2)$), PSNR to the truth, $\|z\|$ against the prior's typical $\sqrt{TD}$, $\max|x|$, the mean pairwise RMS between the restarts, seconds per step; the truth's own J_z from the forward pass; and the CUDA-graph build time.

- **The falsifiable claims.** (i) The pattern of step 11 recurs at every resolution: the data term reaches its noise level for all systems except box inpainting and sparse-view CT, whose restarts end in distinct local minima (visibly different content, J_z spread of hundreds of nats). (ii) $\|z\|$ collapses far below $\sqrt{TD}$ where the data are weak (4× super-resolution, denoising), i.e. the latent MAP is not a posterior sample (step 11, claim iii). (iii) The cost per Adam step grows

as $T\times$ blocks: 0.2 s at $T = 64$, 1–1.5 s at $T = 256$, and of the order of 10 s at $T = 1024$, so the gallery at 128 and 256 px is an hours-long computation for eighteen reconstructions — the motivation for a faster inverse or a different sampler in later steps. (iv) The reconstructions improve in PSNR along the ladder for the pixel-domain systems only insofar as the prior does; for CT the fixed six-fold under-sampling keeps the task equally hard, and the PSNR is a judgement of the prior's detail, not of the sampling.

15.2 Configuration (verbatim)

```
# step14_lidc_gallery.py -- the six measurement systems of measlib on LIDC test slices,
# solved by latent-space MAP (Adam on z, several random restarts) under the step-13
# prior of each resolution. One config for the whole ladder; `--res N` picks the prior.
#
# Every system is y = A x + sigma eps with the EXACT forward operator (pixels, average
# pooling, or the pixel-driven Radon transform), images on the [-1, 1] scale of the
# model window (-1000..400 HU, so 0.05 = 35 HU and 0.25 = 175 HU). The parameters are
# stated for a 64-px image and scaled with the side N so the task stays the same:
# the box covers 3/8 of the side, CT keeps N/4 views (a constant 6x under-sampling of
# the ~pi N / 2 Nyquist number) and a sinogram noise proportional to the ray length.
#
# Deviation, recorded. The first 32- and 64-px galleries ran the six systems one after
# the other with 300 Adam steps each (the step-11 protocol shortened to keep the
# sequential reverse affordable: 2.4 s per step at T = 256, 72 min per gallery). At
# 64 px that was not converged -- for box inpainting all three restarts ended
# thousands of nats above the truth's objective, with the data term far above the noise
# level -- so the script was changed to optimise all systems' restarts as ONE batch of
# 18 (the reverse is latency-bound, so this costs the same per step as one system)
# and `iters` was raised to the step-11 value of 1000. Both first runs are kept under
# outputs/step14_lidc_gallery/first_runs_300_steps/ and quoted in the paper.
#
# Second deviation, recorded. The batched 32- and 64-px galleries ran through
# invlib.FastReverse, whose token step passes the whole KV cache through functionally
# (O(B T^2) memory traffic per pass): 0.28 s per step at T = 64 but 7.7 s at T = 256 with
# the batch of 18, which extrapolates to about a minute per step at T = 1024. The 128-
# and 256-px galleries therefore use invlib.FastReverse2 (`reverse: fast2`), the same
# recursion with the cache written in place and its gradient deferred per chunk of
# token steps. check_fast_reverse2 on the trained 32/64-px priors: x, log q and dJ/dz
# agree with FastReverse to 2e-5 / 6e-7 / 4e-5 (fp32 operation order); a forward+backward
# pass at batch 18 takes 3.1 s at T = 256 (7.7 s before) and 17.2 s at T = 1024.

seed: 20260913
output_root: outputs/step14_lidc_gallery
prior_checkpoint: best         # step-13 model_best.pth (best validation bpd)
priors:
  32: configs/step13_lidc_tarflow_32.yml
  64: configs/step13_lidc_tarflow_64.yml
  128: configs/step13_lidc_tarflow_128.yml
  256: configs/step13_lidc_tarflow_256.yml

restarts: 3                    # independent N(0, I) inits per system; all systems' restarts
                               # run as one batch through the CUDA-graph reverse
iters: 1000                    # the step-11 count (300 in the archived first runs, see above)
reverse: fast2                 # in-place-cache reverse (see above; 32 and 64 px ran with the first one
```

```
chunk: 128                      # token steps per captured graph and per deferred cache-gradient update
lr: 0.1                         # Adam on z, cosine to lr_min (the step-11 sweep: lr 0.1
                                # best, >= 0.5 drifts)
lr_min: 0.005
n_rows: 6                       # one test slice per system (different slices)

systems:
  - {name: denoise, label: "denoising", sigma: 0.25}
  - {name: inpaint_box, label: "inpainting, box", box_frac: 0.375, sigma: 0.05}
  - {name: inpaint_random, label: "inpainting, 70% dropped", drop: 0.7, sigma: 0.05,
     mask_seed: 0}
  - {name: sr_2x, label: "super-resolution 2x", sigma: 0.05}
  - {name: sr_4x, label: "super-resolution 4x", sigma: 0.05}
  # 16 views, sigma 1.0 at 64 px
  - {name: ct, label: "sparse-view CT", views_per_px: 0.25, sigma_per_px: 0.015625}
```

15.3 Results

Each line reports, per system, the lowest-objective restart — the one the method would pick without the truth — with its J_z, data term, PSNR and $\|z\|$; where another restart has a better PSNR it is added in parentheses.

32×32 ($T = 64$, $D = 16$; 3 restarts × 1000 steps, all systems in one batch of 18, 0.28 s per step, 5 min): denoise: J_z 933 (truth 1448), data -130 (noise level 33), PSNR 23.0 dB (best restart 23.3), $\|z\|$ 16 (typical 32), restart spread 0.031; inpaint_box: J_z -300 (truth -80), data -1448 (noise level -1388), PSNR 21.6 dB, $\|z\|$ 20 (typical 32), restart spread 0.147; inpaint_random: J_z 525 (truth 951), data -548 (noise level -481), PSNR 23.5 dB (best restart 25.6), $\|z\|$ 16 (typical 32), restart spread 0.116; sr_2x: J_z 552 (truth 908), data -455 (noise level -404), PSNR 25.9 dB (best restart 29.8), $\|z\|$ 12 (typical 32), restart spread 0.073; sr_4x: J_z 839 (truth 1255), data -118 (noise level -101), PSNR 25.3 dB, $\|z\|$ 6 (typical 32), restart spread 0.030; ct: J_z 1249 (truth 1575), data 235 (noise level 273), PSNR 27.6 dB, $\|z\|$ 12 (typical 32), restart spread 0.058. 64×64 ($T = 256$, $D = 16$; 3 restarts × 1000 steps, all systems in one batch of 18, 7.71 s per step, 128 min): denoise: J_z 4012 (truth 5691), data -134 (noise level 134), PSNR 25.2 dB (best restart 26.4), $\|z\|$ 28 (typical 64), restart spread 0.092; inpaint_box: J_z -1060 (truth -479), data -5657 (noise level -5550), PSNR 18.4 dB, $\|z\|$ 41 (typical 64), restart spread 0.217; inpaint_random: J_z 4435 (truth 3650), data 7 (noise level -1865), PSNR 23.9 dB, $\|z\|$ 36 (typical 64), restart spread 0.192; sr_2x: J_z 2807 (truth 3968), data -1556 (noise level -1615), PSNR 28.1 dB, $\|z\|$ 35 (typical 64), restart spread 0.055; sr_4x: J_z 3605 (truth 4963), data -410 (noise level -404), PSNR 23.5 dB (best restart 23.7), $\|z\|$ 22 (typical 64), restart spread 0.125; ct: J_z 7337 (truth 7121), data 2364 (noise level 2066), PSNR 25.6 dB, $\|z\|$ 49 (typical 64), restart spread 0.121.

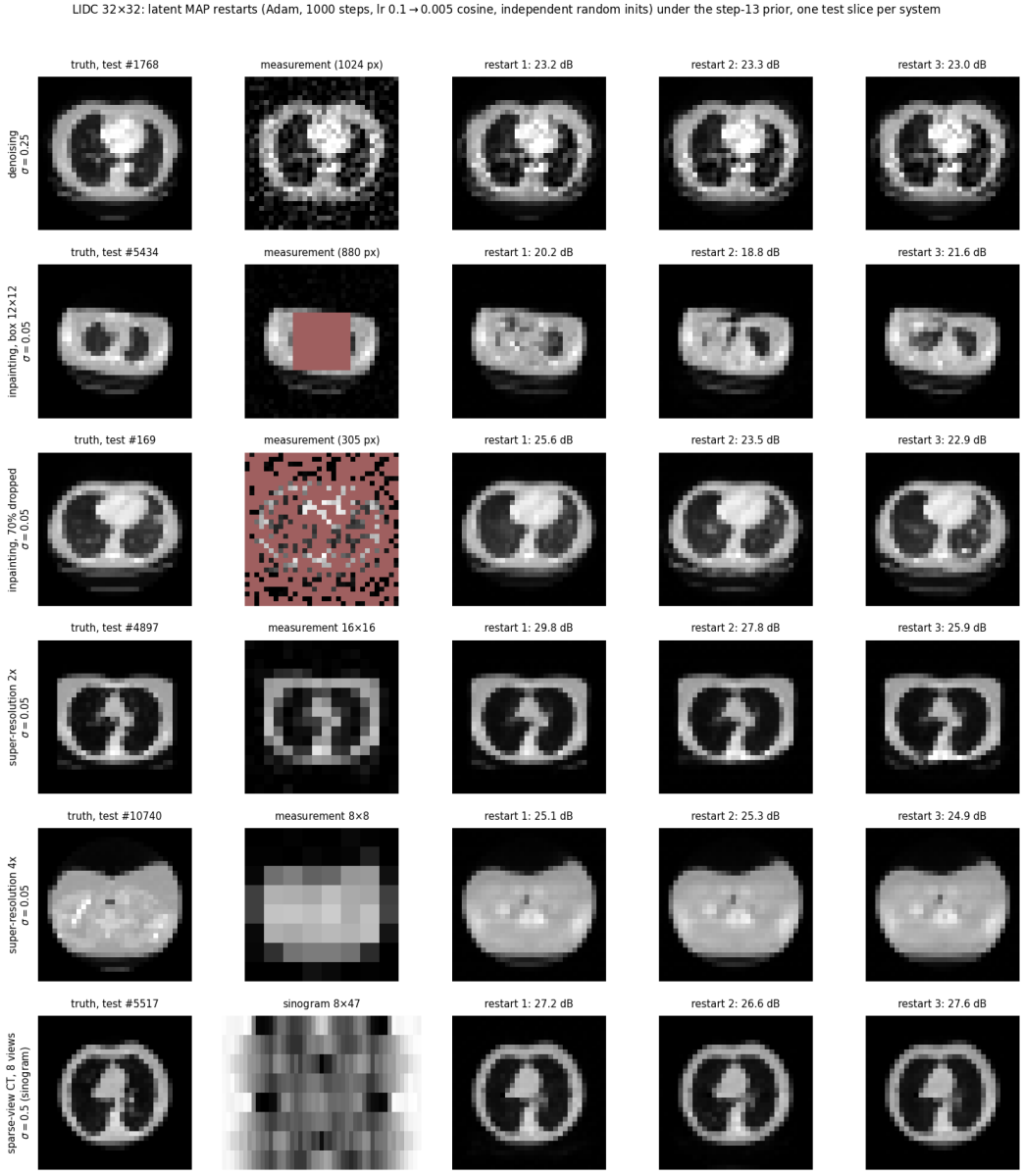


Figure 121: 32 px: truth, measurement (masked pixels in red; the low-resolution image; the 8×47 sinogram) and three latent-MAP restarts with their PSNR, one test slice per system.

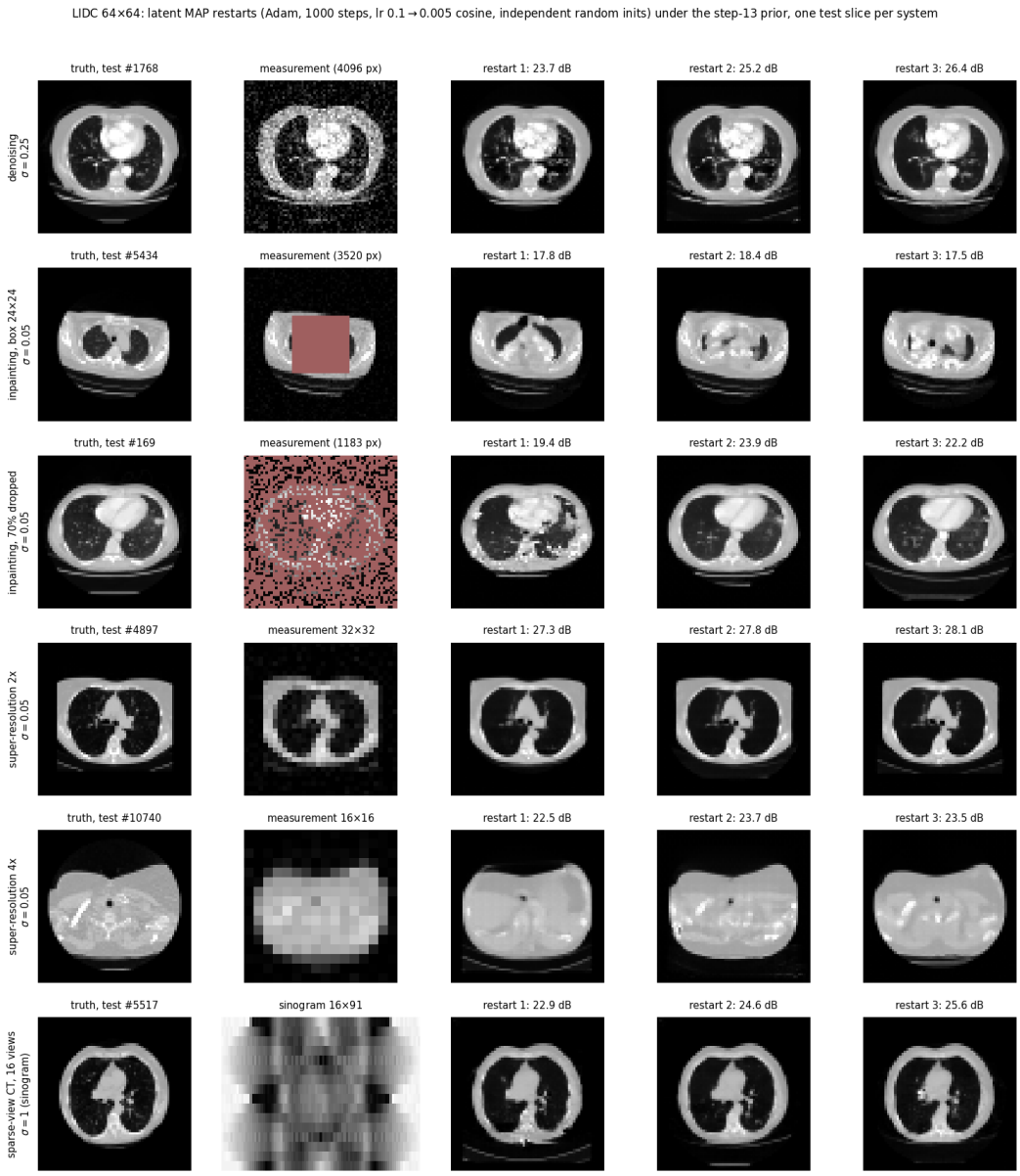


Figure 122: 64 px: the same six systems under the 64-px prior (16 views, $\sigma_{\text{ct}} = 1$).

15.4 Analysis

The rungs are analysed as they finish, each against the four claims and against its archived 300-step first run (`first_runs_300_steps/`, the numbers in parentheses below).

32 px (8 views, $T = 64$).

- **The 300-step runs were not converged; 1000 steps are.** With the batch of 18 and 1000 steps every system's best restart ends with its data term at or below the noise level — denoising −130 against +33, box inpainting −1448 against −1388, random inpainting −548 against −481, 2× SR −455 against −404, 4× SR −118 against −101, CT 235 against 273 — and every objective below the truth's (J_z 933 vs. 1448, −300 vs. −80, 525 vs. 951, 552 vs. 908, 839 vs. 1255, 1249 vs. 1575). At 300 steps box inpainting had stopped at $J_z = +323$ (403 nats above the truth, data term −1015) and random inpainting at 863; in the 1000-step run the best objectives were still falling between steps 300 and 1000 in every system but 4× SR (844 → 839): box inpainting −114 → −300, random inpainting 672 → 525, 2× SR 635 → 552, denoising 999 → 933, CT 1288 → 1249. The sequential 300-step gallery took 5.4 min for 1800 Adam steps, the batched 1000-step one 4.7 min for 1000 (0.28 s per step at batch 18 against 0.18 at batch 3): at $T = 64$ the reverse is indeed latency-bound.

- **(i) half holds.** The multimodality part holds for box inpainting: the three restarts end at $J_z = -91$, −109 and −300 with visibly different lung content inside the 12 × 12 box (restart spread 0.147 RMS, PSNR 18.8–21.6 dB) — distinct local minima, the best 220 nats below the truth. Random inpainting shows the same (525, 585, 743; spread 0.116) and CT less so (1249, 1376, 1481; spread 0.058; PSNR 26.6–27.6). But the "data term above the noise level" part is false at 32 px: with enough steps every restart of every system fits its data to the noise level, box inpainting and CT included. What distinguishes the ill-posed systems is not a data-term residual but the spread of the minima.

- **(ii) holds.** $\|z\|$ at the solutions is 6 (4× SR), 12 (2× SR, CT's best), 15 (denoising), 16–19 (random inpainting) and 20–25 (box inpainting) against the prior's typical $\sqrt{TD} = 32$: the latent MAP sits far inside the typical set, most so where the data constrain least, exactly as in step 11.

- **Denoising: a lower objective and a worse image.** The denoising MAP is the one system whose PSNR *fell* with more optimisation, from 26.7 dB (300 steps, data term −16, at the noise level) to 23.2 dB (1000 steps, data term −130): the optimiser keeps lowering J_z by fitting part of the $\sigma = 0.25$ noise — the residual ends at two thirds of the noise variance — and the prior at 32 px does not charge enough for the speckle this puts into the lung fields to stop it. The mode of the posterior is not its mean, and here the difference is 3.5 dB; the restart spread of 0.031 says all three restarts agree on this noise-fitting solution.

- **Cost (iii).** 0.28 s per Adam step for 18 rows at $T = 64$ (six blocks): 8.4 s of graph build and 282 s of optimisation.

64 px (16 views, $T = 256$).

- **The batch of 18 and 1000 steps fixed the pathological first run, but two systems are still not at their minimum.** Box inpainting, which ended the sequential 300-step run at $J_z = +2016$ (2500 nats above the truth's −479), now has a restart at −1060 — 580 nats

below the truth — with its data term at the noise level (-5657 against -5550). $2\times$ and $4\times$ SR and denoising end below the truth's objective with data terms at the noise level (-1556 vs. -1615; -410 vs. -404; -134 vs. $+134$). But random inpainting ends at $J_z = 4435$ (truth 3650) with a data term of $+7$ against a noise level of -1865: the residual on the 1183 observed pixels has RMS 2.0σ, and sparse-view CT ends at 7337 (truth 7121) with residual RMS 1.2σ (data 2364 vs. 2066). Both were still falling in the last hundred steps (4452 $\rightarrow$ 4435, 7361 $\rightarrow$ 7337) with the learning rate already annealed to 0.005: the optimisation, not the prior, is what stops them, and the truth is a better point of the latent space than any the optimiser found. At equal step count the long schedule was already ahead: at step 300 of the 1000-step run (learning rate still 0.08) the best objectives were -189 (box), 5316 (random) and 8616 (CT), against $+2016$, 7287 and 9666 at the end of the 300-step cosine, whose learning rate had decayed too early to matter.

- **(i) holds, and now in the form stated.** Box inpainting's three restarts end at -1060, -287 and $+291$ with visibly different content in the 24×24 hole (spread 0.217, PSNR 17.5–18.4 dB), and the two worse minima have data terms 310 and 660 nats above the noise level — the "local minima with the data term above the noise level" of the claim. CT is the same on a smaller scale (7337, 7592, 8341; spread 0.121; all data terms above the noise level, PSNR 22.9–25.6). Random inpainting adds a third case (4435, 5851, 6565; spread 0.192). At 32 px every restart of every system reached the noise level; at 64 px the landscape is rougher, and even the well-posed systems show it: the denoising restarts are 200 nats apart (4012, 4208, 4258) with PSNR 23.7–26.4 dB.

- **(ii) holds.** $\|z\|$ at the best restart is 22 ($4\times$ SR), 28 (denoising), 35 ($2\times$ SR), 36 (random inpainting), 41 (box inpainting) and 49 (CT) against the prior's typical $\sqrt{TD} = 64$, the same ordering as at 32 px (weakest data, smallest latent). Two restarts leave the window: box inpainting's worst has a pixel at 2.4 and $4\times$ SR's middle one at 1.6 (the window is $[-1, 1]$).

- **Denoising: the 32-px effect, weaker.** The lowest objective (4012) again belongs to a restart that fits part of the noise — residual variance 87% of σ^2, PSNR 25.2 dB — while the restart with the best PSNR (26.4 dB, residual 92%) is 246 nats worse; at 300 steps the best restart had 25.6 dB at the noise level. The 64-px prior charges more for speckle than the 32-px one did (residual 68% of σ^2 there, 3.5 dB lost against 0.4 dB here).

- **Cost (iii), and the reason for the second reverse.** 7.7 s per Adam step for 18 rows at $T = 256$ (eight blocks), 12.3 s of graph build, 128 min in all: per restart-step 0.43 s against 0.80 s in the sequential run, a factor 1.9, not the factor 6 the "latency-bound" argument promised, because the functional cache copies of the first reverse scale as $B\,T^2$. The Methods bullet above gives the fix used from 128 px on.

- **(iv) is not testable across rungs by PSNR.** The best-restart PSNRs are 25.2 (denoising), 18.4 (box), 23.9 (random), 28.1 ($2\times$ SR), 23.5 ($4\times$ SR) and 25.6 dB (CT) against 23.0, 21.6, 23.5, 25.9, 25.3 and 27.6 at 32 px — three up, three down — but the images differ (a 64-px slice has detail a 32-px one lacks) and the two hardest systems are stuck short of their minima, so the comparison says more about the optimiser than about the prior.

16 Step 15: Latent initialisation, pixel-domain refinement

16.1 Methods

- Code: `step15_two_stage_map.py` (re-uses `step14.make_problem` with the stored measurement, `step11.neg_log_normal`, `step10.draw_samples` and `step13.load_prior`); config: `configs/step15_two_stage_map.yml`; outputs: `outputs/step15_two_stage_map/{data,figures}/` with `two_stage_N.{json,png}`, `two_stage_N_curves.png`, `two_stage_N_solutions.pt` and `two_stage_N_traces.npz` per resolution. Added on 2026-09-04, at Matt's request, while the 128-px prior trained; the 32-px run took 32 s per variant on the shared GPU, the 64-px run 284 s.

- **The estimator.** Two stages. Stage 1 is step 14 itself: the latent MAP, Adam on z with $x = g_\theta(z)$ and $J_z(z) = -\log p(y|g_\theta(z)) - \log \mathcal{N}(z; 0, I)$, three random restarts per system, whose solutions are read from `gallery_N_solutions.pt`. Stage 2 takes each restart's image $x_1 = g_\theta(z_1)$ as the initialisation of Adam on the *pixels* of x under the exact image-space objective of step 11,

$$J_x(x) = -\log p(y|x) - \log p_\theta(x) = -\log p(y|x) + \tfrac{1}{2}\|f_\theta(x)\|^2 + \tfrac{n}{2}\log 2\pi - \log\left|\det \partial f_\theta/\partial x\right|,$$

  with $f_\theta = g_\theta^{-1}$ the parallel transformer forward (`prior(x)` returns z and the per-dimension mean log-determinant, multiplied by $n = N^2$). J_x is the posterior density in x; J_z is not — it is $J_x \circ g_\theta$ *without* the log-determinant — so the two objectives have different minimisers and stage 2 is not a continuation of stage 1 but a change of objective from the point stage 1 found. A step is one forward/backward pass through the prior (no sequential reverse, no checkpointing at 32 px), 32 ms for the batch of 18 against 0.28 s for a latent step, so 1000 pixel steps cost less than a hundred latent ones. The schedule is step 11's image-MAP one: lr 5×10^{-3}, cosine to zero, 1000 steps, betas $(0.9, 0.999)$, non-finite gradients zeroed and counted (none occurred). All 18 rows run as one batch (independent objectives, elementwise optimiser).

- **The problems are the stored ones.** `make_problem` gained an optional `y`: with it the operator is rebuilt from the step-14 spec at side N (the random mask from its `mask_seed`) and the stored measurement is used, so no noise is drawn. The script checks itself against `gallery_N.json`: the data terms of the stored images agree to 1.2×10^{-4} nats, J_z recomputed with $z = f_\theta(x_1)$ to 1.2×10^{-3} nats, and $\max|f_\theta(g_\theta(z_1)) - z_1| = 3.0 \times 10^{-4}$ (the fp32 round trip of the flow).

- **The control.** The same pixel-domain optimisation, same schedule, started from the prior samples $g_\theta(z_0)$ that seeded the latent restarts (z_0 regenerated from the step-14 seeds and pushed through the official reverse): step 11's pixel-only image MAP. The two variants differ only in their initialisation, so the value of the latent stage is read off directly. A sensitivity run repeats the latent-then-pixel variant at lr 10^{-3} and 2×10^{-2} (`lr_sweep`; the main result keeps the step-11 value, fixed before the sweep).

- **What is recorded.** Per system, restart and variant, at the initialisation and at the end: J_x, J_z (the latent objective evaluated at $z = f_\theta(x)$), the data term against its noise level, the log-determinant, $\|f_\theta(x)\|$ against $\sqrt{TD}$, PSNR, $\max|x|$; the restart spread (mean pairwise RMS); the per-step traces of J_x and its three terms; the truth's own values of everything. The figures show, per system, the lowest-J_x restart of each variant before and after stage 2,

and the traces of $J_x - J_x(\text{truth})$ and of the squared residual $\|y - Ax\|^2/(M\sigma^2)$ (1 at the noise level) for all restarts.

- **What step 11 predicts.** This experiment was requested and run in one go, so the expectations below are step 11's findings transcribed, not claims fixed in advance of the design. (a) The pixel stage will lower J_x *below the truth's* in every system, with $\|f_\theta(x)\| \ll \sqrt{TD}$ and the log-determinant above the truth's: the pixel-space mode is a density spike outside the typical set (step 11, claim iii, where the image MAPs sat 4500 nats below the truth at $\|z\| = 36$ against $\sqrt{n} = 64$). (b) The latent initialisation will reach lower J_x, better PSNR and a smaller restart spread than the pixel-only control from prior samples (step 11: the image restarts from $g_\theta(z_0)$ landed in many basins of nearly equal depth and different content, spread 0.208). (c) Whether stage 2 improves the *image* over stage 1 is the open question and the reason for the experiment: the latent MAP's known failure at 32 px is noise fitting (denoising, residual two thirds of σ^2, 3.5 dB lost; step 14), which a log-determinant term that penalises speckle might repair, or might over-repair into the over-smooth spike of (a).

16.2 Configuration (verbatim)

```
# step15_two_stage_map.py -- latent initialisation, pixel refinement. Stage 1 is the
# step-14 gallery (Adam on z, x = g(z), three restarts per system, J_z without the
# log-determinant); stage 2 continues from each restart's image with Adam on the pixels
# of x under the exact image-space objective J_x(x) = -log p(y|x) - log p_theta(x)
# (step 11's image MAP, one parallel forward per step). The control starts the same
# pixel-domain optimisation from the prior samples g(z_0) that seeded the latent
# restarts (step 11's pixel-only image MAP), so the two runs differ only in the
# initialisation. Problems (operators, measurements, test slices) are the stored
# step-14 ones; `--res N` picks the rung.

seed: 20260904
output_root: outputs/step15_two_stage_map
step14_config: configs/step14_lidc_gallery.yml     # priors, systems, seeds; solutions under its output_

pixel:                           # stage 2, both variants
  iters: 1000
  lr: 5.0e-3                     # the step-11 image-MAP schedule: Adam on the [-1, 1] pixels,
  lr_min: 0.0                    # cosine to zero over the run
  betas: [0.9, 0.999]
  log_every: 100
variants: [latent_then_pixel, pixel_only]
# sensitivity of the refined objective to the pixel learning rate (latent_then_pixel only;
# the main result above keeps the step-11 value, chosen before the sweep)
lr_sweep: [1.0e-3, 2.0e-2]
```

16.3 Results

Each line reports, per system, the lowest-J_x restart of the two-stage estimator (before → after stage 2) with the truth's value and the lowest-J_x restart of the pixel-only control; "data" is the data term at the end of stage 2.

32×32 (3 restarts; stage 1 1000 latent steps, stage 2 1000 pixel steps at lr 0.005 cosine, 32 s): denoise: J_x -2268 → -3441 (truth -2472; pixel-only -3439), data 162 (noise level 33; pixel-only 167), PSNR 23.3 → 22.7 dB (pixel-only 22.7), $\|f(x)\|$ 14 (truth 31; pixel-only 14), restart spread

0.001 (pixel-only 0.114); inpaint_box: J_x -4609 → -5228 (truth -4659; pixel-only -5031), data -1419 (noise level -1388; pixel-only -1360), PSNR 20.2 → 20.0 dB (pixel-only 19.0), $\|f(x)\|$ 19 (truth 27; pixel-only 21), restart spread 0.191 (pixel-only 0.178); inpaint_random: J_x -3249 → -3873 (truth -3168; pixel-only -3804), data -481 (noise level -481; pixel-only -481), PSNR 23.5 → 25.8 dB (pixel-only 24.9), $\|f(x)\|$ 17 (truth 31; pixel-only 18), restart spread 0.001 (pixel-only 0.187); sr_2x: J_x -3828 → -4146 (truth -3540; pixel-only -4103), data -381 (noise level -404; pixel-only -365), PSNR 27.8 → 27.6 dB (pixel-only 27.3), $\|f(x)\|$ 15 (truth 28; pixel-only 15), restart spread 0.022 (pixel-only 0.106); sr_4x: J_x -3514 → -3958 (truth -2903; pixel-only -3824), data -47 (noise level -101; pixel-only 60), PSNR 25.1 → 22.0 dB (pixel-only 18.4), $\|f(x)\|$ 13 (truth 29; pixel-only 14), restart spread 0.003 (pixel-only 0.228); ct: J_x -3025 → -3500 (truth -2942; pixel-only -3331), data 261 (noise level 273; pixel-only 340), PSNR 27.2 → 26.2 dB (pixel-only 22.6), $\|f(x)\|$ 14 (truth 27; pixel-only 19), restart spread 0.000 (pixel-only 0.250). 64×64 (3 restarts; stage 1 1000 latent steps, stage 2 1000 pixel steps at lr 0.005 cosine, 284 s): denoise: J_x -12202 → -15975 (truth -11874; pixel-only -13870), data 628 (noise level 134; pixel-only 3152), PSNR 26.4 → 23.1 dB (pixel-only 16.2), $\|f(x)\|$ 39 (truth 61; pixel-only 38), restart spread 0.058 (pixel-only 0.373); inpaint_box: J_x -20439 → -23354 (truth -20146; pixel-only -22467), data -5535 (noise level -5550; pixel-only -5442), PSNR 18.4 → 18.3 dB (pixel-only 17.6), $\|f(x)\|$ 45 (truth 52; pixel-only 46), restart spread 0.145 (pixel-only 0.359); inpaint_random: J_x -13939 → -17422 (truth -14256; pixel-only -15724), data -1647 (noise level -1865; pixel-only -1618), PSNR 23.9 → 26.1 dB (pixel-only 16.8), $\|f(x)\|$ 42 (truth 60; pixel-only 50), restart spread 0.064 (pixel-only 0.298); sr_2x: J_x -16431 → -18363 (truth -15075; pixel-only -17302), data -1187 (noise level -1615; pixel-only -1168), PSNR 27.3 → 27.6 dB (pixel-only 25.5), $\|f(x)\|$ 40 (truth 60; pixel-only 42), restart spread 0.035 (pixel-only 0.087); sr_4x: J_x -16002 → -18900 (truth -12821; pixel-only -18293), data -280 (noise level -404; pixel-only 279), PSNR 23.5 → 23.7 dB (pixel-only 19.4), $\|f(x)\|$ 39 (truth 57; pixel-only 39), restart spread 0.077 (pixel-only 0.283); ct: J_x -11639 → -15030 (truth -12126; pixel-only 35305067380736), data 2244 (noise level 2066; pixel-only 16241), PSNR 24.6 → 27.1 dB (pixel-only 14.7), $\|f(x)\|$ 39 (truth 50; pixel-only 8402984), restart spread 0.021 (pixel-only 0.372).

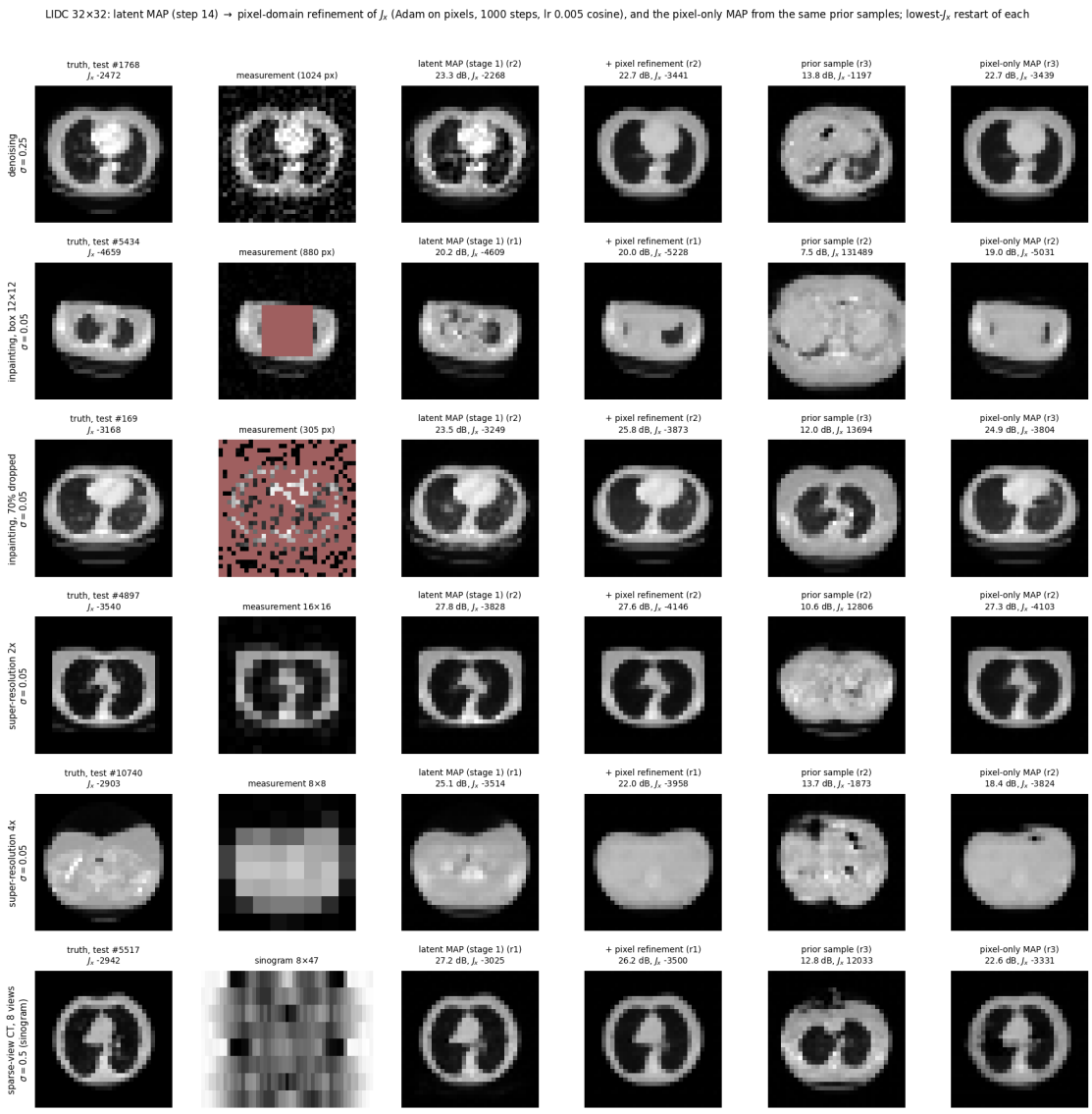


Figure 123: 32 px: truth, measurement, the latent MAP of step 14 and its pixel-domain refinement (lowest-J_x restart), and the prior sample that seeded that system's best pixel-only restart with its pixel-only MAP; PSNR and J_x in the titles.

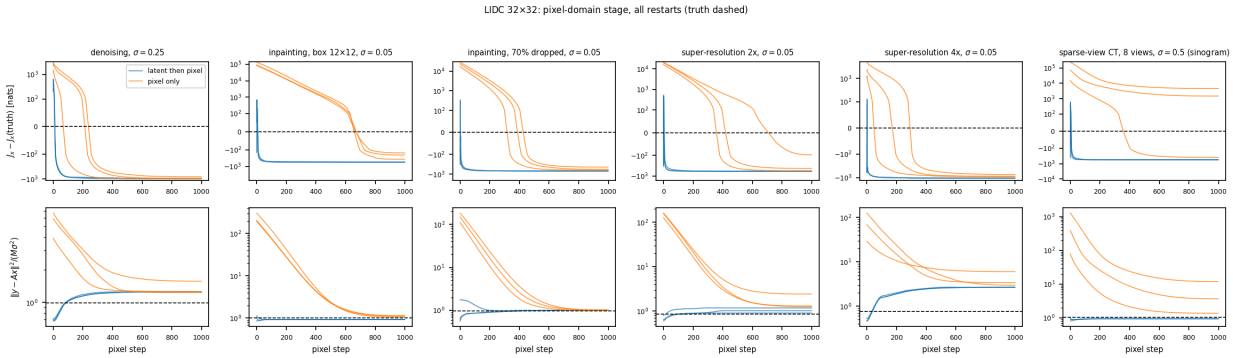


Figure 124: 32 px, the pixel-domain stage for all restarts: $J_x - J_x(\text{truth})$ (top, symmetric log) and the squared residual in units of $M\sigma^2$ (bottom, log; the dashed line is the truth's), latent initialisation in blue, prior-sample initialisation in orange.

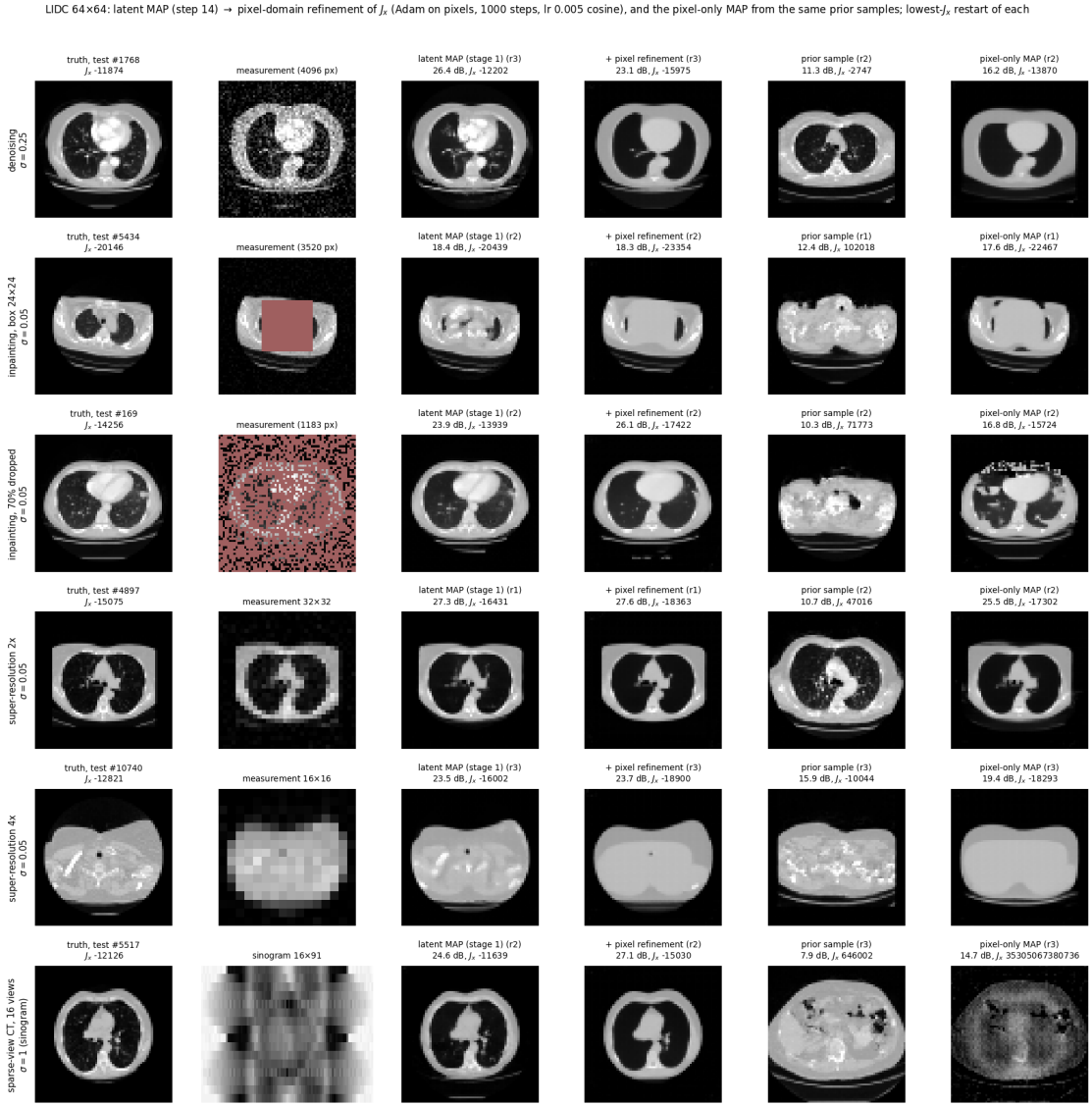


Figure 125: 64 px: the same, from the 64-px latent MAPs of step 14.

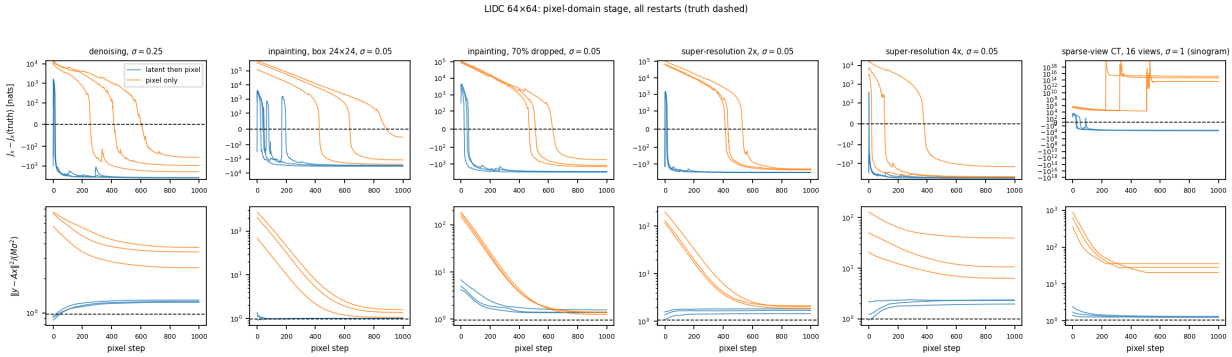


Figure 126: 64 px, the pixel-domain stage for all restarts.

16.4 Analysis

32 px ($T = 64$).

- **(a) holds, in every system.** Stage 2 ends 560–1060 nats below the truth's J_x: -3441 against -2472 (denoising), -5228 against -4659 (box), -3873 against -3168 (random inpainting), -4146 against -3540 (2× SR), -3958 against -2903 (4× SR), -3500 against -2942 (CT); the latent MAPs were already below the truth in five of the six (by 140–614 nats; denoising 204 above), and stage 2 takes off another 144–1173. $\|f_\theta(x)\|$ ends at 13–19 against the truth's 27–31 and $\sqrt{TD} = 32$. Where the nats come from is the log-determinant: in denoising it rises from 3204 at the latent MAP to 4640 (the truth's is 3920), while the data term *worsens* from -114 to $+162$ and J_z, the latent objective evaluated at the refined point, *rises* from 934 to 1198. The same in every system: the log-determinant at the refined points is 280–780 nats above the truth's (4640, 4935, 4478, 4813, 4937, 4801 against 3920, 4579, 4119, 4448, 4158, 4517) and J_z rises in four of six (denoising, random inpainting, both super-resolutions; it falls in box inpainting and CT). The refined point is a place where f_θ expands volume most — a small pixel neighbourhood mapped onto a large latent one — i.e. a spike of the density that carries little mass, and the images show what such a spike looks like for this prior: a smooth slice without lung texture (Figure 123, fourth column).

- **(c): stage 2 makes the image worse in four systems, better in one.** PSNR of the lowest-J_x restart, stage 1 → stage 2: 23.3 → 22.7 (denoising), 20.2 → 20.0 (box; the third restart 21.6 → 23.2, but its J_x is 11 nats higher), 23.5 → 25.8 (random inpainting), 27.8 → 27.6 (2× SR; the best latent restart 29.8 → 28.5), 25.1 → 22.0 (4× SR), 27.2 → 26.2 (CT). The residual explains the split. Stage 1 had fitted below the noise in five systems (squared residual 0.47–0.86 of $M\sigma^2$: the noise fitting of step 14; the box-inpainting restart sat above it at 1.13); stage 2 moves it up — to the truth's level where the data are informative (box 0.93, random inpainting 1.00, CT 0.94; truth 1.00, 0.97, 1.02) and *past* it where they are not (denoising 1.25, 2× SR 1.18, 4× SR 2.7; truth 0.99, 0.85, 0.75). In random inpainting the 305 observed pixels pin the image and the log-determinant only removes the speckle the latent MAP had put into the 719 missing ones: +2.3 dB. In denoising and 4× SR the log-determinant overrules the likelihood: the refined image fits y worse than the truth does, and loses 0.6 and 3.1 dB. The latent MAP's noise-fitting and the pixel MAP's over-smoothing are the two failures of the two objectives, in opposite directions, and neither is the posterior mean (step 10's, at ChestMNIST, beat both MAPs by five dB).

- **The window.** The latent MAPs run outside the model's $[-1, 1]$ range ($\max|x|$ 1.36–1.38 in denoising, 1.6 in one 2× SR restart); every refined image is within 1.02. The density falls steeply outside the range of the training images and the pixel objective sees that directly, while the latent objective sees it only through g_θ and the Gaussian term.

- **One basin from three starts.** In denoising, random inpainting, 4× SR and CT the three refined images are the same to 0.003 RMS or better, although they started from latent solutions 0.03–0.12 apart (step 14's spreads): J_x has one deep basin near the latent solutions in these systems. Box inpainting keeps three minima (spread 0.191, PSNR 19.4–23.2, J_x within 28 nats) — the hole content remains multimodal under both objectives — and 2× SR keeps a small spread (0.022, J_x within 23 nats).

- **(b) holds: the latent initialisation matters for the optimisation.** From the prior samples (J_x between -1873 and $+236{,}915$ at the start, data terms up to 2×10^5) the same

1000 steps end 2–200 nats above the two-stage values in five systems (denoising −3439, box −5031, random −3804, 2× SR −4103, 4× SR −3824) with the best PSNRs 0.0–3.6 dB lower (22.7, 19.0, 24.9, 27.3, 18.4) and spreads 0.11–0.25 against ≤ 0.02 for the four single-basin systems; in CT two of the three pixel-only restarts fail outright ($J_x = -1453$ and $+1626$, data terms 773 and 2313 against a noise level of 273, $\max|x|$ 1.9) and the third reaches −3331 at 22.6 dB against −3500 and 26.2 dB. The traces (Figure 124) show the pixel-only runs still descending in box inpainting and CT when the learning rate reaches zero: they are limited by the schedule, the two-stage runs are converged by step 400. The ordering by image quality is therefore latent MAP $\geq$ latent-then-pixel $>$ pixel-only in five systems, with random inpainting the exception where the middle one wins.

- **Learning rate.** At 2×10^{-2} the refined J_x agrees with the 5×10^{-3} run to within 15 nats in every system (lower by 4 in box inpainting, higher by 14 in 2× SR); at 10^{-3} it is short by 0–80 nats and the images are 0–2 dB better — because they are less converged, i.e. nearer the latent MAP, not because the optimiser found a better minimum. The main run is converged.

- **What it means.** The two-stage procedure is the best optimiser of J_x in this paper — the lowest values from any start, all restarts agreeing, 32 s — and what it shows is that minimising J_x is not what one wants: the density in pixel coordinates has spikes at smooth images that are 280–780 nats more probable than any real slice, and a good optimiser finds them. The latent MAP is the better estimator at 32 px not because it optimises better but because its objective, lacking the log-determinant, does not see the spikes — and it has its own opposite failure. A tempered log-determinant, or the refined point as the starting point of a sampler or of the step-10 posterior mean, would be new estimators, to be claimed before they are run.

64 px ($T = 256$).

- **(a) holds again, four times larger.** Stage 2 ends 2,900–6,100 nats below the truth's J_x: −15,975 against −11,874 (denoising), −23,354 against −20,146 (box), −17,422 against −14,256 (random inpainting), −18,363 against −15,075 (2× SR), −18,900 against −12,821 (4× SR), −15,030 against −12,126 (CT). The log-determinant at the refined points is 2,500–5,400 nats above the truth's (21,136, 22,588, 20,431, 21,720, 23,139, 21,815 against 17,565, 19,667, 17,906, 19,043, 17,784, 19,247) and $\|f_\theta(x)\|$ ends at 39–45 against the truth's 50–61 and $\sqrt{TD} = 64$. The typical-set gap scales with n ($n/2 = 2048$ nats here against 512 at 32 px) and so does the spike's depth. Every refined image is within $\max|x| \leq 1.21$ (latent MAPs up to 2.36).

- **(c) reverses: stage 2 improves the image in four systems and hurts one.** PSNR of the lowest-J_x restart, stage 1 → stage 2: 26.4 → 23.1 (denoising; every restart loses 2.5–3.3 dB), 18.4 → 18.3 (box), 23.9 → 26.1 (random inpainting), 27.3 → 27.6 (2× SR), 23.5 → 23.7 (4× SR; the image is a featureless lung outline at both stages), 24.6 → 27.1 (CT). This is not the 32-px picture, and the residuals say why. At 32 px stage 1 had over-fitted the data in five systems; at 64 px the latent MAPs of step 14 (1000 Adam steps at $T = 256$) are *under*-fitted in three: their squared residuals are 4.2 (random inpainting), 1.6 (2× SR) and 1.7 (CT) times $M\sigma^2$ against the truth's 0.96, 1.07 and 1.04, i.e. stage 1 had not converged, and stage 2 finishes its job — the residual falls to 1.37, 1.83 and 1.25 and J_z, the latent objective, *falls* too (random inpainting 4435 → 3008, CT 7592 → 6785). So the two systems with the largest gains (+2.2 and +2.5 dB) are the ones where the pixel stage is partly a better optimiser of the *latent* objective, not evidence that the log-determinant helps. Where stage

1 was converged the 32-px pattern returns: denoising (stage-1 residual 0.92) goes to 1.24 and loses 3.3 dB with the log-determinant up 3,570 nats and the data term up from −33 to +628 against a noise level of 134; 4× SR (0.95) goes to 1.97 and gains nothing.

- **Basins.** The refined restarts agree less than at 32 px: spreads 0.021 (CT), 0.035 (2× SR), 0.058 (denoising), 0.064 (random inpainting), 0.077 (4× SR), 0.145 (box; PSNR 18.3–18.9), with J_x within 110–460 nats across restarts. With four times the dimensions the pixel objective keeps more than one spike near the latent solutions.

- **(b) holds, more strongly.** The pixel-only control from prior samples ends 600–2,100 nats above the two-stage values in five systems (denoising −13,870, box −22,467, random −15,724, 2× SR −17,302, 4× SR −18,293) with the lowest-J_x restart 0.7–9.3 dB worse (16.2, 17.6, 16.8, 25.5, 19.4) and spreads 0.09–0.37; in denoising its residual is 2.47 $M\sigma^2$ and the image is a blur (Figure 125, last column). In CT all three pixel-only restarts diverge: from J_x between 2.5×10^5 and 6.5×10^5 at the start (data terms $2.6–6.6 \times 10^5$) the lowest value reached along the way is +11,725, and the runs end at J_x 3.5×10^{13} to 1.3×10^{15} with $\|f_\theta(x)\|$ of 10^7, $\max|x|$ 1.7 and 13–15 dB: the step-11 image MAP from a prior sample does not work for CT at 64 px at this learning rate. The ordering by image quality is latent-then-pixel ≥ latent MAP > pixel-only in five systems, latent MAP first in denoising.

- **Learning rate.** 2×10^{-2} is unstable at 64 px: 12 of the 18 rows diverge (J_x up to 10^{25}), one ends at +47,000, and the five that survive end 1,500–4,600 nats above the main run. 10^{-3} is short by 60–2,400 nats (most in random inpainting) and, being nearer the latent MAP, is 1.4–2.2 dB better in denoising and 1.4–3.5 dB worse in random inpainting. The main run at 5×10^{-3} is converged to within 20–95 nats over its last 400 steps (Figure 126) and is the one reported.

- **What changes with n.** The spike is deeper (the log-determinant excess grows from 280–780 to 2,500–5,400 nats) but its image is no worse relative to the latent MAP than at 32 px, because at 64 px the latent MAP is the weaker optimiser: 1000 latent steps at $T = 256$ leave three systems under-fitted, and the cheap pixel stage (0.28 s per step for the batch of 18) recovers what stage 1 left. A fair comparison of the two objectives at 64 px needs a converged stage 1 (more latent steps, or a smaller learning-rate floor) — a change to step 14's configuration, to be claimed before it is run. What holds at both resolutions is the denoising case, the one system where stage 1 is converged and the data are least informative: there the log-determinant costs 0.6 dB at 32 px and 3.3 dB at 64 px, growing with n as the typical-set argument says it should.

17 Step 16: Random walks on the posterior, in latent coordinates

17.1 Methods

- Code: `step16_latent_langevin.py` (re-uses `step14.make_problem` with the stored measurements, `step13.load_prior`, `invlib.FastReverse`); configs: `configs/step16_latent_langevin.yml` (the main run) and `configs/step16_latent_langevin_stiff.yml` (the follow-up, tag `_stiff`); outputs: `outputs/step16_latent_langevin/{data,figures}/` with `langevin_N.json`, `langevin_N_frames.npz` (every recorded state and the per-iteration traces), `langevin_N_solutions.pt`, the stills `langevin_N.png`, `langevin_N_variants.png`,

`langevin_N_traces.png` and one video per system, `langevin_N_{system}.mp4`. Added on 2026-09-04 at Matt's request ("instead of MAP, generate mp4 videos of a random walk on the posterior for each measurement model, in the latent domain only"), run at 32 px while the 128-px prior trained.

- **The target.** In latent coordinates the posterior is

$$p(z \mid y) \;\propto\; p\big(y \mid g_\theta(z)\big)\,\mathcal{N}(z; 0, I)$$

exactly: $x = g_\theta(z)$ is a bijection, so the change of variables from $p(x \mid y)$ to z multiplies by $|\det \partial g_\theta/\partial z|$, and the prior density $p_\theta(g_\theta(z)) = \mathcal{N}(z)\,|\det \partial f_\theta/\partial x|$ divides by the same factor. No log-determinant appears — the latent objective J_z of step 14, which is *not* a density in x, is the exact negative log-density of the posterior in z, and a Markov chain with that target, pushed through g_θ, samples $p(x \mid y)$. The two MAP failures of steps 14 and 15 (the latent MAP's noise fitting, the pixel MAP's density spikes) are properties of modes; samples from the same posterior have neither by construction, which is the hypothesis Matt put forward: "MAP for a Gaussian is also the mean of the distribution, so maybe that's why it's coming out blurry — it's averaging all possible things". The experiment shows what the samples look like, and what their running mean looks like next to them.

- **The sampler.** Hamiltonian Monte Carlo on z with the energy

$$U(z) \;=\; \Big[-\log p\big(y \mid g_\theta(z)\big) + \tfrac{\|z\|^2}{2\tau^2}\Big]/T,$$

$T = \tau = 1$ the posterior itself, $\tau < 1$ a cooled prior (the latent truncation of step 10's samplers, as a target rather than a post-hoc clamp), $T < 1$ the tempered posterior. Each iteration draws a momentum $p \sim \mathcal{N}(0, I)$, integrates L leapfrog steps of size ε and accepts the end point with probability $\min(1, e^{H_0 - H_1})$; a non-finite trajectory is rejected. ε is per chain, scaled by $U(0.8, 1.2)$ every iteration, and adapted during the first part of the burn-in by $\log \varepsilon \mathrel{+}= r_k\,(\alpha_k - 0.65)$ with α_k the acceptance probability and r_k decaying linearly from 0.2 to 0.05, then frozen. $L = 1$ would be MALA. The gradient of U is one differentiable reverse (`invlib.FastReverse`, the compiled per-token CUDA graphs of step 11) and one forward of the measurement model; all systems × variants × chains run as one batch.

- **Deviation, recorded: MALA first.** The first pilot (200 steps, batch 90, five variants including chains started from a prior sample) used MALA. Its step size adapted down to $h = 6 \times 10^{-5}$ with acceptance still ≈ 0 for denoising, both inpaintings and 2× super-resolution; only CT and 4× SR reached 0.3–0.4; from a prior sample nothing moved at all ($U \sim 10^4$–10^5, every proposal rejected). The posterior in z is stiff along the measured directions (curvature $\sim s_i^2/\sigma^2$ for a Jacobian singular value s_i of g_θ) and unit-curvature along the rest, and MALA's cost per independent sample grows with that condition number where HMC's grows with its square root. Hence HMC, and no prior-sample variant. The pilot's log and json are kept under `outputs/step16_latent_langevin/pilot_mala/`; a display bug of the pilot (the residual column divided by σ^2 twice) affected only the printed residuals, not the objective.

- **Initialisations and variants (the operating-point sweep).** Chains start from the step-14 latent MAP (the lowest-J_z restart, read from `gallery_N_solutions.pt`) or from a latent maximum-likelihood point: Adam on the data term alone, 300 steps at lr 0.02, from the MAP, which walks out along the measured directions until it over-fits the noise (residuals 0.15–0.69 $M\sigma^2$) and lands on the typical shell ($\|z\|$ 24–34 against $\sqrt{n} = 32$). The main run:

three variants × six systems × three chains = 54 rows, {posterior from the MAP, posterior from the MLE, cooled prior $\tau = 0.7$ from the MAP}, 160 iterations of $L = 16$ leapfrog steps, burn-in 60 (adaptation 50), $\varepsilon_0 = 10^{-2}$. The follow-up (`_stiff`): the four stiff systems only, from the MAP and the MLE, two chains, 30 iterations of $L = 128$, burn-in 10 (adaptation 8), $\varepsilon_0 = 2.5 \times 10^{-3}$ (where the main run's adaptation ended for those systems) — 3,840 gradients on a batch of 16, sized to finish in about 40 minutes next to the 128-px training. Chains of one (system, variant) share the initial state and differ in their noise.

- **What is recorded.** After every iteration: U, the data term, $\|z\|$, PSNR of $g_\theta(z)$, acceptance and its probability, ε, and the image itself (fp16) for every row. Per (system, variant): acceptance in burn-in and sampling, U against $U(f_\theta(x_{\text{true}}))$, the residual $\|y - Ax\|^2/(M\sigma^2)$, $\|z\|$, PSNR of the last state of each chain, of the samples on average and of the posterior mean over the sampling phase (all chains pooled), the pixel-wise posterior std, the distance moved from the initialisation and the spread of the chains. The videos show, per system, the three chains of each variant, the running posterior mean (sampling phase) with its PSNR, and the running std, one frame per iteration.

- **Claims, made before the run.** (i) From the latent MAP the chains leave it: $\|z\|$ grows from the MAP's 6–25 towards $\sqrt{n} = 32$, the residual rises from the MAP's noise-fitted 0.5–0.9 $M\sigma^2$ towards 1, and the samples show lung texture that the MAP lacks. (ii) The running posterior mean is smoother than any sample and its PSNR is higher than the samples' and than the MAP's — the MMSE estimator, which is what Matt's "it's averaging all possible things" describes; whether it is blurrier *than the MAP* is the question. (iii) The stiff systems mix slowly at $L = 16$ (the chain moves $L\varepsilon \approx 0.05$ per iteration along the unmeasured directions and must cross ~ 1); the $L = 128$ follow-up moves them further. (iv) The MLE start is already on the shell and its chains reach a stable $\|z\|$ sooner; from the two starts the sampling-phase means agree where the chains mix and differ where they do not. (v) $\tau = 0.7$ pulls $\|z\|$ to $\approx 0.7\sqrt{n}$ and gives smoother samples with higher PSNR than $\tau = 1$ — the usual truncation trade-off, with no guarantee about which the eye prefers.

17.2 Configuration (verbatim)

```
# step16_latent_langevin.py -- random walks on the posterior, in latent space only.
# The target is p(z | y) \propto p(y | g(z)) N(z; 0, I) (exact: x = g(z) is a bijection,
# so the Jacobian cancels and no log-determinant appears). The chain is Hamiltonian
# Monte Carlo on z -- ‘leapfrog‘ steps of size eps per iteration, eps adapted per chain
# during the burn-in towards ‘target_accept‘ and frozen afterwards; ‘leapfrog: 1‘ is
# MALA -- and the state x = g(z) is recorded after every iteration for one mp4 per
# system. The problems (operators, measurements, test slices) are the stored step-14
# ones; ‘--res N‘ picks the rung.
#
# Operating-point sweep (all in one batch: systems x variants x chains): the exact
# posterior from two initialisations -- the step-14 latent MAP (lowest-J_z restart) and
# a latent maximum-likelihood point (Adam on the data term alone, started from the
# MAP) -- and a cooled target from the MAP, prior temperature tau < 1
# (||z||^2 / (2 tau^2), latent truncation); an overall temperature T < 1 (U / T, the
# tempered posterior) is also available. T = tau = 1 is the posterior itself.
#
# Deviation, recorded. The first pilot (200 steps, batch 90) used MALA with five
# variants including chains started from a prior sample: the step size adapted down to
# 6e-5 with acceptance still ~0 for denoising, inpainting and 2x super-resolution (the
```

```
# posterior in z is stiff along the measured directions, wide along the rest), only CT
# and 4x super-resolution reached acceptance 0.3-0.4, and from a prior sample nothing
# moved at all (U ~ 1e4-1e5, every proposal rejected). Hence HMC (its cost per
# independent sample scales with the square root of the condition number, MALA's with
# the condition number), and no prior-sample variant. The pilot's log is kept under
# outputs/step16_latent_langevin/pilot_mala/.

seed: 20260904
output_root: outputs/step16_latent_langevin
step14_config: configs/step14_lidc_gallery.yml     # priors, systems, seeds; solutions under its output_

chains: 3                       # independent chains per (system, variant); same init, different noise
iterations: 160                 # HMC iterations (each `leapfrog` gradient evaluations)
leapfrog: 16                    # leapfrog steps per iteration (1 = MALA)
burn_in: 60                     # iterations before this are not used for the posterior mean/std
adapt_iters: 50                 # step-size adaptation window (within the burn-in)
target_accept: 0.65             # HMC's usual target (0.574 for MALA)
eps0: 1.0e-2                    # initial leapfrog step size (the MALA pilot's h = eps^2/2 ~ 6e-5)
adapt_rate: 0.2                 # log eps += rate * (acceptance probability - target) per iteration;
                                # the rate decays linearly to a quarter over adapt_iters
eps_jitter: 0.2                 # eps scaled by U(1 - j, 1 + j) each iteration (breaks periodic orbits)

mle:                            # the latent maximum-likelihood initialisation
  steps: 300                    # Adam on the data term alone, from the latent MAP
  lr: 2.0e-2

variants:
  - {name: posterior_from_map, init: map, temperature: 1.0, prior_scale: 1.0}
  - {name: posterior_from_mle, init: mle, temperature: 1.0, prior_scale: 1.0}
  - {name: cool_prior_0.7,     init: map, temperature: 1.0, prior_scale: 0.7}

video:
  fps: 8
  dpi: 100

# step16_latent_langevin.py, second configuration: long trajectories for the stiff systems.
# The main run (configs/step16_latent_langevin.yml, 16 leapfrog steps) adapted the step
# size to 1.5e-3 -- 4e-3 for denoising, both inpaintings and 2x super-resolution (the
# measured directions are stiff), so one iteration moves the chain by L*eps ~ 0.05 along
# the unmeasured, prior-scale directions and 100 iterations diffuse ~0.5 of the unit they
# must cross: those chains explore the neighbourhood of the MAP, not the posterior. Same
# sampler, same target, eight times longer trajectories (128 leapfrog steps, ~0.3 per
# iteration), fewer iterations; only those four systems, from the latent MAP and from the
# latent MLE. Output files carry the tag `_stiff`. Trimmed to two chains and 30 iterations
# (3,840 gradients, batch 16) so that it finishes in ~40 min while sharing the GPU with the
# 128-px training; the adaptation starts where the main run's ended, so ten iterations of
# burn-in are enough to settle the step size.

seed: 20260904
output_root: outputs/step16_latent_langevin
step14_config: configs/step14_lidc_gallery.yml
tag: _stiff
systems: [denoise, inpaint_box, inpaint_random, sr_2x]
```

```
chains: 2
iterations: 30
leapfrog: 128
burn_in: 10
adapt_iters: 8
target_accept: 0.65
eps0: 2.5e-3                    # where the main run's adaptation ended for these systems
adapt_rate: 0.2
eps_jitter: 0.2

mle:
  steps: 300
  lr: 2.0e-2

variants:
  - {name: posterior_from_map, init: map, temperature: 1.0, prior_scale: 1.0}
  - {name: posterior_from_mle, init: mle, temperature: 1.0, prior_scale: 1.0}

video:
  fps: 4
  dpi: 100

# step16_latent_langevin.py, third configuration: the two inpaintings to convergence.
# The '_stiff' run (128 leapfrog steps, 30 iterations) brought denoising and 2x
# super-resolution to the shell but left both inpaintings climbing: box from ||z|| 20 to
# 26--28, random from 16 to 24--26, both still rising linearly at iteration 30, and the
# random chains at acceptance 0.38--0.45 because the adaptation froze after 8 iterations
# before the step size had come down. Same sampler, same target, same trajectories, three
# times the iterations, a longer adaptation, and a smaller initial step for the random
# chains' sake; only the two inpaintings, from the latent MAP and from the latent MLE.
# Output files carry the tag '_inpaint'. Batch 8 (2 systems x 2 variants x 2 chains),
# 11,520 gradients at ~0.45 s each while sharing the GPU with the 128-px training.

seed: 20260904
output_root: outputs/step16_latent_langevin
step14_config: configs/step14_lidc_gallery.yml
tag: _inpaint
systems: [inpaint_box, inpaint_random]

chains: 2
iterations: 90
leapfrog: 128
burn_in: 30
adapt_iters: 25
target_accept: 0.65
eps0: 1.5e-3                    # between the box (1.4e-3) and random (1.8e-3) values of the _stiff run
adapt_rate: 0.2
eps_jitter: 0.2

mle:
  steps: 300
  lr: 2.0e-2

variants:
```

```
  - {name: posterior_from_map, init: map, temperature: 1.0, prior_scale: 1.0}
  - {name: posterior_from_mle, init: mle, temperature: 1.0, prior_scale: 1.0}

video:
  fps: 8
  dpi: 100

# step16_latent_langevin.py, fourth configuration: one random walk per system, quickly.
# Matt: "a single random walk with 32 samples in an animation as frames for each system;
# the priority is a fast result; 32, 64 and 128 px; denoising, 4x super-resolution, CT".
# One chain from the latent MAP, no variants, 64 leapfrog steps per iteration (a
# trajectory of L*eps ~ 0.2 per iteration for denoising, ~1 for CT and 4x SR), 24
# iterations of burn-in and 32 recorded samples; the videos show the 32 samples only.
# Batch 3: 3,584 gradients, ~0.2-0.35 s each at 32 px on a shared GPU (12-20 min); at
# 64 and 128 px the reverse is the in-place-cache one of step 14 (‘reverse: fast2‘; the
# 32-px run used the first reverse), ~2 s and ~12 s per gradient.
# Output files carry the tag ‘_walk‘.

seed: 20260904
output_root: outputs/step16_latent_langevin
step14_config: configs/step14_lidc_gallery.yml
tag: _walk
systems: [denoise, sr_4x, ct]
reverse: fast2
chunk: 128

chains: 1
iterations: 56
leapfrog: 64
burn_in: 24
adapt_iters: 20
target_accept: 0.65
eps0: 5.0e-3                    # between the 32-px main run's denoising (3e-3) and CT (1.5e-2) values
adapt_rate: 0.2
eps_jitter: 0.2

variants:
  - {name: posterior_from_map, init: map, temperature: 1.0, prior_scale: 1.0}

video:
  fps: 4
  dpi: 100
  sampling_only: true
```

17.3 Results

Each line reports, per system and variant, the sampling-phase acceptance, $\|z\|$ of the final states, the PSNR of the samples on average and of the posterior mean.
`langevin_32`: 32×32 ($n = 1024$, $\sqrt{n} = 32$; batch 54, 3 chains, 160 HMC iterations × 16 leapfrog steps, burn-in 60, 1.21 s per gradient, 52 min).
denoise (#1768): posterior_from_map: acc 0.90, $\|z\|$ 23, samples 23.3 dB, mean 25.0 dB; posterior_from_mle: acc 0.78, $\|z\|$ 33, samples 22.4 dB, mean 24.4 dB; cool_prior_0.7: acc 0.74, $\|z\|$ 20, samples 23.4 dB, mean 24.9 dB

inpaint_box (#5434): posterior_from_map: acc 0.56, $\|z\|$ 21, samples 21.1 dB, mean 21.2 dB; posterior_from_mle: acc 0.43, $\|z\|$ 33, samples 21.3 dB, mean 21.7 dB; cool_prior_0.7: acc 0.62, $\|z\|$ 21, samples 21.6 dB, mean 21.7 dB
inpaint_random (#169): posterior_from_map: acc 0.42, $\|z\|$ 18, samples 23.4 dB, mean 23.6 dB; posterior_from_mle: acc 0.53, $\|z\|$ 25, samples 23.7 dB, mean 23.9 dB; cool_prior_0.7: acc 0.50, $\|z\|$ 18, samples 23.5 dB, mean 23.7 dB
sr_2x (#4897): posterior_from_map: acc 0.63, $\|z\|$ 20, samples 25.9 dB, mean 26.8 dB; posterior_from_mle: acc 0.67, $\|z\|$ 33, samples 24.8 dB, mean 25.6 dB; cool_prior_0.7: acc 0.53, $\|z\|$ 17, samples 25.6 dB, mean 26.1 dB
sr_4x (#10740): posterior_from_map: acc 0.71, $\|z\|$ 30, samples 23.4 dB, mean 25.6 dB; posterior_from_mle: acc 0.56, $\|z\|$ 31, samples 23.0 dB, mean 24.9 dB; cool_prior_0.7: acc 0.61, $\|z\|$ 22, samples 24.2 dB, mean 25.5 dB
ct (#5517): posterior_from_map: acc 0.55, $\|z\|$ 32, samples 25.3 dB, mean 27.0 dB; posterior_from_mle: acc 0.70, $\|z\|$ 33, samples 25.1 dB, mean 27.0 dB; cool_prior_0.7: acc 0.41, $\|z\|$ 24, samples 26.2 dB, mean 27.6 dB.
`langevin_32_inpaint`: 32×32 ($n = 1024$, $\sqrt{n} = 32$; batch 8, 2 chains, 90 HMC iterations × 128 leapfrog steps, burn-in 30, 0.43 s per gradient, 82 min).
inpaint_box (#5434): posterior_from_map: acc 0.96, $\|z\|$ 33, samples 22.1 dB, mean 23.8 dB; posterior_from_mle: acc 0.93, $\|z\|$ 34, samples 21.4 dB, mean 23.5 dB
inpaint_random (#169): posterior_from_map: acc 0.25, $\|z\|$ 25, samples 24.1 dB, mean 24.9 dB; posterior_from_mle: acc 0.57, $\|z\|$ 30, samples 24.3 dB, mean 25.3 dB.
`langevin_32_stiff`: 32×32 ($n = 1024$, $\sqrt{n} = 32$; batch 16, 2 chains, 30 HMC iterations × 128 leapfrog steps, burn-in 10, 0.54 s per gradient, 35 min).
denoise (#1768): posterior_from_map: acc 0.80, $\|z\|$ 32, samples 23.9 dB, mean 26.1 dB; posterior_from_mle: acc 0.95, $\|z\|$ 32, samples 24.3 dB, mean 26.7 dB
inpaint_box (#5434): posterior_from_map: acc 0.77, $\|z\|$ 27, samples 22.1 dB, mean 22.7 dB; posterior_from_mle: acc 0.57, $\|z\|$ 33, samples 21.4 dB, mean 22.0 dB
inpaint_random (#169): posterior_from_map: acc 0.38, $\|z\|$ 25, samples 23.6 dB, mean 24.3 dB; posterior_from_mle: acc 0.40, $\|z\|$ 28, samples 23.4 dB, mean 23.9 dB
sr_2x (#4897): posterior_from_map: acc 0.80, $\|z\|$ 32, samples 26.8 dB, mean 29.0 dB; posterior_from_mle: acc 0.45, $\|z\|$ 33, samples 25.1 dB, mean 26.6 dB.
`langevin_32_walk`: 32×32 ($n = 1024$, $\sqrt{n} = 32$; batch 3, 1 chains, 56 HMC iterations × 64 leapfrog steps, burn-in 24, 0.40 s per gradient, 24 min).
denoise (#1768): posterior_from_map: acc 0.97, $\|z\|$ 31, samples 24.0 dB, mean 25.2 dB
sr_4x (#10740): posterior_from_map: acc 0.91, $\|z\|$ 33, samples 22.8 dB, mean 24.5 dB
ct (#5517): posterior_from_map: acc 0.97, $\|z\|$ 32, samples 25.1 dB, mean 27.0 dB.

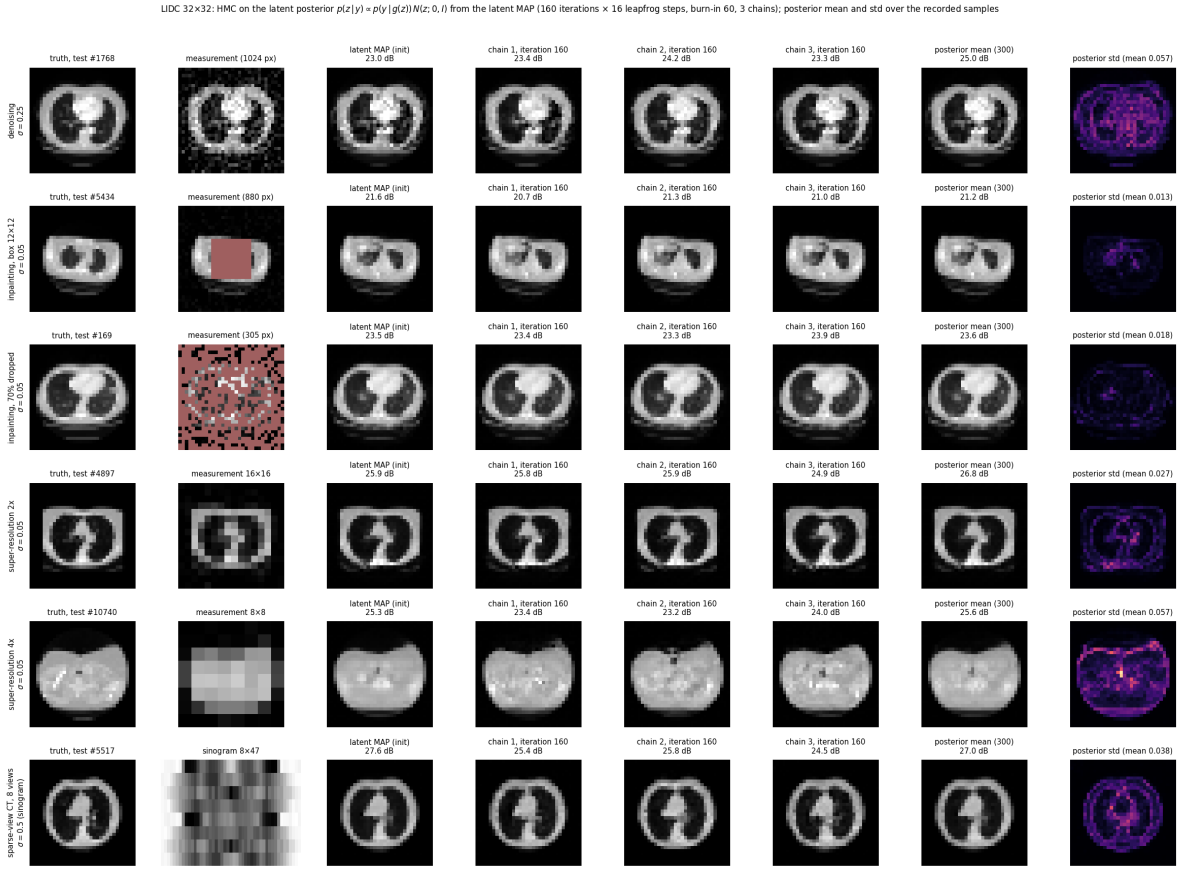


Figure 127: 32 px, the posterior from the latent MAP ($L = 16$): truth, measurement, the initial state, the last state of the three chains, the posterior mean and std over the sampling phase.

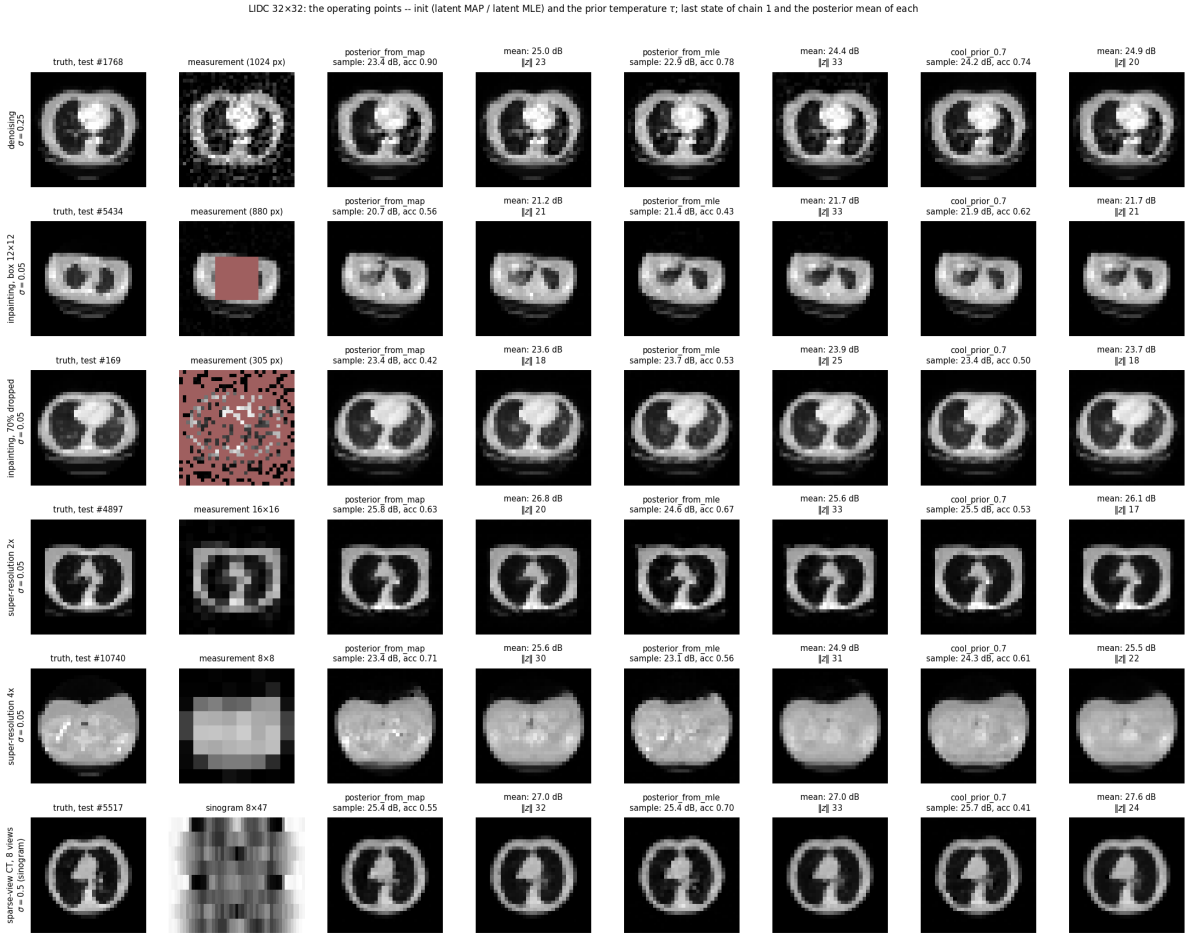


Figure 128: 32 px, the operating points: for each variant the last state of chain 1 and the posterior mean.

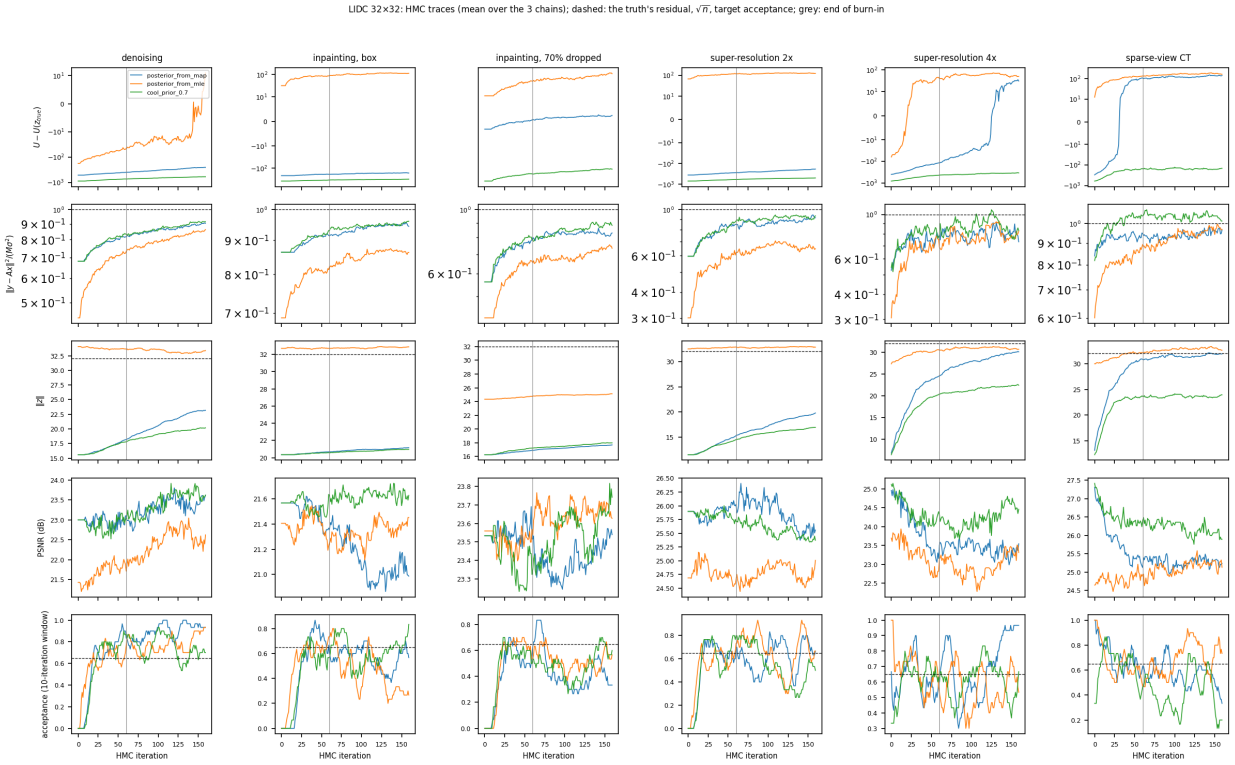


Figure 129: 32 px, traces (mean over chains): $U - U(z_{\text{true}})$, the residual in units of $M\sigma^2$, $\|z\|$, PSNR and the windowed acceptance.

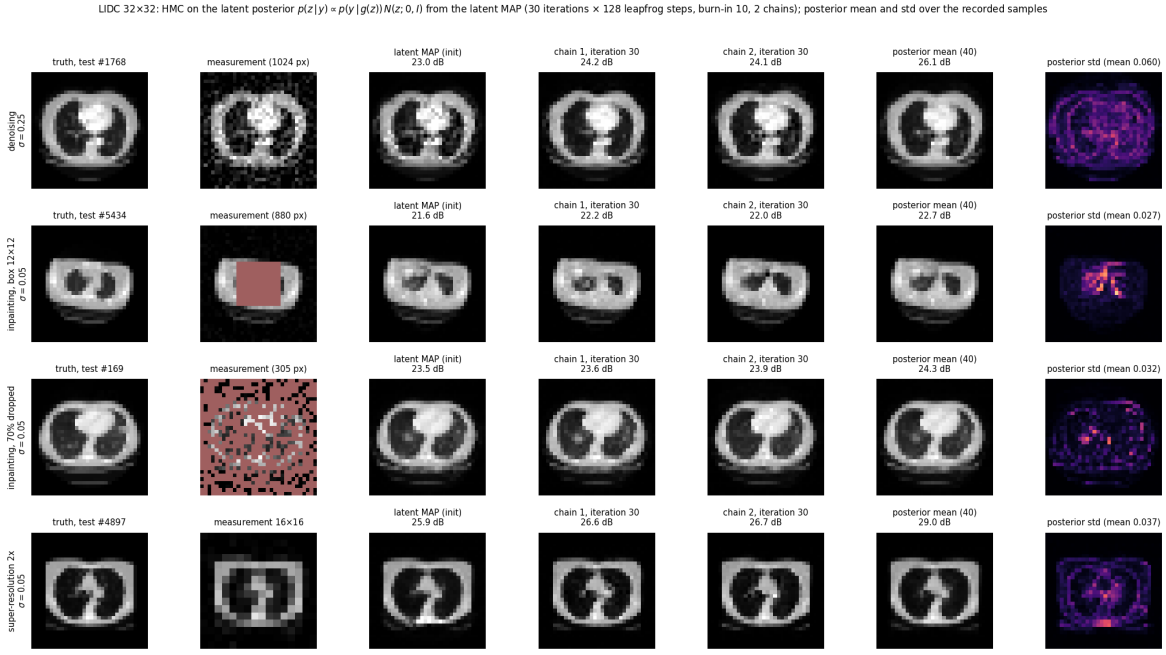


Figure 130: 32 px, the stiff systems with $L = 128$ leapfrog steps per iteration.

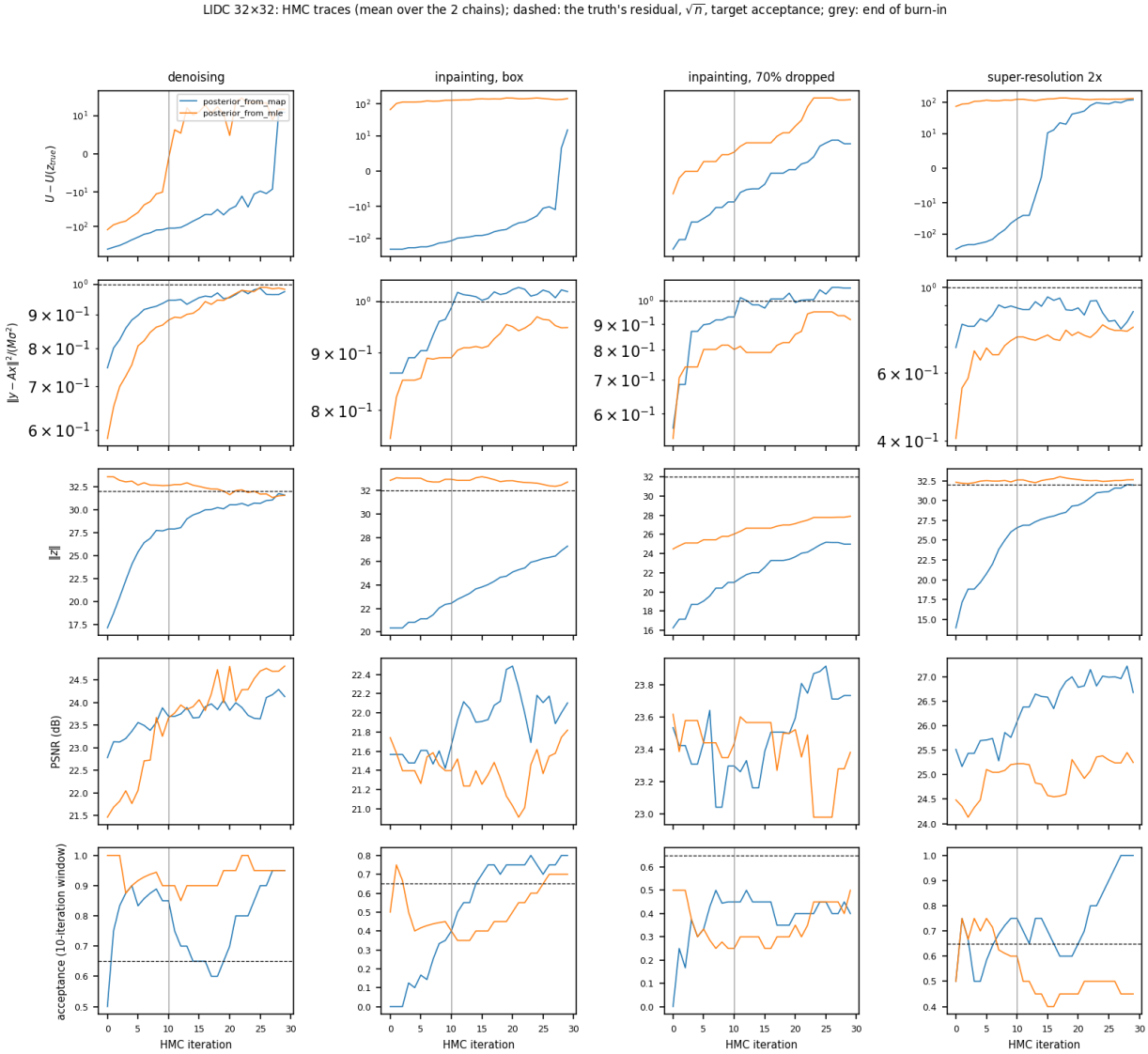


Figure 131: 32 px, traces of the $L = 128$ run.

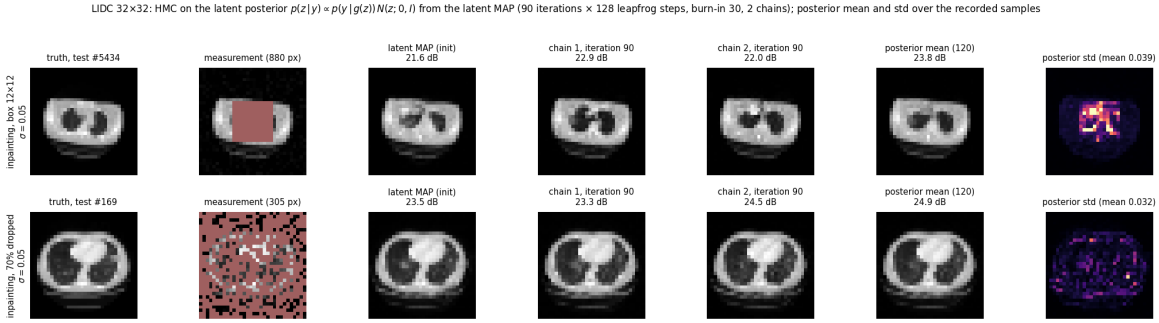
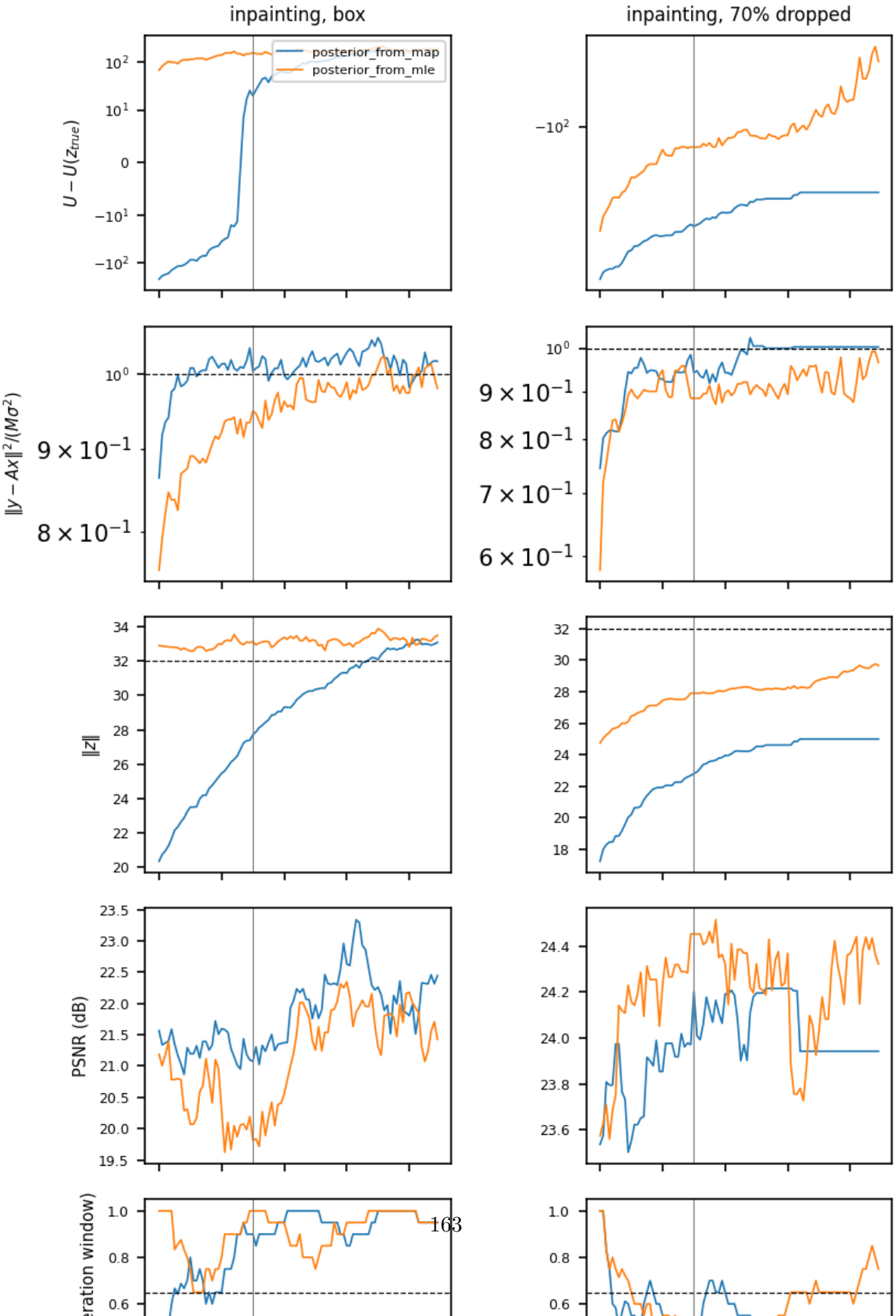


Figure 132: 32 px, the two inpaintings to convergence ($L = 128$, 90 iterations, two chains from the latent MAP): truth, measurement, the initial state, the two last states, the posterior mean and std.

2: HMC traces (mean over the 2 chains); dashed: the truth's residual, $\sqrt{n}$, target acceptance; grey: enc



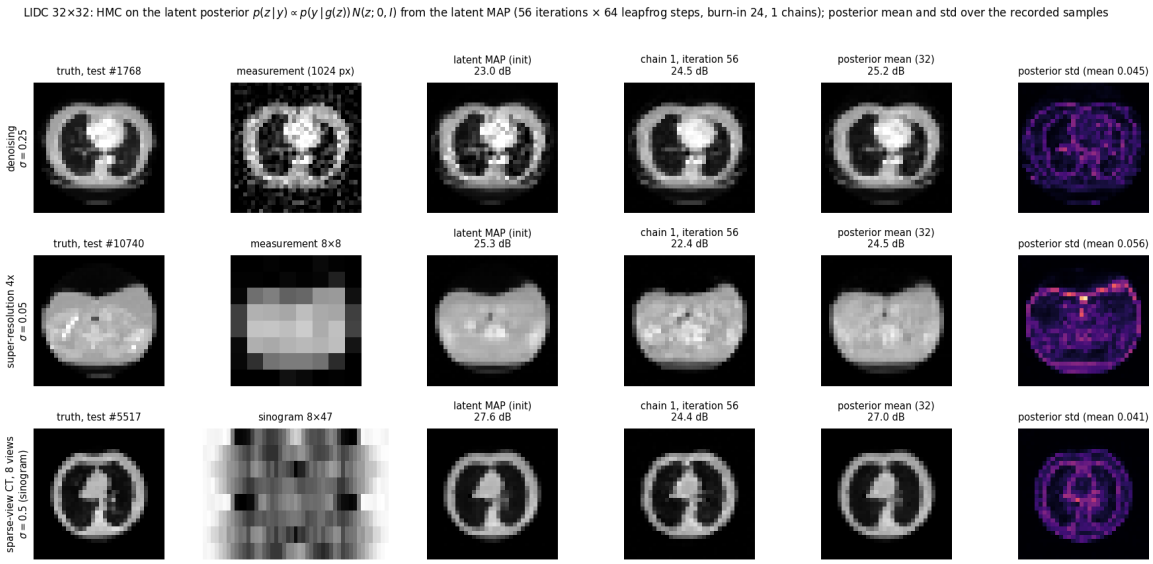


Figure 134: 32 px, one random walk per system ($L = 64$, 24 burn-in iterations, 32 recorded samples, from the latent MAP): truth, measurement, the initial state, the last state, the mean and std of the 32 samples. The 32 samples are the frames of `langevin_32_walk_{denoise,sr_4x,ct}.mp4`.

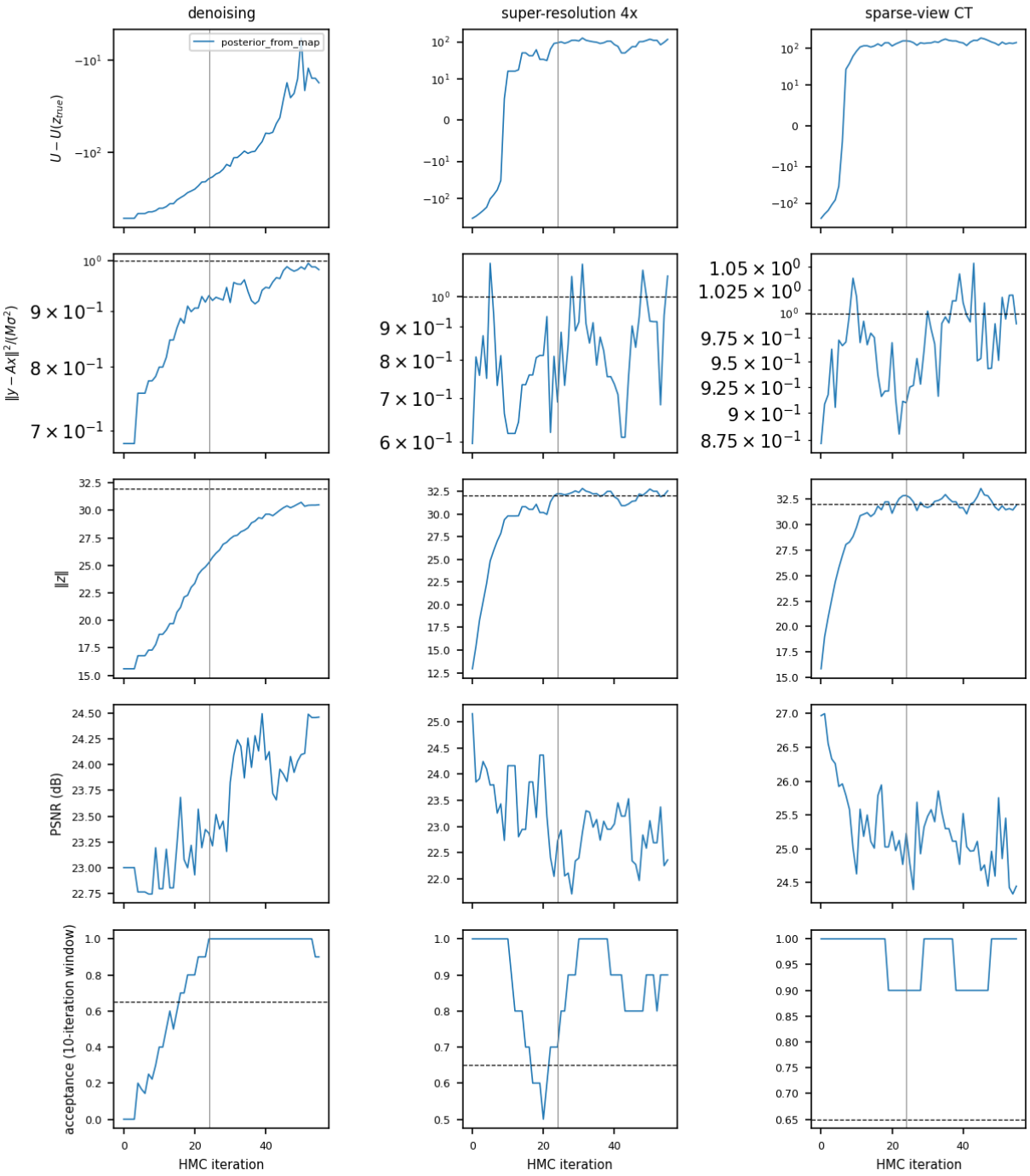


Figure 135: 32 px, traces of the random walks.

17.4 Analysis

- **The sampler works; the cost is the flow's reverse.** Every one of the 54 chains reached the target acceptance: 0.41–0.90 in the sampling phase, with the step size adapted per system to 1.2–1.6×10^{-3} (box), 1.6–2.2×10^{-3} (random), 3–4×10^{-3} (denoising, 2× SR), 4.5×10^{-3}–1.8×10^{-2} (4× SR) and 1.1–2.2×10^{-2} (CT) — a ten-fold spread that is the stiffness of the measured directions, and which the MALA pilot could not live with. One gradient of U (a differentiable reverse of the flow for the batch of 54) cost 1.21 s next to the 128-px training, 19.3 s per iteration of 16 leapfrog steps, 52 minutes for 2,560 gradients; the MLE initialisation (300 Adam steps on a batch of 6) took 119 s. A chain decorrelates along the unmeasured, unit-curvature directions after travelling a distance of order one, i.e. after $\sim 1/\varepsilon$ gradients: 60–80 gradients for CT, 300–800 for the stiff four. That single number decides everything below.

- **Claim (i), leaving the MAP: confirmed for CT and 4× SR, not reached for the other four.** From the latent MAP, $\|z\|$ went $12.1 \to 31.6$–32.3 for CT and $5.7 \to 27.7$–31.4 for 4× SR, the residual from 0.84 to 0.92–1.01 and from 0.53 to 0.69–0.77 $M\sigma^2$, and the chains moved 27–31 from their start — against $\sqrt{32^2 + \|z_0\|^2} \approx 32$–$34$ for an independent point of the shell, i.e. they have forgotten where they began. Their U ends at or above the truth's (CT 763–780 vs. 634; 4× SR 273–384 vs. 314): a typical posterior point, not a dense one. The four stiff systems did not get there in 160 iterations: $\|z\|$ $15.6 \to 21.8$–24.6 (denoising), $20.3 \to 20.9$–21.6 (box), $16.3 \to 17.0$–18.5 (random), $11.5 \to 18.1$–21.1 (2× SR); their U is still 100–350 nats *below* the truth's (denoising 223–289 vs. 507, box -1204 to -1171 vs. -1021, random -370 to -334 vs. 10, 2× SR -246 to -191 vs. -33), and the box and random chains moved only 5–9 from their start. The traces say the same in two ways: $\|z\|$ of denoising and 2× SR is still rising with no plateau at iteration 160, and box and random are flat at a value that is not the shell — the chain is trapped by the step size, not converged. Those four are the `_stiff` run's business. The residuals do move in the claimed direction everywhere (0.56–0.86 at the MAP to 0.78–0.96 at the end), which is the noise-fitting of the latent MAP being undone.

- **Claim (ii), the mean and the samples — Matt's question answered.** Per row, the posterior mean beats the samples by 0.1–2.2 dB (denoising 23.3 vs. 25.0, box 21.1 vs. 21.2, random 23.4 vs. 23.6, 2× SR 25.9 vs. 26.8, 4× SR 23.4 vs. 25.6, CT 25.3 vs. 27.0; samples first), the MMSE ordering. Against the latent MAP (step 14: 23.0, 21.6, 23.5, 25.9, 25.3, 27.6 dB) the posterior mean is *higher* in four of six (denoising +2.0, random +0.1, 2× SR +0.9, 4× SR +0.3) and lower in box (-0.4) and CT (-0.6). The samples themselves: within 0.5 dB of the MAP where the chains stayed near it (denoising +0.3, random -0.1, 2× SR 0.0, box -0.5) and 1.9–2.3 dB below it where they reached the shell (4× SR, CT) — the price of being a draw rather than an estimate. What the figures show is the thing the numbers cannot: the chain finals have speckled parenchyma, vessel dots and a crisp chest wall; the latent MAP is smooth; the posterior mean is smooth again. So the hypothesis "the MAP is the mean, hence the blur" is half right and half wrong for this model. Right: the mean *is* smooth, and it is the higher-PSNR object. Wrong about the mechanism: the latent MAP is not blurry because it averages — it is a mode, not a mean — it is blurry because it sits at $\|z\|$ 6–20, and the generator maps small-norm latents to smooth images (step 10's temperature effect, arrived at by optimisation instead of by choice). A posterior sample at $\|z\| \approx 32$ has the texture back, and costs 0–2.3 dB of PSNR for it. If one wants a single sharp image, neither the MAP nor the mean is it; a sample is, and a sample is what the videos show.

- **Claim (iii), stiffness: confirmed**, as the numbers of (i) already say; the per-iteration displacement $L\varepsilon \approx 0.02$–$0.07$ for the stiff four is the whole explanation, and the first iteration of the $L = 128$ run moved denoising from $\|z\|$ 15.6 to 17.1 and 2× SR from 11.5 to 13.9 — what took the $L = 16$ run 20–30 iterations.

- **Claim (iv), the MLE start: on the shell from the first step, and a mixing check that passes where it should.** The MLE chains stay at $\|z\|$ 32.4–33.7 throughout (24.8–25.5 for random, whose MLE sat at 24.3). Where the MAP chains reached the shell the two starts agree: CT posterior means 27.0 vs. 27.0 dB, chain U 762–830 from both, $\|z\|$ 31.5–33.7 from both; 4× SR 25.6 vs. 24.9 dB, U 273–406. Where they did not, the two starts have not met (box: $\|z\|$ 21 vs. 33 and U −1190 vs. −900; 2× SR: 20 vs. 33 and −220 vs. +90), which is the honest statement that neither has converged. The MLE start also keeps a trace of its noise-fitting: residuals 0.60–0.69 (2× SR), 0.66–0.80 (random), 0.85–0.87 (box) at the end, still rising in the traces, against 0.83–0.96 from the MAP. Its chains adapted to a larger step size than the MAP chains on denoising and box (5–8×10^{-3} vs. 3–4×10^{-3}; 3.3×10^{-3} vs. 1.4×10^{-3}): the posterior is less curved on the shell than in the basin. The PSNR of the MLE-start means is 0.3–1.2 dB below the MAP-start means everywhere except box and random (where it is 0.3–0.5 dB above), so for a single run the MAP start is the better operating point and the MLE start is the check on it.

- **Claim (v), $\tau = 0.7$: confirmed where it could act.** For CT and 4× SR the cooled chains settle at $\|z\|$ 22–24 $\approx 0.7\sqrt{n}$, and give the best numbers in the table: samples 26.2 dB (CT; +0.9 over $\tau = 1$) and 24.2 dB (4× SR; +0.8), means 27.6 and 25.5 dB. For the stiff four the cooling changed nothing (within 0.3 dB, same $\|z\|$) because those chains never reached the radius at which the prior term differs. The posterior std maps (0.013–0.057 in the $[-1, 1]$ pixel units, largest for denoising and 4× SR, smallest for the inpaintings where the std is confined to the missing pixels) shrink under $\tau = 0.7$ where the cooling acted (CT 0.038 to 0.031, 4× SR 0.057 to 0.041) and not where it did not (denoising 0.057 to 0.053, box and random unchanged), as a narrower prior should make them.

- **Operating point, as of the main run.** HMC, $L = 16$, ε adapted to 0.65 acceptance, from the latent MAP, $T = 1$: correct and converged for CT and 4× SR in 160 iterations (52 min shared); $\tau = 0.7$ if one is willing to trade a narrower posterior for +0.8 dB. For denoising, both inpaintings and 2× SR the same sampler needs longer trajectories — the `_stiff` run, below. What the videos show, checked frame by frame on CT and on the last frame of denoising: three chains that differ from each other in the parenchymal detail and agree on the anatomy, a running mean that settles within the first 30 sampling iterations, a running std concentrated on vessels and the chest wall (CT) or spread over the lung fields (denoising).

- **The $L = 128$ run (`_stiff`): the stiff four reach the shell, two of them within the budget.** Same sampler, same target, 128 leapfrog steps per iteration, 30 iterations, two chains, from the MAP and from the MLE; 35 min shared with the 128-px training (0.54 s per gradient at batch 16). The adapted step sizes are the main run's (2.5–3.5×10^{-3} for denoising and 2× SR, 1.4–2.3×10^{-3} for box, 1.7–2.0×10^{-3} for random), so one iteration now moves $L\varepsilon \approx 0.2$–0.4 instead of 0.02–0.07, and the first iteration did what the main run's first 20–30 did (claim (iii)). Denoising and 2× SR converge: from $\|z\|$ 15.6 and 11.5 both chains sit at 31.1–32.1 and 32.0 by iteration 25, having moved 29–32 units — the whole way to the shell; the residual ratios are 0.96–0.99 and 0.81–0.92; the chain energies bracket the truth's for denoising (U 494 and 545 against 507) and sit 116–130 nats above it for 2× SR (U 84 and 97

against −33, with data terms −428 and −414 bracketing the truth's −423) — the CT and 4× SR picture of the main run. The MLE start agrees on denoising: $\|z\|$ 30.7–32.4, U 491–555, means 26.7 vs. 26.1 dB, and its chains ended 40–41 units from where they started, farther than their own norm, i.e. at an unrelated point of the shell. On 2× SR the MLE start has not forgotten its noise-fitting: one chain still has residual 0.68, a data term 21 nats below the truth's and a pixel at 1.69, and the two means differ (26.6 vs. 29.0 dB), so for 2× SR only the MAP start is converged. Box and random inpainting are not there in 30 iterations. The MAP chains climbed from $\|z\|$ 20.3 to 26.4–28.2 (box) and from 16.3 to 23.5–26.4 (random), still rising linearly in the traces; the box energies straddle the truth's (−978 and −1034 against −1021) but the MLE chains sit at −848 to −903 and $\|z\|$ 32–33, so the two starts are 5–6 units and 100 nats apart; random is 130–200 nats below the truth from both starts, with acceptance 0.38–0.45 at $\varepsilon \approx 2 \times 10^{-3}$ (the adaptation froze after 8 iterations, before the step size had come down to the 0.65 target). At the present rate box needs roughly 20 more iterations and random 40–60, i.e. another 25–70 min at this batch. The reconstructions change the picture of claim (ii) for these systems: the samples on the shell are *above* the latent MAP in PSNR — denoising 24.1–24.2 against 23.0, box 22.0–22.2 against 21.6, random 23.6–23.9 against 23.5, 2× SR 26.6–26.7 against 25.9 — and the posterior means (40 samples each) are 26.1, 22.7, 24.3 and 29.0 dB: +3.1, +1.1, +0.8 and +3.1 dB over the MAP, and +1.1, +1.5, +0.7 and +2.2 dB over the main run's means, which averaged a neighbourhood of the MAP rather than the posterior. The gain is largest where the MAP sits deepest in the basin ($\|z\|$ 11.5 for 2× SR, 15.6 for denoising), so the −1.9 to −2.3 dB of CT and 4× SR is not a rule: on one image per system, the sign depends on the problem. The videos show what the stills do — two chains that agree on the anatomy and differ in the parenchyma; for box inpainting the two chains draw visibly different lung boundaries inside the missing square and the std map is confined to it (mean std 0.027, against 0.060 for denoising, 0.032 for random and 0.037 for 2× SR). The RMS distance between the two chains' final images is 0.10–0.14 in $[-1, 1]$ units.

- **Operating point, revised.** HMC from the latent MAP, $T = \tau = 1$, ε adapted to 0.65 acceptance, with L chosen by the stiffness of the system: $L = 16$ for CT and 4× SR (converged in 160 iterations; 52 min for three chains on six systems), $L = 128$ for denoising and 2× SR (30 iterations, the last 25 on the shell; 35 min for two chains on four systems), $L = 128$ and 70 iterations for box inpainting (the `_inpaint` run below); random inpainting is the one system this sampler has not brought to the shell from the MAP. The MLE start is the mixing check; it passes on denoising and, in the main run, on CT and 4× SR. The cooled prior $\tau = 0.7$ is the option for a sharper single sample at the price of a narrower posterior, tested on CT and 4× SR only. All PSNR numbers in this section are for one image per system.

- **The inpaintings, three times longer (`_inpaint`): box converges, random does not.** Same sampler, $L = 128$, 90 iterations with 30 of burn-in and a 25-iteration adaptation from $\varepsilon_0 = 1.5 \times 10^{-3}$, two chains from the MAP and two from the MLE; batch 8, 82 min at 0.43 s per gradient (Figures 132 and 133). *Box.* The MAP chains climb from $\|z\|$ 20.3 to 32.7 and 33.5 — the traces rise linearly to about iteration 65 and then level off — having moved 32 units, their whole starting distance from the shell; the energies cross the truth's at iteration 28 and end at −846 and −817 against −1021, i.e. 175–204 nats above it, with the residual at 1.02 and the data term (−1380) at the noise level (−1388); acceptance 0.96 at $\varepsilon = 1.4$–1.6×10^{-3}. The MLE chains, which start on the shell (32.7) at residual 0.69, forget the noise-fitting over the first 40 iterations (residual 0.97–0.99 at the end) and finish at $\|z\|$ 33.4–33.6, U −841 and −828 — within 20 nats of the MAP chains — 40–41 units

from where they started. Four chains from two starts 12 units apart agree on $\|z\|$ to 1 unit and on U to 30 nats; that is the mixing check passing for box inpainting, which the `_stiff` run's 30 iterations could not show. The final images are 22.0–22.9 dB from the MAP start and 21.3–21.5 from the MLE start (MAP 21.6), the means over 120 samples 23.8 and 23.5 dB (+2.2 and +1.9 over the MAP), and the two chains of a pair are 0.17 RMS apart in the image. *Random (70% dropped).* The MAP chains climb from $\|z\|$ 16.3 to 24.5–25.5 and stop: the acceptance, 0.52 in burn-in, falls to 0.25 over the sampling phase and to 0 after iteration 70, with the step size frozen at $1.0–1.2 \times 10^{-3}$ since the adaptation ended at iteration 25; after iteration 70 no proposal is accepted. The energies stop at -149 and -187, 160–200 nats *below* the truth's 10 — the chains are still inside the basin, not on the shell — and the residual sits at 0.96–1.05. The MLE chains do better: from 24.3 to 29.2–30.1, energies -35 and -57 and still rising at the end, acceptance 0.57 at $1.1–1.4 \times 10^{-3}$. So for this system the two starts have *not* met (5 units and 100–130 nats apart), the MAP-start chains are not samples of the posterior, and the correct statement is that random inpainting at 32 px needs a smaller step than the adaptation reached — of order 5×10^{-4}, judging by the collapse — and correspondingly more iterations, or a different proposal. The PSNRs are reported for completeness and not as posterior statistics: samples 24.1 (MAP start) and 24.3 dB (MLE start), means 24.9 and 25.3 dB against the MAP's 23.5.

- **One walk per system, 32 samples as frames (`_walk`).** On the request for a quick animation at each rung rather than a study: a single chain from the latent MAP, $L = 64$, 24 burn-in iterations and 32 recorded ones, for denoising, 4× SR and CT; batch 3, 24 min at 0.40 s per gradient (Figure 134; the videos are the 32 samples at 4 frames per second). All three chains reach the shell within the burn-in — $\|z\|$ 15.6 → 30.5 (denoising), 5.7 → 32.5 (4× SR), 12.1 → 31.9 (CT), each having moved 29–32 units — at adapted step sizes of 3.2×10^{-3}, 7.8×10^{-3} and 1.1×10^{-2} and sampling-phase acceptances of 0.97, 0.91 and 0.97 (above the 0.65 target: the adaptation's 20 iterations ended before the step had grown into it, so the walk is shorter per iteration than it could be). Residuals 0.98, 1.08 and 0.99; energies 490 (truth 507), 431 (314) and 780 (634): denoising brackets the truth, 4× SR and CT sit 117 and 146 nats above it, the main run's picture. PSNR of the last state / the samples on average / the 32-sample mean: 24.5 / 24.0 / 25.2 dB for denoising (MAP 23.0), 22.4 / 22.8 / 24.5 for 4× SR (MAP 25.3), 24.4 / 25.1 / 27.0 for CT (MAP 27.6); std maps 0.045, 0.056 and 0.041 on average, concentrated on the lung boundaries and the mediastinum for denoising and CT and on the body outline and the mediastinum for 4× SR. With 32 samples from one chain at these step sizes the frames are correlated (a displacement of $L\varepsilon \approx 0.2–0.7$ per iteration against a shell of radius 32) and the "posterior mean" is a 32-sample average, so these numbers are the animation's, not the study's; they agree with the main run's three-chain means (25.0, 25.6, 27.0 dB) to 0.2 dB on denoising and CT and fall 1.1 dB short on 4× SR.

18 Reproducibility

- Global seed `20260831` (config) seeds torch, numpy, and every `random_split`; splits are therefore identical across runs and machines.
- `outputs/` and dataset caches are gitignored; the committed code + config + this PDF are the record, and any output can be regenerated with the commands above.
- Provenance for the run in this PDF:

```
{
  "step": "step1_datasets",
  "command": "python step1_datasets.py",
  "config_file": "/dev_ws/invertible_normalizing_flow_20260831/configs/step1_datasets.yml",
  "config": {
    "seed": 20260831,
    "data_root": "/data/image_benchmarks",
    "output_root": "outputs/step1_datasets",
    "loader": {
      "batch_size": 64,
      "num_workers": 4,
      "pin_memory": true
    },
    "split": {
      "val_fraction": 0.1,
      "test_fraction": 0.1
    },
    "figures": {
      "samples_per_split": 8,
      "stats_max_batches": 20
    },
    "datasets": {
      "mnist": {
        "enabled": true
      },
      "fashion_mnist": {
        "enabled": true
      },
      "cifar10": {
        "enabled": true
      },
      "cifar100": {
        "enabled": true
      },
      "svhn": {
        "enabled": true
      },
      "medmnist": {
        "enabled": true,
        "size": 28,
        "flags": [
          "pathmnist",
          "bloodmnist",
          "dermamnist",
          "pneumoniamnist",
          "breastmnist",
          "retinamnist",
          "organamnist"
        ]
      },
      "celeba": {
        "enabled": true,
        "image_size": 64
      },
      "imagenette": {
        "enabled": true,
        "variant": "160px",
        "image_size": 64
      },
      "imagenet": {
        "enabled": true,
        "image_size": 64
      },
      "afhq": {
        "enabled": true,
        "version": "v1",
        "image_size": 64,
        "attempt_download": true
```

```
      },
      "ffhq": {
        "enabled": true,
        "image_size": 128
      },
      "celeba_hq": {
        "enabled": true,
        "image_size": 256
      },
      "lsun_bedroom": {
        "enabled": true,
        "image_size": 64
      }
    }
  },
  "timestamp_utc": "2026-08-31T21:22:10.697662+00:00",
  "versions": {
    "python": "3.10.12",
    "torch": "2.11.0+cu128",
    "torchvision": "0.26.0+cu128",
    "numpy": "2.2.6",
    "medmnist": "3.0.2"
  },
  "cuda_available": true
}
```


- Provenance for step 2 (includes the pinned ml-tarflow commit):


```
{
  "step": "step2_tarflow",
  "command": "python step2_tarflow.py --collect",
  "config": {
    "seed": 20260831,
    "tarflow_repo": "/dev_ws/ml-tarflow",
    "output_root": "outputs/step2_tarflow",
    "runs": {
      "mnist_toy": {
        "device": "cuda:0",
        "dataset": "mnist",
        "conditional": true,
        "img_size": 28,
        "channel_size": 1,
        "hflip": false,
        "patch_size": 4,
        "channels": 128,
        "blocks": 4,
        "layers_per_block": 4,
        "noise_type": "gaussian",
        "noise_std": 0.1,
        "cfg": 0,
        "batch_size": 256,
        "lr": 0.002,
        "epochs": 100,
        "sample_freq": 10,
        "sample_rows": 10,
        "sample_nrow": 10,
        "eval_bpd": false,
        "eval_freq": 0,
        "classifier_eval": true
      },
      "cifar10_uniform": {
        "device": "cuda:1",
        "dataset": "cifar10",
        "conditional": false,
        "img_size": 32,
        "channel_size": 3,
        "hflip": true,
```

```
        "patch_size": 2,
        "channels": 256,
        "blocks": 8,
        "layers_per_block": 4,
        "noise_type": "uniform",
        "cfg": 0,
        "batch_size": 128,
        "lr": 0.0002,
        "epochs": 60,
        "sample_freq": 10,
        "sample_rows": 8,
        "sample_nrow": 8,
        "eval_bpd": true,
        "eval_freq": 5,
        "classifier_eval": false
      }
    }
  },
  "tarflow_repo": "https://github.com/apple/ml-tarflow",
  "tarflow_commit": "313120e9dfaec546b3f464d0fa87866eb0c309fa",
  "paper": "Zhai et al., Normalizing Flows are Capable Generative Models, arXiv:2412.06329",
  "timestamp_utc": "2026-08-31T22:39:49.279035+00:00",
  "versions": {
    "python": "3.10.12",
    "torch": "2.11.0+cu128",
    "torchvision": "0.26.0+cu128"
  },
  "results": [
    "mnist_toy",
    "cifar10_uniform"
  ]
}
```


- Provenance for steps 3 and 4:


```
{
  "step": "step3_invertible_modules",
  "config": {
    "seed": 20260901,
    "output_root": "outputs/step3_invertible_modules",
    "tarflow_repo": "/dev_ws/ml-tarflow",
    "dims": {
      "flat": 64,
      "image": [
        2,
        8,
        8
      ],
      "seq": 64
    },
    "feature_domain": {
      "img_size": 32,
      "patch_size": 4
    },
    "batch": 64,
    "timing_reps": 25,
    "viz_patch": 4,
    "tolerances": {
      "recon_fp32": 0.0001,
      "logdet": 0.001,
      "orthogonality": 1e-05,
      "metablock": 0.001
    }
  },
  "command": "python step3_invertible_modules.py",
  "timestamp_utc": "2026-09-01T12:36:49.070649+00:00",
  "versions": {
```

```
    "python": "3.10.12",
    "torch": "2.11.0+cu128"
  },
  "device": "cpu",
  "n_pass": 26,
  "n_total": 26
}

{
  "step": "step4_tarflow_ablation",
  "config": {
    "seed": 20260901,
    "tarflow_repo": "/dev_ws/ml-tarflow",
    "output_root": "outputs/step4_tarflow_ablation",
    "dataset": "mnist",
    "img_size": 32,
    "channel_size": 1,
    "patch_size": 4,
    "noise_type": "uniform",
    "grad_clip": 1.0,
    "model": {
      "channels": 128,
      "blocks": 4,
      "layers_per_block": 2
    },
    "sample_nrow": 6,
    "budget": {
      "full": {
        "epochs": 120,
        "train_subset": 0,
        "val_subset": 4000,
        "test_subset": 10000,
        "batch": 128,
        "lr": 0.001,
        "sample": true
      },
      "reduced": {
        "epochs": 6,
        "train_subset": 12000,
        "val_subset": 2000,
        "test_subset": 4000,
        "batch": 128,
        "lr": 0.001,
        "sample": true
      }
    },
    "variants": [
      "baseline_flip",
      "identity_only",
      "random_perm",
      "haar",
      "hadamard",
      "dct",
      "hartley",
      "rand_ortho",
      "householder",
      "cayley"
    ]
  },
  "profile": "full",
  "device": "cuda:0",
  "command": "python step4_tarflow_ablation.py",
  "tarflow_repo": "https://github.com/apple/ml-tarflow",
  "tarflow_commit": "313120e9dfaec546b3f464d0fa87866eb0c309fa",
  "timestamp_utc": "2026-09-01T16:49:41.545975+00:00",
  "versions": {
    "python": "3.10.12",
    "torch": "2.11.0+cu128"
```

```
  },
  "ranking": [
    "baseline_flip",
    "cayley",
    "haar",
    "random_perm",
    "householder",
    "hadamard",
    "dct",
    "rand_ortho",
    "hartley",
    "identity_only"
  ]
}
```


- Provenance for steps 7 and 8 (both include the pinned ml-tarflow commit; step 7's is that of the third attempt, the archived attempts carry their own):


```
{
  "step": "step7_system_bridge",
  "config": {
    "seed": 20260901,
    "tarflow_repo": "/dev_ws/ml-tarflow",
    "output_root": "outputs/step7_system_bridge",
    "dataset": "mnist",
    "img_size": 32,
    "channel_size": 1,
    "patch_size": 4,
    "noise_type": "uniform",
    "grad_clip": 1.0,
    "loss_reject_mult": 5.0,
    "loss_reject_abs": 5.0,
    "scale_bound": 8.0,
    "train_target": "expected",
    "max_rewinds": 3,
    "model": {
      "channels": 128,
      "blocks": 4,
      "layers_per_block": 2
    },
    "budget": {
      "full": {
        "epochs": 120,
        "train_subset": 0,
        "val_subset": 4000,
        "test_subset": 10000,
        "batch": 128,
        "lr": 0.001
      },
      "reduced": {
        "epochs": 6,
        "train_subset": 12000,
        "val_subset": 2000,
        "test_subset": 4000,
        "batch": 128,
        "lr": 0.001
      }
    },
    "systems": {
      "denoise": {
        "kind": "denoise",
        "sigma": 0.2,
        "beta": 1.0
      },
      "inpaint_box": {
        "kind": "inpaint_box",
        "sigma": 0.05,
```

```
        "box": 12,
        "beta": 1.0
      },
      "inpaint_random": {
        "kind": "inpaint_random",
        "sigma": 0.05,
        "drop": 0.5,
        "seed": 0,
        "beta": 1.0
      },
      "sr_2x": {
        "kind": "sr_2x",
        "sigma": 0.05,
        "beta": 1.0
      },
      "sr_4x": {
        "kind": "sr_4x",
        "sigma": 0.05,
        "beta": 1.0
      },
      "denoise_gauss": {
        "kind": "denoise",
        "sigma": 0.2,
        "beta": 1.0,
        "noise_type": "gaussian",
        "noise_std": 0.05,
        "scale_bound": 0,
        "train_target": "sampled",
        "max_rewinds": 0
      },
      "inpaint_random_gauss": {
        "kind": "inpaint_random",
        "sigma": 0.05,
        "drop": 0.5,
        "seed": 0,
        "beta": 1.0,
        "noise_type": "gaussian",
        "noise_std": 0.05,
        "train_target": "sampled",
        "max_rewinds": 0
      }
    }
  },
  "profile": "full",
  "device": "cuda:0",
  "started": "2026-09-01T20:13:30.184206+00:00"
}

{
  "step": "step8_conditional_flow",
  "config": {
    "seed": 20260901,
    "tarflow_repo": "/dev_ws/ml-tarflow",
    "output_root": "outputs/step8_conditional_flow",
    "dataset": "mnist",
    "img_size": 32,
    "channel_size": 1,
    "patch_size": 4,
    "noise_type": "uniform",
    "grad_clip": 1.0,
    "loss_reject_mult": 5.0,
    "loss_reject_abs": 5.0,
    "scale_bound": 8.0,
    "max_rewinds": 2,
    "model": {
      "channels": 128,
      "blocks": 4,
      "layers_per_block": 2
```

```
    },
    "budget": {
      "full": {
        "epochs": 120,
        "train_subset": 0,
        "val_subset": 4000,
        "test_subset": 10000,
        "batch": 128,
        "lr": 0.001
      },
      "reduced": {
        "epochs": 6,
        "train_subset": 12000,
        "val_subset": 2000,
        "test_subset": 4000,
        "batch": 128,
        "lr": 0.001
      }
    },
    "systems": {
      "ct_sparse": {
        "kind": "ct",
        "sigma": 0.5,
        "n_angles": 8,
        "rcond": 0.01
      },
      "sr_4x": {
        "kind": "sr_4x",
        "sigma": 0.05
      },
      "inpaint_box": {
        "kind": "inpaint_box",
        "sigma": 0.05,
        "box": 12
      },
      "inpaint_random": {
        "kind": "inpaint_random",
        "sigma": 0.05,
        "drop": 0.5,
        "seed": 0
      },
      "sr_2x": {
        "kind": "sr_2x",
        "sigma": 0.05
      },
      "denoise": {
        "kind": "denoise",
        "sigma": 0.2
      },
      "sr_4x_unbounded": {
        "kind": "sr_4x",
        "sigma": 0.05,
        "scale_bound": 0
      }
    }
  },
  "profile": "full",
  "device": "cuda:0",
  "tarflow_commit": "313120e9dfaec546b3f464d0fa87866eb0c309fa",
  "started": "2026-09-01T21:27:24.702384+00:00"
}
```


- Provenance for steps 9 and 10 (both include the pinned ml-tarflow commit; step 10 also records the step-9 checkpoint it used):


```
{
  "step": "step9_chestmnist_tarflow",
```

```
  "command": "python step9_chestmnist_tarflow.py",
  "config": {
    "seed": 20260903,
    "tarflow_repo": "/dev_ws/ml-tarflow",
    "output_root": "outputs/step9_chestmnist_tarflow",
    "dataset": "medmnist_chestmnist",
    "medmnist_size": 64,
    "img_size": 64,
    "channel_size": 1,
    "patch_size": 8,
    "noise_type": "uniform",
    "hflip": false,
    "model": {
      "channels": 768,
      "blocks": 8,
      "layers_per_block": 8
    },
    "scale_bound": 8.0,
    "train": {
      "epochs": 40,
      "batch": 64,
      "lr": 0.0001,
      "weight_decay": 0.0001,
      "grad_clip": 1.0,
      "eval_every": 1,
      "early_stop_patience": 4,
      "sample_every": 2,
      "sample_nrow": 8,
      "sample_rows": 8,
      "test_subset": 0
    },
    "nn_check_samples": 16
  },
  "tarflow_repo": "https://github.com/apple/ml-tarflow",
  "tarflow_commit": "313120e9dfaec546b3f464d0fa87866eb0c309fa",
  "dataset": "MedMNIST v2 ChestMNIST, size-64 release (Yang et al. 2023)",
  "timestamp_utc": "2026-09-03T06:46:17.350032+00:00",
  "versions": {
    "python": "3.10.12",
    "torch": "2.11.0+cu128",
    "torchvision": "0.26.0+cu128"
  }
}

{
  "config": {
    "seed": 20260910,
    "tarflow_repo": "/dev_ws/ml-tarflow",
    "output_root": "outputs/step10_variational_posterior",
    "prior_config": "configs/step9_chestmnist_tarflow.yml",
    "prior_checkpoint": "best",
    "measurement": {
      "name": "inpaint_box",
      "box": 24,
      "sigma": 0.05
    },
    "test_index": null,
    "fit": {
      "steps": 2000,
      "batch": 32,
      "lr": 1e-05,
      "betas": [
        0.9,
        0.99
      ],
      "grad_clip": 1.0,
      "log_every": 10,
      "eval_every": 100,
```

```
      "n_eval": 64,
      "ckpt_every": 50
    },
    "eval": {
      "n_samples": 128,
      "outlier_thresh": 2.0,
      "n_grid": 32
    },
    "walk": {
      "n_frames": 600,
      "fps": 30,
      "angle": 0.08,
      "batch": 32
    }
  },
  "prior_config": {
    "seed": 20260903,
    "tarflow_repo": "/dev_ws/ml-tarflow",
    "output_root": "outputs/step9_chestmnist_tarflow",
    "dataset": "medmnist_chestmnist",
    "medmnist_size": 64,
    "img_size": 64,
    "channel_size": 1,
    "patch_size": 8,
    "noise_type": "uniform",
    "hflip": false,
    "model": {
      "channels": 768,
      "blocks": 8,
      "layers_per_block": 8
    },
    "scale_bound": 8.0,
    "train": {
      "epochs": 40,
      "batch": 64,
      "lr": 0.0001,
      "weight_decay": 0.0001,
      "grad_clip": 1.0,
      "eval_every": 1,
      "early_stop_patience": 4,
      "sample_every": 2,
      "sample_nrow": 8,
      "sample_rows": 8,
      "test_subset": 0
    },
    "nn_check_samples": 16
  },
  "tarflow_commit": "313120e9dfaec546b3f464d0fa87866eb0c309fa",
  "torch": "2.11.0+cu128",
  "timestamp": "2026-09-03T08:53:07.535519+00:00",
  "argv": [
    "step10_variational_posterior.py"
  ]
}
```